

THE AMI TOOL KIT

VERSION 1.0

Contents

1 Introduction: What is Ami 15

1.1 History 22
1.2 Reliance on ANSI C/libc 22
1.3 Coining 22
1.4 Module format 22
1.5 Overview of Ami libraries and modularity 23
1.6 Services 24
1.7 Advanced User I/O and Presentation Management 24
1.8 Naming 26
1.9 Advanced device libraries 27
1.9.1 sound 27
1.9.2 network 27

2 Using C++ With Ami 29

2.1 Two Modes for C++ Programs 29
2.2 C++ Extension layer 29
2.3 Libraries covered by a C++ Extension layer 30
2.4 Library documentation for C++ 30

3 System Services Library 31

3.1 Filenames and Paths 31
3.2 Predefined paths 32
3.3 Time and Date 32
3.4 Directory Structures 34
3.5 File Attributes and Permissions 36
3.6 Environment Strings 36
3.7 Creating or Removing Paths 37
3.8 Option Character 37
3.9 Path Character 37

3.10 Location..... 38
3.11 Internationalization..... 42
3.12 Executing Other Programs..... 45
3.13 Threads and thread management..... 45
3.13.1 Concurrency Locks..... 47
3.13.2 Semaphores..... 48
3.14 C++ services interface..... 48
3.15 Methods in services..... 51
3.16 Functions and procedures in services..... 51
4 Terminal Interface Library..... 57
4.1 ANSI C Compatible Mode..... 60
4.2 Specifying the Main Window..... 60
4.3 Basic Cursor Positioning..... 61
4.4 Automatic Mode..... 61
4.5 Tabbing..... 62
4.6 Scrolling..... 62
4.7 Colors..... 62
4.8 Attributes..... 63
4.9 Multiple Surface Buffering..... 63
4.10 Advanced Input..... 64
4.11 Buffer Follow mode..... 69
4.12 Focus events..... 70
4.13 Hover events..... 70
4.14 Legacy Input..... 70
4.15 Event callbacks..... 70
4.16 Timers..... 72
4.17 The Frame Timer..... 73
4.18 Mouse..... 73
4.19 Joysticks..... 74
4.20 Function Keys..... 75
4.21 Automatic “hold” Mode..... 75
4.22 Direct Writes..... 76

4.23 Printers....76
4.24 Remote display....77
4.25 Advanced Input in C++....77
4.26 Terminal Objects In C++....77
4.27 Event Callbacks in C++....81
4.28 Procedures, functions and methods in terminal....85
4.29 Events in terminal....90

5 Character Windows Management Library....97

5.1 Screen Appearance....98
5.2 Rooted vs. non-rooted windows surfaces....98
5.3 Window Modes....99
5.4 Buffered Mode....99
5.5 Unbuffered Mode....99
5.6 Buffer Follow Mode....100
5.7 Delayed Window Display....101
5.8 Window Frames....101
5.9 Multiple Windows....102
5.10 Parent/Child Windows....102
5.11 Moving and Sizing Windows....103
5.12 Z Ordering....103
5.13 Class Window Handling....104
5.14 Parallel Windows....104
5.15 Menus....105
5.16 Setting Menu Active....107
5.17 Setting Menu States....107
5.18 Standard Menus....107
5.19 Menu Sublisting....108
5.20 Advanced Windowing....109
5.21 Events....109
5.22 C++ windows interface....110
5.23 Procedures and Functions in windows character mode....111
5.24 Events In windows character mode....114

6 Graphical Interface Library 117

6.1 Terminal model 117
6.2 Graphics Coordinates 119
6.3 Character Drawing 119
6.4 String Sizes and Kerning 121
6.5 Justification 121
6.6 Effects 121
6.7 Tabs 122
6.8 Colors 122
6.9 Drawing Modes 122
6.10 Drawing Graphics 123
6.11 Figures 123
6.12 Predefined Pictures 126
6.13 Scrolling 126
6.14 Clipping 126
6.15 View Transformation 126
6.16 Mouse Graphical Position 126
6.17 Animation 127
6.18 Copy between buffers 127
6.19 Buffer Follow mode 127
6.20 Printers 128
6.21 Remote display 128
6.22 Declarations 128
6.23 C++ graphics interface 129
6.24 Functions in graphics 130
6.25 Events In graphics 138

7 Graphical Windows Management Library 139

7.1 Screen Appearance 139
7.2 Rooted vs. non-rooted windows surfaces 139
7.3 Window Modes 140
7.4 Buffered Mode 140
7.5 Unbuffered Mode 140

7.6 Buffer Follow Mode....141
7.7 Defacto transparency....142
7.8 Delayed Window Display....142
7.9 Window Frames....142
7.10 Multiple Windows....143
7.11 Parent/Child Windows....144
7.12 Moving and Sizing Windows....145
7.13 Z Ordering....145
7.14 Class Window Handling....146
7.15 Parallel Windows....146
7.16 Menus....147
7.17 Setting Menu Active....149
7.18 Setting Menu States....150
7.19 Standard Menus....150
7.20 Menu Sublisting....151
7.21 Advanced Windowing....152
7.22 Events....152
7.23 C++ windows interface....153
7.24 Procedures and Functions in windows....155
7.25 Events In windows....159

8 Widgets Library....161

8.1 Tiles, Layers and Looks....161
8.2 Background Colors and Placement....162
8.3 Sizes....162
8.4 Logical Widget Identifiers....162
8.5 Killing, Selecting, Enabling and Getting Text to and from Widgets....162
8.6 Resizing and repositioning a widget....163
8.7 Types of widgets....163
8.8 Z ordering....163
8.9 Controls....164
8.9.1 Buttons....164
8.9.2 Checkboxes....165

8.9.3 Radio Buttons....165
8.9.4 Scrollbars....165
8.9.5 Number Select Box....167
8.9.6 Edit Boxes....167
8.9.7 List Boxes....168
8.9.8 Drop Down Box....169
8.9.9 Drop/Edit Box....169
8.9.10 Sliders....170
8.9.11 Tabbed Bars....171
8.10 Components....172
8.10.1 Background Box....172
8.10.2 Group Boxes....173
8.10.3 Progress Bars....173
8.11 Dialogs....174
8.11.1 Alert Dialogs....174
8.11.2 Color Query Dialogs....175
8.11.3 Open File Query Dialogs....175
8.11.4 Save File Query Dialogs....176
8.11.5 Find Query Dialogs....177
8.11.6 Search and Replace Query Dialogs....178
8.11.7 Font Query Dialogs....178
8.12 Events....180
8.13 C++ widgets interface....180
8.14 Procedures and functions in widgets....182
8.15 Events In widgets....196
9 Sound Library....199
9.1 Ports....200
9.2 MIDI Output: Composition Interface....200

9.3 Notes....201
9.4 Channels and Instruments....202
9.4.1 Piano Group....204
9.4.2 Chromatic percussion Group....204
9.4.3 Organ Group....204
9.4.4 Guitar Group....204
9.4.5 Bass Group....205
9.4.6 Solo strings Group....205
9.4.7 Ensemble Group....205
9.4.8 Brass Group....205
9.4.9 Reed Group....206
9.4.10 Pipe Group....206
9.4.11 Synth lead Group....206
9.4.12 Synth pad Group....206
9.4.13 Synth effects Group....207
9.4.14 Ethnic Group....207
9.4.15 Percussive Group....207
9.4.16 Sound effects Group....208
9.4.17 Drum Channel....208
9.5 Volume....209
9.6 Time and the Sequencer....210
9.7 Effects....212
9.8 Pitch Changes....213
9.9 MIDI Input/Output: Structure Interface....213
9.10 Prerecorded MIDI Files....215
9.11 Waveform Input/Output....215
9.12 Prerecorded Waveform Files....220
9.13 C++ sound interface....221
Functions and Procedures in sound....223

10 Networking Library....233
10.1 IPv4 and IPv6 addresses....233
10.2 Standard stream channels....233
10.3 Secure channels....234
10.4 Message based communications....234
10.5 Reliable messaging....234
10.6 Serving Connections....235
10.7 Managing certificates....235
10.8 C++ network interface....237
Functions and Procedures in network....239
11 Option: Command Line Option Processing....243
11.1 Functions and Procedures in Option....247
12 Config: The configuration database....249
12.1 Functions and Procedures in config....256
13 Print modules....257
13.1 Including a print subsystem....257
12.5 pdfgraph....258
12.6 txtterminal....258
14 Remote mode....259
14.1 graph_server....260
15 The remote display protocol....263
12.7 Introduction....263
12.8 Architecture....263
12.9 Transport....264
12.10 Wire format....264
12.11 Menu trees....265
12.12 Event records....265
12.13 Session....265
12.14 Calls, queries and ordering....265

12.15 Windows....266
12.16 Events and client-local operations....266
12.17 File transfer....266
12.18 The module partition....267
12.19 Sound....267
12.20 Future work....267
12.21 Message catalog....267

16 Widget Creation....277

16.1 How the overrides work on stock widgets....277
16.2 New widgets....278
16.3 The widget base....278
16.4 An example: the kick button....279

17 Sound module plugins....283

17.1 Functions in sound plug-ins....284

18 Example Applications....287

19 Libc and alternatives....291

19.1 stdio....291
19.2 Linker overriding....293
19.3 The STDIO_BYPASS flag....293

20 Directory layout....295

21 Installing and building Ami....297

21.1 Standard make targets....298

22 Building user applications....301

22.1 Models and forms....302
22.2 Building outside the tree....302
22.3 How the makefile links the program....303
22.4 C++ programs....304
22.5 Platforms....304

23 Testing Ami....307

23.1 The regression....307
23.2 The tests....307
23.3 Judging text....308
23.4 Judging terminal listings....309
23.5 Judging screens....309
23.6 Driving the tests: the event file....310
23.7 The standards....312
23.8 The tests that want a person....313
23.9 When a test dies....313
23.10 The test report....314

24 Windows Specific Details....315

24.1 Terminal....315
24.1.1 Transparent mode....315
24.1.2 Configuration settings....316
24.1.3 Redirected files....316
24.1.4 Bypass input....317
24.2 Graphics....317
24.2.1 Configuration settings....317
24.2.2 Bypass input....318
24.2.3 Delayed output....318
24.3 Printing....318

25 Linux Specific details....319

25.1 Terminal....319
25.1.1 Configuration settings....319
25.1.2 Redirected files....320
25.1.3 Bypass input....321
25.1.4 Remote operation....321
25.2 Graphics....322
25.2.1 The X backend....322

25.2.2 The Wayland backend....323
25.2.2.1 Gnome....323
25.2.2.2 Plasma....323
25.2.3 Configuration settings....323
25.2.4 The desktop packages....324
25.2.5 The widget theme....324
25.2.6 Redirected files....325
25.2.7 Bypass input....326
25.2.8 Delayed output....326
25.3 Sound....327
25.3.1 ALSA device names....327
25.4 Network....328
26 Mac OS X specific details....329
26.1 What is the Mac's own....329
26.2 Terminal....329
26.3 Graphics....329
26.3.1 The main thread....330
26.3.2 Widgets....330
26.3.3 Configuration settings....330
26.4 Sound....330
27 License....333
27.1 Code in Ami that is not mine....333
27.2 What the tree carries besides Ami....334
27.3 What a program links....334

1 Introduction: What is Ami



Ami is a general purpose tool kit designed for C, but enabled with bindings for several languages and thus not restricted to a particular language.

Ami is unlike many graphical TKs in use today in that:

1. It does not force you to “change models” from standard line oriented or even terminal oriented code. Instead of introducing a new, windowed graphical paradigm, Ami works with the code you have, immediately adapts it to any target environment, and then lets you add terminal, or graphical, or advanced widget controls at will, complementing the existing code.
2. Ami does not force you to use or emulate an object oriented code model. Although it is common to add an object model layer atop Ami, it is not required.
3. Ami does not force you to use callbacks. Callbacks can break the linear structure of the code, and they are optional in Ami. Further, several languages allow for nested functions, and a nested function cannot cleanly be passed as a callback.
4. Ami is designed to be language agnostic. It does not use void pointers, rely on casting to represent multiple types, etc. Only the constructs that make sense for all languages are represented.
5. You can use any part of Ami you wish. Each module of Ami is separate and does not rely on the others. You can use just one module, or some of the modules, or all of it. It’s your choice.

The first and most visible aspect of Ami is that any C program, graphical or not, can be immediately compiled and run with it. The only dependency is the C standard ANSI library for C, commonly called the “whitebook” standard.

The standard “hello, world” program is used as an example. This program, like any other program written in C that does not already include graphics or terminal controls, can be redirected to any supported operating system, and any model of line oriented, terminal oriented, full graphical display, and windowed environment of any of text mode or graphics mode. If the same CPU is targeted, only a relink is required, not a full recompile.

The hello world program:

```
#include <stdio.h>

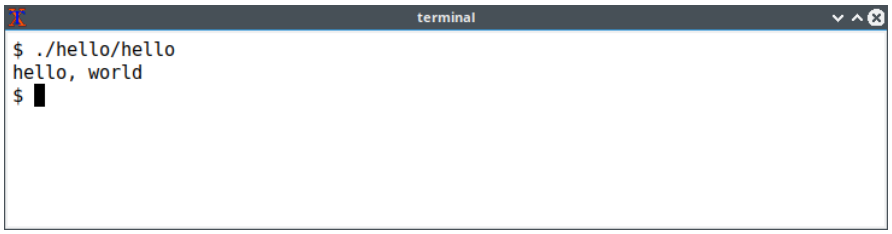
int main(void)

{

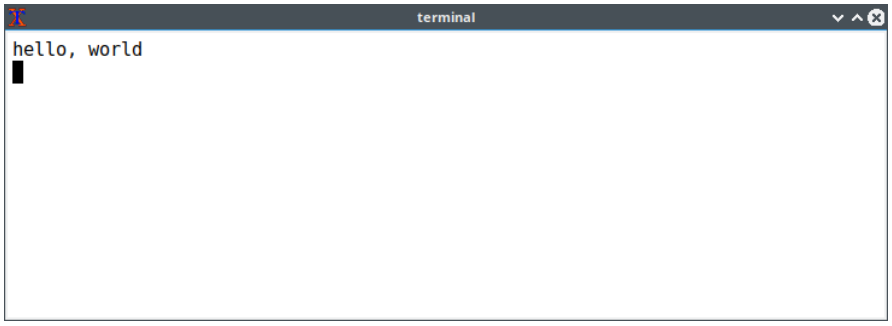
    printf("hello, world\n");

    return (0);

}
```



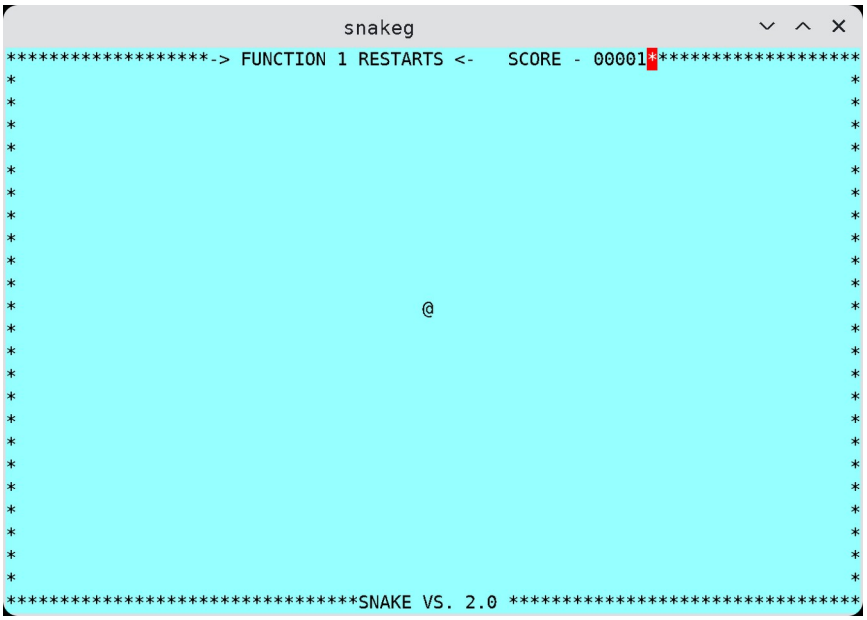
Hello world on the command line.



Hello world on a terminal.



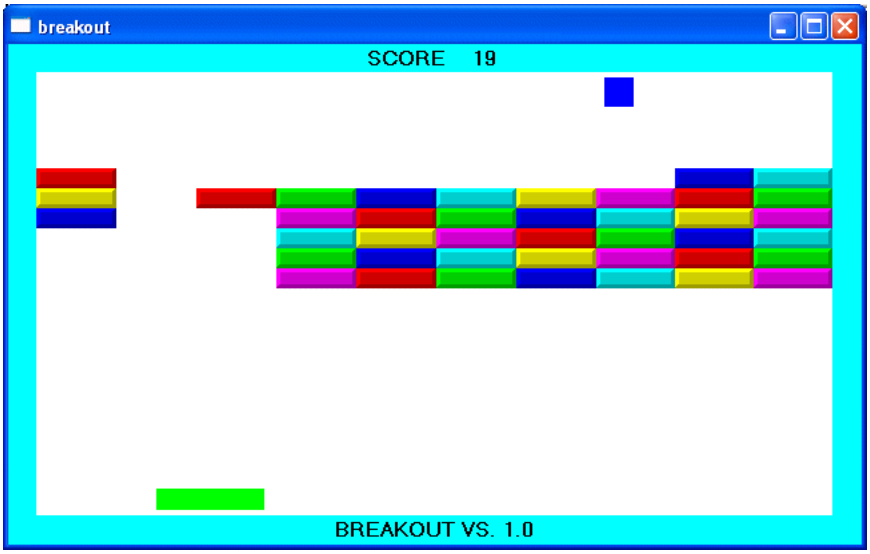
Hello, world in a graphical window.



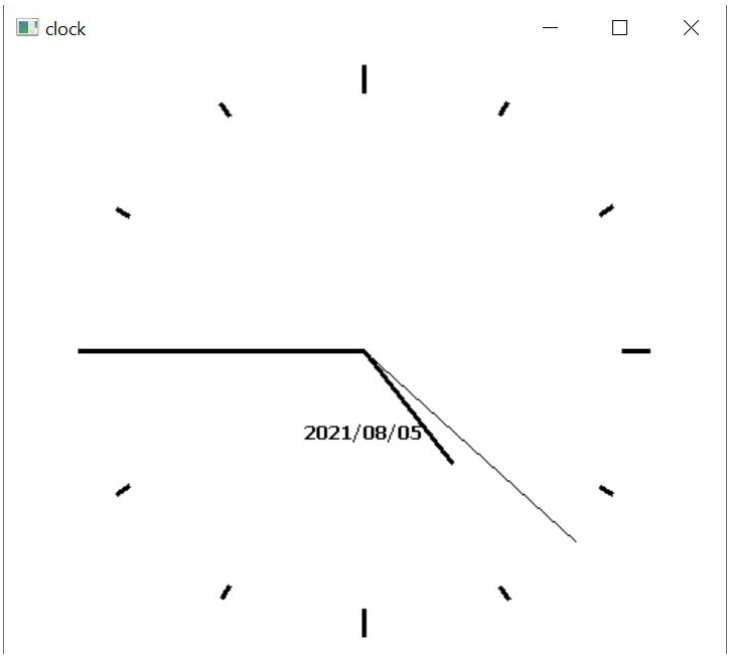
Console game.



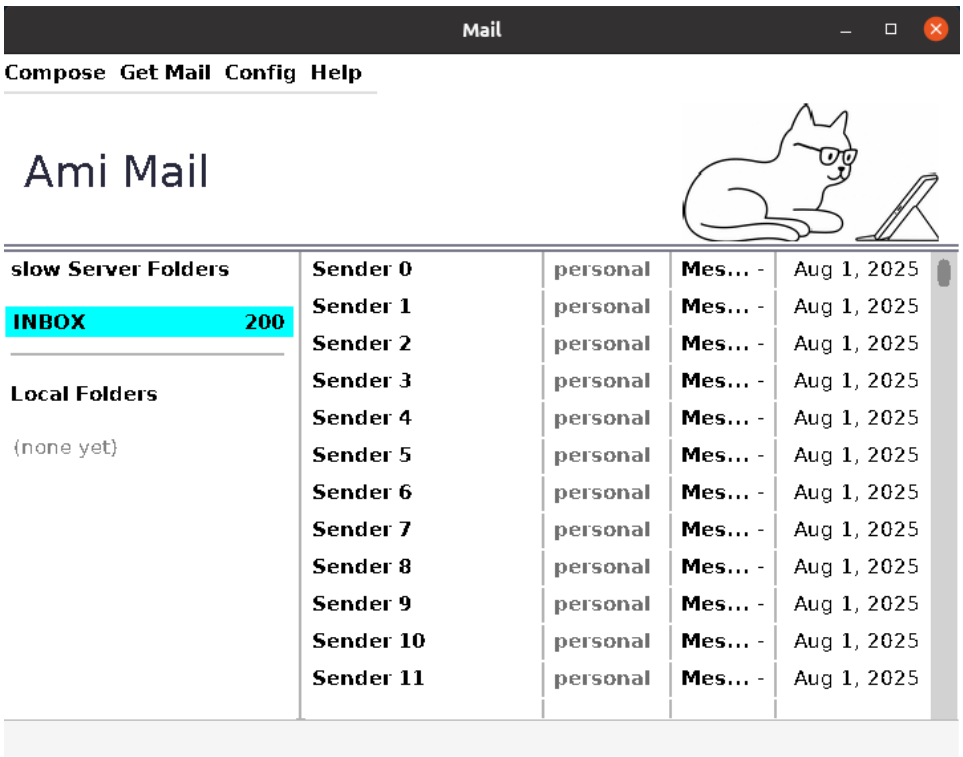
Breakout character game with sound and joystick



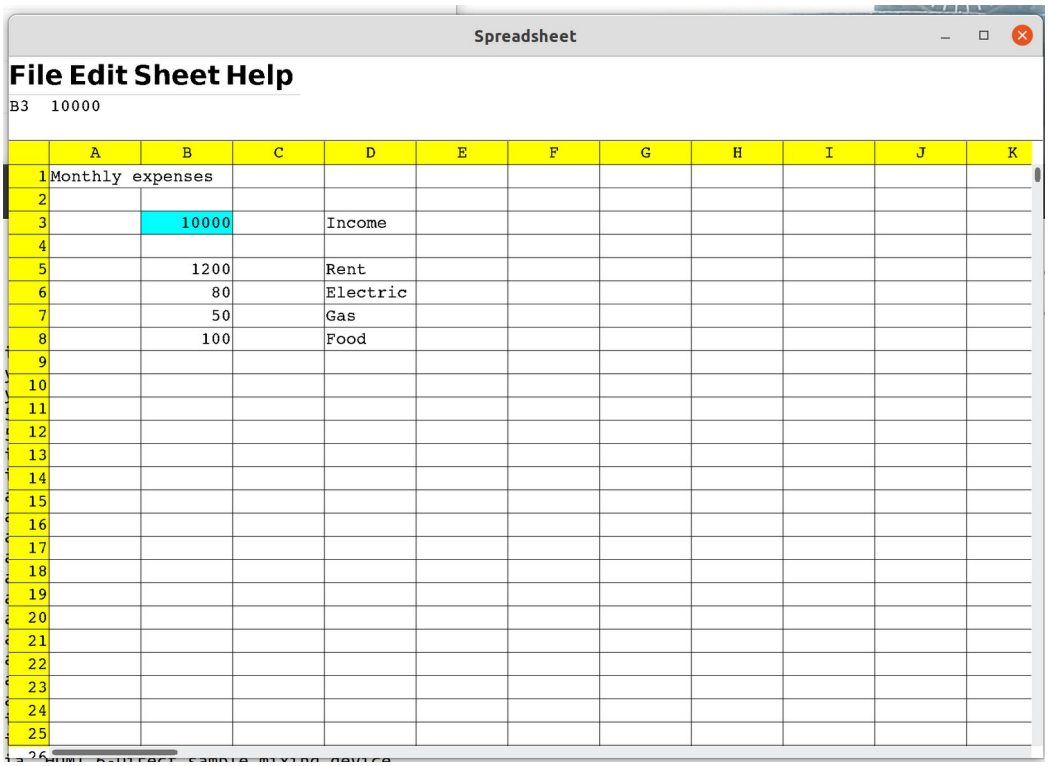
Breakout graphical game with sound and joystick.



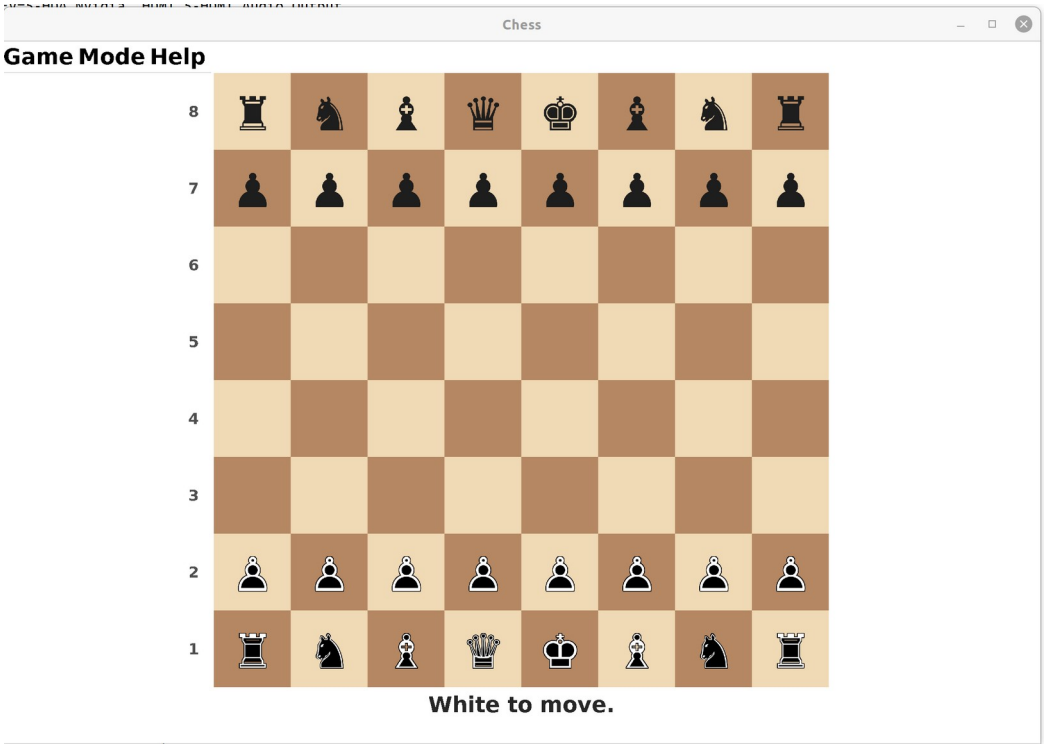
Clock program, resizable.



Email client program



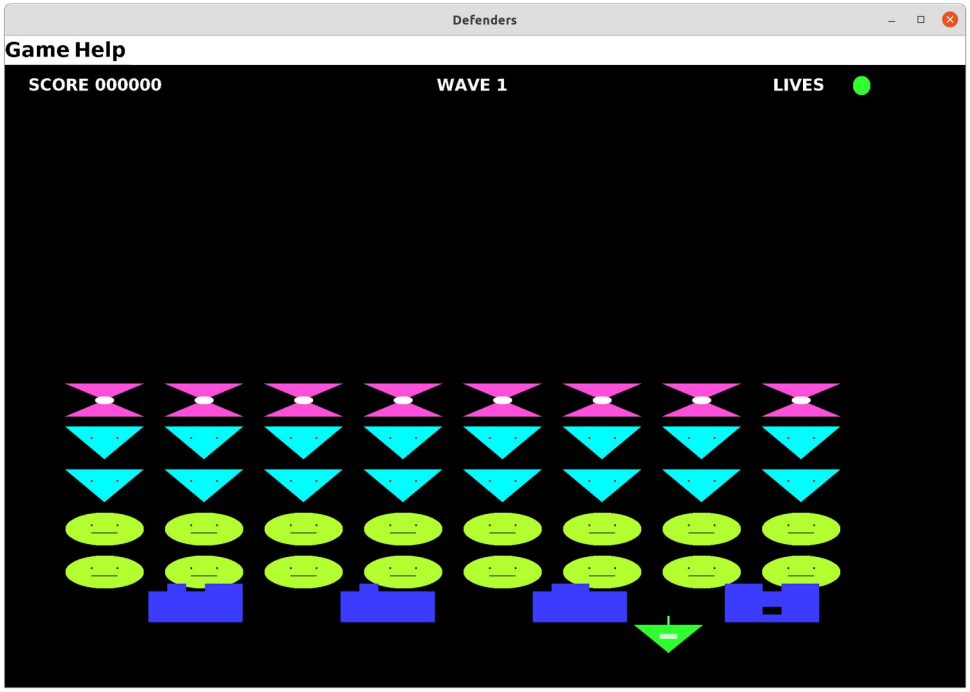
Spreadsheet



Chess game



Backgammon game



Defenders game

Ami is different from other TKs in that it tries to provide a complete runtime environment for the programs it ports, rather than just providing graphics. I don't believe you should have to make a collection of unrelated modules just to get programs to run.

1.1 History

In 1996, most people making TKs were working with the idea that text based interfaces were going to be obsolete very soon. Everything would be point and click. Graphical interfaces would discard any backward compatibility, since text mode would be useless very soon. There were a couple of libraries that did feature upward compatibility for text mode programs, but all they did was emulate text mode programs in Windows, with virtually no path to go beyond that.

In the years since, there has been a renaissance for text processing, as programmers realized that command line interfaces (CLIs) were more powerful and more concise in the majority of cases. Indeed, Linux features command line operations for virtually everything that its graphical applications can do, and when a feature comes out in Windows, usually the first thing programmers request is a command line version of the same functionality. It's not just about the power of CLIs, but also the fact that CLIs are scriptable, and have the ability to be included as components into larger programs without change.

It is for these reasons that I think Ami fits well with modern architectures. It's not so much that I think a lot of text mode programs will be converted to full graphical using Ami (although that is certainly possible), but more that Ami makes it effortless to cross from text mode programs to graphical mode programs (and back).

At the same time, Ami is a good graphical model as well. It is dramatically simpler than direct programming to the common operating system interfaces such as Win32 and Xwindows (as indeed most TKs are), and it is built on a series of timeless paradigms that fit together well.

1.2 Reliance on ANSI C/libc

The base Ami code relies on both ANSI C as well as the standard C library, generally referred to as libc. This means that systems that support both of these standards will be able to host Ami.

1.3 Coining

To prevent namespace collisions, each of the calls in Ami are coined with the prefix "ami_" such as:

```
ami_list();
```

Language bindings that allow for separate namespaces (such as C++) usually drop the ami_ prefix.

1.4 Module format

Each module of Ami has both a .c file and a .h file, each of which has an include guard (prevents multiple inclusions of the same file). Each module has a startup section and an ending section, run before the program begins and after it ends respectively. It is used to set up global variables used in each module.

All of this, of course, is simply good programming practices, and it is covered in the document "C Language Coding Standard", which is included with the Ami release.

1.5 Overview of Ami libraries and modularity

The modules in Ami are divided into two basic types:

1. Basic extensions used by programs regardless of extra devices or capabilities available.
2. Extensions that introduce new device capabilities.

The following extension libraries will be shown in Ami:

Section	Name	Description
3	Service library	Directory lists, file name handling, paths, environment, program execution, date and time, internationalization.
4	Terminal library	Output to text surface, advanced input.
5	Character windows management library	Management of multiple character mode windows.
6	Graphical library	Output to graphical surface.
7	Windowing management library	Management of multiple windows.
8	Widget library	Buttons, lists, dialogs relevant to screen based programs.
9	Sound library	Midi and wave input and output.
10	Network library	Network program access.

Typically the windowing library and the widget library are combined with lower level modules. For example, the graphical library, the windowing library and the widget library are combined on systems that are capable of all three functions.

1.6 Services

services.c/.h contains a series of operating system support extensions such as filename/path handling, directory listings, time and date, environment strings, executing other programs (processes), executing threads, and similar functions.

1.7 Advanced User I/O and Presentation Management

The advanced user I/O modules are a family that implement a series of advancing levels that match advancing levels in I/O capabilities, including:

Terminal character presentation

Gives the ability to place characters on a grid for a typical fixed size character display.

Graphical presentation

Gives the ability to draw figures, and place characters with proportional fonts.

Window management

Divides the display surface into a series of independent windows.

Widget placement and management

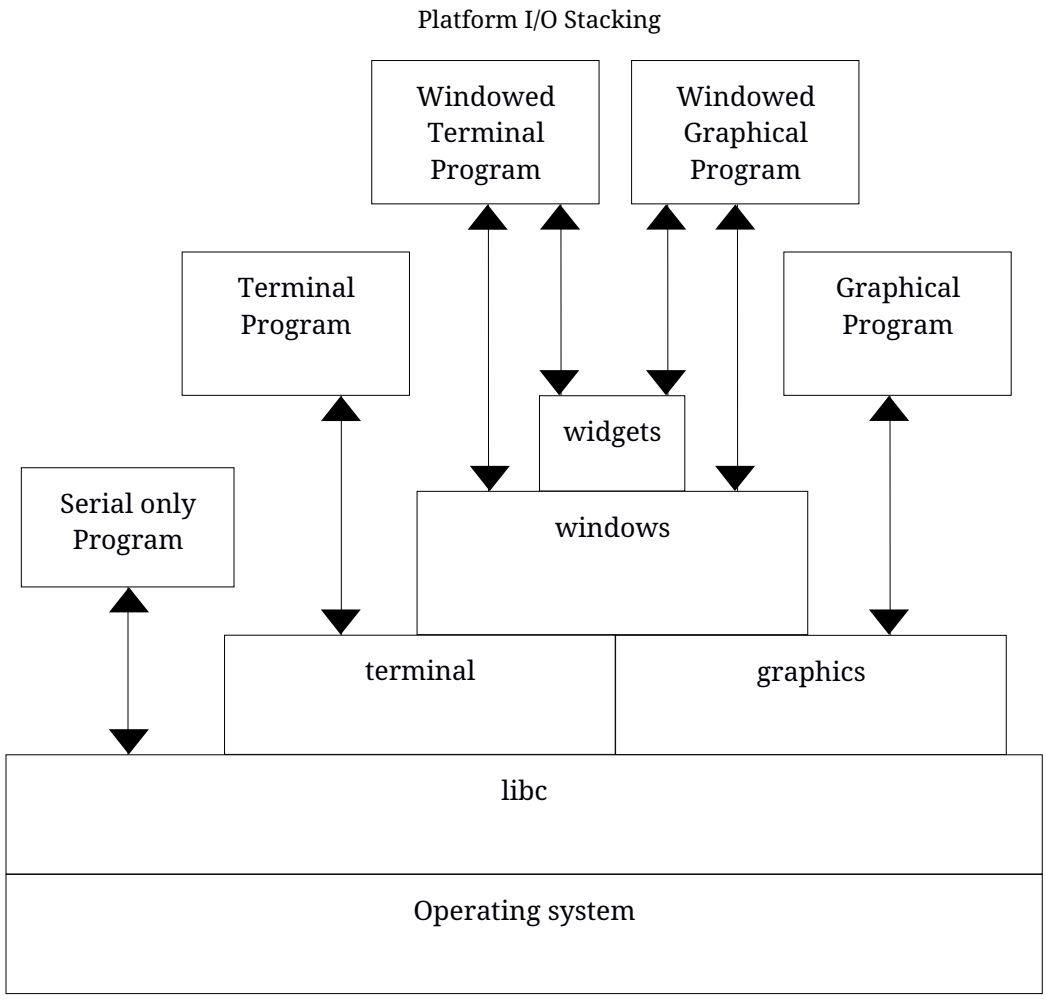
Gives a library of common controls, layered components and dialogs.

All of the supported display modes are upward compatible, and fixed size character presentation is always available in any graphic mode. The result is that there are two possible stacking levels of presentation:

```
Terminal  -> Single fixed display -> Windowed display -> Widgets
Graphical -> Single fixed display -> Windowed display -> Widgets
```

The terminal, graphical, window management and widget libraries are a set of libraries that implement a series of screen surface operating standards in an ascending sequence of capability:

These layers are all forward and backward compatible, so that any lower level can be used in any higher level. For example, an ANSI C program that only inputs and outputs in terms of characters formatted in terms of lines can be run under any level all the way up to a multiple windowing environment with widgets without source change.



The primary method for performing this is the use of the "character grid", which shows where the cells of screen characters exist in a graphical system. The character grid is only relevant to characters as a figure, and not to lines or other figures, and output to the grid is the default. A program may use

arbitrary character placement by explicitly specifying it, and it can turn off the automatic character wrapping features of the grid.

Similarly, all text and graphical windows are buffered in a multiple windowing system, so that they need not be aware of the windows management unless they wish to be.

Serial I/O is the name given here to the default method of ANSI C which goes back to its origin, and includes the **printf()**, **fscanf()**, and similar library procedures.

Terminal I/O is so named after the terminals that were used with computers for several decades, and for text mode still in use even in windowed operating systems.

From here, two distinct branches exist, one with, and one without graphics capability. For example, it is possible to have both multiple windows, widgets and dialogs all while staying within the capability of a character I/O only terminal. Therefore, there exists both a graphics version of the windows management library and the windows library, and one of each with character only ability.

The names of the libraries, as they appear in an include statement, are as follows:

Name	Contents	Call model
terminal	Terminal I/O	Terminal
graphics	Graphical I/O	Graphical
windows	Managed windowing	Terminal and graphical
widgets	Widgets and dialogs	Terminal and graphical

The serial level does not require an explicit library, since that is the normal I/O method specified in ANSI C.

The key to understanding the variation in libraries is that a use of a particular library does not necessarily lead to a completely different block of code. A system that only has graphical output modes will implement the terminal library by aliasing that to the graphical library. It is also common for a graphical or terminal mode library to include all of the windows management and widget/dialog functions in the same module. Thus, a use of the windows management libraries does not cause a specific library to be linked as much as flag that the program will be using these features.

1.8 Naming

The functionality of the I/O modules nest, which effectively means that each of the I/O modules extend each other. In Ami this is done by simply carrying the same function names forward into the extention module. For example, references to calls in **terminal** are rerouted to **graphics**, since graphics contains all of the procedures and functions of terminal.

These combinations of Ami modules are possible:

Module	Extended to
terminal	**windows**
terminal	**widgets**
graphics	**widgets**

Each of the I/O modules has its own interface specification, which includes a superset of the base module. This allows each module to be fully checked against its own specification. A program that specifies **terminal** only sees **terminal** definitions, a program that specifies graphics only sees **graphics** definitions, and so forth. At the lowest level (assembly language or “linker” level), aliases are provided for the module names, and so a program compiled to the **terminal** standard can be linked to a graphics module, etc.

1.9 Advanced device libraries

1.9.1 sound

sound gives the ability to drive input and output MIDI messages, perform sequencing, and input and output wave files.

1.9.2 network

network allows access to a network using the ANSI C library file model. It supports TCP/IP, SSL, and messaging.

2 Using C++ With Ami



C++ can be used with Ami: everything that works with C works with C++ as well.

2.1 Two Modes for C++ Programs

C++ can be used at two different levels, which I refer to as C+ and C++ levels. These levels use the file endings .cp and .cpp, respectively[1]. The C+ level uses namespaces and overloads for functions. The C++ level implements this plus object orientation.

The reason for the two levels is that many programmers use C++ as an “extended C”, which is to say, they use procedurally oriented code supplemented by the features of the C++ language. For Ami, this means the namespace feature of C++, and the ability to overload functions.

The reason for two different file endings, .cp and .cpp, is that there are examples in the Ami tree that use both the C+ mode and the C++ mode for different versions of same program. So having the different file endings makes it clear what mode is being used in the source file.

2.2 C++ Extension layer

The C+ and C++ support is implemented as a wrapper in C++ to the C libraries in Ami. The code for the wrappers is in the cpp directory. The header files used are in the hpp directory. To include a particular library in C++, you use the hpp version of the header file:

```
#include <stdio.h>
#include <stdlib.h>
#include <terminal.hpp>

using namespace terminal;

int main()
{

    evtrec er;
    printf("hello c++ world\n");
    do { event(stdin, &er);
```

[1] You don’t have to use this convention. It is used to separate different Ami example programs that do the same thing, but use different language modes.

```
        if (er.etype == etterm) exit(1);
    } while (er.etype != etenter);

}
```

In this case, the terminal.hpp file is the analog of the terminal.h file used in C. In C++, you only use the hpp version of the file.

To implement the functionality, a thin layer of C++ code is used to form the C++ interface from the underlying C libraries.

Each header file contains a namespace corresponding to the library name. In the example for terminal.hpp, it contains a namespace **terminal**. In the client code using it, you either open up the namespace using:

```
using namespace terminal;
```

Or you use C++ namespace notation to access the declarations in the file, like:

```
#include <stdio.h>
#include <stdlib.h>
#include <terminal.hpp>

int main()
{

    terminal::evtrec er;

    printf("hello c++ world\n");
    do { terminal::event(stdin, &er);
        if (er.etype == terminal::etterm) exit(1);
    } while (er.etype != terminal::etenter);

}
```

Or any mix of the above.

2.3 Libraries covered by a C++ Extension layer

The wrappers now cover five libraries: terminal, graphics, sound, services and network. Where an interface holds state, the wrapper offers objects for it: the term and graph objects hold the single surface, window and widget the many-window display, synth and wave the sound ports, net and msg the connections, and thread, mutex and signal the threading of services. Each is described in a C++ section of its library's chapter, ahead of the function listing.

2.4 Library documentation for C++

Because C++ can use all of the functionality of C for Ami, C++ features are documented as part of the C level documentation for libraries. It is clearly marked what the extended functionality for C++ is.

3 System Services Library



services.c/.h/.cpp/.hpp contains common operating system related tasks, including directory access, time and date, files and paths, file attributes, environment strings, the local option character, execution of external programs, location, internationalization and multitasking.

3.1 Filenames and Paths

A file specification is composed of a path, name and extension:

<path><name><ext>

The exact format of a file specification changes with the operating system. The routine **brknam()** takes a file specification and breaks it down into its path, name and extension components. The routine **maknam()** does the opposite, creating a composite name from the three components. When a file specification has no path, it means that it refers to the default path. When a file specification needs to be printed or saved in an absolute format, with the path always specified, the **fulnam()** routine is used to "normalize" the file specification by filling out the complete path.

To parse file specifications from the user, the routine **filchar()** returns a character set[2] of all possible characters in a filename. This can be used to get a sequence of characters from a string or file that can constitute a file specification. Once the potential file specification has been loaded into a string, it can be checked for validity as a file specification by **validfile()**, and as a path by **validpath()**.

Filenames are based on the idea that there is a set of characters that may be used for filenames in a particular installation, and the characters outside that set are used to delimit between filenames. This characteristic of filenames allows a program to load the filename into a string, then check its proper structure using subsequent calls. The program may remove certain characters from the set of filename characters to be able to parse command lines.

A common method used to represent filenames in command lines and other text when the characters allowed in a filename are essentially unlimited is to quote the filename. Using this method, the filename appears as:

“myfile”

or

[2]Unsigned integer used as a bit set.

‘myfile’

This can work together with limited character set filenames by recognizing the leading quote. If both types of quotes are allowed, then the leading quote should match the trailing quote. Additionally, the program should be able to recognize a “quote image” of the forms:

Character sequence	Result
“”	Double quote “
‘’	Single quote ‘
\”	Forced double quote “
\’	Forced single quote ‘

A robust program would recognize all of these forms.

Note that **services** does not contain any method to parse filenames.

If a file specification contains wildcards, this can be determined by **wild()**. **services** does not define what wildcard characters or specifiers are used, or their format. **wild()** simply indicates that the filename contains a wildcard specification that may result in multiple files being indicated by the same file specification. A specification that ends in the path character names everything in that directory, the same as that path followed by *, and is wild.

3.2 Predefined paths

To find common objects that a program needs, three predefined paths are provided:

Program path

> Is the path that the program was executed from. This is used to find data that accompanied the program, and system wide option files. **getpgm()** is used to read the path.

User Path

> This is the path for the current user's home directory. This is used to store options that only apply to the current user. **getusr()** is used to read the path.

Current Path

> The default path is used to find options that apply only to the current file being worked on. Unlike the other path types, the current path can be both read and set. Setting the current path will set the default path used to finish incomplete file specifications. **getcur()** is used to read the path, and **setcur()** is used to set it.
>
> Note that if the system has no concept of a current path, this property is used to form full path names from default path names in **services**.

3.3 Time and Date

Time is kept in two different formats in **services**. The first is seconds time, and the second is "clock" time. Seconds time is literally a count of the number of seconds since a fixed reference time. Clock time is a free running clock that ticks every 100 microseconds.

Seconds time is returned by the **time()** function. It is in a format called "S2000", and it is the number of seconds relative to midnight on the morning of January 1, year 2000. This means that S2000 has a negative value for years before 2000, and a positive value after that. S2000 time is dependent on the representation of a **long**, which follows the size of the word in the current CPU. This results in the following limits according to bit size:

Bits	Farthest year into the past	Farthest year into the future
32	1932 AD	2068 AD
64	292471206678 BC	292471210678 AD

This represents a reasonable range of time for 32 bits, and by the year 2068 it is likely that the time will universally be 64 bits or better. Note that it does not matter how many bits are returned by time, so that transition will occur seamlessly. Because 64 bits is already in excess of what is needed for computer based timekeeping, it is likely that time in 64 or more bits will actually contain fewer bits, despite the **long** representation.

S2000 time is self-relative, meaning that times relative to the year 2000 are measured by a fixed number of seconds. S2000 only matched UTC time once, exactly at midnight on the morning of January 1, year 2000. The host keeps the current time aligned to UTC as each leap second is announced, so the time and date read now are correct. A difference between two S2000 times counts Gregorian seconds, 86400 to the day, and is short by the leap seconds inserted between them, so it grows less accurate the farther the two times lie from each other and from the present. A table of leap seconds can be used to find accurate past differences, but obviously that cannot be used to correct future differences.

There are two types of calculations that can be applied to UTC, static and dynamic. The static calculation typically finds UTC by applying leap year corrections, then a fixed table of leap second years. The static calculation will fall out of accuracy from the time that the system is released. It is impossible for it to be otherwise, since UTC is based on astronomical observation.

The leap year part of that is arithmetic, and it never goes out of date. The rule is the Gregorian one: a year that divides by four is a leap year, unless it is a century year, unless that century divides by four hundred. **services** has it as a macro:

```
/* every fourth year, except a century year, unless the century
  itself divides by four hundred */
#define LEAPYEAR(y) ((y & 3) == 0 && y % 100 != 0 || y % 400 == 0)
```

So 2000 was a leap year, by the four hundred rule; 1900 was not and 2100 will not be; 2024 was. The rule gives the length of a year and of February, and that is all the calendar needs from it:

```
yd = LEAPYEAR(y) ? 366 : 365;    /* days in the year */
days[1] = 28 + LEAPYEAR(y);      /* days in February */
```

To find the date of an S2000 time, **dates()** walks out from 2000 a year at a time, forward for a positive time and back for a negative one, taking each year's days off the count until fewer than a year's worth remain; the year it stops in is the year, and the days left place the month and the day by the same table:

```
y = t < 0 ? 1999 : 2000;          /* the year each side of 2000 */
t = labs(t);                     /* seconds from it, either way */
```

```
while (t / DAYSEC > (LEAPYEAR(y) ? 366 : 365)) {

   t -= (LEAPYEAR(y) ? 366 : 365) * DAYSEC;
   if (y >= 2000) y++; else y--;

}
```

Nothing in this depends on observation: the calendar fixes it, and no table of leap years is ever needed. What falls out of accuracy is the other correction, the leap seconds, which are declared a few months ahead as the Earth's rotation is measured, and which a fixed table can only carry up to the day it was written.

The dynamic calculation relies on receiving a current table of corrections from the network or other communications method. This is the same calculation as fixed time, but with a continually updated table of leap seconds.

When an S2000 time is not available, it is customary to set it to –**LONG_MAX**, which makes it clear that the time was not set.

The time returned by **time()** is in GMT or "universal" time. To convert to local time, the function **local()** is used. It takes the given GMT S2000 time, and offsets it by the local time zone offset, and by daylight savings time, and returns the adjusted local time.

The time can be placed, in character format, to a string by **times()**, and written to either an output file, or the **stdout** file by **writetime()**. The date can be placed, in character format, to a string by **dates()**, and written to either an output file, or the standard output by **writedate()**. These procedures format the time and date using the current internationalization settings of the host.

clock() time is typically derived from a free running counter kept in the host computer, and it is represented by **long** values. It may or may not be synchronized to the values returned by time. For this reason, **clock()** should be treated as self-relative and not compared to **time()** in any way.

clock() time is treated differently from **time()**. Since it free runs, you must be prepared for it to "wrap", or suddenly start counting up from zero. This can be determined by checking if any stored time is greater, or later in time, than the current clock value. The function **elapsed()** takes a reference time, and determines how much time has passed from that, including compensation for wraparound. The exact count at which the **clock()** count wraps is system dependent.

The total amount of time that can be represented with **clock()** is determined not only by the bit size of the **clock()** return value, but also by the size of the counter the host computer maintains. All that is guaranteed is that clock will be able to keep a unique time for at least 24 hours.

The actual increment of time for each tick of the clock is determined by the host computer. If the host cannot time to 100 microsecond accuracy, then the **clock()** time will increment in multiples > 1, or effectively a running approximation of the actual timer. For this reason, there may not be an exact time length between successive counts of the timer.

3.4 Directory Structures

The **list()** procedure takes a file specification, including wildcards, and returns a linked list of all of the matching directory entries:

```
/* attributes */
typedef enum {

    ami_atexec, /* is an executable file type */
    ami_atarc,  /* has been archived since last modification */
    ami_atsys,  /* is a system special file */
    ami_atdir,  /* is a directory special file */
    ami_atloop  /* contains hierarchy loop */

} ami_attribute;
typedef long ami_attrset; /* attributes in a set */

/* permissions */
typedef enum {

    ami_pmread,  /* may be read */
    ami_pmwrite, /* may be written */
    ami_pmexec,  /* may be executed */
    ami_pmdel,   /* may be deleted */
    ami_pmvis,   /* may be seen in directory listings */
    ami_pmcopy,  /* may be copied */
    ami_pmren    /* may be renamed/moved */

} ami_permission;
typedef long ami_permset; /* permissions in a set */

/* standard directory format */
typedef struct ami_filrec {

    char*             name;    /* name of file (zero terminated) */
    long              namel;   /* length of filename */
    long long         size;    /* size of file */
    long long         alloc;   /* allocation of file */
    ami_attrset        attr;    /* attributes */
    long              create;  /* time of creation */
    long              modify;  /* time of last modification */
    long              access;  /* time of last access */
    long              backup;  /* time of last backup */
    ami_permset        user;    /* user permissions */
    ami_permset        group;   /* group permissions */
    ami_permset        other;   /* other permissions */
    struct ami_filrec* next;    /* next entry in list */

} ami_filrec;
typedef ami_filrec* ami_filptr; /* pointer to file records */
```

Each directory file entry has the name of the file, along with a series of descriptive data for the file. These are divided into attributes and permissions. An attribute is a characteristic of the file, and

generally does not change. Permissions indicate what can be done with the file, and are divided into the user, group and other permissions.

Not all attributes nor all permissions are available on every operating system. If a particular permission or attribute is not implemented on a given operating system, then setting it will have no effect, and reading it will always return unset.

The size of the file is its size in bytes. The allocation is the total space it occupies on the storage medium, which may be different from its size for several reasons. The blocking may be such that the size is rounded up to the nearest block. The operating system may have the ability to reserve space for the file beyond what it is currently using, or may not release space back to the free space pool if the file is truncated.

The times of interesting events in the file's life are available, in "S2000" format (already discussed). If a particular time is not available, then it is set to –**LONG_MAX**.

The file structure is a collection of items that may be implemented on any given operating system. The way to prevent the need to decide what is and what is not implemented on a particular system is to focus on what is essential for all systems. For example, the size of a file is usually present, as well as the last modification time. The last modification time can be used to determine when to back up files, by comparing it to the date of the backup copy of the same file, or to the modification date of the archive containing the file. Similarly, a "make" style program can determine when to remake a file by looking at the modification time.

3.5 File Attributes and Permissions

The attributes of a file can be set by name with the **setatr()** command. It takes a set of attributes and the filename, and sets all the given attributes on the file. The routine **resatr()** resets attributes. The user permissions for a file are set and reset by **setuper()** and **resuper()**. The group permissions are set and reset by **setgper()** and **resgper()**. The other permissions for a file are set and reset by **setoper()** and **resoper()**.

3.6 Environment Strings

The environment is a collection of strings that is kept by the executive, and passed to programs when they are started. Each string has a name and a value, both of which are arbitrary strings. An environment string can be retrieved by name by **getenv()**, and set by **setenv()**. The entire environment string set can be retrieved at one time by the **allenv()** routine, which uses a linked list to represent the environment strings:

```
/* environment strings */
typedef struct ami_envrec {

    char* name;    /* name of string (zero terminated) */
    char* data;    /* data in string (zero terminated) */
    struct ami_envrec *next; /* next entry in list */

} ami_envrec;
typedef ami_envrec* ami_envptr; /* pointer to environment record */
```

Both the characters and format of the name of an environmental variable and the data it contains are operating system dependent. However, a subset of naming conventions will work on the vast majority of systems. It starts with a character in the sequence 'A'..'Z', 'a'..'z', '_', followed by any number of characters in the sequence 'A'..'Z', 'a'..'z', '_', '0'..'9'. The data in the string can be a series of any valid characters.

The reason for retrieving the entire environment is to pass it on to other programs in an **exec()** statement.

Although most available operating systems implement the environment string concept, it has fallen out of favor in modern programs. Placing the needed configuration strings for a particular program into a place where the executive will use it both requires special calls, and places individual program data into a pool where it can be deleted or corrupted.

Some current systems use a repository concept where program data is kept in a central tree structured database. This is not covered in **services**, and would be covered in another library.

A better method is to use a file containing the configuration information for the program in its startup directory, then optionally another version in the user directory, and finally one in the current directory. This allows options that affect all runs on the current machine to be kept in one file, while the options for a particular user are kept in another file, and lastly the options in use in the current project in the current directory. This system has the advantage that it addresses concerns in a multiuser operating system, and allows each program to maintain its own startup data. This system is documented in 12 Config: The configuration database

For systems that do not implement an environment string capability, Ami will commonly keep a file in the user path area that contains the strings. This is then used just for that program, that is, the set of environment strings are kept just for the accessing program.

3.7 Creating or Removing Paths

A file path, or directory, is created by **makpth()**, and removed by **rempth()**. If the directory has files in it, then it cannot be removed until all the files (and directories) under it have been removed. This means by implication that each section of a path must be removed separately, and a tree structured delete would have to repeatedly remove the contents of one element of the path, then remove the path section itself, and so on.

3.8 Option Character

When parsing commands, the option character for the current operating system is found with **optchr()**. **services** does not define the exact format of command line options. The option character is an aid to portability. This is used in 11 Option: Command Line Option Processing.

3.9 Path Character

The path character is used to separate path components, usually directories in a tree structured file system, within a path for the given system. It can be found with the **pthchr()** function. **services** does not define the contents, structure or meaning of a path. The path division character is an aid to portability.

3.10 Location

The functions **latitude()** and **longitude()** exist to give the location of the host computer in geographic coordinates, and return longs. The measurements are ratioed to **LONG_MAX**.

The **longitude()** is 0 at the Prime Meridian, a line passing through Greenwich, UK. The longitudes 0 to **LONG_MAX** are east of the line (or in the future, timewise), and longitudes 0 to –**LONG_MAX** are west of the line. Thus **LONG_MAX** and –**LONG_MAX** meet on the opposite side of the world from Greenwich. This means for a 32 bit integer that there is 0.0000000838190317 degrees for each step or 9.3306920025 millimeters per step.

The **latitude()** is 0 at the equator and **LONG_MAX** at the north pole, and –**LONG_MAX** at the south pole.

The **longitude()** and **latitude()** in minutes and seconds can be found with:

```
void find_dms(long longitude, long latitude,
              long* degrees_longitude, long* minutes_longitude,
              long* seconds_longitude,
              long* degrees_latitude, long* minutes_latitude,
              long* seconds_latitude,
              long* west, long* north)

{

    double lo = (double)longitude*180.0/LONG_MAX;
    double la = (double)latitude*90.0/LONG_MAX;

    *west = lo < 0;
    *north = la >= 0;
    lo = fabs(lo);
    la = fabs(la);
    *degrees_longitude = lo;
    *minutes_longitude = (long)(lo*60)%60;
    *seconds_longitude = (long)(lo*3600)%60;
    *degrees_latitude = la;
    *minutes_latitude = (long)(la*60)%60;
    *seconds_latitude = (long)(la*3600)%60;

}
```

The shape of the world is approximated as an ellipsoid. This means that the circumference of the earth at the equator is a longer distance than the circumference of the earth on the prime meridian (by about 68 kilometers).

The altitude of the host in MSL or Mean Sea Level is given by **altitude()**. It is 0 for the mean surface of the ocean (sea level averaged over a long period of time), and **LONG_MAX** at 100 kilometers in height (the altitude at which space begins). It is –**LONG_MAX** 100 kilometers in depth.

The altitude of 0 (MSL) is typically defined as height above a model of the geoid, or idealized model of the earth. This means it would have to be calculated against the latitude and longitude to find the actual distance from true center of the earth. However, in the majority of cases it can be accepted as a relative measurement.

The location of the host can be entered by the user or determined automatically by instrumentation (GPS). The host could even be mobile, in which case it is possible to determine speed and direction from the coordinates against time.

If there is no location, it is indicated by longitude equal to –**LONG_MAX**. This value of **longitude()** is redundant to **LONG_MAX**. Both **longitude()** and **latitude()** are considered unavailable by the value of **longitude()** being –**LONG_MAX.**

altitude() is available separate from **longitude()** and **latitude()**. It is not available when **altitude()** is –**LONG_MAX**.

The country of location is found with **countrys()**, which gives a string corresponding to the English name of the country. The countries of the world, and their associated codes, are given by ISO 3166-1. At this writing, the following countries exist, in alphabetical order:

#	Country	#	Country	#	Country	#	Country
004	Afghanistan	214	Dominican Republic	430	Liberia	663	Saint Martin (French part)
248	Åland Islands	218	Ecuador	434	Libya	666	Saint Pierre and Miquelon
008	Albania	818	Egypt	438	Liechtenstein	670	Saint Vincent and the Grenadines
012	Algeria	222	El Salvador	440	Lithuania	882	Samoa
016	American Samoa	226	Equatorial Guinea	442	Luxembourg	674	San Marino
020	Andorra	232	Eritrea	446	Macao	678	Sao Tome and Principe
024	Angola	233	Estonia	450	Madagascar	682	Saudi Arabia
660	Anguilla	748	Eswatini	454	Malawi	686	Senegal
010	Antarctica	231	Ethiopia	458	Malaysia	688	Serbia
028	Antigua and Barbuda	238	Falkland Islands (Malvinas)	462	Maldives	690	Seychelles
032	Argentina	234	Faroe Islands	466	Mali	694	Sierra Leone
051	Armenia	242	Fiji	470	Malta	702	Singapore
533	Aruba	246	Finland	584	Marshall Islands	534	Sint Maarten (Dutch part)
036	Australia	250	France	474	Martinique	703	Slovakia
040	Austria	254	French Guiana	478	Mauritania	705	Slovenia
031	Azerbaijan	258	French Polynesia	480	Mauritius	090	Solomon Islands
044	Bahamas	260	French Southern Territories	175	Mayotte	706	Somalia
048	Bahrain	266	Gabon	484	Mexico	710	South Africa
050	Bangladesh	270	Gambia	583	Micronesia (Federated States of)	239	South Georgia and the South Sandwich Islands
052	Barbados	268	Georgia	498	Moldova, Republic of	728	South Sudan
112	Belarus	276	Germany	492	Monaco	724	Spain
056	Belgium	288	Ghana	496	Mongolia	144	Sri Lanka
084	Belize	292	Gibraltar	499	Montenegro	729	Sudan
204	Benin	300	Greece	500	Montserrat	740	Suriname
060	Bermuda	304	Greenland	504	Morocco	744	Svalbard and Jan Mayen
064	Bhutan	308	Grenada	508	Mozambique	752	Sweden
068	Bolivia (Plurinational State of)	312	Guadeloupe	104	Myanmar	756	Switzerland
535	Bonaire, Sint Eustatius and Saba	316	Guam	516	Namibia	760	Syrian Arab Republic
070	Bosnia and Herzegovina	320	Guatemala	520	Nauru	158	Taiwan, Province of China

#	Country	#	Country	#	Country	#	Country
072	Botswana	831	Guernsey	524	Nepal	762	Tajikistan
074	Bouvet Island	324	Guinea	528	Netherlands	834	Tanzania, United Republic of
076	Brazil	624	Guinea-Bissau	540	New Caledonia	764	Thailand
086	British Indian Ocean Territory	328	Guyana	554	New Zealand	626	Timor-Leste
096	Brunei Darussalam	332	Haiti	558	Nicaragua	768	Togo
100	Bulgaria	334	Heard Island and McDonald Islands	562	Niger	772	Tokelau
854	Burkina Faso	336	Holy See	566	Nigeria	776	Tonga
108	Burundi	340	Honduras	570	Niue	780	Trinidad and Tobago
132	Cabo Verde	344	Hong Kong	574	Norfolk Island	788	Tunisia
116	Cambodia	348	Hungary	807	North Macedonia	792	Turkey
120	Cameroon	352	Iceland	580	Northern Mariana Islands	795	Turkmenistan
124	Canada	356	India	578	Norway	796	Turks and Caicos Islands
136	Cayman Islands	360	Indonesia	512	Oman	798	Tuvalu
140	Central African Republic	364	Iran (Islamic Republic of)	586	Pakistan	800	Uganda
148	Chad	368	Iraq	585	Palau	804	Ukraine
152	Chile	372	Ireland	275	Palestine, State of	784	United Arab Emirates
156	China	833	Isle of Man	591	Panama	826	United Kingdom of Great Britain and Northern Ireland
162	Christmas Island	376	Israel	598	Papua New Guinea	840	United States of America
166	Cocos (Keeling) Islands	380	Italy	600	Paraguay	581	United States Minor Outlying Islands
170	Colombia	388	Jamaica	604	Peru	858	Uruguay
174	Comoros	392	Japan	608	Philippines	860	Uzbekistan
178	Congo	832	Jersey	612	Pitcairn	548	Vanuatu
180	Congo, Democratic Republic of the	400	Jordan	616	Poland	862	Venezuela (Bolivarian Republic of)
184	Cook Islands	398	Kazakhstan	620	Portugal	704	Viet Nam
188	Costa Rica	404	Kenya	630	Puerto Rico	092	Virgin Islands (British)
384	Côte d'Ivoire	296	Kiribati	634	Qatar	850	Virgin Islands (U.S.)

#	Country	#	Country	#	Country	#	Country
191	Croatia	408	Korea (Democratic People's Republic of)	638	Réunion	876	Wallis and Futuna
192	Cuba	410	Korea, Republic of	642	Romania	732	Western Sahara
531	Curaçao	414	Kuwait	643	Russian Federation	887	Yemen
196	Cyprus	417	Kyrgyzstan	646	Rwanda	894	Zambia
203	Czechia	418	Lao People's Democratic Republic	652	Saint Barthélemy	716	Zimbabwe
208	Denmark	428	Latvia	654	Saint Helena, Ascension and Tristan da Cunha		
262	Djibouti	422	Lebanon	659	Saint Kitts and Nevis		
212	Dominica	426	Lesotho	662	Saint Lucia		

If the country is not set, an empty string is returned.

The ordinal number of the country is given by **country()**, which returns the number of the country from the table above. These ordinal numbers are given by the standard ISO 3166-1.

The current time zone is given by **timezone()**. It gives the offset in seconds, in the range of -12 to +14 hours from GMT or Greenwich Mean Time. The function **daysave()** is true if daylight savings is in effect in the current host location.

If 24 hour time is used in the current host location, the function **time24hour** will return 1, otherwise 0. The accepted format for 24 hour time is with hours from 0 to 23.

Both the current time zone and daylight savings time are factored into the calculation of **local()**, and that is the preferred method to find local time.

3.11 Internationalization

The current language in use on the host can be found with **languages()**, which gives a string corresponding to the English name of the language. The languages used are according to the ISO 639-1 standard:

#	Name	#	Name	#	Name	#	Name
1	Abkhaz	47	Finnish	93	Lingala	139	Samoan
2	Afar	48	French	94	Lao	140	Sango
3	Afrikaans	49	Fula, Fulah, Pulaar, Pular	95	Lithuanian	141	Serbian
4	Akan	50	Galician	96	Luba-Katanga	142	Scottish Gaelic, Gaelic
5	Albanian	51	Georgian	97	Latvian	143	Shona
6	Amharic	52	German	98	Manx	144	Sinhalese, Sinhala
7	Arabic	53	Greek (modern)	99	Macedonian	145	Slovak
8	Aragonese	54	Guarana	100	Malagasy	146	Slovene
9	Armenian	55	Gujarati	101	Malay	147	Somali
10	Assamese	56	Haitian, Haitian Creole	102	Malayalam	148	Southern Sotho
11	Avaric	57	Hausa	103	Maltese	149	Spanish
12	Avestan	58	Hebrew (modern)	104	Maori	150	Sundanese
13	Aymara	59	Herero	105	Marathi	151	Swahili
14	Azerbaijani	60	Hindi	106	Marshallese	152	Swati
15	Bambara	61	Hiri Motu	107	Mongolian	153	Swedish
16	Bashkir	62	Hungarian	108	Nauruan	154	Tamil
17	Basque	63	Interlingua	109	Navajo, Navaho	155	Telugu
18	Belarusian	64	Indonesian	110	Northern Ndebele	156	Tajik
19	Bengali, Bangla	65	Interlingue	111	Nepali	157	Thai
20	Bihari	66	Irish	112	Ndonga	158	Tigrinya
21	Bislama	67	Igbo	113	Norwegian Bokmal	159	Tibetan Standard, Tibetan, Central
22	Bosnian	68	Inupiaq	114	Norwegian Nynorsk	160	Turkmen
23	Breton	69	Ido	115	Norwegian	161	Tagalog
24	Bulgarian	70	Icelandic	116	Nuosu	162	Tswana
25	Burmese	71	Italian	117	Southern Ndebele	163	Tonga (Tonga Islands)
26	Catalan	72	Inuktitut	118	Occitan	164	Turkish
27	Chamorro	73	Japanese	119	Ojibwe, Ojibwa	165	Tsonga
28	Chechen	74	Javanese	120	Old Church Slavonic, Church Slavonic, Old Bulgarian	166	Tatar
29	Chichewa, Chewa, Nyanja	75	Kalaallisut, Greenlandic	121	Oromo	167	Twi
30	Chinese	76	Kannada	122	Oriya	168	Tahitian
31	Chuvash	77	Kanuri	123	Ossetian, Ossetic	169	Uyghur
32	Cornish	78	Kashmiri	124	(Eastern) Punjabi	170	Ukrainian
33	Corsican	79	Kazakh	125	Pali	171	Urdu
34	Cree	80	Khmer	126	Persian (Farsi)	172	Uzbek
35	Croatian	81	Kikuyu, Gikuyu	127	Polish	173	Venda
36	Czech	82	Kinyarwanda	128	Pashto, Pushto	174	Vietnamese
37	Danish	83	Kyrgyz	129	Portuguese	175	Volapuk
38	Divehi, Dhivehi, Maldivian	84	Komi	130	Quechua	176	Walloon
39	Dutch	85	Kongo	131	Romansh	177	Welsh
40	Dzongkha	86	Korean	132	Kirundi	178	Wolof
41	English	87	Kurdish	133	Romanian	179	Western Frisian

42	Esperanto	88	Kwanyama, Kuanyama	134	Russian	180	Xhosa
43	Estonian	89	Latin	135	Sanskrit	181	Yiddish
44	Ewe	90	Luxembourgish, Letzeburgesch	136	Sardinian	182	Yoruba
45	Faroese	91	Ganda	137	Sindhi	183	Zhuang, Chuang
46	Fijian	92	Limburgish, Limburgan, Limburger	138	Northern Sami	184	Zulu

The ordinal number of the language is given by **language()**, which returns the number of the language from the table above. The ISO 639-1 standard does not specify language ordinal numbers, these are specific to **services**. Note that even if the languages are added to or subtracted to in future implementations, the ordinal numbers will not be changed.

The decimal point character in use can be found with **decimal()**. The current number separator in use is defined with **numbersep()**.

The time and date formats can be derived from the country of the host. The main difference between formats is the order of the elements in time and date. The functions **timeorder()** and **dateorder()** give the ordering:

dateorder Code	Date format
1	year-month-day (ISO 8601 standard format)
2	year-day-month
3	month-day-year
4	month-year-day
5	day-month-year
6	day-year-month

timeorder Code	Time format
1	Hour:minute:second (ISO 8601 standard format.
2	Hour:second:minute
3	Minute:hour:second
4	Minute:second:hour
5	Second:hour:minute
6	Second:minute:hour

The separator character for fields in the date is given by **datesep()**, which is '-' in ISO 8601 date formats. The separator character for fields in time is given by **timesep()**, which is ':' in ISO 8601 time formats.

The number of digits in each section of the time and date formats is:

Section	Digits
Year	4
Month	2
Day	2
Hour	2
Minute	2
Second	2

If the time and date format is not set, it defaults to the ISO 8601 standard for time and date formatting. This is the correct format for output that can be read across international boundaries.

Note that the procedures **times()**, **dates()**, **writetime()** and **writedate()** automatically use the internationalization settings of the host to arrive at the host computer's natural time and date formatting.

The symbol for the currency used in the country of host is given by **currchr()**.

3.12 Executing Other Programs

An external program can be executed by **exec()**, which takes the command line for the program, including the program name, and all of its parameters and other options on the same line after one or more spaces:

cmd parameter parameter... parameter

This is passed as a string to the **exec()** routine. When programs are executed this way, the executing program does not need to await the finish of the program, nor can it find out if the program ran correctly. If this is required, the routine **execw()** is used. **execw()** will wait for the program to complete, then place the error return for the program in a variable. This variable will be 0 if the program ran correctly, or non-zero if it didn't. The exact numerical meaning of the error is up to the program executed.

If the system cannot execute programs in parallel, **exec()** is equivalent to **execw()**, but without the return code.

If the environment is to be set for the executed program, the call **execе()** can be used, which takes an environment list. This allows the environment to be retrieved from the current environment or created as new, modified or added to as needed, then passed to the executed program. **execew()** does the same thing, but waits for the program to finish, and returns an error code.

If a command line concept does not exist on the target system, Ami can pass it via another means, such as a file or environmental variable. Alternately, it could simply be ignored.

3.13 Threads and thread management

A program as executed by **exec**() exists as a complete process. A process is a thread of execution, along with the data that the thread executes or uses. This includes the program code, data, variables, dynamic space, etc. It also includes environment variables and a set of open files. When the program is executed, only the environment variables can be passed on.

A process can have any number of threads, even though it starts with only one. **services** provides the means to create more than one thread, to coordinate their execution, and to signal events between

threads. Threads can execute anywhere in the code that the main/process thread can, and can access the same data as the other threads in the process. This means that the threads must be coordinated with each other so that data corruption cannot occur.

Threads can run by “time sharing” the CPU between the threads, or they can be done by giving a thread its own CPU or CPU core to run, or a thread can be a combination of the two. It is even possible to move a thread between these two modes dynamically. What is important to know is how threads are implemented is entirely transparent to the programs using **services**. It is also possible that the performance of a program can be enhanced by breaking program work into multiple threads running on multiple concurrent CPUs.

For each thread there exists a thread logical id, from 1 to N where N is the maximum number of threads supported, usually 100. The exact logical number of the thread is chosen for you, and thus allocation of logical thread ids is performed by **services**.

There is one reserved thread number, which is thread number 1, the main thread running the process. When the main thread returns, the program ends, and every thread ends with it: a program that needs the work of its threads complete sees it complete before the main thread returns.

A new thread is created by **newthread(threadmain)**. This function creates a thread and returns the logical id of the thread. The thread begins by executing the function “threadmain”, which is of the form:

```
void threadmain(void);
```

The created thread will run until either:

1. The threadmain function exits.
2. The process terminates.

There are many methods by which the thread can communicate with other threads, including the main thread.

Threads are asked to terminate rather than killed: a flag under a lock, a signal, and the thread answering for itself. The asking side sets the flag and signals; the thread folds the flag into its wait:

```
lock(l);
stop = 1;
sendsig(s);
unlock(l);

lock(l);
while (!work && !stop) waitsig(l, s);
if (!stop) { <the work> }
unlock(l);
if (stop) return;
```

services does not provide a method to kill one thread by another thread because this action is inherently unsafe: a thread stopped at an arbitrary point leaves its locks held and its shared data half written. If another thread must be killed, it usually indicates that the application is in error, and the best procedure is to let the thread be killed along with the process when it terminates.

Thread numbers can be reused after the thread terminates.

3.13.1 Concurrency Locks

Concurrency locks are the basic means to coordinate accesses between threads. The fundamental rule of threading is that concurrency locks must be used whenever two different threads can access the same variable data[3].

As with threads, locks are controlled by a logical id, from 1 to N, where N is usually 100. **services** assigns the logical numbers for you.

A concurrency lock is created by **initlock()**. It returns the logical lock number. A lock is destroyed by **deinitlock(ln)**, which takes **ln** as the logical lock number returned by **initlock()**. The lock entry is freed, and the logical lock number can be reassigned by **initlock()**.

A concurrency lock is acquired by **lock(ln)**, where **ln** is the logical id of the lock. It is released by **unlock(ln)**, again where **ln** is the logical lock id. When a lock is acquired, all other threads attempting to acquire the lock are suspended until the lock is released. Generally this is done by “fairlocking”, which means that the locks are given out in the order the threads arrive.

When locking, it is possible for threads to deadlock if multiple locks need to be acquired. A typical deadlock situation is:

Thread A:

```
lock(lock_a);
lock(lock_b);
```

Thread B:

```
lock(lock_b);
lock(lock_a);
```

If thread A takes lock_a and thread B takes lock_b, then both threads will deadlock awaiting each other.

A simple way to avoid deadlock is to always take locks in order. A better way is to use hierarchical locking. Using this method, we have a master lock:

Thread A:

```
lock(lock_m);
lock(lock_a);
lock(lock_b);
```

Thread B:

```
lock(lock_m);
lock(lock_b);
lock(lock_a);
```

[3] In addition, the compiler must be told of parallel access possibilities via the “volatile” keyword.

The master lock m is taken any time both A and B must be both acquired. What is nice about this scheme is that it can be used with individual lock requests (lock a and lock b) at the same time. It also can be implemented with simple lock primitives.

3.13.2 Semaphores

The main advantage of threads is that they can share any and all of the data of a process. This means that they can communicate using any data in memory, of any size. However, threads need to know when other threads have finished arranging data. Semaphores are able to send simple signals between threads. A semaphore is like a doorbell. It sends an alert between two threads, but what the exact meaning of the doorbell sounding is up to the threads that are communicating to determine.

Like threads and locks, semaphores have logical identifiers from 1 to N, where N is usually 100. **services** assigns the logical identifiers for you.

Semaphores are tied to locks, because for a thread to determine unambiguously what a signal means, the determination must be made within a locked section. A thread looks at data that communicates between threads within a locked region, then accepts a signal that the data is updated while in the locked region.

A signal is created by **initsig()**. It returns the logical signal number. When a signal is no longer needed it is released by **deinitsig**(**sn**), where **sn** is the logical signal number. The signal is sent by **sendsig(sn)**, where **sn** is the logical signal number. Any number of threads can wait for a given signal, and **sendsig()** releases one or more of them to run and process the signal. The threads released must be in a runnable state.

Because usually only one thread can use a signal at a time, the **sendsigone(sn)** call can be used, where **sn** is the logical signal number. As with locks, ideally the first thread to wait on the signal is the one released to run. It is also possible that the underlying system treats **sendsigone()** as equivalent to **sendsig()**, and will wake up one or more threads.

Threads wait on signals using **waitsig(ln, sn)**, where **ln** is a logical lock identifier, and **sn** is a logical signal number. **waitsig()** must be called within the lock **ln**. It releases the lock, waits for the signal, and then reasserts the lock and continues. The lock is released synchronously with checking for a signal.

The typical flow for signaling is:

```
lock(l);
while (<data condition is not yet true>)
    waitsig(l, s);
<perform data service>
unlock(l);
```

3.14 C++ services interface

Services has a C++ wrapper, in cpp/services.cpp with its declarations in hpp/services.hpp, following the pattern of the other wrappers. A C++ program includes <services.hpp> and works inside the services namespace, and the wrapper is carried in the standard libraries, so nothing extra is linked.

The namespace carries the whole of the services interface with the prefixes stripped: every function without its ami_, the attribute and permission types, the file and environment records laid out identically

to their C forms, and the set operator macros as inline functions. Most of services are stateless calls: files, paths, the environment, time, measurement -- and the namespace serves those directly: **time()**, **getusr()**, **list()**.

The threading pieces are the exception, and become objects. A thread starts on construction, created as by **newthread()**. A mutex is made by its constructor, freed by its destructor, and locks and unlocks by the C verbs -- the class is called mutex rather than lock only because C++ will not let a class and its method share a name. A signal likewise lives from constructor to destructor, sends with **sendsig()** or **sendsigone()**, and waits on a mutex with **waitsig(m)**.

Here is the worker handshake, as a whole program:

```
#include <iostream>

#include <services.hpp>

using namespace services;

mutex mx;
signal sg;
int done;

void worker(void)

{

    /* the work */
    mx.lock();
    done = 1;
    sg.sendsig();
    mx.unlock();

}

int main(void)

{

    /* newthread(worker): the thread starts here */
    thread t(worker);

    mx.lock();
    while (!done) sg.waitsig(mx);
    mx.unlock();

    std::cout << "the worker is done" << std::endl;

    return (0);

}
```

The objects hold identifiers, not hooks, so any number of each may exist, and a lock or signal is given back by the destructor: an identifier can never leak past an early return or outlive what it names.

3.15 Methods in services

void mutex::lock(void);

Acquire the lock the object holds, as lock() does by identifier.

void mutex::unlock(void);

Release the lock the object holds.

void signal::sendsig(void);

Flag an event on the signal the object holds: every waiter is set to run.

void signal::sendsigone(void);

Flag an event on the signal the object holds: one waiter is set to run.

void signal::waitsig(mutex& m);

Release the mutex m, wait for the signal the object holds, and reacquire m before returning, as waitsig() does by identifiers.

thread::thread(void (*threadmain)(void));

Construct a thread. The thread is created as by newthread(), begins executing threadmain, and the identifier is kept by the object.

3.16 Functions and procedures in services

void [ami_]allenv([ami_]envrec **el);

Returns a complete list of the strings in the environment to **el**.

long [ami_]altitude(void);

Returns the binary height above the earth's normalized surface, from 0 (ground) to 100km or space at LONG_MAX. Returns –LONG_MAX if the altitude is not available.

void [ami_]bakupd(char* fn);

Set backup time current. Given a file by name **fn**, sets the backup time for the file as current. Backup programs should also reset the archive bit to show that backup has occurred.

void [ami_]brknam(char* fn, char* p, long pl, char* n, long nl, char* e, long el);

Break down file specification. Breaks the file specification **fn** down into path **p**, name **n**, and extension **e**, with maximum lengths **pl**, **nl**, and **el**. Note that any one of the resulting components could be blank, if it does not exist in the name.

long [ami_]clock(void);

Returns 100 microsecond, free running time.

long [ami_]country(void);

Returns the ISO 3166-1 numeric code for the current country.

void [ami_]countrys(char* s, long sl, long c);

Returns the string corresponding to the current country. This is an empty string if no position is known. The resulting string must fit into the string **s** in the space given as **sl**.

char [ami_]currchr(void);

Returns the currency symbol for the current country.

long [ami_]dateorder(void);

Returns the the date order code used to specify the local time format in use. See the table for valid codes.

void [ami_]dates(char* s, long sl, long t);

Place date from S2000 time **t** in string **s**, with maximum length **sl**.

char [ami_]datesep(void);

Returns the character used to separate date components in the current system.

long [ami_]daysave(void);

Returns 1 if daylight savings is in effect at the current location, otherwise 0.

char [ami_]decimal(void);

Returns the character used to separate the fractional part from the rest of numbers (the decimal point).

void [ami_]deinitlock(long ln);

Releases a concurrency lock by logical id **ln**.

void [ami_]deinitsig(long sn);

Releases a concurrency signal by logical id **sn**.

long [ami_]elapsed(long r);

Given a stored **clock** time **r**, will check the current **clock** time and find the total number of 100 microsecond ticks since the given time, then return that. Accounts for timer wraparound.

void [ami_]exec(char* cmd);

Execute external program, with parameters, from the string **cmd**. Does not wait for the program to finish, and cannot detect if it finished with an error.

void [ami_]exece(char* cmd, [ami_]envrec *el);

Execute external program, with parameters from the string **cmd** and full environment. The environment is passed as a list in **el**. Does not wait for the program to finish, and cannot detect if it finished with an error.

void [ami_]execew(char* cmd, [ami_]envrec *el, long *e);

Execute external program, with parameters from the string **cmd** and full environment. The environment is passed as a list in **el**. Waits for the program to finish, and returns its error code in **e**. The error code is 0 for no error, otherwise the error is a code specified by the program executed.

void [ami_]execw(char* cmd, long *e);

Execute external program, with parameters from the string **cmd**. Waits for the program to finish, and returns its error code in **e**. The error code is 0 for no error, otherwise the error is a code specified by the program executed.

void [ami_]filchr([ami_]chrset fc);

Returns the set of valid filename characters in **fc**.

void [ami_]fulnam(char* fn, long fnl);

Create full file specification from a partial file specification **fn**. Given a file specification with an incomplete path (either by using the default path, or mnemonic shortcuts for things like parent directory), creates a fully pathed name of standard form. This can "normalize" file specifications, for comparisons, and to store the complete path for the file. The fully pathed result is returned in **fn**, with maximum length **fnl**.

void [ami_]getcur(char* fn, long fnl);

Get current path to **fn** with maximum length **fnl**.

void [ami_]getenv(char* ls, char* ds, long dsl);

Finds and returns an environment string. **ls** contains the name of the string to look up, **ds** with maximum length **dsl** contains the resulting string as found. If there is no environment string by that name, the return is empty.

void [ami_]getpgm(char* p, long pl);

Get the program path to **p** with maximum length pl. Returns the program path, which is the path the program running was loaded from.

void [ami_]getusr(char* fn, long fnl);

Get the user path to **fn** with maximum length **fnl**. The user path is a path specific to each user.

long [ami_]initlock(void);

Creates a new concurrency lock and returns the logical id for it.

long [ami_]initsig(void);

Creates a new concurrency signal and returns the logical id for it.

long [ami_]language(void);

Returns the ordinal number of the language in effect for the current location.

void [ami_]languages(char* s, long sl, long l);

Returns the name of the language l, as numbered by language(). The name string is placed at s, a buffer with length sl, by the critical buffer convention: a name that exactly fills the buffer is not zero terminated, a shorter one is, and a name that cannot fit at all is an error.

long [ami_]latitude(void);

Returns the latitude of the current location as a binary number, where 0 is the equator and LONG_MAX is the north pole, and –LONG_MAX is the south pole.

If longitude is –LONG_MAX, indicating that the position is invalid, then latitude is also invalid.

void [ami_]list(char* fn, [ami_]filrec **l);

Form a file list from the filename **fn**, and return in the file entry list **l**. The filename **fn** may contain wildcards, and may be fully pathed, or refer to the current directory. If no files are found, then the list pointer is returned NULL.

long [ami_]local(long t);

Converts the given GMT S2000 time **t** to local time, using the time zone offset and daylight savings status in the host computer, and returns the result.

void [ami_]lock(long ln);

The given lock by logical id **ln** is aquired. Generally fairlocking can be assumed, which means that for N waiters on the lock, they will be given the lock first come first served.

long [ami_]longitude(void);

Returns the longitude of the current location as a binary number, where 0 is the prime meridian (Greenwich, England), and LONG_MAX is the other side of the world eastward, and –LONG_MAX is the other side of the world, westward.

If the current position is not available, then –LONG_MAX is returned. Note that the back side of the planet is already indicated by LONG_MAX.

void [ami_]maknam(char* fn, long fnl, char* p, char* n, char* e);

Create file specification from components. Creates file specification **fn** from path **p**, name **n**, and extension **e**. Components may be blank, but the path and the name cannot both be blank.

void [ami_]makpth(char* fn);

Make path using **fn**. Creates a new path or directory. If the path already exists, it's an error.

long [ami_]newthread(void (*threadmain)(void));

Start new thread. Expects a function, **threadmain**, to start as the new thread. A logical thread number is created from 1 to N, where N is the maximum number of threads supported. The new thread number is allocated and returned to the caller. The thread is created and executes at the given function. The thread will run until it returns from the called function or is killed.

char [ami_]numbersep(void);

Returns the character used to separate parts of a number, every 3 digits.

char [ami_]optchr(void);

Find the option character. Returns the character that is used to introduce options in command lines on the current system.

char [ami_]pthchr(void);

Returns the character used to separate components in a path in the current system.

void [ami_]remenv(char* sn);

Remove a string **sn** from the environment. The string is found by name, and removed from the environment. No error results if the string does not exist.

void [ami_]rempth(char* fn);

Remove path **fn**. Removes a path, or directory. The directory must exist, and must be empty of any files or other directories, or an error results.

void [ami_]resatr(char* fn, [ami_]attrset a);

Reset attributes. Given a file by name **fn**, the attributes in the set **a** are set false for the file.

void [ami_]resgper(char* fn, [ami_]permset p);

Reset group permissions. Given a file by name **fn**, the permissions in the set **p** are set false for the file.

void [ami_]resoper(char* fn, [ami_]permset p);

Reset other permissions. Given a file by name **fn**, the permissions in the set **p** are set false for the file.

void [ami_]resuper(char* fn, [ami_]permset p);

Reset user permissions. Given a file by name **fn**, the permissions in the set **p** are set false for the file.

void [ami_]sendsig(long sn);

Flags an event to the given signal by logical id **sn**. Either all waiters on the signal or just one is set to run by a signal.

void [ami_]sendsigone(long sn);

Flags an event to the given signal by logical id **sn**. Only one thread is signaled to run. This version of signal is used when only one waiter can use the event signaled.

Note that it is possible for this call to be equivalent to **sendsig()** on a given implementation. Thus programs should check if the signaled event is still active, and not just assume it.

void [ami_]setatr(char* fn, [ami_]attrset a);

Set attributes. Given a file by name **fn**, the attributes in the set **a** are set true for the file.

void [ami_]setcur(char* fn);

Set current path from string **fn**. Sets the default path for all file specifications.

void [ami_]setenv(char* sn, char* sd);

Finds and sets an environment string. **sn** contains the string name to set, and **sd** contains the contents to set it to. If there is no string by that name, it is created, otherwise the old string is replaced.

void [ami_]setgper(char* fn, [ami_]permset p);

Set group permissions. Given a file by name **fn**, the permissions in the set **p** are set true for the file.

void [ami_]setoper(char* fn, [ami_]permset p);

Set other permissions. Given a file by name **fn**, the permissions in the set **p** are set true for the file.

void [ami_]setuper(char* fn, [ami_]permset p);

Set user permissions. Given a file by name **fn**, the permissions in the set **p** are set true for the file.

long [ami_]time(void);

Returns current S2000 time in GMT

long [ami_]time24hour(void);

Returns 1 if 24 hour time is in effect at the current location, otherwise 0.

long [ami_]timeorder(void);

Returns the time order code used to specify the local time format in use. See the table for valid codes.

void [ami_]times(char* s, long sl, long t);

Place time from S2000 time **t** in string **s**, with maximum length **sl**.

char [ami_]timesep(void);

Returns the character used to separate time components in the current system.

long [ami_]timezone(void);

Returns the offset, in seconds, of the current time zone from GMT, in the range of -12 to +14 hours.

void [ami_]unlock(long ln);

The concurrency lock by logical id **ln** is released. The next thread queued for the lock and that is in a runnable state is set to run.

long [ami_]validfile(char* s);

Parses and checks the file specification **s** for a valid filename on the current system. Returns 1 if valid, otherwise 0.

long [ami_]validpath(char* s);

Parses and checks the file specification **s** for a valid path on the current system. Returns 1 if valid, otherwise 0.

void [ami_]waitsig(long ln, long sn);

Wait for a given signal. Accepts a concurrency lock logical id **ln**, and a signal logical id **sn**. Releases the concurrency lock and waits for the given signal, then returns when the signal flags.

The purpose of accepting a lock number is that it is released and the signal is checked atomically. This means other threads that are not signaling will not run, and thus the wait and signal operations are synchronized together.

long [ami_]wild(char* s);

Checks if the file specification **s** contains wildcards. Returns 1 if so, otherwise 0. A specification ending in the path character names everything in that directory and is wild.

void [ami_]writedate(FILE *f, long t);

Write date from S2000 time **t** to text file **f**.

void [ami_]writetime(FILE *f, long t);

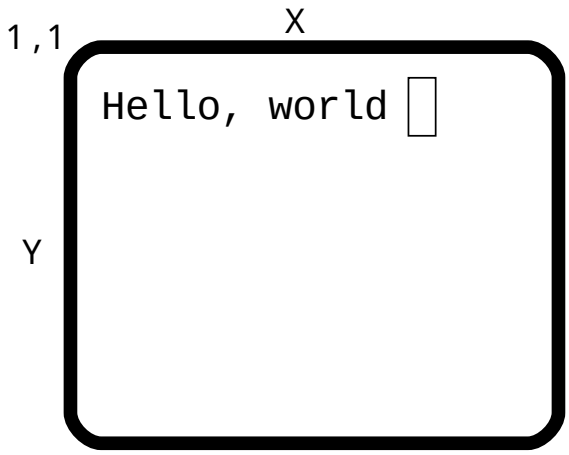
Write time from S2000 time **t** to text file **f**.

4 Terminal Interface Library



Standard ANSI C uses an I/O paradigm that is serial, or more correctly "line oriented". Each line is built up in sections, then output to the I/O device with an appended "end of line". The end of line causes the current line to be completed, and the next line begins.

The next level of paradigm is the presentation of lines onto a 2d text surface. This can simply be an emulation of the serial only system or “virtual paper” using a screen that scrolls up from the bottom. The next step is to allow full addressing of the 2d surface and allowing text to be placed anywhere within that surface. To this is added colors, different text presentation modes, and finally advanced, multiple device input.



terminal starts in ANSI C compatible mode, then allows the program to move to a full addressable surface without automatic scrolling. In advanced mode, **terminal** emulates an infinite virtual surface where the terminal exists as a window clipped to the origin. This is the most consistent model of such a surface.

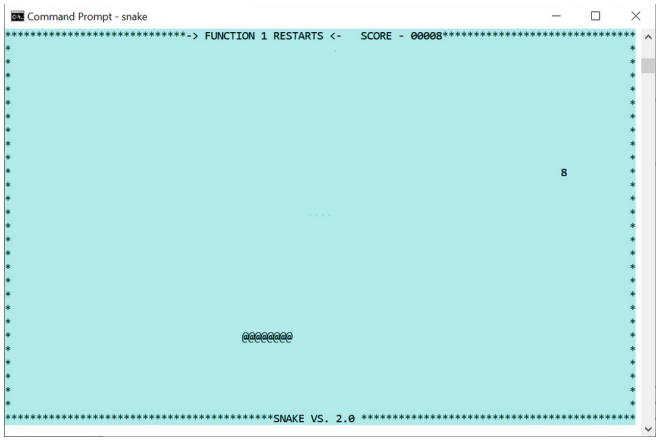
Because **terminal** overrides the interface between the program and the operating system it runs on, all of the ANSI C defined serial I/O works compatibly with the **terminal** presented surface. It is also modal, meaning that changes to character modes affect all further output to the terminal.

terminal introduces the concept that several devices can be used for input at the same time. The normal user keyboard is supplemented by a mouse, timers and a joystick. These are all implemented via the event concept, which unifies the input and removes the need to poll multiple devices.

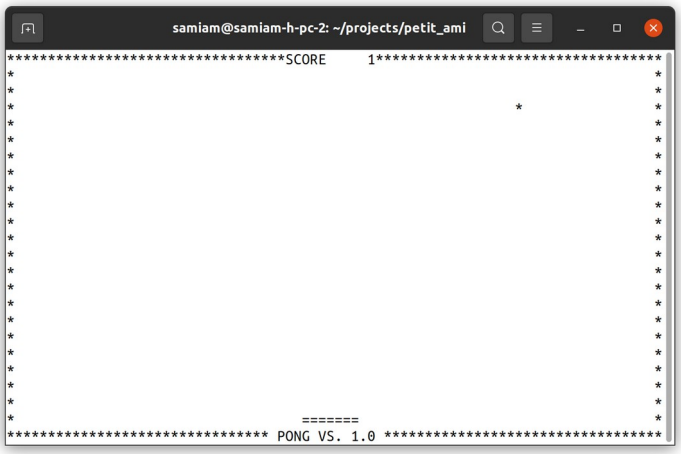
terminal gives a set of logical events for common control keys from the keyboard that allow the program to avoid direct recognition of implementation dependent key codes.

In addition to the default presentation surface, **terminal** is capable of switching between multiple display surfaces. This capability has several uses.

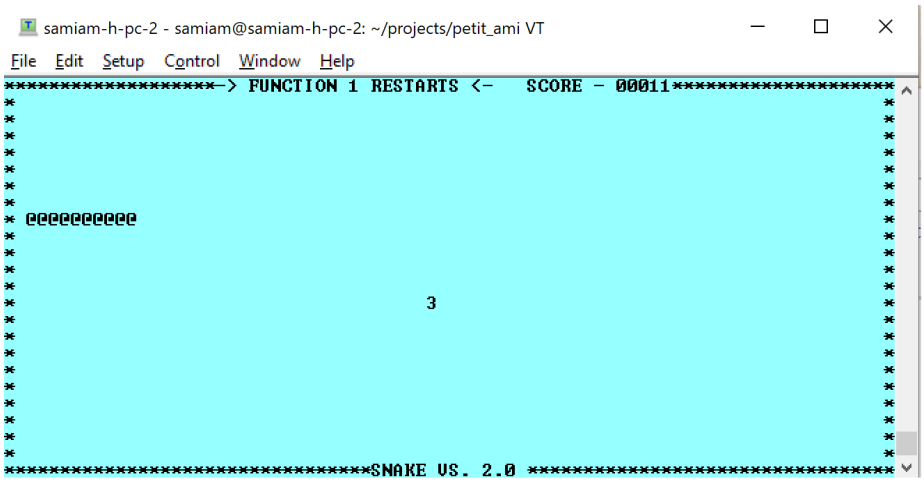
Examples:



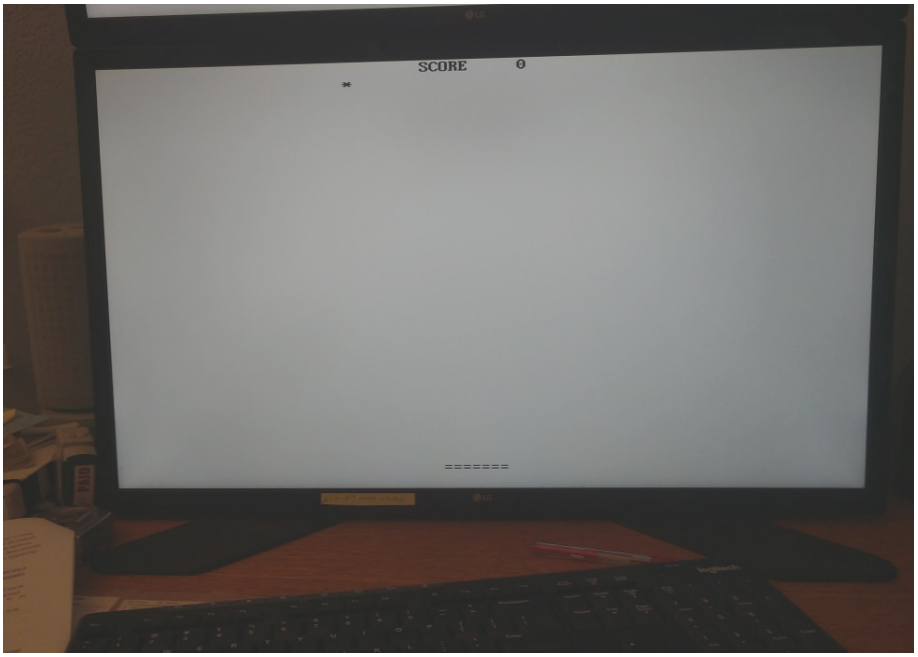
Terminal programs can appear in any console or terminal window, on multiple operating systems, or even on an actual terminal. Here on a Windows console.



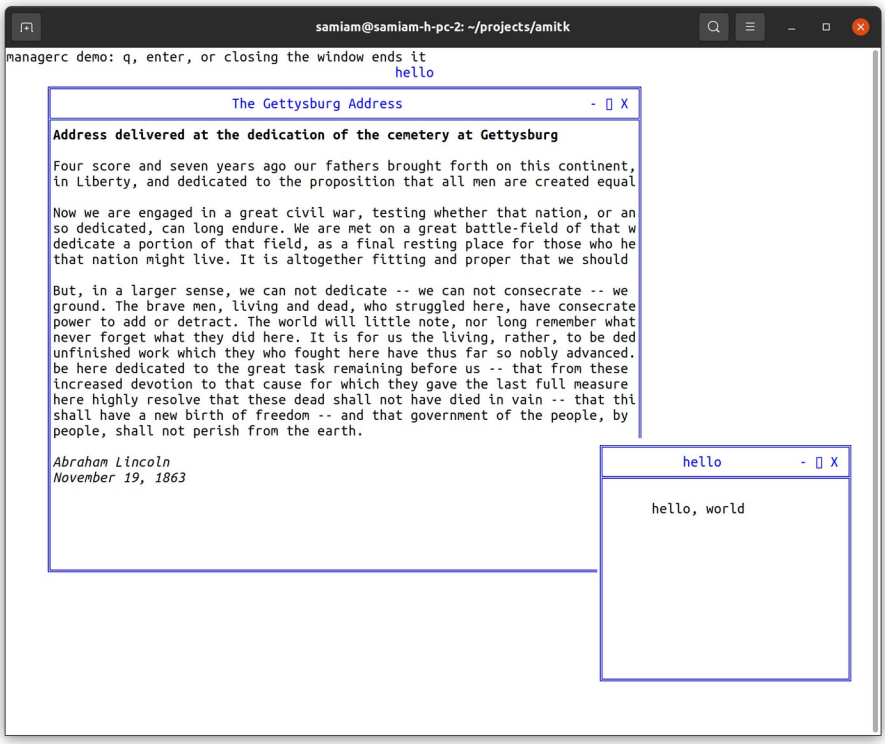
Here terminal is on a Linux xterm.



And here remoted by ssh from Linux to TeraTerm on Windows. The xterm escape character protocol makes an excellent lightweight remote serial display protocol.



Playing pong on a Linux CLI (no Xwindows display). xterm is implemented directly to the console.



Multiple managed windows on a character terminal, by the windowc module. The window frames, title bars and controls are drawn entirely with characters, so this works anywhere a terminal does.

These examples only scratch the surface. xterm can be used via an RS232 or other serial link, which could go to a remote system or embedded system.

4.1 ANSI C Compatible Mode

To write data to the **terminal** screen, ANSI C I/O calls are used. For example, a **putchar(c)** statement places each character on the screen, then moves the cursor to the right, obeying any automatic line wrapping. If a **putchar(c)** is called at the end of the line, then the cursor is returned to the left side of the screen, one line down. If the end of the line is reached, and automatic scrolling is enabled, then the screen will scroll upwards.

The '\f' output character, which causes a printer to move to the next (blank) page, is emulated by clearing the screen and moving the cursor to home position at (1,1).

All of the ANSI C I/O calls are supported in **terminal** mode, including **printf(f,...)**, **fprintf(f,...)**, **fscanf(f,...)**, **fputc(c,f)**, **fgetc(f)**, etc. The C++ stream calls are also supported.

4.2 Specifying the Main Window

All calls in **terminal** take a file to specify which window is supplying input or getting output, typically **stdin** or **stdout**. This is for upward compatibility reasons. For the majority of terminal calls, **stdout** is used, except for **event()**, which gets input from **stdin**.

4.3 Basic Cursor Positioning

The cursor is the point where text is entered onto the screen. It's usually marked with a blinking block or underline. To move the cursor one character up, down, left or right, the **up(f)**, **down(f)**, **left(f)** and **right(f)** procedures are used. To move the cursor anywhere on the screen, the **cursor(f,x,y)** call is used. To find out where the cursor currently is, the functions **curx(f)** and **cury(f)** return the current x and y coordinates of it.

The character cells on the screen are labeled from 1 to N, where in x 1 is the left side of the screen, and N is the right side of the screen. In y, 1 is the top of the screen, and N is the bottom of the screen.

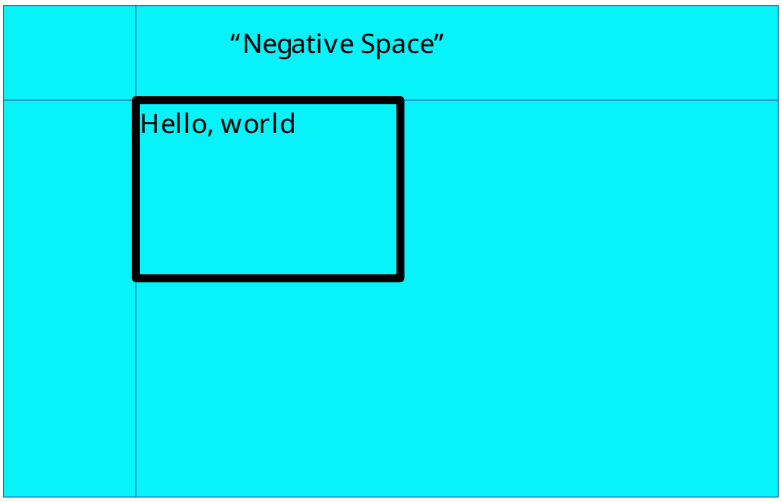
The actual size of the screen is system specific, and can be found by **maxx(f)** and **maxy(f)**, which return the maximum index in x and y for the screen. When a terminal is emulated in a windowing system, it is usually by default 25 lines of 80 characters each, because that was a very common size in terminals.

The cursor can be moved to the home, or 1,1 position, by the **home(f)** procedure.

4.4 Automatic Mode

Automatic mode is the mode that causes **terminal** to scroll upwards when the bottom of the screen is reached, and a newline character ('\n') is written. Line wrap, or the wrapping of characters back to the left at the next line if text is written off the right hand side, is an automatic mode. The automatic mode is useful for emulating ANSI C serial mode programs, but rapidly gets in the way for advanced programs.

Automatic mode can be turned off with **auto(f,e)**. Turning **auto(f,e)** off converts (or actually, reveals) the screen as a character surface that goes from –**LONG_MAX** to +**LONG_MAX** in both x and y, and has its origin at 1,1, and the screen is a "viewport" on this surface that extends from 1,1 to **maxx(f)**, **maxy(f)**. Text can be drawn anywhere, including off screen, but the characters outside of the 1,1 to **maxx(f)**, **maxy(f)** box will not be seen, and are "clipped out" of view. It can be determined if the cursor lies within the screen's bounds by **curbnd(f)**.



The "virtual screen" that appears with **auto(f,e)** off is a good match for today's windowing environments. Text can be drawn without worrying if it will cause the line to wrap, or the screen to scroll. And if a line of text happens to extend off the screen, that does not cause an error.

4.5 Tabbing

terminal will keep track of tabs set in x, for any character position. When **terminal** starts, the tabs are set to every 8th position on the screen, i.e., positions x= 9, 17, etc.

Outputting a tab character will cause the cursor to move to the next tab position on the line. If the cursor is at a tab position, then it will move to the next one. Tab positions can be set by **settab(f,t)**, and cleared by **restab(f,t)**. **clrtab(f)** clears all set tabs.

4.6 Scrolling

The screen can be scrolled by the **scroll(f,x,y)** procedure, which implements arbitrary direction scrolling. If the **y** value given is positive, then the screen data scrolls up. If the **y** value is negative, then the screen data scrolls down. If the **x** value is positive, the screen data scrolls left, and if it's negative, the screen data scrolls right. If either **x** or **y** is 0, then there is no movement in that direction.

Value	Cursor Direction	Screen data
+y	Down	Moves up
-y	Up	Moves down
+x	Right	Moves left
-x	Left	Moves right

4.7 Colors

There are two colors to set for text. One is the foreground, and the other is the background. The foreground is the color within the character itself. The background is the space behind the character. The possible colors are chosen from the two sets of primary colors:

```
typedef enum { ami_black, ami_white, ami_red, ami_green, ami_blue,
                ami_cyan, ami_yellow, ami_magenta } ami_color;
```

Characters are written in the currently set foreground and background colors. The foreground color is set by **fcolor(f,c)**, and the background color by **bcolor(f,c)**.

The current background color is also used to set the color of any blanked out areas caused by other commands. For example, outputting '\f' (form feed) clears the screen to the background color. Scrolls, either programmer selected or automatic, use the background color for any uncovered areas that are blanked out.

Many terminals have the ability to set colors from a full color pallete. fcolorc(f, r, g, b) will set the foreground color to an rgb or red, green, blue value, and bcolorc(f, r, g, b) will set the background color. The colors are ratioed to 0..LONG_MAX.

4.8 Attributes

terminal provides many attribute controls, each of which can be switched on or off individually.

Attribute	Result
Reverse	Enables reverse video.
underline	Underlines each character.
superscript	Gives a smaller and higher character for superscripting.
subscript	Gives a smaller and lower character for subscripting.
italic	Prints in italic or slanted characters.
bold	Prints in extra dark, or bold characters.
Strikeout	Prints characters with a horizontal bar through them.
blink	Blinks each character on and off.

For programs to maintain portability, the programmer should assume two things. First, that only a single attribute at one time can be set. This would mean that setting a second attribute after a first one is set, would cause the first attribute to be unset.

Second, the programmer should not assume that any one particular attribute is available. Many older terminals do not have more than one or two attributes. **superscript(f,e)**, **subscript(f,e)**, **italic(f,e)** and **strikeout(f,e)** are rare in standalone terminals. **superscript(f,e)**, **subscript(f,e)**, **italic(f,e)** and **bold(f,e)** are rare today on graphical windowing systems because they alter the geometry of the characters such that it is difficult or impossible to emulate a character cell in such a system. **blink(f,e)** is also rare in graphical systems because of the computational work required.

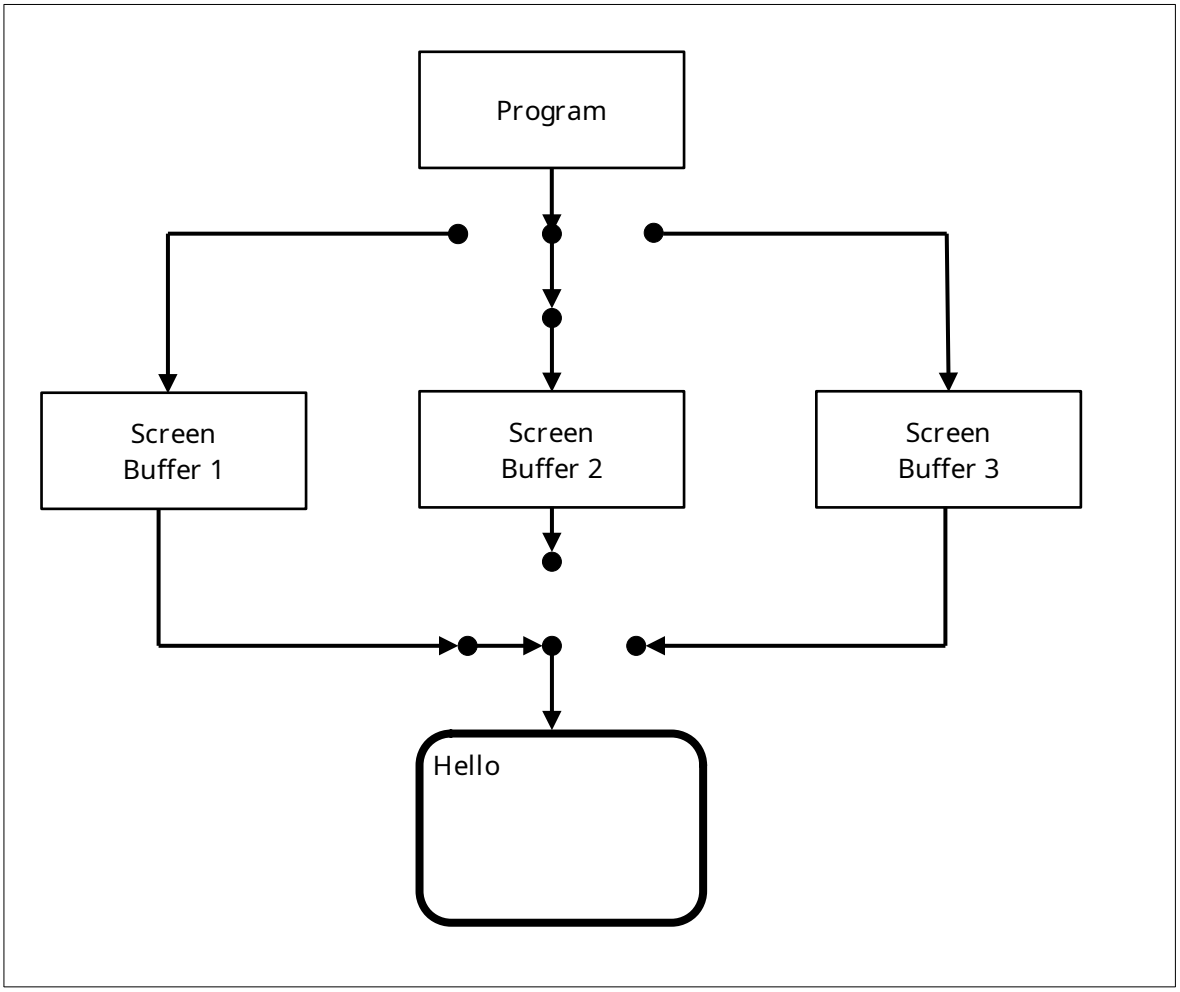
For this reason, there is a general mode, **standout(f,e)**, that can be enabled or disabled just like an attribute. What **standout(f,e)** does is enable an attribute the terminal does have, so that the program does not have to select a specific attribute. Most commonly this is implemented as reverse video.

4.9 Multiple Surface Buffering

terminal implements the ability to have logical screens in buffers. This is a copy of the characters on the screen, saved in a buffer of the same size as the screen. Logical buffers have many uses. Multiple screens can be kept, and quickly switched to the user. A program, like an editor, that wants to save the

screen as it was before it started, can switch away to a second screen to do its work, and then switch back to the original screen to restore its contents. Buffers can also be used to accomplish animation.

The **select(f,u,d)** procedure sets both the current screen to be displayed, and also the screen that updates are to go to. When **terminal** starts, these are both set to the same screen, number 1. Any screen can be selected, and the display screen and the update screen can be different. If the update screen is not the one in display, then all of the write statements will place characters in the buffer, but not on screen. This "split mode" is typically used for animation.



Implementations are free to limit the total number of logical screens available. The user should assume no more than 10 logical screens are available.

4.10 Advanced Input

terminal implements an advanced output model that is upward compatible with the ANSI C serial model. Similarly, an advanced input model is also implemented that meets the needs of newer systems.

Today's systems deal with multiple input devices, not just the keyboard. Devices include a mouse, a joystick, timers, and other future devices. The standard method on small computers was to poll for such

devices, or accept interrupts from them. The difficulty with those solutions was that these methods did not fit well with modern multitasking systems.

One method would be to use a multitask thread per device. However, this complicates small programs unnecessarily. The common solution today is "events", and the "event loop". The event model takes advantage of the fact that user input devices don't generate a lot of high bandwidth data. Each of the devices attached to the input from the user have their data packaged as "messages", which are small records, that are a description of the event. These events are then placed in a queue, and the program pulls the events, one at a time, from the queue and acts on them.

The description of an event record is:

```
/** events */
typedef enum {

    /** ANSI character returned */     ami_etchar,
    /** cursor up one line */          ami_etup,
    /** down one line */               ami_etdown,
    /** left one character */          ami_etleft,
    /** right one character */         ami_etright,
    /** left one word */               ami_etleftw,
    /** right one word */              ami_etrightw,
    /** home of document */            ami_ethome,
    /** home of screen */              ami_ethomes,
    /** home of line */                ami_ethomel,
    /** end of document */             ami_etend,
    /** end of screen */               ami_etends,
    /** end of line */                 ami_etendl,
    /** scroll left one character */   ami_etscrl,
    /** scroll right one character */  ami_etscrr,
    /** scroll up one line */          ami_etscru,
    /** scroll down one line */        ami_etscrd,
    /** page down */                   ami_etpagd,
    /** page up */                     ami_etpagu,
    /** tab */                         ami_ettab,
    /** enter line */                  ami_etenter,
    /** insert block */                ami_etinsert,
    /** insert line */                 ami_etinsertl,
    /** insert toggle */               ami_etinsertt,
    /** delete block */                ami_etdel,
    /** delete line */                 ami_etdell,
    /** delete character forward */    ami_etdelcf,
    /** delete character backward */   ami_etdelcb,
    /** copy block */                  ami_etcopy,
    /** copy line */                   ami_etcopyl,
    /** cancel current operation */    ami_etcan,
    /** stop current operation */      ami_etstop,
    /** continue current operation */  ami_etcont,
    /** print document */              ami_etprint,
    /** print block */                 ami_etprintb,
    /** print screen */                ami_etprints,
    /** function key */                ami_etfun,
    /** display menu */                ami_etmenu,
    /** mouse button assertion */      ami_etmouba,
    /** mouse button deassertion */    ami_etmoubd,
    /** mouse move */                  ami_etmoumov,
    /** timer matures */               ami_ettim,
    /** joystick button assertion */   ami_etjoyba,
    /** joystick button deassertion */ ami_etjoybd,
    /** joystick move */               ami_etjoymov,
    /** window was resized */          ami_etresize,
```

```
    /** window has focus */            ami_etfocus,
    /** window lost focus */           ami_etnofocus,
    /** window being hovered */        ami_ethover,
    /** window stopped being hovered */ ami_etnohover,
    /** terminate program */           ami_etterm,
    /** frame sync */                  ami_etframe,
    /** window redraw */               ami_etredraw,
    /** window minimized */            ami_etmin,
    /** window maximized */            ami_etmax,
    /** window normalized */           ami_etnorm,
    /** menu item selected */          ami_etmenus,

    /* Reserved extra code areas, these are module defined. */
    ami_etsys = 0x1000, /* start of base system reserved codes */
    ami_etman = 0x2000, /* start of window management reserved codes
*/
    ami_etwidget = 0x3000, /* start of widget reserved codes */
    ami_etuser = 0x4000 /* start of user defined codes */

} ami_evtcod;

/** event record */

typedef struct {

    /* identifier of window for event */ long winid;
    /* event type */                     ami_evtcod etype;
    /* event was handled */              long handled;
    union {

        /* these events require parameter data */

        /** etchar: ANSI character returned */  char echar;
        /** ettim: timer handle that matured */ long timnum;
        /** etmoumov: */
        struct {

            /** mouse number */   long mmoun;
            /** mouse movement */ long moupx, moupy;

        };
        /** etmouba */
        struct {

            /** mouse handle */  long amoun;
            /** button number */ long amoubn;

        };
        /** etmoubd */
```

```
        struct {

            /** mouse handle */  long dmoun;
            /** button number */ long dmoubn;

        };
        /** etjoyba */
        struct {

            /** joystick number */ long ajoyn;
            /** button number */   long ajoybn;

        };
        /** etjoybd */
        struct {

            /** joystick number */ long djoyn;
            /** button number */   long djoybn;

        };
        /** etjoymov */
        struct {

            /** joystick number */      long mjoyn;
            /** joystick coordinates */ long joypx, joypy, joypz;
                                        long joyp4, joyp5, joyp6;

        };
        /** function key */ long fkey;
        /** etresize */
        struct {
            long rszx, rszy;
        };
        /* ami_etmenus */
        long menuid; /* menu item selected */

    };

} ami_evtrec, *ami_evtptr;
```

The event codes from **etredraw** on serve the window management and widget levels, and the reserved areas hold their extensions and user defined events; see the windows management and widgets chapters.

The next event for the program is retrieved via the **event(f,er)** call. The basis of the event system is that events must be retrieved often enough that the input queue does not fill up. Thus, an "event model" program will be centered on the event loop that gets the next event, acts on it, and returns to the top for more events.

An important principle of event handling is that events that are not defined for a compliant program are ignored. That is, if an event not defined for **terminal** is received, it will be ignored. This is the basis of upward compatibility. If a new event is defined, existing programs will simply ignore it.

A typical event loop is as follows.

```
#include <stdio.h>
#include <terminal.h>

int main()

{

    ami_evtrec er;

    do {

        ami_event(stdin, &er); /* get next event */
        switch (er.etype) {

            case ami_etchar:
                if (er.echar == 'g') /* perform character action */;
                break;
            case ami_etmoumov: /* perform mouse move action */
                break;
            default: /* do nothing */
                break;

        }

    } while (er.etype != ami_etterm);

}
```

Note the final default case that does nothing.

4.11 Buffer Follow mode

Some terminal implementations can exist in windowed environments, and that window can be resized. For this reason, terminal provides a mode called “buffer following” that allows a program to resize its output to “follow” the window size onscreen. The function **sizbuf(f,x,y)** will change the size of the buffers to the size in x and y. When the buffer is resized, the functions **maxx(f)** and **maxy(f)** will change to reflect the new size. If the buffer is larger than the window size, it will be clipped, and the system may or may not provide a way for the user to examine the contents of the buffer via scrolling or other means. The function **autobuf(f, e)** will enable scrolling of the buffer in this case. This is usually turned off in buffer follow mode to prevent any scroll bars from appearing briefly during resize.

If the buffer is smaller than the window size, it is presented at the origin, and the unoccupied space in the window is cleared to the background color. After a **sizbuf(f,x,y)** call, all of the screen buffers are resized, and any previous contents are lost, and the buffer is cleared to spaces. The exception to this rule

is the current buffer in display, whose contents are copied to the new display buffer for as many character cells are in common in x and y. The reason for this is to suppress “flash”, or the time between the redrawing of the new, resized buffer and the program refresh of it.

When the window size is changed, an **etresize** message will be sent to the window. This message carries the new size of the window. The message won’t be sent on a system that does not perform windowing (a full screen terminal), and the program can ignore it if it does not implement buffer following. To implement buffer following, the program would take the **rszx** and **rszy** size parameters from the message and use the **sizbuf(f,x,y)** function to resize the buffer. The **maxx(f)** and **maxy(f)** functions will then “follow” the new window size and the program should rewrite the update buffer for all active screens.

The main thing to understand about buffer follow mode is that it is optional. The client program can ignore a change in the onscreen display size, and keep maintaining a buffered virtual screen, or it can change its buffer to follow the screen.

4.12 Focus events

A terminal that is hosted by a windowing system may or may not have focus. Focus indicates the terminal will get input. The focus events only apply if the terminal is within a window on a windowed system. Focus is always true on a non-windowed system.

There are two focus events, **etfocus** and **etnofocus**. **etfocus** indicates the input is active to the terminal. **etnofocus** indicates no input will occur. The events only are sent when the state of focus changes. Typically, the focus state will be accompanied by a change in the cursor.

4.13 Hover events

A terminal that is hosted by a windowing system may or may not have hover. Hover indicates the mouse is over the terminal window. The hover events only apply if the terminal is within a window on a windowed system. Hover is always true on a non-windowed system.

There are two hover events, **ethover** and **etnohover**. **ethover** indicates the mouse is over the terminal window. **etnohover** indicates the mouse is no longer over the terminal window. The events are only sent when the state of hover changes.

4.14 Legacy Input

terminal mode supports all of the ANSI C input methods (**fgetc(f)**, **fscanf(f,...)**, etc.). It does this by calling **event(f,er)** for you, and discarding all events not related to line character input. Typically you want to get your own input with **event(f,er)**. The legacy mode can be quite useful for inputting lines of characters from the user, because **terminal** mode often features line editing functions.

4.15 Event callbacks

An alternative to setting up an event loop, or a complement to it, is to arrange callbacks for individual events. Events are overridden with the call **eventover(e, eh, oeh)**. The event to be overridden is specified, along with a function to be called when the event occurs. The old routine that was executed on the callback is also returned, and normally that is executed within the handler routine:

```
/** event function pointer */
typedef void (*ami_pevthan)(ami_evtrec*);

#include <stdio.h>

ami_pevthan oldhandler; /* previous event handler */

void myevent(ami_evtptr er)

{

    /* perform event handling using event record er */
    oldhandler(er); /* pass back to previous handler */

}

int main()

{
    ami_evtrec er; /* event record */

    /* override event handler */
    ami_eventover(ami_etchar, myevent, &oldhandler);
    do { ami_event(stdin, &er); } while (er.etype != ami_etterm);

}
```

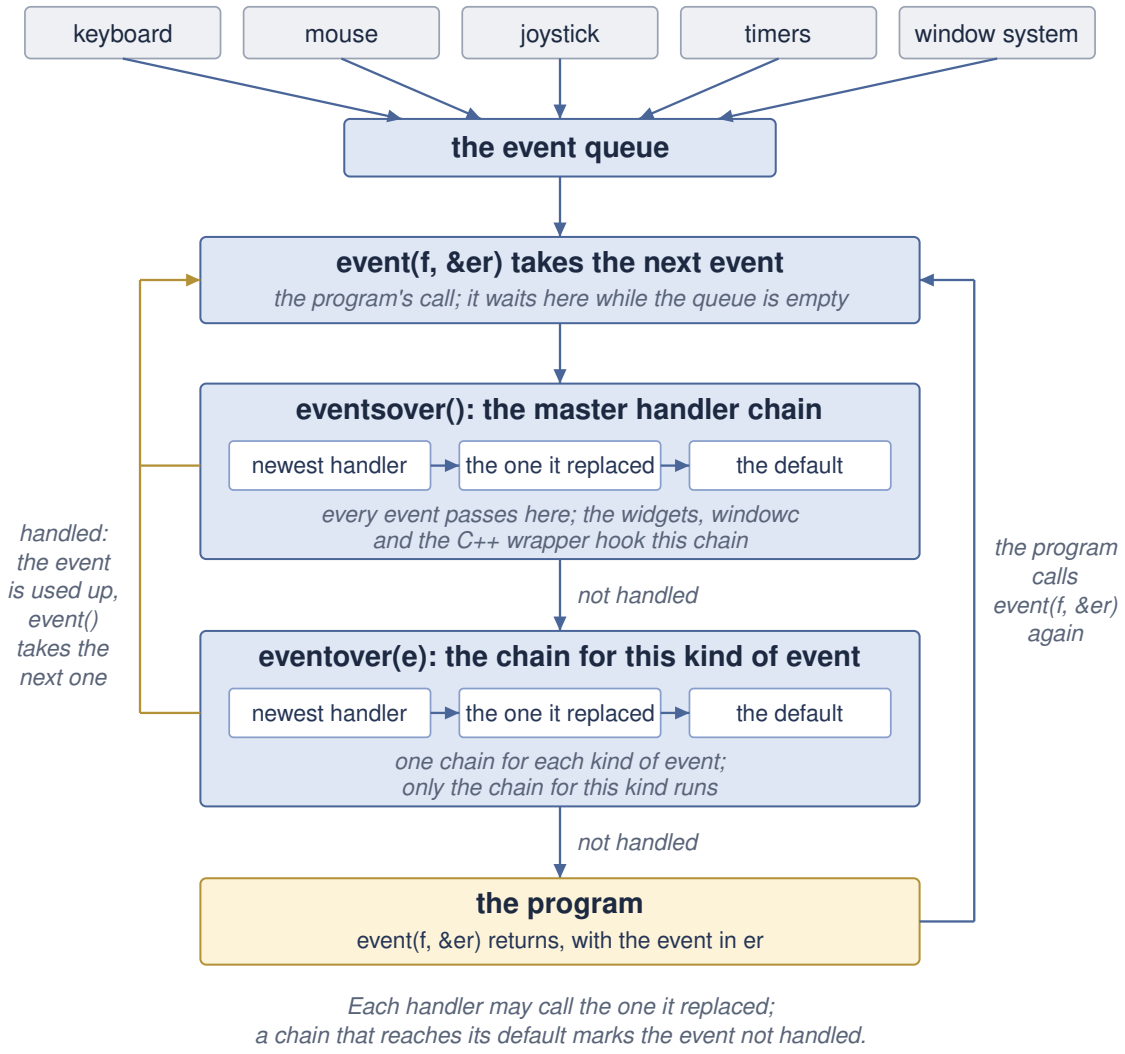
event(f,er) directly calls the overriding function, and the event is processed there. If the overriding procedure does not want to handle the event, then the inherited version of the procedure is called in the overrider. This accomplishes two things. First, any other overriders that are chained to the procedure are executed, so that they may handle the event. Finally, if none of the overriders wish to handle the event, the chain ends with the default implementation, which then flags that the event is to be returned to the caller of **event(f,er)**. In this way, if none of the overriders handle the event, it is returned as a normal record back to the **event(f,er)** caller.

There is no parallel execution implied in such event callbacks. The **event(f,er)** function still must be called to activate the event procedures, and all such procedures run in the context of the current process.

The event override mechanism is a way to break the rigid formalism of event loop design. In the event loop model, the event loop must be changed anytime there is new code that needs to handle events. Using event procedures, new handlers can be added without such modification. This enhances modularity, since the new code may be in a different module. This aids the extensibility of the system.

Event procedures are also a way to obfuscate a program by making it less obvious where the flow of control is in the program. The advantage to the event loop model is that it creates clear flow of control, even when handling asynchronous user generated events.

Another type of event override is **eventsover(eh, oeh)**. This call overrides all of the events. This can be used to place a filter of all events.



The road of an event: from its source to the queue, through the master handlers and then the handlers for its kind, and back to the program from event() when none of them takes it.

4.16 Timers

Timers allow a **terminal** program to perform periodic events, such as screen updates, and keeping track of time. From 1 to 10 timers are available, numbered 1..10. Each timer is given a time to measure, in 100 Microsecond counts (see 3.3 “Time and Date” in **services** for more details). When the timer is done, it sends an event to the event queue.

Timers can either simply stop when their time is done, or they can automatically start timing their original set time again. This is the "recurrent" mode, and it's useful when an activity needs to be performed periodically for the life of the program. For example, updating the screen on an active program, or checking when to update a clock display.

Timers are set by **timer(f,i,t,r)**. A timer can be stopped or "killed" by **killtimer()**. Killing a timer that is not active will not generate an error. This allows a single run timer to be killed without timing out during the call.

Due to the queuing nature of the event system, there is no guarantee about the accuracy of a timer. It can arrive later than its set time. If the program is late getting back to the event queue, or is busy with other events that occur before the timer, receipt of the timer event can be very late. All that receiving a timer event does is indicate that the time it measured has passed.

An example of timer use is the display of a clock, using the **services** time call. If a recurrent timer is set to go off on every second, the program should ***not*** simply advance the second on each timer event. Instead, the timer event should tell the program to read time to determine if the second has changed, and what value it currently has.

Implementations are free to limit the number of timers. Users should assume no more than 10 timers are available.

4.17 The Frame Timer

When performing animation, it is common to flip between screen buffers with select. The ideal time to perform a buffer flip is during the retrace time for the display, or the time it takes the display drawing hardware to reset to the top of the screen, and start drawing from the top again. Hiding the buffer flip in the retrace time can be key to making animations appear smooth.

The frame timer is a timer much like the standard timers, except that it is set automatically by the implementation. It is simply enabled or disabled, and gives an **etframe** event when it times out. On hardware that is capable, the **etframe** event is triggered by the beginning of the retrace cycle.

If an interrupt for the start of the retrace cycle is not available, then the frame timer is simply defaulted to a reasonable rate of redraw for animation, for example, 30 times per second.

4.18 Mouse

The mouse gives a position x,y on the screen, as well as from 1 to n buttons on it. A mouse actually gives its position as relative movements in x and y, but the system converts this to a screen position. The function **mouse(f)** returns the number of mice attached to the system.

Mice generate two events. First, when the mouse moves, it generates position changes via the **etmoumov** event. The program does not have to worry about where it is going. Each time the position changes, a new x, y position is posted as an event.

The second event generated by a mouse is mouse button asserts and deasserts, **etmouba** and **etmoubd**. An "assert" means a press of the button, and a "deassert" is the release of the button. Note that instead of an on/off status check as polled by the program, the assert and deassert events give exact notice of when the button changes state, and what it is changing to.

When there is more than one mouse per system, the default behavior should be to treat them as separate, but equal controls on the screen from the same user. When a mouse moves or changes button state, it gets control of the program. This matches the common use of multiple pointing devices, where two devices are alternated by one user. For example, a trackball and a mouse may be just two different input methods from one user.

Alternately, a second mouse could be a remote mouse over a network. This "collaborative computing" model allows two users to look at the same document, with separate mice. Advanced implementations of multiple mice such as these should be selected by the user.

Mice are subject to “rate limiting”. This means that the number of events per second reported by the mouse can be limited to no more than what the human viewer can perceive. This prevents the number of mouse events from affecting program performance.

4.19 Joysticks

From 0 to 4 joysticks may be supported. Each joystick can have from 0 to 3 "axes" of directions of travel. In addition, each joystick can have 0 to 4 buttons. The function **joystick(f)** returns the number of joysticks in the system. The function **joyaxis(f,j)** gives the number of axes on a given joystick. The function **joybutton(f,j)** gives the number of buttons on a given joystick.

The messages **etjoyba** or joystick button assert, and **etjoybd** or joystick button deassert, give events for the assertion and deassertion of the buttons on a joystick. When any axis on a joystick moves, it generates a **etjoymov**, or joystick movement, event. This event gives the relative setting of each axis of the joystick.

Unlike a mouse, a joystick is entirely relative: it reports deflection from center, not position. Each axis is represented by an integer. If the axis is not implemented it always reads 0. If the axis is deflected left, up or in, depending on the type of axis, then it is negative. If it is right, down, or out, it is positive. The axis is determined in its amount of deflection. If it is in the middle, it is 0. If it is deflected to the maximum, it is **LONG_MAX**, with the sign giving the direction.

By convention, the axes on a joystick are:

1. Left/right, or slider.
2. Up/down
3. In/out

Joysticks are "rate limited" devices. This means that no matter how fast the joystick is "slewed", it will only produce an event about every 10 milliseconds at maximum rate. The reason for this is that humans cannot generally perceive events like movement of a pointer across the screen at a faster rate than this, so there is no point in updating faster than that, and flooding the input event queue.



A typical USB joystick with several axis and several buttons.

4.20 Function Keys

The system may have function keys, which are keys whose function are determined by the program. The number of function keys implemented are found with **funkey(f)**. Function key messages are sent by the message **etfun**. Typically at least 10 function keys are implemented, but **terminal** may implement many more. The Ami standard method for handling these is to give up to 10 predefined functions to the user, then let the user program the equivalent function for the rest of the function keys.

4.21 Automatic "hold" Mode

terminal implements a feature to help with legacy programs designed to the ANSI C serial I/O model. When a program exits that is unaware of the terminal model, the terminal window can abruptly close. This means that any printout from the program is lost.

The automatic hold mode keeps the window open unless an **etterm** event is received, until the user specifically closes the window. This mode is enabled by default in systems where it is valuable, i.e., windowing systems.

There are times when the automatic hold mode can be a problem. For example, if a user displayed menu features a "quit" option, it would be incorrect to hold that window after the user has already closed the program.

To solve this, the **autohold(e)** procedure can be used to set automatic hold mode off or on. With automatic hold mode off, the window exits immediately when the program does. Note that **autohold(e)** does not take a file or window parameter.

4.22 Direct Writes

terminal accepts all standard ANSI C output methods, **putchar(c)**, **printf(fm,...)**, **fprintf(f,fm,...)** and others. However, it can be faster to output characters to the console directly, bypassing the normal file protocol layers inherent in the system. The procedure **wrtstr(f,s)** outputs a character string directly to the terminal without any interpretation of control characters. It cannot be used with **auto(f,e)** on.

4.23 Printers

terminal can be used to operate a printer using a subset of the **terminal** functionality. To model a printer, **terminal** uses a one page buffer that can be written using normal **terminal** commands to write to this "virtual screen" contained within the buffer. When the form-feed character is output ('\f'), the contents of the buffer is output to the printer as a whole page, and then a new page can be written.

When **terminal** is connected to a printer file, the following conditions are true:

1. There is no input file associated with the printer, neither the event loop nor the virtual event methods function. **event(f,er)** gives an error. None of the input devices work, and timer, mouse, joystick, function keys, the frame timer and the **autohold(e)** procedure all give errors.
2. The **select(f,u,d)** call does not function, and gives an error.
3. The dimensions given by **maxx(f)** and **maxy(f)** reflect the size of the printed page.
4. The exact set of attributes will be dependent on the printer. **blink(f,e)**, of course, is never available.
5. The exact size of the output buffer can be selected by **sizbuf(f, x, y)**.

A printer device is viewed as accepting a series of pages to be printed. The last page should be followed by outputting a form-feed ('\f'), to insure that a partial page is not left in the page buffer. **terminal** may automatically add a form-feed if the printer file is closed with a page still active.

Since printer device mode is a subset of **terminal** functionality, it is possible for any program designed to output to a printer to have its output viewed on a terminal screen. Each page will then be presented to the user in turn.

The utility of using **terminal** to perform printer output, vs. using direct output to a printer, is that each page can be fully and easily formatted before outputting it.

A print file is opened with **openprint(f, n)**. The name **n** is either a filename, which receives the printed pages as plain text, or a printer, recognized by the name. The exact format of a printer name is operating system specific.

A printer receives the document from the spooler as a single job when the print file is closed; a file receives each page as it completes.

The page buffer is 80 by 25 characters, and **sizbuf(f, x, y)** resizes it: the buffer size is the print page size. Output composes onto the page, cursor movement included, until the form-feed completes it; the page is then written as formatted text, with the position of the text on each line carried by left padding with spaces. The colors and the text attributes are accepted and ignored: plain text carries neither[4].

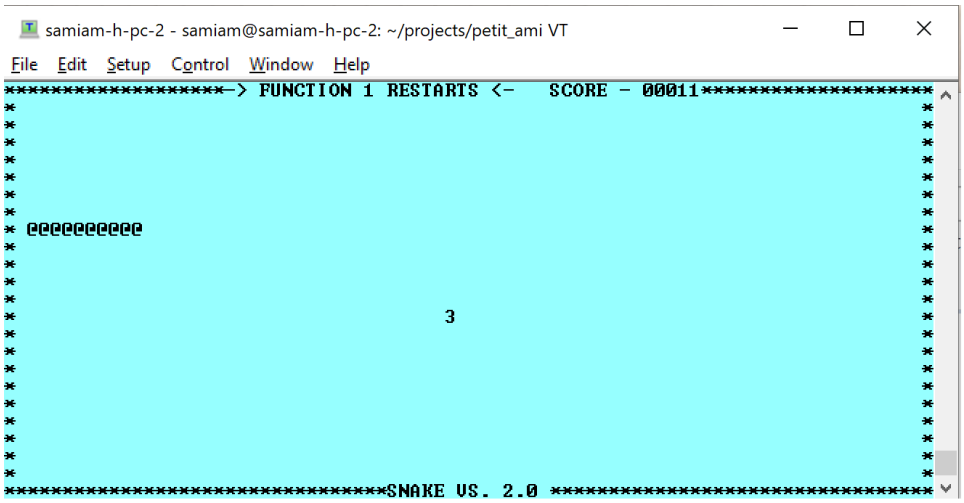
[4]It is possible to use a full graphics print module, in which this condition may not apply.

4.24 Remote display

terminal is compatible with the use of “remote display” of presentation text. In this mode, the output from **terminal** is encoded and sent over a communications channel. This is done for both input and output functionality, and thus remote display mode is implemented without restriction of functionality.

The exact format of the data passing over the communications channel to allow remote mode to function is system dependent.

Terminal is designed to operate efficiently with such remote displays. For example, there is no ability to read characters from the display.



An xterm ssh display is effectively a remote display. However, terminal may define its own remote display protocol, or even the Xwindow protocol can be used. Of interest in this mode is that some features such as timers don’t matter where they are implemented, server or client.

4.25 Advanced Input in C++

The event record in C++ is the same as for C, but the “ami_” prefix is not used. The terminal.hpp file contains a namespace terminal, and the namespace is joined by:

```
using namespace terminal;
```

Alternately, symbols from the terminal namespace can be specified individually:

```
terminal::event(stdin, &er);
```

The event type, the event record, and every other declaration are those of the C interface with the ami_ prefix dropped; the bracketed [ami_] in the listings of this chapter marks exactly what falls away.

4.26 Terminal Objects In C++

All of the procedures, functions and other declarations in **terminal** are also available in a terminal object of the form:

```
/* object based interface */
class term {

FILE*      infile;
FILE*      outfile;

public:

/* constructor */
term();

/* destructor */
~term();

/* copying is refused: two objects would free one event hook */
term(const term&) = delete;
term& operator=(const term&) = delete;

/* methods */
void cursor(long x, long y);
long  maxx(void);
long  maxy(void);
void home(void);
void del(void);
void up(void);
void down(void);
void left(void);
void right(void);
void blink(long e);
void reverse(long e);
void underline(long e);
void superscript(long e);
void subscript(long e);
void italic(long e);
void bold(long e);
void strikeout(long e);
void standout(long e);
void fcolor(color c);
void bcolor(color c);
void autom(long e);
void curvis(long e);
void scroll(long x, long y);
long  curx(void);
long  cury(void);
long  curbnd(void);
void select(long u, long d);
void event(evtrec* er);
void timer(long i, long t, long r);
void killtimer(long i);
```

```
long  mouse(void);
long  mousebutton(long m);
long  joystick(void);
long  joybutton(long j);
long  joyaxis(long j);
void settab(long t);
void restab(long t);
void clrtab(void);
long  funkey(void);
void frametimer(long e);
void autohold(long e);
void wrtstr(char *s);
void wrtstr(char *s, long n);
void wrtstrn(char *s, long n);
void sizbuf(long x, long y);
static void termCB(evtrec* er);

/* virtual callbacks */
virtual long evchar(char c);
virtual long evup(void);
virtual long evdown(void);
virtual long evleft(void);
virtual long evright(void);
virtual long evleftw(void);
virtual long evrightw(void);
virtual long evhome(void);
virtual long evhomes(void);
virtual long evhomel(void);
virtual long evend(void);
virtual long evends(void);
virtual long evendl(void);
virtual long evscrl(void);
virtual long evscrr(void);
virtual long evscru(void);
virtual long evscrd(void);
virtual long evpagd(void);
virtual long evpagu(void);
virtual long evtab(void);
virtual long eventer(void);
virtual long evinsert(void);
virtual long evinsertl(void);
virtual long evinsertt(void);
virtual long evdel(void);
virtual long evdell(void);
virtual long evdelcf(void);
virtual long evdelcb(void);
virtual long evcopy(void);
virtual long evcopyl(void);
virtual long evcan(void);
```

```
virtual long evstop(void);
virtual long evcont(void);
virtual long evprint(void);
virtual long evprintb(void);
virtual long evprints(void);
virtual long evfun(long k);
virtual long evmenu(void);
virtual long evmouba(long m, long b);
virtual long evmoubd(long m, long b);
virtual long evmoumov(long m, long x, long y);
virtual long evtim(long t);
virtual long evjoyba(long j, long b);
virtual long evjoybd(long j, long b);
virtual long evjoymov(long j, long x, long y, long z);
virtual long evresize(long rszx, long rszy);
virtual long evfocus(void);
virtual long evnofocus(void);
virtual long evhover(void);
virtual long evnohover(void);
virtual long evterm(void);
virtual long evframe(void);

}; /* class term */
```

Each method is the terminal call of the same name with the file parameter left off; the descriptions appear in 4.28 “Procedures, functions and methods in terminal”.

The file is **stdin** for input and **stdout** for output. The constructor binds the object to them, so a **term** object is the main window and no other, and there is one **term** object at a time: a second would take the events from the first, and the constructor refuses it. Nothing is attached by the program; by the time **main** runs, **stdin** and **stdout** are the terminal, and the object names them. **printf** and the rest of stdio reach the same window through **stdout**, which is why the example below prints with **printf** and reads with **ti.event()**.

The event method takes an event record just as event(f,er) does, with the file left off.

A **term** object can be created as follows:

```
#include <stdio.h>
#include <stdlib.h>

#include <terminal.hpp>

using namespace terminal;

int main()

{

    term   ti; /* terminal object */
```

```
    evtrec er; /* event record */

    printf("hello c++ world\n");
    do { ti.event(&er);
        if (er.etype == etterm) exit(1);
    } while (er.etype != etenter);

}
```

As in the procedural interface to terminal, events in the **term** class can be registered as callbacks via the virtual procedures. However, just as in the procedural interface, such callbacks do not function unless the event method for the **term** object is called.

One term object exists at a time. The events come back to the object through a single hook on the event chain, so a second object would not share the events with the first: it would take them all. The constructor refuses a second object with a message rather than allow it. The destructor takes the hook back out and restores the event chain as it was, so a term object can be made and destroyed in the course of a program, and no event is ever delivered to an object that no longer exists. The graph object of the graphics library keeps the same rule.

4.27 Event Callbacks in C++

In C++, an alternative to receiving events as structures are the event based virtual methods:

```
/* virtual callbacks */
virtual long evchar(char c);
virtual long evup(void);
virtual long evdown(void);
virtual long evleft(void);
virtual long evright(void);
virtual long evleftw(void);
virtual long evrightw(void);
virtual long evhome(void);
virtual long evhomes(void);
virtual long evhomel(void);
virtual long evend(void);
virtual long evends(void);
virtual long evendl(void);
virtual long evscrl(void);
virtual long evscrr(void);
virtual long evscru(void);
virtual long evscrd(void);
virtual long evpagd(void);
virtual long evpagu(void);
virtual long evtab(void);
virtual long eventer(void);
virtual long evinsert(void);
virtual long evinsertl(void);
virtual long evinsertt(void);
virtual long evdel(void);
virtual long evdell(void);
virtual long evdelcf(void);
virtual long evdelcb(void);
virtual long evcopy(void);
virtual long evcopyl(void);
virtual long evcan(void);
virtual long evstop(void);
virtual long evcont(void);
virtual long evprint(void);
virtual long evprintb(void);
virtual long evprints(void);
virtual long evfun(long k);
virtual long evmenu(void);
virtual long evmouba(long m, long b);
virtual long evmoubd(long m, long b);
virtual long evmoumov(long m, long x, long y);
virtual long evtim(long t);
virtual long evjoyba(long j, long b);
virtual long evjoybd(long j, long b);
virtual long evjoymov(long j, long x, long y, long z);
virtual long evresize(long rszx, long rszy);
virtual long evfocus(void);
virtual long evnofocus(void);
```

```
virtual long evhover(void);
virtual long evnohover(void);
virtual long evterm(void);
virtual long evframe(void);
```

Each event method corresponds to an event from the **evtrec** structure. When **event()** has an event to return to the calling program, it first calls the default implementation of the corresponding virtual method, which flags that the event is unhandled, and should be returned to the caller.

The alternative method is activated by overriding the virtual procedure corresponding to the event that is to be received directly. **event()** directly calls the overriding procedure, and the event is processed there. If the overriding procedure handles the event, it returns true (non-zero). If the overriding procedure does not want to handle the event, then the inherited version of the procedure is called in the overrider. This accomplishes two things. First, any other overriders that are chained to the procedure are executed, so that they may handle the event. Finally, if none of the overriders wish to handle the event, the chain ends with the default implementation, which then flags that the event is to be returned to the caller of **event()**. In this way, if none of the overriders handle the event, it is returned as a normal record back to the **event()** caller.

There is no parallel execution implied in such event callbacks. The **event()** function still must be called to activate the event procedures, and all such procedures run in the context of the current process.

The event procedures are prefixed with “ev” (event) to prevent them from colliding with the names of the normal **terminal** functions.

This is an example of using event callbacks in C++:

```
#include <stdlib.h>
#include <iostream>

#include <terminal.hpp>

using namespace terminal;

class myterm: public term

{

    public:
    long evterm(void) {

        std::cout << "Terminating!" << std::endl;
        exit(1);

    }

};

int main()

{

    myterm ti;
    evtrec er;

    std::cout << "Hello c++ world" << std::endl;
    do { ti.event(&er); } while (1);

}
```

The event procedures are a way to break the rigid formalism of event loop design. In the event loop model, the event loop must be changed anytime there is new code that needs to handle events. Using event procedures, new handlers can be added without such modification. This enhances modularity, since the new code may be in a different module. This aids the extensibility of the system.

Event procedures are also a way to obfuscate a program by making it less obvious where the flow of control is in the program. The advantage to the event loop model is that it creates clear flow of control, even when handling asynchronous user generated events.

4.28 Procedures, functions and methods in terminal

For C++ optional parts of the specification of each function is denoted by []. The contents within the brackets are optional in the C++ interface. For all functions using an input file, the default for the file, if the parameter is not specified, is **stdin**. For all functions using an output file, the default for the file, if the parameter is not specified, is **stdout**.

void [ami_]auto([FILE* f,] long e);

Turns automatic mode on if **e** is 1, or off if **e** is 0, in output surface file **f**. Note that the autom() name must be used in C++, as auto is a reserved word there.

void [ami_]autohold(long e);

Sets the state of automatic hold. If **e** is 1, **autohold** is enabled. If **e** is 0, **autohold** is disabled. **autohold** determines if the window will exit immediately if the program self terminates. If an exit was not ordered via the user interface, the display is held until it is.

void [ami_]bcolor([FILE* f,] [ami_]color c);

Sets the background, or space color, to the color **c** in output surface file **f**.

void [ami_]bcolorc([FILE* f,] long r, long g, long b);

Sets the background color by components r, g and b, each 0 to LONG_MAX, rendered by approximation as fcolorc() is.

void [ami_]blink([FILE* f,] long e);

If **e** is 1, causes all further characters written to the screen to appear in blinking text in output surface file **f**. if **e** is 0, blink is turned off.

void [ami_]bold([FILE* f,] long e);

If **e** is 1, causes all further characters written to the screen to appear in bold text in output surface file **f**. If **e** is 0, bold mode is turned off.

void [ami_]clrtab[(FILE* f)];

Clear all tabs. All tabs are removed from the tabbing table in output surface file **f**.

long [ami_]curbnd[(FILE* f)];

Check cursor in bounds. Returns true if the cursor is currently within the bounds of the screen in output surface file **f**.

void [ami_]cursor([FILE* f,] long x, long y);

Set cursor location for output surface file **f** in **x** and **y**.

void [ami_]curvis([FILE* f,] long e);

Turns cursor visibility on if **e** is 1, or off if **e** is 0, in output surface file **f**.

long [ami_]curx[(FILE* f)];

Find the current **x** location of the cursor in output surface file **f**.

long [ami_]cury[(FILE* f)];

Find the current **y** location of the cursor in output surface file **f**.

void [ami_]del[(FILE* f)];

Back up cursor by one character in output surface file **f**, and erase character at that location. If the cursor is at the left side of the screen, and automatic mode is on, the cursor will be moved up one line, and to the right of screen. If the cursor is at the top of the screen, extreme left, then the screen will be scrolled down one line, and the cursor moves to the right side of the screen. If automatic mode is off, the cursor simply moves left.

void [ami_]down[(FILE* f)];

Move cursor down one line in output surface file **f**. If the cursor is already at the bottom line, and automatic mode is on, the screen will be scrolled up, and the cursor remains at the same position. If automatic mode is off, the cursor simply moves down one line.

void [ami_]errorover([ami_]errhan nfp, [ami_]errhan* ofp);

Hooks the library error handler. The new handler nfp receives the error code, and the old handler is returned in ofp for chaining. The handler must not return: it ends the program or passes the code on.

void [ami_]event([FILE* f,] [ami_]evtrec* er);

Get next event. Retrieves the next event from the input queue from the terminal input file **f** to event record **er**. If there is no event ready, the program will wait.

void [ami_]eventover([ami_]evtcod e, [ami_]pevthan eh, [ami_]pevthan* oeh);

Overrides the event **e** with the new handler function pointer **eh**, and returns the old event handler function pointer in **oeh**. The event handler function call is hooked, meaning that each new event function handler that overrides the original can chain to the last. If the event handler function does not care to process the event, it calls the old event handler to continue the chain. If no overrider wants to handle the event, it goes back to the caller of **event().**

void [ami_]eventsover([ami_]pevthan eh, [ami_]pevthan* oeh);

Overrides all events with the new handler function pointer **eh**, and returns the old event handler function pointer in **oeh**. The event handler function call is hooked, meaning that each new event function handler that overrides the original can chain to the last. If the event handler function does not care to process the event, it calls the old event handler to continue the chain. If no overrider wants to handle the event, it goes back to the caller of **event().**

void [ami_]fcolor([FILE* f,] [ami_]color c);

Sets the foreground, or text color, to the color **c** in output surface file **f**.

void [ami_]fcolorc([FILE* f,] long r, long g, long b);

Sets the foreground color by components r, g and b, each 0 to LONG_MAX. The color is rendered by approximation: the display shows the nearest color it can, which on a primary color terminal is the nearest primary.

void [ami_]frametimer([FILE* f,] long e);

Enables or disables the framing timer in output surface file **f**. If **e** is 1, the frame timer is enabled. If **e** is 0, it is disabled. The frame timer gives frame timer events, which occur approximately on each refresh of the display screen. It may be tied to the refresh hardware, or may be simulated via a timer.

long [ami_]funkey[(FILE* f)];

Returns the number of function keys available in output surface file **f**.

void [ami_]home[(FILE* f)];

Send cursor to 1,1 location (upper left of screen) in output surface file **f**.

void [ami_]italic([FILE* f,] long e);

If **e** is 1, causes all further characters written to the screen to appear in italic text in output surface file **f**. If **e** is 0, italic mode is turned off.

long [ami_]joyaxis([FILE* f,] long j);

Returns the number of axes on the given joystick **j** in output surface file **f**.

long [ami_]joybutton([FILE* f,] long j);

Returns the number of buttons on the given joystick **j** in output surface file **f**.

long [ami_]joystick[(FILE* f)];

Returns the number of joysticks in the system in output surface file **f**.

void [ami_]killtimer([FILE* f,] long i);

Stop timer in output surface file **f**. Stops the timer **i**. If the timer is not active, no error is reported.

void [ami_]left[(FILE* f)];

Back up cursor by one character in output surface file **f**. If the cursor is at the left side of the screen, and automatic mode is on, the cursor will be moved up one line, and to the right of screen. If the cursor is at the top of the screen, left, then the screen will be scrolled down one line, and the cursor moves to the right side of the screen. If automatic mode is off, simply moves left one character.

long [ami_]maxx[(FILE* f)];

Find maximum screen location **x** in output surface file **f**.

long [ami_]maxy[(FILE* f)];

Find maximum screen location **y** in output surface file **f**.

long [ami_]mouse[(FILE *f)];

Returns the number of mice in output surface file **f**.

long [ami_]mousebutton([FILE* f,] long m);

Returns the number of buttons on a given mouse **m** in output surface file **f**.

void [ami_]openprint(FILE** f, char* n);

Opens a print file to **f**. The name **n** is either a filename, which receives the printed pages as plain text, or a printer, recognized by the ‘:’ in the name: **lp0:**, **lp1:**, and so on are the printers by number as the system lists them, and **lp:** is the system default printer. The print file is written with the normal terminal calls onto the page buffer, 80 by 25 characters until **sizbuf()** resizes it; outputting a form-feed (‘\f’) completes the page. There is no input side: the event and input

device calls give errors. Closing the file completes the document, ejecting a partial page, and sends a printer document to the spooler as a single job.

void [ami_]restab([FILE* f,] long t);

Reset tab. Removes the tab at location **t** in output surface file **f**. If there is not a tab set there, it is not an error.

void [ami_]reverse([FILE* f,] long e);

If **e** is 1, causes all further characters written to the screen to appear in reverse text in output surface file **f**. If **e** is 0, reverse mode is turned off.

void [ami_]right[(FILE* f)];

Move forward by one character in output surface file **f**. If the cursor is at the right side of the screen, and automatic mode is on, the cursor will be moved down one line, and to the left of screen. If the cursor is at the bottom of the screen, right, then the screen will be scrolled up one line, and the cursor moves to the left side of the screen. If automatic mode is off, simply moves right one character.

void [ami_]scroll([FILE* f,] long x, long y);

Scroll in arbitrary directions. The update screen in surface file **f** is scrolled according to the differences in **x** and **y**. Uncovered areas on the screen appear in the current background color.

void [ami_]select([FILE *f,] long u, long d);

Select buffer to update and display. Selects the active buffer for update **u**, and display **d** in output surface file **f**. The update buffer will receive the result of all writes. The display buffer will be shown on screen.

terminal provides at least 10 screen buffers, numbered 1 to n.

void [ami_]settab([FILE* f,] long t);

Set new tab. Sets a new tab location at **t** in output surface file **f**.

void [ami_]sizbuf([FILE* f,] long x, long y);

Sets the size of the buffer used to draw into. **x** and **y** indicate the width and height, respectively, of the buffer surface in window **f**.

void [ami_]standout([FILE* f,] long e);

If **e** is 1, causes all further characters written to the screen to appear in standout text in output surface file **f**. Standout is assigned to the first mode possible from the following order:

Reverse.
Underline.
Bold
Italic.
Strikeout.
Blink.

If none of those modes are available, **standout** is a no-op. If **e** is 0, standout mode is turned off.

void [ami_]strikeout([FILE* f,] long e);

If **e** is 1, causes all further characters written to the screen to appear in strikeout text in output surface file **f**. If **e** is 0, strikeout mode is turned off.

void [ami_]subscript([FILE* f,] long e);

If **e** is 1, causes all further characters written to the screen to appear in subscript text in output surface file **f**. If **e** is 0, subscript mode is turned off.

void [ami_]superscript([FILE* f,] long e);

If **e** is 1, causes all further characters written to the screen to appear in superscript text in output surface file **f**. If **e** is 0, superscript mode is turned off.

void [ami_]timer([FILE* f,] long i, long t, long r);

Set timer active in output surface file **f**. The timer **i** will be set to run for time **t**. If the repeat flag **r** is set, then the timer will automatically repeat when the time expires. If the timer is already in use, then it will cease its current timing, and perform the new time.

void [ami_]title([FILE* f,] char *ts);

Sets the title of the window or terminal surface f to the string ts. On a hosted terminal, the host window or terminal title is set if the host allows it.

void [ami_]titlen([FILE* f,] char *ts, long n);

As title(), but takes exactly n characters of ts. The string need not be zero terminated.

void [ami_]underline([FILE* f,] long e);

If **e** is 1, causes all further characters written to the screen to appear in underlined text in output surface file **f**. If **e** is 0, underline mode is turned off.

void [ami_]up[(FILE* f)];

Move cursor up one line in output surface file **f**. If the cursor is already at the top line, and automatic mode is on, the screen will be scrolled down, and the cursor remains at the same position. If automatic mode is off, the cursor simply moves up one line.

void [ami_]wrtstr([FILE* f,] char *s);

Writes the string **s** directly to the output surface file **f**. No control character interpretation is done. This procedure is used to perform efficient writes to the display surface without per-character overhead.

It is an error to call this routine when **auto()** is enabled.

void [ami_]wrtstrn([FILE* f,] char *s, long n);

As wrtstr(), but writes exactly n characters of s. The string need not be zero terminated.

4.29 Events in terminal

For each item, both the event structure section and the virtual function are presented. See the description of the event record (4.10 “Advanced Input” or 4.25 “Advanced Input in C++”) for the format of the entire record. For events, the parameters are fields of the structure. For overrides, they are formal parameters.

Event: [ami_]etchar
Override: long evchar(char echar);

Returns a keyboard character **echar**.

Event: [ami_]ettim
Override: long evtim(long timnum);

Indicates the timer according to the timer handle **timnum** has expired.

Event: [ami_]etmoumov
Override: long evmoumov(long mmoun, long moupx, long moupy);

The mouse with handle **mmoun** has moved, to the position indicated by **moupx** and **moupy**.

Event: [ami_]etmouba
Override: long evmouba(long amoun, long amoubn);

The mouse with handle **amoun** asserted the button **amoubn**.

Event: [ami_]etmoubd
Override: long evmoubd(long dmoun, long dmoubn);

The mouse with handle **dmoun** deasserted the button **dmoubn**.

Event: [ami_]etjoyba
Override: long evjoyba(long ajoyn, long ajoybn);

The joystick with handle **ajoyn** asserted the button **ajoybn**.

Event: [ami_]etjoybd
Override: long evjoybd(long djoyn, long djoybn);

The joystick with handle **djoyn** deasserted the button **djoybn**.

Event: [ami_]etjoymov
Override: long evjoymov(long mjoyn, long joypx, long joypy, long joypz);

The joystick with handle **mjoyn** moved, and the coordinates **joypx**, **joypy** and **joypz**. The values of each axis are between –**LONG_MAX**..**LONG_MAX**. The number of axis actually present in the given joystick are given by the function **joyaxis()**. The value returned by an unimplemented axis is undefined.

Event: [ami_]etfun
Override: long evfun(long fkey);

A function key was sent from the keyboard, with **fkey** giving the number of the key.

Event: [ami_]etup
Override: long evup(void);

The key for move cursor up was sent from the keyboard.

Event: [ami_]etdown
Override: long evdown(void);

The key for move cursor down was sent from the keyboard.

Event: [ami_]etleft
Override: long evleft(void);

The key for move cursor left was sent from the keyboard.

Event: [ami_]etright
Override: long evright(void);

The key for move cursor right was sent from the keyboard.

Event: [ami_]etleftw
Override: long evleftw(void);

The key for move cursor left word was sent from the keyboard. This indicates the cursor should be moved left one “word”, or over any series of non-space characters.

Event: [ami_]etrightw
Override: long evrightw(void);

The key for move cursor right word was sent from the keyboard. This indicates the cursor should be moved right one “word”, or over any series of non-space characters.

Event: [ami_]ethome
Override: long evhome(void);

The key for move cursor to the home position in the document (top extreme left) was sent from the keyboard.

Event: [ami_]ethomes
Override: long evhomes(void);

The key for move cursor to the home position in the screen (top extreme left) was sent from the keyboard.

Event: [ami_]ethomel
Override: long evhomel(void);

The key for move cursor to the home position in the line (extreme left) was sent from the keyboard.

Event: [ami_]etend
Override: long evend(void);

The key for move cursor to the end position in the document (bottom extreme right) was sent from the keyboard.

Event: [ami_]etends
Override: long evends(void);

The key for move cursor to the end position in the screen (bottom extreme right) was sent from the keyboard.

Event: [ami_]etendl
Override: long evendl(void);

The key for move cursor to the end position in the line (extreme right) was sent from the keyboard.

Event: [ami_]etscrl
Override: long evscrl(void);

The key for scroll screen left one character was sent from the keyboard.

Event: [ami_]etscrr
Override: long evscrr(void);

The key for scroll screen right one character was sent from the keyboard.

Event: [ami_]etscru
Override: long evscru(void);

The key for scroll screen up one character was sent from the keyboard.

Event: [ami_]etscrd
Override: long evscrd(void);

The key for scroll screen down one character was sent from the keyboard.

Event: [ami_]etpagd
Override: long evpagd(void);

The key for page down was sent from the keyboard.

Event: [ami_]etpagu
Override: long evpagu(void);

The key for page up was sent from the keyboard.

Event: [ami_]ettab
Override: long evtab(void);

The key for enter tab was sent from the keyboard.

Event: [ami_]etenter
Override: long eventer(void);

The key for enter line was sent from the keyboard.

Event: [ami_]etinsert
Override: long evinsert(void);

The key for insert block was sent from the keyboard.

Event: [ami_]etinsertl
Override: long evinsertl(void);

The key for insert line was sent from the keyboard.

Event: [ami_]etinsertt
Override: long evinsertt(void);

The key for insert toggle was sent from the keyboard. The action should be to toggle the state of the insert/overwrite flag used to determine if new typed text overwrites previous text on screen, or inserts new text between existing characters.

Event: [ami_]etdel
Override: long evdel(void);

The key for delete block was sent from the keyboard.

Event: [ami_]etdell
Override: long evdell(void);

The key for delete line was sent from the keyboard.

Event: [ami_]etdelcf
Override: long evdelcf(void);

The key for delete character forward was sent from the keyboard. This indicates the character to the right of the cursor should be deleted.

Event: [ami_]etdelcb
Override: long evdelcb(void);

The key for delete character backward was sent from the keyboard. This indicates the character to the left of the cursor should be deleted.

Event: [ami_]etcopy
Override: long evcopy(void);

The key for copy block was sent from the keyboard. This indicates the currently selected block should be copied.

Event: [ami_]etcopyl
Override: long evcopyl(void);

The key for copy line was sent from the keyboard. This indicates the current line should be copied.

Event: [ami_]etcan
Override: long evcan(void);

The key for cancel current operation was sent from the keyboard. This indicates the operation in progress should be canceled.

Event: [ami_]etstop
Override: long evstop(void);

The key for stop current operation was sent from the keyboard. This indicates the operation in progress should be stopped.

Event: [ami_]etcont
Override: long evcont(void);

The key for continue current operation was sent from the keyboard. This indicates the operation in progress should be continued if stopped previously.

Event: [ami_]etprint
Override: long evprint(void);

The key for print current document was sent from the keyboard. This indicates the current document should be printed in entirety.

Event: [ami_]etprintb
Override: long evprintb(void);

The key for print current block was sent from the keyboard. This indicates the current selected bock, if it exists, should be printed.

Event: [ami_]etprints
Override: long evprints(void);

The key for print current screen was sent from the keyboard. This indicates the current screen should be printed.

Event: [ami_]etmenu
Override: long evmenu(void);

The key for display menu was sent from the keyboard. This indicates the menu, if any, should be displayed.

Event: [ami_]etresize
Override: long evresize(long rszx, long rszy);

The window for terminal has resized. The new width is **rszx**, and the new height is **rszy**. The **etresize** event will only be sent if terminal is in a windowed system. Since terminal is a buffered model, it can be ignored, since the drawing surface, and the geometry parameters **maxx()** and **maxy()** will not be affected. If the client chooses to resize to follow the window resize, **sizbuf()** is used to change the geometry.

Event: [ami_]etfocus
Override: long evfocus(void);

The window hosting terminal gained the input focus; see 4.13 “Focus events”. Sent only on a windowed system, when the focus state changes.

Event: [ami_]etnofocus
Override: long evnofocus(void);

The window hosting terminal lost the input focus. Sent only on a windowed system, when the focus state changes.

Event: [ami_]ethover
Override: long evhover(void);

The mouse moved onto the window hosting terminal; see 4.14 “Hover events”. Sent only on a windowed system, when the hover state changes.

Event: [ami_]etnohover
Override: long evnohover(void);

The mouse moved off the window hosting terminal. Sent only on a windowed system, when the hover state changes.

Event: [ami_]etterm
Override: long evterm(void);

The key for terminate program was sent from the keyboard. This indicates the program should be exited.

If this event is received, then the user ordered the exit. This means that automatic hold mode, if enabled, will be bypassed, and the program closed immediately.

Event: [ami_]etframe
Override: long evframe(void);

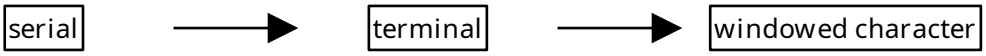
The frame timer matured; see 4.19 “The Frame Timer”. Sent while the frame timer is enabled, approximately at each refresh of the display.

5 Character Windows Management Library

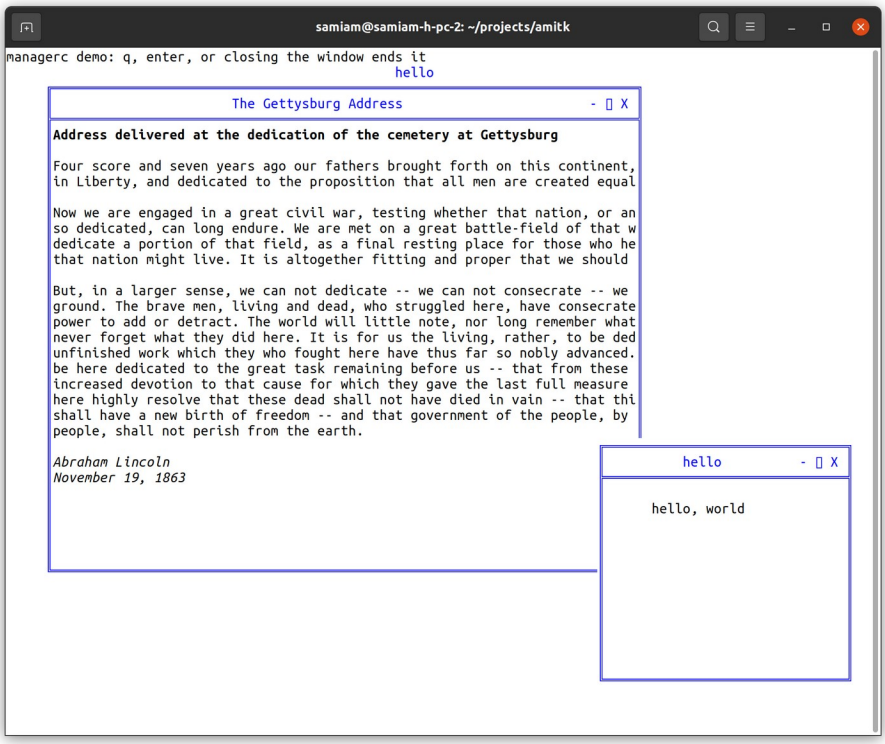


windows on a character system is completely upward compatible with terminal. Given a single fixed screen, **windows** subdivides the screen into virtual windows, which can be set to any size or position. A **terminal** compatible program sees its window as a "virtual screen", and does not know that some or all of the contents of that screen may be hidden. A **windows** aware program can participate in the benefits of a windowed environment.

This chapter describes that management level as a character system presents it: the interface of terminalw.h, which is the terminal interface with the window management calls added. A program that needs only the terminal surface includes terminal.h and does not carry the manager; a windowed character program includes terminalw.h and links the manager carrying library. The combined character and graphical form of this level appears in the windows management chapter.



Promotion of programs: a serial program runs under terminal, and a terminal program runs windowed.



Character based windows.

5.1 Screen Appearance

The idea of windows management is to take a program that thinks it is talking to an ordinary terminal, and allow the presentation of multiple such windows within a single screen.

By default, **windows** emulates a **terminal** interface for each window that is identical to the behavior of a full screen window from the program's point of view. A standard size screen is implemented for the program of 80 characters by 25 lines (even if the real display has a different size). **windows** accepts program writes, and places that information in a buffer that looks like an ideal screen. Then, the contents of that buffer are placed on the screen in various arrangements to complete the desktop for the user.

5.2 Rooted vs. non-rooted windows surfaces

Terminal windows can appear in either a fixed display surface or as part of a managed window system. An example of the former would be an application that runs alone with a window manager on a stand-alone console display. An example of the latter is within a fully windows driven display with multiple programs running. A rooted display means the program window runs full screen without a window frame and cannot be resized or minimized. Any child windows or dialogs appear as windows within that rooted display surface.

On a non-rooted display, parentless windows appear as peer windows on the home desktop, as do dialogs.

5.3 Window Modes

A window can be any actual size on the display. A window can also be off the display entirely, so that it accepts changes to its buffer, but does not display those changes. In addition, a window can be active, inactive, or overlapped, or even completely covered by other windows on the display.

5.4 Buffered Mode

A window comes up by default in the "buffered" mode. In buffered mode, the program has a "virtual display surface" that it writes to. **windows** maintains this surface in memory, and maintains the actual onscreen window as a scrolled window on that. The program is unconcerned with what part of its screen is being displayed, or even if it is displayed at all. The user operates the window using the frame controls.

Buffered mode is designed to allow the program to be unconcerned with the management of the display. However, the program can set the size of the virtual display using **sizbuf(f, x, y)**. When a buffer is resized, its contents are cleared to the current background color.

The buffer is a display surface that the buffer handler keeps for the program. The buffered mode allows a program to "think" it has a window of size N, but the appearance of the window on the actual display is a window on that virtual display that is maintained by the buffer handler.

Because from the program's point of view the window size does not change, and is always a complete window, the resize events are performed transparently to the program. Similarly, the minimize and maximize events are handled for the program (see 5.5 “Unbuffered Mode”). Discovery is when a window or part of a window is uncovered on the display.

When in buffered mode, the window manager may use scroll bars to display within a screen view that is smaller than the buffer.

A non-rooted window shows them when its buffer is larger than its client area: a bar on the right for a taller buffer, and one along the bottom for a wider one. The bars are the window manager’s own, and the user scrolls the view over the buffer with them; the program receives no events from them. They can be turned off for a window with **autobuf(f, e)**, and off for every window by the option scrollbuffer under windowc in the configuration.

Although buffered mode automatically handles window status events (covered in 5.5 “Unbuffered Mode”), these events are still sent through. For maximum compatibility, the using program should ignore any events not relevant to the mode it is in.

5.5 Unbuffered Mode

The program can leave buffered mode by using **buffer(f, b)**. Unbuffered mode has no display buffer, and no default actions for events. Instead, the onscreen appearance of the window is set dynamically by the program.

When unbuffered mode is entered, the buffer for the window is discarded, and it becomes entirely up to the program to manage its own window by watching events. The size of the window no longer reflects the buffer, but instead is the size of the onscreen window. In contrast to buffered mode, this can change many times as the window is resized under user control.

Buffered mode can be reentered at any time.

When running unbuffered, it is assumed that the program will manage the update of the on-screen display surface itself. When running unbuffered, the program handles events that are normally handled automatically.

The etredraw event indicates that some part of the window needs to be redrawn on the screen. At the character level the event carries no update rectangle: the program redraws the client area.

The **etresize** event indicates that the onscreen window has changed its size. Its purpose is to let the program update any calculations based on the size of the window. This event can be ignored, used to update stored **maxx and maxy** values, or it could even cause the entire arrangement of the screen to be reformed.

etresize is not normally used to redraw areas of the screen. If a resize event causes new screen area to be uncovered, then one or more **etredraw** events will also be sent. There is no way to determine exactly which redraw operations correspond to the resize event, so some redundancy is possible with applications that repack their window's contents on a resize.

The **etmin** event indicates that the window has been minimized. When a window is minimized, it will not be displayed, and won't receive **etredraw** events. On the desktop, the window is represented by a small icon. An alternative form of minimization is for the window to receive an **etmin** message, followed by an etresize message, and continued **etredraw** messages. This is a hint for the window to display vital information in a much smaller format that fits into the minimized icon.

The **etmax** event indicates that the window has been maximized, or covers the entire desktop. If the window needs to be updated, this event may be accompanied by **etresize** and **etredraw** events.

The **etnorm** event indicates a normal window mode has been resumed from **etmin** or **etmax**.

In many modern computers, the screen buffers are in fast video memory and managed by graphics hardware. Thus it can be more efficient for the underlying system to process all writes to an offscreen buffer and then copy back to the user screen. Thus the unbuffered mode may in fact have no real effect. The main difference for the client program is that **etredraw** events are rarely, if ever sent, since the system has a copy of the onscreen contents.

5.6 Buffer Follow Mode

windows has an intermediate mode between buffered and unbuffered mode. With faster computers and more memory, programs often retain the screen buffers and simply resize them as the window is resized. To allow for this mode, the **etresize** message also contains rszx and rszy parameters that have the new size of the window. These parameters are normally ignored. For buffer follow mode, the rszx and rszy parameters are used to resize the buffer using **sizbuf(f, x, y)**. The new buffer sizes are reflected in **maxx(f) and maxy(f)** and the program can continue to use the screen buffers as before. Note that the **sizbuf(f,x,y)** call resizes all of the screen buffers and clears them to the background color.

Buffer mode gives client programs the ability to treat the screen as a virtual screen whose geometry does not change. The **etresize** message, which is normally ignored in buffered mode, gives the status of what the actual display window is doing, and the client program has the option of reconfiguring the buffer to “follow” what the display window is doing.

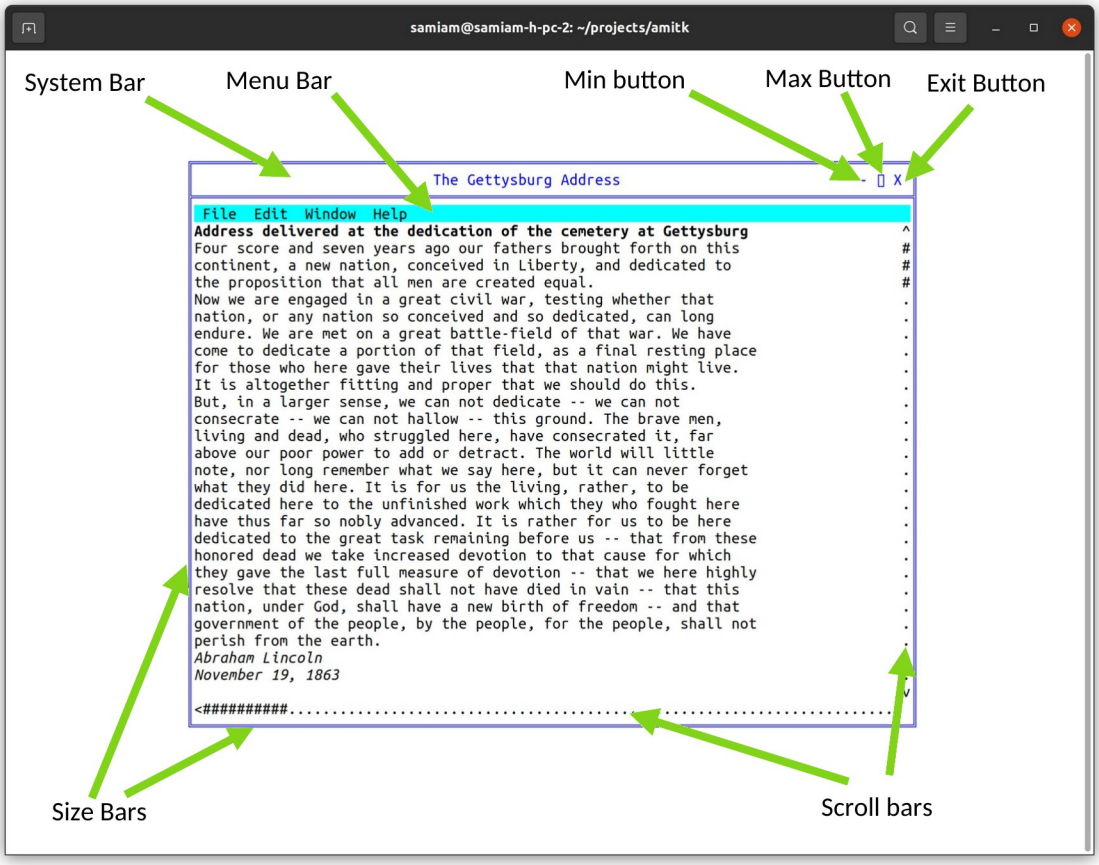
5.7 Delayed Window Display

When a window is created, it is not displayed until an actual write operation is made to it. This allows the window to be changed in configuration by the program before it is displayed, and prevents the window being rapidly changed on screen as that happens. For example, the program might want to take the window out of buffered mode, change its size, remove the frame, or add menus.

5.8 Window Frames

Under most window systems, each window has a "window frame", which has various window management functions. These include:

- Move bars to resize and move the window.
- A title bar that indicates what program is running in the window.
- A "minimize" button.
- A "maximize" button.
- A "menu" bar.



A window has several system components.

When a window is in buffered mode, none of the frame features are under the control of the program, but instead are managed by the buffering software.

The appearance, or even the existence, of any of these items can be customized by the program. The frame can be enabled or disabled by **frame(f, e),** where **f** is the window file, and **e** is true to enable the frame. The title can be set by title(f, ts). The title, minimize and maximize buttons are considered part of the "system bar", which can be enabled or disabled by **sysbar(f, e)**. The resize bars are enabled or disabled by **sizable(f, e)**.

The frame control functions **frame**, **title**, **sysbar** and **sizable** are optional within an implementation. The system may not allow an application to, for example, disable its sizing bars, or it may not have a set of system controls on the window. The program must be ready for these commands not to have an effect. For example, after the **sizable** function is set false, it should still be ready to receive a new size. If, for example, the size of the graphic to be presented is a fixed size, it would be filled out or clipped to fit the resulting window.

5.9 Multiple Windows

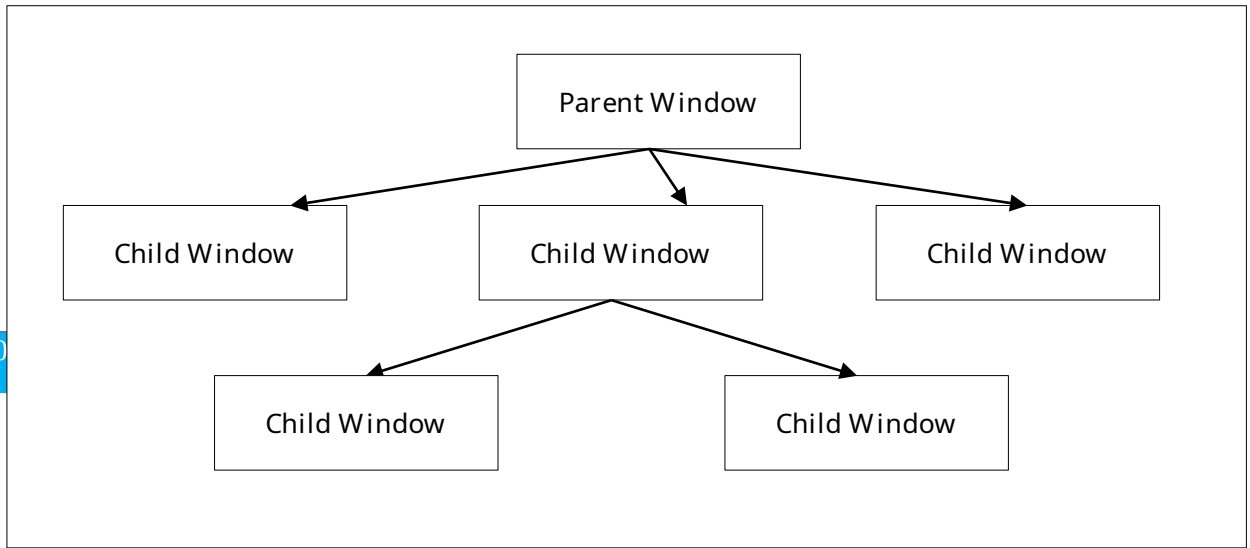
The **stdout** file is used as the output and the **stdin** file as input to the main window by default. A window can also be explicitly opened by **openwin(inw,outw,par,id)**. When a new window is opened, it is placed on the desktop with the other windows. The files **inw** and **outw** specify the input and output files attached to the window. The input file receives all user input and events from the window, and all of the write and other drawing operations are sent to the output file. The **id** is a number, from 1 to N, that specifies the logical window. The logical window number 1 is reserved for the **stdin** and **stdout** pair. The **par** parameter is NULL for a parentless window. If a window is parentless and the main window is rooted, it is placed alongside the child windows.

The output side of a window must always be unique, but the input side can be shared. Multiple windows can be attached to a single input file, and that input file will receive all of the input and events from all attached windows. The logical window number is returned with each event, so that the source of the input or event can be determined. This forms the basis of “class window handling” (5.13 “Class Window Handling”), or using one input handler to handle multiple windows.

Windows can be closed using the standard ANSI C **fclose()** call. Only the output side of a window is used to close that window. The input side is automatically closed when there is no longer a window that references it.

5.10 Parent/Child Windows

Windows systems are tree structured, and each window is a child of a parent window, with the exception of a "root" or master window, which is usually the window that contains the entire desktop. The methods introduced create windows that live side by side on the desktop, and have the desktop as their parents. However, it is possible to nest windows within each other.



A window is opened as a child by **openwin(inw, outw, par, id)**, which is the same **openwin()** call with a parent specified. The parent file **par** is the text file that is attached to the output window of the parent. To be a child window means to move as one within it. It maintains its position within the parent, even as the parent itself moves. It always is in front of the Z ordering for the parent. It is minimized, maximized and closed with the parent.

All windows have their position given in relative terms to the parent's client area origin. Any position operation is also relative to the parent.

The common use of the parent is to create "sibling" windows, or windows equivalent to the default program window in status, that are positioned independently within the parent. A program starts as the child of an external parent window. This could be an actual program, such as a program acting as a manager, or it could be the desktop root. There is no difference between these for the program.

Programs do not have direct access to the parent window in which the program was created. That parent window is usually the desktop.

5.11 Moving and Sizing Windows

Moving a window is done with the **setpos(f, x, y)** procedure. The position is given in parent coordinates. When a window is on the desktop, it is necessary to find the size of the desktop, which is found with **scnsiz(f, x, y)**. On a character system the desktop is measured in the same character cells as the windows on it.

Sizing a window is done with **setsiz(f, x, y)** procedure. This sets the size of the entire window, and so does not indicate the resulting size of its client area. To find out what window size is needed for a given client area, before actually sizing the window, the function **winclient(f, cx, cy, wx, wy, ms)** is used. It returns the window size needed to contain that client. winclient takes a set of the modes of the window to enable it to make the size determination.

The mode set declaration is:

```
/* windows mode sets */
typedef enum {

    ami_wmframe, /* frame on/off */
    ami_wmsize,  /* size bars on/off */
    ami_wmsysbar /* system bar on/off */

} ami_winmod;
typedef long ami_winmodset;
```

To find the current size of a window, in parent terms, the procedure **getsiz(f, x, y)** is used. This procedure is useful when you need to find the size of a window on the desktop.

5.12 Z Ordering

The Z ordering, or back to front ordering of windows, is determined by default to be newest created windows to the front, oldest to the back. This order can be modified by the users when they select windows towards the back to come to the front. The program can also reorder windows at will by sending them to the front or back. Note that the Z ordering is always relative within a parent.

To send a window to the front, the call **front(f)** is used. To send a window to the back, the call **back(f)** is used. To achieve a specific order within a set of windows, they should be sent to the front in the opposite order from which they are to appear, i.e., the backmost first, and the frontmost last. Alternately, the windows can be sent to the back in the order they appear.

There are other reasons besides reordering windows that these functions might be useful. When an error is encountered, it is common to send a window containing the error message to the front of the parent Z order, and to send the entire window to the front. Placing an active display or a background pattern in a frameless maximized window can create wallpaper. Placing the same window in the back can be a screen saver.

5.13 Class Window Handling

windows handles windows as a set of two files, the input and output. However, these don't need to always be a set. It is possible to reuse an already open input file as the input side of any new window. This is possible because the window id supplied when the window is opened is passed with all messages to the event procedure.

Allowing a single program thread to handle multiple open windows allows the creation of multiple window programs without the need to create a multitask program. For each message, the id is examined, and the action performed on the appropriate output file window.

Because the output file is unique, it is always the handle used to refer to the window in management calls.

5.14 Parallel Windows

Parallel tasking is a natural match to a multiwindowing system. With a task created to operate each window, the program is simplified, and user response is generally better.

To increase the responsiveness of user interfaces, the recommended program design for windowing programs is to create two tasks for each window, the "foreground" and the "background" tasks. The foreground task performs the event loop and the redraw, resize, move and other user interface tasks. The background task would handle computation tasks and other tasks related to performing actions or changing the drawing surface. The foreground task determines if an action that requires the background task is required, then sends a command to the background task via a queue or other IPC (InterProcess Communication).

The reason for this construction is that the things the user sees, such as keeping the client area redrawn, or obeying a move, resize, minimize or similar command always have a task waiting to perform them. This means that elementary user desktop management appears responsive to the user, while data manipulation and computation tasks simply delay client area updates. Users will be much more willing to tolerate client area slowdowns than having a window apparently lock stubbornly in position. Ideally, the client area should also have an indication that computation is taking place. The most modern method for this is a progress indicator that gives estimates as to when a task will complete, if the task will take longer than about 1 second.

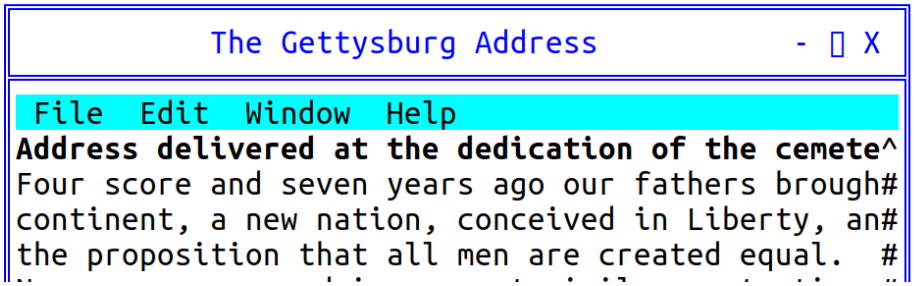
If the work performed in a client area is trivial, only a foreground task need be provided. But be aware that it is very easy to slip into a mode where essential updates are held off. Calling a file procedure, waiting on a timer, or other simple task compromises the response time for window management. Better

to leave the window in buffered mode, or structure with foreground/background from program creation than to have to add it later.

5.15 Menus

The menu is a series of buttons labeled with text for user action. The buttons can either have a direct effect on the program, or they can activate a series of submenus in a tree structure.

The exact location of a menu is system dependent. It may or may not affect the client area.



A menu on a window.

A menu is described to **windows** by constructing a data structure:

```
/* menu */
typedef struct ami_menurec* ami_menuptr;
typedef struct ami_menurec {

        ami_menuptr next;   /* next menu item in list */
        ami_menuptr branch; /* menu branch */
        long        onoff;  /* on/off highlight */
        long        oneof;  /* "one of" highlight */
        long        bar;    /* place bar under */
        long        id;     /* id of menu item */
        char*      face;   /* text to place on button */

} ami_menurec;
```

The menu is activated by **menu(f, m)** where **m** is the data structure for the menu.

Menu buttons are highlighted when pointed to. Additionally, a button can have "on/off" highlighting. In this mode, a state is kept for the button that is flipped on or off as the button is pressed. The highlighting for on and off states is different. The data structure that defines a menu is not used or updated after it is used to create a menu.

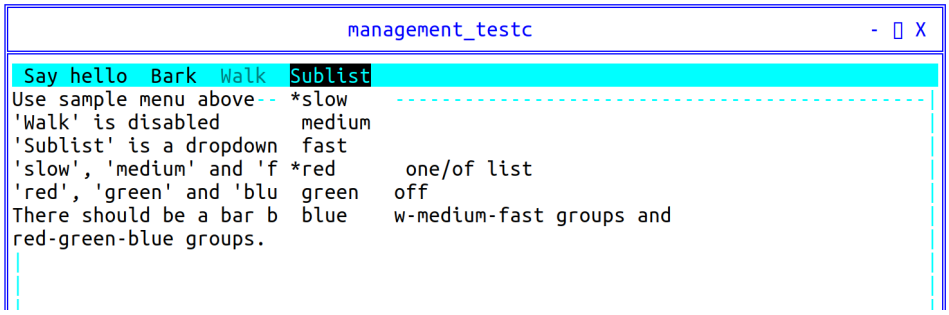
The **onoff** field true selects on/off highlighting. After the menu is activated, the state of the button can be changed with **menusel(f, id, e)**. The button state is not automatically changed by a user menu select. The program must specifically change the state of the button.

Another highlight mode is "one of" or "checklist" highlighting. In this case, a group of related buttons are joined by setting the **oneof** flag on each item in the list but the last. The highlighting for a button that is selected is different from one that is not, and only one item will be active in the list.

Like on/off buttons, user selection does not automatically change the state of the buttons in the list. It must be specifically changed by a **menusel()** call.

Typically, neither on/off nor one/of switching is available for top level (horizontal) menus, and the program should not count on them.

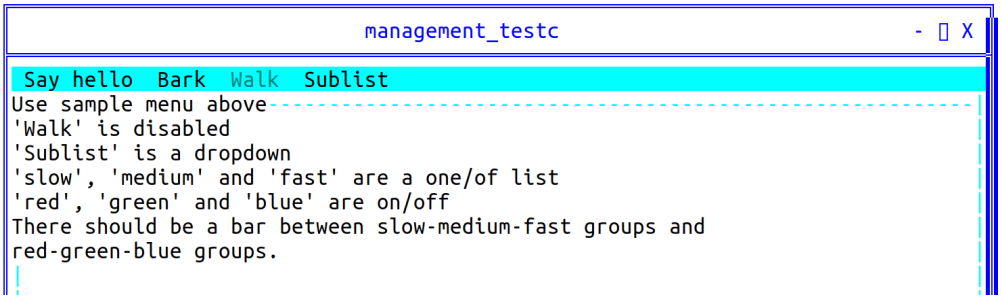
Menu items can be grouped in a vertical list by setting the **bar** field active. This causes a horizontal bar to be drawn under the menu item. Vertical lists are described next in "menu sublisting".



Selected menu items “slow” and “red”.

A button can be enabled or disabled. A disabled button is highlighted specially, typically as greyed out. It indicates a button that is entirely inactive, because that function is not available. This is done to allow the same menu to be used regardless of the availability of the functions on the menu. Also, some functions come and go. For example, a "save" button (for "save file") might only be active if the file has changed, and needs to be saved.

The enable status can be changed after the menu is activated by **menuena()**.



The “walk” menu item is disabled.

Each button in a menu has a numeric **id**. This is used by several functions to change the state of buttons. It is an error to have two buttons with the same id.

The **face** text string indicates what text will appear on the face of the button. The system will automatically size the buttons to fit the largest text of a button, so care should be taken not to have a single button's text so long as to cause a lot of white space in the menu.

5.16 Setting Menu Active

A menu is constructed by the program as a list using the menu data structure, then set active by calling **menu()**, which can also turn the menu back off.

When a menu data structure is activated, all of the fields are copied and placed into an internal form. After the activation, the menu data has no function, and changing its fields will have no effect. The menu data structure can be recycled immediately after the **menu()** call.

5.17 Setting Menu States

The check state of a button is set with menusel(f, id, e) after the menu is active. For a button in a one of group, setting it also clears the other members of the group. Enabling and disabling a button of any kind, greyed out when disabled, is done with menuena(f, id, e).

5.18 Standard Menus

Besides presenting a menu in the method standard for the implementation, it is also likely that there exists a standard arrangement of commonly used buttons in the menu. **windows** defines a series of such standard buttons.

```
/* standardized menu entries */
#define AMI_SMNEW         1 /* new file */
#define AMI_SMOPEN        2 /* open file */
#define AMI_SMCLOSE       3 /* close file */
#define AMI_SMSAVE        4 /* save file */
#define AMI_SMSAVEAS      5 /* save file as name */
#define AMI_SMPAGESET     6 /* page setup */
#define AMI_SMPRINT       7 /* print */
#define AMI_SMEXIT        8 /* exit program */
#define AMI_SMUNDO        9 /* undo edit */
#define AMI_SMCUT        10 /* cut selection */
#define AMI_SMPASTE      11 /* paste selection */
#define AMI_SMDELETE     12 /* delete selection */
#define AMI_SMFIND       13 /* find text */
#define AMI_SMFINDNEXT   14 /* find next */
#define AMI_SMREPLACE    15 /* replace text */
#define AMI_SMGOTO       16 /* goto line */
#define AMI_SMSELECTALL  17 /* select all text */
#define AMI_SMNEWWINDOW  18 /* new window */
#define AMI_SMTILEHORIZ  19 /* tile child windows horizontally */
#define AMI_SMTILEVERT   20 /* tile child windows vertically */
#define AMI_SMCASCADE    21 /* cascade windows */
#define AMI_SMCLOSEALL   22 /* close all windows */
#define AMI_SMHELPTOPIC  23 /* help topics */
#define AMI_SMABOUT      24 /* about this program */
#define AMI_SMMAX        24 /* maximum defined standard menu entries
*/

/* standard menu selector */
typedef long ami_stdmenusel;
```

Standard menu button definitions should be used whenever possible. To create a menu using standard menu buttons, the procedure **stdmenu()** is used. The arrangement of the menu is chosen to match the implementation, and the non-standard buttons provided are integrated into the menu in the normal place for the implementation. When standard buttons are used, the button ids are set to the values shown above, so these values should not be used by the program.

Note that **stdmenu()** simply constructs a menu, it does not set it active.

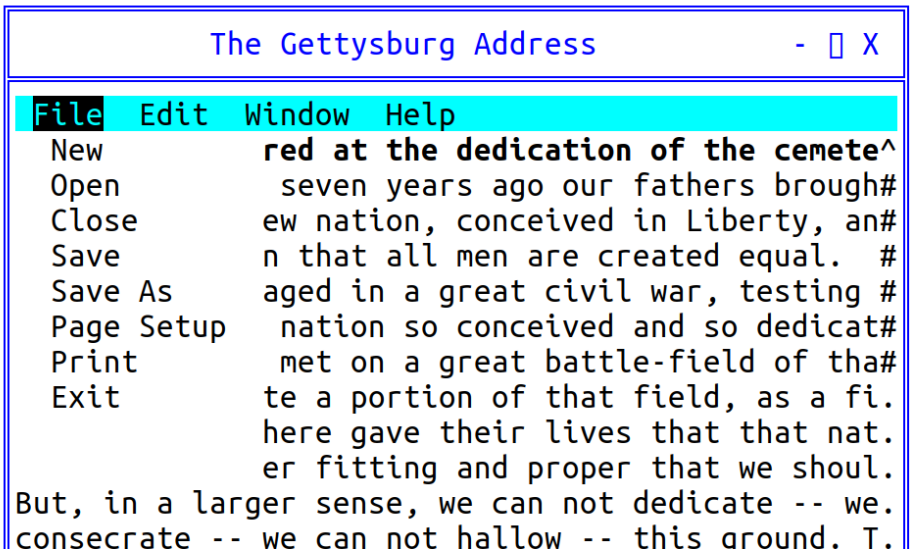
5.19 Menu Sublisting

When there is not enough room to represent the entire menu on a window, or anytime the size of a menu must be compressed, the tree structure of menus can be used with "pulldown" menus. A pulldown menu is a vertical menu list that appears under the selected button. This is done by placing a submenu in the **branch** pointer of the menu structure. The branch defines the pulldown menu items. The branch menu item is specially indicated to show that it is a branch, and not an immediate button, usually with an arrow pointing towards where the branch pulldown will appear. Any number of levels of pulldown

menus can be created. The levels under the topmost branch are generally placed to the right of the branch button.

If a branch pulldown cannot be placed where it should, due to the proximity of the button to the edge of the display, it instead goes back in the other direction, up or down, or both, until space is found for it. If the entire pulldown cannot be displayed, it is truncated. This would only happen if the pulldown was too large for the display.

When a branch button is pressed, no event is generated for it. The button is strictly used to activate the pulldown.



A pulldown menu.

5.20 Advanced Windowing

The features of a window besides the client area are designed to be the minimum features implemented across platforms. To implement more advanced windows appearances with custom menus, controls and status lines, the standard method is to turn off frames and menus for child windows, and use them as building blocks for more advanced client areas with multiple subwindows and features.

It is recommended that at least one menu appear for a program, even if the functions duplicate some of the advanced controls provided. The reason for this is that the menu has different presentation methods in different systems, so providing the standard "top" menu will help programs' portability.

5.21 Events

New event types are added for the management mode. As usual, events beyond the definition here should be ignored.

The event type and the event record are those of terminal (4.10 “Advanced Input”); the codes and fields for the management level are already declared there. The events of the management level are:

```
/** window redraw */ ami_etredraw,
/** window minimized */ ami_etmin,
```

```
/** window maximized */ ami_etmax,
/** window normalized */ ami_etnorm,
/** menu item selected */ ami_etmenus,
```

The etresize event carries the new window size in the rszx and rszy fields of the record, and etmenus carries the selected menu item in menuid. The widget codes that follow these in the declaration belong to the widget level, described in the widgets chapter.

5.22 C++ windows interface

The window object of the C++ interface belongs to the graphical library, where every window and widget is an object and events route to the window they name. At the character level the management calls are used from C++ as they are from C, with the ami_ prefix; the terminal namespace covers the terminal surface only.

5.23 Procedures and Functions in windows character mode

void [ami_]autobuf(FILE* f, long e);

Enables or disables the automatic scroll bars for a buffered window **f**. If **e** is true, the default, the window shows scroll bars when its buffer is larger than its client area, a vertical bar for a taller buffer and a horizontal one for a wider buffer, and the user scrolls the view over the buffer with them. If **e** is false, the window never shows them. The bars belong to the window manager, and the program receives no events from them. A program that keeps its buffer the size of its client area has no use for the bars, and turns them off so that they do not appear for a moment while the window shrinks. The option scrollbuffer under windowc in the configuration turns the bars off for every window.

void [ami_]back(FILE* f);

Sends the window **f** to the back of the parent Z order.

void [ami_]buffer(FILE* f, long e);

Engages or removes window **f** from buffered mode, according to the boolean **e**. In buffered mode, all of the drawing for a window is performed on a memory buffer, then copied to the screen. The screen view of the buffer can be all or part of the buffer, and multiple buffers can be managed.

If the buffer mode is enabled, the size of the buffer is set to the previous size.

void [ami_]frame(FILE* f, long e);

Enables or disables the appearance of the frame in window **f**, according to Boolean **e**, which includes the minimize, maximize, title, size, move and close controls. If the frame is removed, the user will be unable to operate the frame controls.

void [ami_]front(FILE* f);

Sends the window **f** to the front of the parent Z order.

void [ami_]getsiz(FILE* f, long* x, long* y);

Finds the size of a window in parent coordinate terms for window **f**, to size **x** and **y**. The size is in characters.

void [ami_]menu(FILE* f, [ami_]menuptr m);

Sets up the menu bar for window **f** from the given list **m**, which contains a menu bar definition structure.

If the menu pointer is NULL, then the menu is removed.

The menu data structure is copied during the call, so the menu structure can be reused or freed.

void [ami_]menuena(FILE* f, long id, long onoff);

Enables or disables an on/off menu button for window **f**. **id** refers to a button **id** that was specified in the menu data structure. If onoff is true, the button is enabled, otherwise disabled. The highlighting of the button will change to match.

void [ami_]menusel(FILE* f, long id, long select);

Sets the check state of a menu button in window f. id refers to a button id that was specified in the menu data structure. If select is true, the button is checked, otherwise unchecked. For a button in a one of group, checking it clears the other members of the group. The highlighting of the buttons will change to match.

void [ami_]openwin(FILE** infile, FILE** outfile, FILE* parent, long wid);

Opens a new window. The input file **infile** will get messages pertaining to the window, and the output file **outfile** will be used to draw to it. The input file may already be open elsewhere, in which case it will get all messages for all open windows opened with it. If a previous **infile** is not used, it must be NULL. The window will be placed at 1,1 within the parent and held out of display. The output file is always used as the window handle, since it is unique to the window.

The window identifier **id** is a number from 1 to n that is returned in messages concerning the window.

Only the output side of a window pair is used in procedures or functions that operate on the window once opened with **openwin()**.

A window is closed with the standard ANSI C fclose() procedure, used on the output file for the window. The input side of the window is automatically closed when there are no longer any windows that reference it.

If the optional parent window is specified, the new window is created as a child of the given parent, otherwise it must be NULL. This means that it will always be displayed within the parent, and be clipped to it.

void [ami_]scnsiz(FILE* f, long* x, long* y);

Finds the size of the user screen or desktop for window **f** to the size **x** and **y**. The size is in characters.

void [ami_]setpos(FILE* f, long x, long y);

Sets the position, within the parent, of a child window **f**, using position **x** and **y**. If the window is on the desktop, then the window position is relative to the desktop. The position is in characters.

The position is set in terms of the parent's coordinates.

void [ami_]setsiz(FILE* f, long x, long y);

Sets the size of window **f** in parent coordinate terms, to size **x** and **y**. The size is in characters.

void [ami_]sizable(FILE* f, long e);

Enables or disables the sizing bars for a window **f**. If **e** is true, the sizing bar is enabled, otherwise disabled. If the frame is not enabled, then the size bars will not appear regardless of the sizable status, but it will be recorded. If the size bars are removed, the user will be unable to resize the window.

void [ami_]sizbuf(FILE* f, long x, long y);

Sets the size of the buffer used to draw into. **x** and **y** indicate the width and height, respectively, of the buffer surface in window **f**. It is an error if buffering is not enabled. The buffer size is in characters.

void [ami_]stdmenu([ami_]stdmenusel sms, [ami_]menuptr* sm, [ami_]menuptr pm);

Constructs a standard menu and returns that in **sm**. **sms** contains the set of desired standard buttons. **pm** contains a menu containing non-standard buttons to be added to the menu. The menu is constructed using the desired standard buttons, and the non-standard buttons placed into the menu at a standard location.

If **pm** contains no menu items, it should be NULL.

void [ami_]sysbar(FILE* f, long e);

Enables or disables the system control bar for a window **f**. If **e** is true, the system bar is enabled, otherwise disabled. The system bar normally includes the title, minimum, maximum, and other control buttons. If the frame is not enabled, then the system bar will not appear regardless of the **sysbar()** status, but the size of it will be recorded.

void [ami_]title(FILE* f, char* ts);

Sets the title of the window **f** to the string **ts**. If the title is too long for the current window size, an implementation defined method will be used to make it fit, for example, it is clipped.

void [ami_]winclient(FILE* f, long cx, long cy, long* wx, long* wy, [ami_]winmodset ms);

Determines the window size needed for a given client size within window **f**. Given a desired client size of **cx** and **cy**, in width and height, the necessary window size to achieve that will be returned in wx and wy. The measurements are in characters.

The set of modes **ms** is used to determine the needed window size.

5.24 Events In windows character mode

See the description of the event record (5.21”Events”) for the format of the event record.

Event: [ami_]etresize

The window was resized. The new size in parent terms can be found with **getsiz(f, x, y)**.

Event: [ami_]etredraw

Signifies that the window should be redrawn. At the character level the event carries no update rectangle: the redraw is satisfied by redrawing the window client area.

Event: [ami_]etmin

Signifies that the window was minimized. This may not need any action, but is simply for information.

Event: [ami_]etmax

Signifies that the window was maximized. This may not need any action, but is simply for information. If the window needs to be redrawn as a result of the change, a separate redraw event will be sent

Event: [ami_]etnorm

The window was normalized back to its original size. This may not need any action, but is simply for information. If the window needs to be redrawn as a result of the change, a separate redraw event will be sent.

Event: [ami_]etmenus

A menu item was selected. The **id** parameter gives which menu item was selected, which is a logical identifier number selected when the menu structure was constructed.

6 Graphical Interface Library



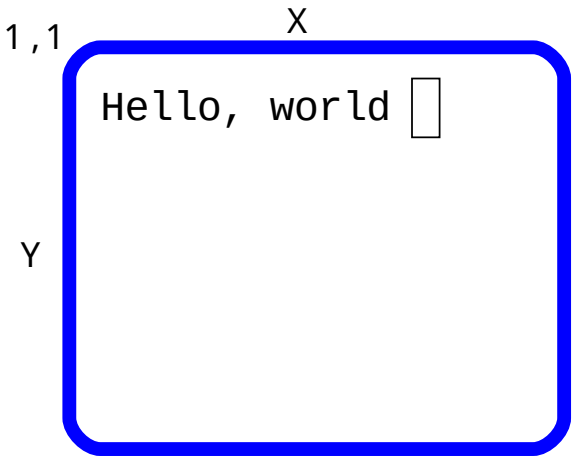
The graphical library **graphics** extends the **terminal** model by adding graphical output procedures.

Since it is completely upward compatible with **terminal**, and standard ANSI C serial output modes, any program from ANSI C, or terminal compliant ANSI C will run under **graphics**. In the most advanced modes of **graphics**, ordinary ANSI C I/O statements can still be used to output text, so all of the input and output formatting functions of ANSI C still work.

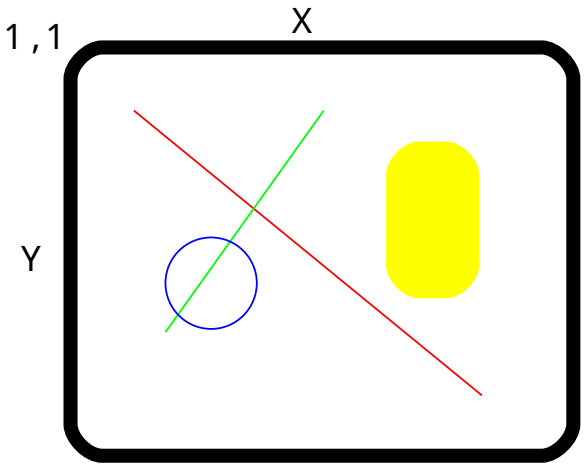
graphics will handle any graphics task. Besides drawing features, it supports double page animation. Combined with the sound library **sound**, full graphical games are possible.

6.1 Terminal model

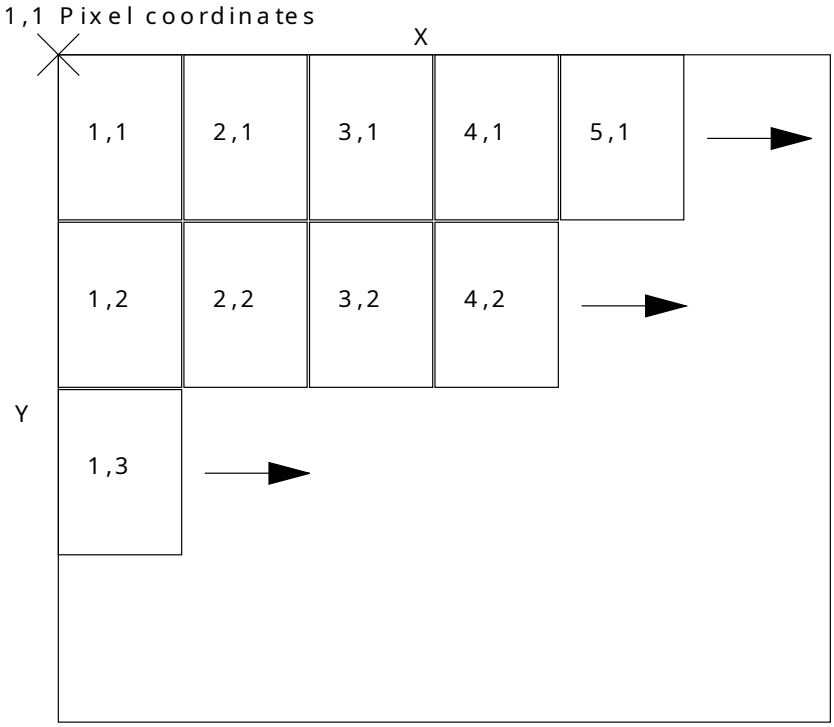
graphics emulates **terminal** by setting up a "character grid" across the pixel based screen. Each "cell" on the grid matches the pixel height and width of a character. The character font is set to a fixed font by default when **graphics** starts. This gives every character in the font the same height and width. The text drawing mode is set such that both the foreground (the inside of the characters) and the background (the space behind the characters) are drawn, overwriting any content of each character cell as it is written.



Is implemented in a graphics coordinate screen:



Each increment is called a picture element, or pixel, and represents the smallest drawable dot on the screen. Each character of the terminal mode is mapped onto a square area of the drawing surface as follows:



This mode can be kept, while drawing other figures on the same text surface. Alternately, the cursor can be set to any arbitrary pixel on the screen, and text written anywhere, down to the pixel. Also, the font can be changed to a proportional one for a more pleasing look. Because automatic mode relies on characters being neatly placed on the grid, it must be turned off before the cursor leaves the grid or becomes a proportional font.

When **auto()** is turned off, the grid is still useful. Although the character spacing varies, the line spacing is still valid. This means that the grid is no longer useful in the x direction, but it is still useful in the y direction. In addition, the origin in x is still valid. The result is that a series of lines printed with end-of-lines will do the right thing with **auto()** off, namely present a series of left justified lines at the x origin (flush left) with the correct spacing.

6.2 Graphics Coordinates

The total size of the graphics screen is found by **maxxg(f)** and **maxyg(f)**, which return the maximum pixel index in x and y. The pixel coordinates on the screen are from 1,1 to **maxxg(f)**, **maxyg(f)**. The cursor can be set to any pixel position by **cursorg(f, x, y)**. The current location of the cursor in pixel terms is found by **curxg(f)** and **curyg(f)**.

6.3 Character Drawing

There can be any number of fonts available on the system, including both fixed space fonts, and proportional fonts that vary in the width of characters. The number of fonts in the system can be found with the **fonts(f)** function. Fonts are chosen by logical number with **font(f, c)**, where **c** is the font code, 1 to **fonts(f)**. There are two methods to determine what font is assigned to a particular font code. The first is the standard font codes, the second is the font name system. Standard fonts are numbers for commonly used fonts in the system. These are commonly available fonts the program may need.

The standard font codes are:

Terminal Font: **AMI_FONT_TERM: 1**

> This is the default font set up by **graphics** when it starts. It's a fixed font. It also cannot be superscripted, subscripted, bold or italic, because these modes change the size of the font.

Book Font: **AMI_FONT_BOOK: 2**

> This is a serif font, and is good for general purpose text such as what a paragraph in a book is written in. This is the most common proportional font.

Sign Font: **AMI_FONT_SIGN: 3**

> This is a no serif font (sans serif), and is best for headings, titles and similar uses, as in road signs and other signs. It's a proportional font.

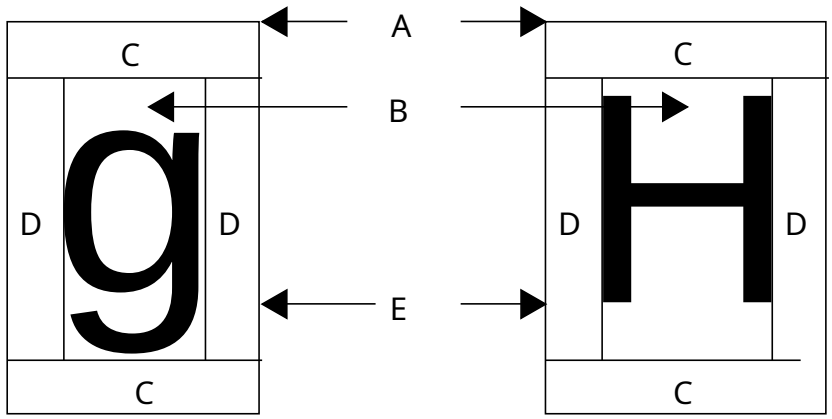
Technical Font: **AMI_FONT_TECH: 4**

> The technical font is a fixed font that is guaranteed to be able to scale to any arbitrary size. This font is used to label drawings and engineering documents. Before the technology existed to create arbitrary fonts in any point size, the technical font was done with stroked vector graphics. Now, it is more likely to be equivalent to the sign font.

The first four fonts corresponding to the standard fonts are always present, so **fonts()** will always be 4 or greater.

The name of an installed font can be found by **fontnam(f, fc, fns)**, where **fc** is the font code, 1 to **fonts(f)**, and **fns** returns the descriptive string for the font, such as "Helvetica". The application should

use the standard fonts by default, then present the system fonts, by name to the user, and let the user choose one of them.



The Parts of the box are:

Label	Part
A	Character box
B	Character bounding box
C	Line space
D	Character space
E	Baseline

The size of each character is set by **fontsiz(f, n)**, where n is the height of the character cell in pixels. Every glyph of the font fits the cell: the em square is scaled down until ascenders and descenders stand inside, and **chrsizy(f)** returns exactly n. The reason character sizes are set by their height is because proportional fonts vary in the width of the character. The height never varies. The current height of the font is found by **chrsizy(f)**. Its width is found with **chrsizx(f)**. When a proportional font is active, **chrsizx(f)** returns the width of a space in that font, which is always as wide or wider than the widest character in that font.

Besides the basic font, extra space can be added between lines (known as "leading" in typography, for the lead strips used between type lines) with **chrspcy(f, n)**. n is the number of pixels of extra space to add between lines. Extra space between characters is added with **chrspcx(f, n)**, where n is the number of pixels of extra space to add between characters.

The graphical cursor is placed at the upper left of the character box for the next character to be drawn. If it is needed to know exactly where the character will rest if drawn on a line. The offset from the top of the character box to the baseline of the text is found with the function **baseline(f)**.

In typography, fonts and characters are measured by points, of which there are 28.35 points per centimeter. Point measurement implies that the exact size of objects drawn on the screen is known. This can be determined by the functions **dpmx(f)** and **dpmy(f)**, which return the "dots per meter" or pixels in one meter for both x and y. The reason it can be two different measures for the two different axes is that the display may not have square pixels or a 1:1 aspect ratio.

The pixels covered by a given point size are:

```
ami_dpmy(f)/2835*point size
```

Feeding that height to **fontsiz()** no longer yields type at that point size: **fontsiz()** sizes the cell, and the em square scales down inside it. For exact typographic sizes use **setpoints()** below.

The screen aspect ratio can also be found from these calls, it is:

```
ami_dpmx(f)/ami_dpmy(f)
```

The point size can also be set directly with setpoints(f, ps), which takes the size in points and converts through the display metrics as above. The cell height that results is read back with **chrsizy(f)**, and the point size standing — set by **setpoints(f, ps)**, or produced by a **fontsiz(f, n)** cell fit — is read with **points(f)**. The path, or angle at which characters are drawn, is set by path(f, a). The angle is 0 up, running clockwise through the full circle as a ratio of LONG_MAX; the normal setting is LONG_MAX/4, 90 degrees, on the grid. The path cannot be changed while auto() is on.

6.4 String Sizes and Kerning

When writing text to the screen in proportional font, it is often needed to know exactly how much space the string will take up on the screen, down to the pixel. This amount of space can change not only because of the variable width of proportional fonts, but also because of an effect called "kerning". When a string of characters is draw together, the system can apply "kerning" to characters in the string. Kerning means to fit the individual letters together, like puzzle pieces, to come up with a tighter spacing than is normally possible. For example, "A" and "V" together as "AV" can typically be kerned together to take less space because they can overlap. To find the exact number of pixels in x that a string will occupy, the function **strsiz(f, s)** is used, where s is the string. The size of the string in y does not change, and can be found with **chrsizy**. To find the exact position, in pixels offset from the beginning of the string in x, of a given character, use **chrpos(f, s, n)**, where **s** is the string, and **n** is the index of the character to find the x offset of, from 0 to the length of the string.

6.5 Justification

Justification is the spreading of spacing through a string of characters to fit a given space. If the string will fit into the space is found with **strsiz(f, s)**, and checking if the resulting pixels required are less than or equal to the space they will occupy as justified. The character string is written in justified mode with **writejust(f, s, n)**, where **s** is the string to write, and **n** is the number of pixels to fit it in. If the number of pixels allowed for is not enough, the string will be larger than the requested number. The offset, in x, of a given justified character, is found with **justpos(f, s, p, n)**, where **s** is the string, **p** is the offset of the character you are interested in, and **n** is the total number of pixels to fit the string in, as in **writejust()**.

6.6 Effects

graphics expands the effects in **terminal**. For smaller character baselines, **condensed(f, b**), is used. For larger character baselines, **extended(f, b)** is used**.**

In addition to normal **bold(f,b)**, there are also **light(f,b)**, **xlight(f, b)**, and **xbold(f,b)** effects. For lighter than normal, extra light, and extra bold modes.

Characters will have an embossed look with **hollow(f, b)** and **raised(f, b)**. Hollow makes the character look sunken, and raised makes it look as if coming off the page.

6.7 Tabs

graphics extends the character level of tabbing in **terminal** with procedures that can set tabs on an individual pixel. **settabg(f, x)** sets a tab at the pixel **x**. **restabg(f, x)** resets the tab at pixel **x**. The **terminal** procedure **clrtab(f)** clears all tabs, including pixel level tabs.

6.8 Colors

The simple eight colors from **terminal** are still available, with the addition of two new calls that allow access to the full range of colors an advanced graphic system provides. **fcolorg(f, r, g, b)** sets the foreground color from values of red **r**, green **g**, and blue **b**. Similarly, **bcolorg(f, r, g, b)** sets the background color from rgb values. The values of the colors are ratioed. This means that instead of an absolute number, the possible colors are ratioed from 0 to LONG_MAX, where 0 is dark, and LONG_MAX is saturated color. Color ratios allow the true color range implemented by the system to be hidden.

6.9 Drawing Modes

When colors, background or foreground, are drawn on the screen, they can be in a number of mixing modes. Mixing modes govern how the new color is laid over the old. The modes are mutually exclusive, so the setting of a new mix mode for a given color deactivates the old mode. If the old color is simply to be overwritten, then **fover(f)** or **bover(f)** is appropriate. If the new color is to be xor'ed with the old color, then **fxor(f)** or **bxor(f)** is used. If the new color is to be ignored, leaving the old color underneath intact, use **finvis(f)** or **binvis(f).** There is also **fand(f), band(f), for(f)** and **bor(f).**

There might not seem to be a use for an "invisible" color, but there are actually several uses. First, if the background is set invisible, the text can be overlaid on another pattern. In fact, this is the most common drawing mode, and most graphical programmers will prefer to turn the background off and lay the backgrounds themselves. Similarly, leaving the background on, then setting the foreground invisible can be used to "stencil" letters with arbitrary patterns inside the letters.

xor(f) mode is good for several things. First, if a series of figures, say rectangles, are to be laid, but the intersections between them are to be left visible, **xor()** is the right mode. Second, **xor()** can be used to place, and then remove a pattern, even a complex one, easily. This is used to allow the user to place figures by dragging them with the mouse to a new location. It can be used to draw "rubber band" boxes around selections.

xor() mode has two rules of interest:

1. Any two colors, even the same colors, xored together, will give a third color, with the exception of black.
2. Having xored a drawing into the viewplane, xoring the same color and drawing into the viewplane again will restore the old drawing.

xor() can be used for several special effects. However, **xor()** does not tolerate inaccuracy. Xoring something back off the screen has to be done the same way it was put on, with the same parameters. Also, the mode of drawing in **graphics** is not compatible with some uses of the **xor()** mode. Drawing a rectangle, for example, will result in "corner errors", because the rectangles are built from lines, and lines start and end at the coordinates of the box corners. Because one line starts at the same point another ends, they will xor mix together. The result is a recognizable point of off-color on each corner.

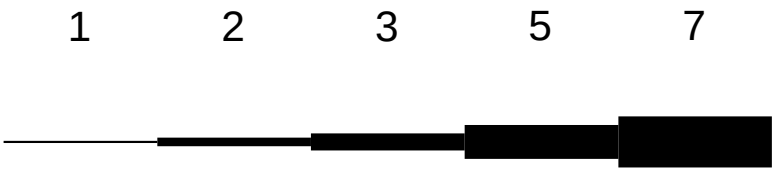
The best way to use xor is to xor a single figure onto the screen, then xor it back off. If complex figures are to be xored on and off the screen, xor is run backwards, that is, from the last figure drawn back to the first.

and() and **or()** modes are used to create stencils and other effects. For example, a drawing and'ed with a black stencil will remain black in the stencil area, and intact elsewhere.

6.10 Drawing Graphics

A graphics element that is not a character is referred to as a "figure". What **graphics** tries to do is provide a small toolset, that does not include figures that you could reasonably construct from the lower level figures. For example, there is no circle figure in **graphics**, because that is simply a special case of **ellipse**.

A parameter that applies to almost all figures is the width of lines. The width of a line usually defaults to 1, but may be more if a single line is unusable on the current display. This can easily happen on a very high resolution display. The line width can be set by the procedure **linewidth(f, n)**, where **n** is the number of pixels for the line to use. There is no limit on the width of a line, and in fact, lines are a defacto way to draw arbitrary angle filled rectangles.



When the line width is set to an even number of pixels, an effect called "even line uncertainty" exists. If you draw a line between two points, and the line width is say, 2, one of the 2 pixels is going to be on the line, and the other could be to either side of it. It depends on how the math happens to round off.

To prevent this, line width should always be set to an odd number.

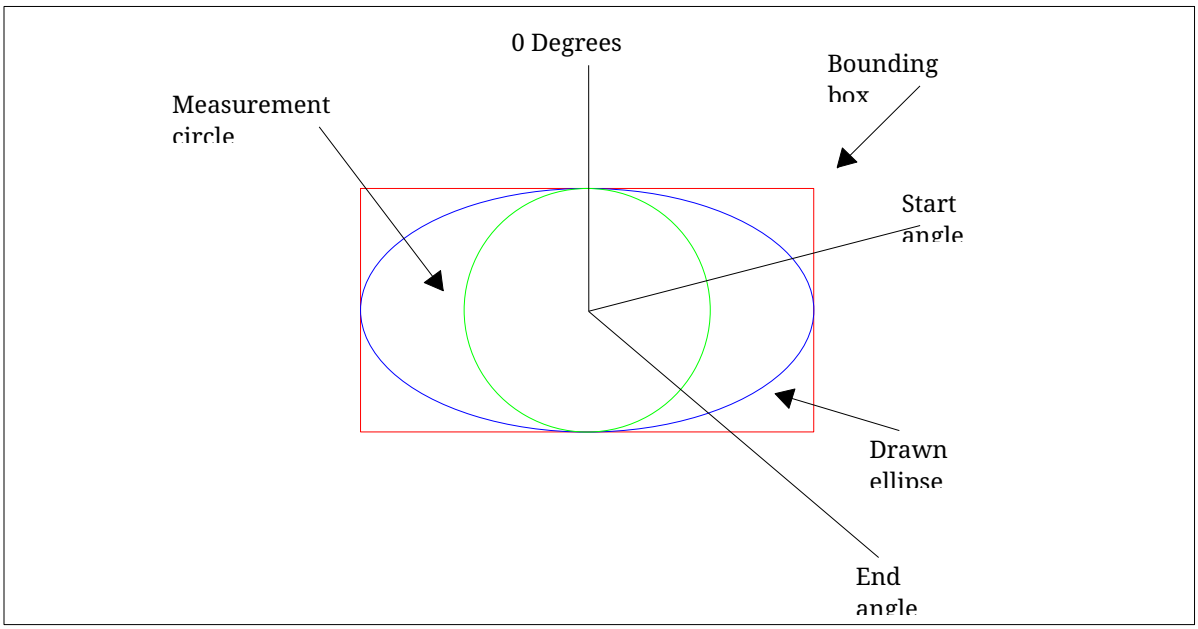
Lines are solid by default. The style is set with **linestyle(f, s)**, choosing from **lssolid**, **lsdash** and **lsdot**.

6.11 Figures

The fundamental figure in graphics is the line. A line is drawn, in the current **linewidth()**, by **line(f, x1, y1, x2, y2)**. A rectangle is drawn with **rect(f, x1, y1, x2, y2)**, whose borders have the current **linewidth()**. A filled rectangle is drawn with **frect(f, x1, y1, x2, y2)**, whose interior is the foreground color. An ellipse is drawn with **ellipse(f, x1, y1, x2, y2)**. The x and y parameters define a rectangle that contains the figure. The procedure **fellipse(f, x1, y1, x2, y2)** draws a filled ellipse.

The procedure **arc(f, x1, y1, x2, y2, rs, re)** draws an arc line around the ellipse formed by the rectangle formed by the x and y parameters. The start and end points of the arc are described by a special ratio notation that gives the angle. The angles in a 360 degree circle are described by a number from 0 to **LONG_MAX**. 0 is the 0 degree, or top center, of the ellipse. The angles around the circle clockwise then go from 0 to **LONG_MAX**, at which time a full 360 degrees have been traversed. For example, **LONG_MAX** div 2 is 180 degrees, **LONG_MAX** div 4 is 90 degrees, etc. The parameter **rs**

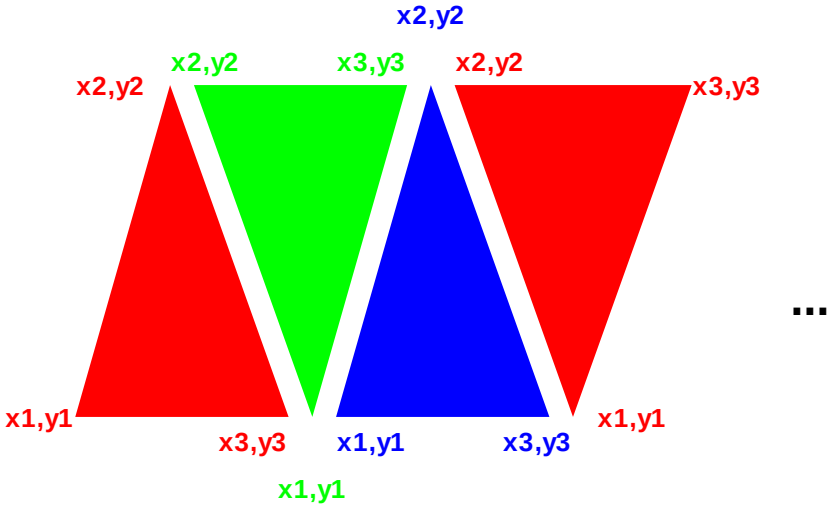
gives the angle where the arc starts. The parameter **re** gives the angle where the arc ends. Arcs can be specified to cross **LONG_MAX** back to zero, or use negative degrees, or any combination.



The procedure **farc(f, x1, y1, x2, y2, rs, re)** draws a filled arc. The procedure fchord(f, x1, y1, x2, y2, rs, re) draws a filled chord, the section of the ellipse closed by the line joining the arc ends.

Rectangles with rounded corners can be drawn with **rrect(f, x1, y1, x2, y2, xs, ys)**. The x and y parameters describe the bounding box, The xs and ys parameters describe the size of the ellipses that are placed in the corners to round the edges of the box. To draw a filled rounded rectangle, use **frrect(f, x1, y1, x2, y2, xs, ys)**.

The general purpose shape **ftriangle(f, x1, y1, x2, y2, x3, y3)** draws a filled triangle. Parting with convention, **graphics** does not give complex polygon procedures. Rather, you can build up polygons from triangles, and in any case, a high speed drawing engine in hardware would accept triangles only, so the lower level software would have to break up the polygon for you.



...

Example of polygon filled shape draw:

```
typedef struct { long x, y; } point;

void poly(FILE* f, point points[] /* ends with 0,0*/)

{

    int p = 0;


    /* count points */
    while (points[p].x || points[p].y) p++;
    if (p < 3) return; /* minimum 3 points */
    p = 1; /* index 2nd point */
    while ((points[p].x || points[p].y) &&
           (points[p+1].x || points[p+1].y)) {

        /* connect origin, next two points */
        ami_ftriangle(f, points[0].x, points[0].y,
                      points[p].x, points[p].y,
                      points[p+1].x, points[p+1].y);
        p++;

    }

}
```

Single pixels can be set with **setpixel(f, x, y)**.

6.12 Predefined Pictures

A picture, or a bitmap, is defined outside the program by a drawing application. Its format is typically operating system specific. **graphics** considers pictures to be a cached resource. A picture is loaded from a file by **loadpict(f, p, fs)**. The string **fs** indicates the file name for the picture file. **p** is a logical picture number, from 1 to n, and indicates how you want to refer to the picture while its loaded into memory.

A logical picture is drawn onto the screen with **picture(f, p, x1, y1, x2, y2)**. The parameter **p** indicates the logical picture to draw. The x and y parameters indicate the box that the picture is to be drawn into. **graphics** will scale or stretch the picture as needed to make the picture fit into the space given.

In order to determine the parameters of a picture, such as native size and aspect, the functions **pictsizx(f, p)** and **pictsizy(f, p)** are used. These give the native size of the picture in x and y, and the aspect ratio of the picture is then found with **pictsizx(f, p)/pictsizy(f, p)**.

When the picture is no longer needed, **delpict(f, p)** removes it from cache.

6.13 Scrolling

As in **terminal**, **graphics** can scroll in arbitrary directions. It can also scroll down to the pixel, using the call **scrollg(f, x, y)**. The parameters work the same way as the character position parameters of scroll, except that pixels are specified instead of characters.

6.14 Clipping

Clipping is automatic in **graphics**. Any figure drawn is clipped to the edges of the screen. If a figure is drawn entirely outside the screen bounds, it is clipped out.

6.15 View Transformation

The drawing surface can be viewed through an offset and a scale. **viewoffg(f, x, y)** sets the offset of the viewport into the logical drawing space, in pixels, from -**LONG_MAX** to **LONG_MAX**. **viewscale(f, x, y)** sets the scale of the view in x and y: 1.0 presents the logical space at actual size, smaller values shrink the presentation, and larger expand it. Character sizes follow the scale, so text drawn on a scaled view keeps its proportion to the drawing.

The functions **scalex(f, x)** and **scaley(f, y)** map a physical pixel coordinate back to the logical drawing space, applying the inverse of the current offset and scale. Mouse positions stay in physical pixels, so a program hit testing against a scaled drawing puts the mouse coordinates through these to land in its own space.

6.16 Mouse Graphical Position

A new event, **etmoumovg**, exists that gives mouse movements in pixels, not just characters. The old **etmoumov** still occurs, and carries the character grid message. The **etmoumovg** message happens when the mouse moves a pixel, and the **etmoumov** message happens when the mouse moves a whole character cell. If you don't need the **etmoumov[g]** message, you simply ignore it.

6.17 Animation

In **terminal**, the **select()** call was introduced, that switches between multiple screen buffers. This call is tailor made for double buffer animation, it works in **graphics** the same way. Double buffer animation works by having two screen buffers. One of them is drawn, and the other one is displayed. When a "frame" is drawn, i.e., the picture in the buffer currently being drawn is complete, the drawing and display buffers are swapped, and then drawing begins on what used to be the display buffer, clearing it if necessary.

Double buffering removes any of the ongoing drawing operations from the active screen. Although computers do this quite fast, seeing drawing operations in progress tends to produce annoying flashing effects, sparkles and other effects during the drawing. In addition, although the worst interactions with the painting of the graphics card memory to the display screen have disappeared, there can still be odd effects from the fact that the display is being refreshed across the image being drawn.

graphics supports more than double buffering (triple, quad or better). However, the advantages of this typically diminish as the amount of data being managed grows without a compensating gain in drawing speeds.

To reduce flicker effects the buffer is flipped when the display enters its retrace period. Syncing with the display eliminates the effects that occur when writing to the display while the display is drawing across it. The retrace period is when the refresh cycle has finished the last lines on the screen, and will head back to the top of the screen. This gives a short time while the display is not doing anything, so if the buffer swap can occur within the retrace, no effects will appear on the screen at all.

The solution is the **etframe** message. **etframe** is sent when the display enters refresh. If the system does not allow notification for retrace, the **etframe** message is generated by a timer that keeps either the screen refresh rate, or 30 cycles per second if that cannot be determined.

6.18 Copy between buffers

Blocks of pixels can be copied between buffers with procedure **blockcopyg(f, s, d, sx1, sy1, sx2, sy2, dx1, dy1, dx2, dy2)**. This copies a pixel block from a source bounding box to a destination box in the same or a different buffer. It is capable of both resizing the block, as well as using the write mode to place the pixels.

Copying between buffers is a very powerful technique to speed animation. Pictures can be constructed and processed in a non-displayed buffer, then copied quickly to a target buffer for display. A program with a lot of animation will typically have an offline buffer just to keep sprites and other drawn objects ready to present. The drawing mode can be used to create stencils and other drawing tools.

6.19 Buffer Follow mode

As in the **terminal** mode, the buffer follow mode can be used. When the program receives a **etresize**, in addition to the rszx and **rszy** size parameters, it receives the **rszxg** and **rszyg** size parameters. These can be used to set the buffer size in graphical terms with **sizbufg(f,x,y).** Then the functions **maxx(f)** and **maxy(f)** will return the character size, and the functions **maxxg(f)** and **maxyg(f)** will return the size in graphics terms. As in **terminal**, this effectively destroys the existing buffers and creates new ones that are cleared to the background color.

6.20 Printers

As with **terminal**, **graphics** can output to a printer if the printer is graphics capable. The one page buffer contains sufficient pixels to allow a complete page with graphics to be rendered in the buffer, then output to the printer by outputting a '\f' (form-feed) operation.

See **terminal** for a list of restrictions on printer operation.

A print file is opened with **openprint(f, n)**. The name **n** is either a filename, which receives the finished document in .pdf form, or a printer, recognized by the ':' in the name: **lp0:**, **lp1:**, and so on are the printers by number as the system lists them, and **lp:** is the system default printer.

The page is US letter at 600 DPI, 5100 by 6600 pixels, which presents 85 by 66 characters in the default 12 point terminal font. The print file is written with the normal terminal and graphics calls, and **title(f, ts)** sets the document title. Outputting a form-feed ('\f') completes the page and begins a new one. Closing the file completes the document: a partial page is ejected, and a printer receives the document from the spooler as a single job.

6.21 Remote display

Graphics is capable of using a remote display as in **terminal**. The same comments as in **terminal** apply to **graphics**.

6.22 Declarations

The declarations for graphics are those of terminal (4.10 "Advanced Input"), with graphical additions: the standard font codes, the line styles, the graphical mouse movement, and the display size events. The additions are:

```
/* standard fonts */
#define AMI_FONT_TERM 1 /* terminal (fixed space) font */
#define AMI_FONT_BOOK 2 /* serif font */
#define AMI_FONT_SIGN 3 /* san-serif font */
#define AMI_FONT_TECH 4 /* technical (scalable) font */

/* line styles */
typedef enum { ami_lssolid, ami_lsdash, ami_lsdot } ami_lstyle;

/** mouse move graphical */ ami_etmoumovg,
/** enlarge what is displayed */ ami_etusize,
/** reduce what is displayed */ ami_etdsize,
```

In the event record, the etmoumovg event carries the mouse number mmoung and the pixel position moupxg and moupyg, and the resize record carries rszxg and rszyg, the new size in pixels, beside rszx and rszy.

6.23 C++ graphics interface

Graphics has a C++ wrapper, in cpp/graphics.cpp with its declarations in hpp/graphics.hpp, following the pattern of the other wrappers. A C++ program includes <graphics.hpp> and works inside the graphics namespace, and the wrapper is carried in the graphical library, so nothing extra is linked.

The namespace carries the graphical interface with the prefixes stripped: the functions, the color and event types, and the event record laid out identically to its C form. The one exception is the widget creators, which appear as the widget classes rather than as free functions, since C++ will not let a class and a function share a name; see the widgets chapter. The C interface keeps them all.

The graph object is the single surface object, the graphical mirror of the terminal library's term: it holds the main window, every call is a method, and the event virtuals -- evchar() to evdsize() -- fire as events arrive, each returning nonzero to consume the event or zero to let it pass to the ordinary event loop. One graph object exists at a time, refused rather than allowed twice, and its destructor restores the event chain, the same law the term object keeps. A graph object serves a program that lives on the main window alone; a program of several windows wants the window objects of the windows management chapter, which know which window each event belongs to.

6.24 Functions in graphics

See **terminal** for the basic text functions. These are all implemented in **graphics**.

For all of the following functions, If the screen file **f** is not present, the default is the standard **stdout** or standard **stdin** file.

void [ami_]arc(FILE* f, long x1, long y1, long x2, long y2, long sa, long ea);

Draw an arc. Draws an arc on an ellipse, whose bounding box is **x1**,**y1** to **x2**,**y2**. The arc starts on the ellipse from the angle **sa**, to the angle **ea**. The angles are given in 360 degree to **LONG_MAX** ratio form. Uses the current color and mode in output surface file **f**.

void [ami_]band(FILE* f);

Set and mode background. Sets all new drawing to and old colors with new colors on the background in output surface file **f**.

long [ami_]baseline(FILE* f);

Find baseline of current font. Finds the baseline, or line on which all characters of the current font sit upon, in terms of offset from the character bounding box origin in y in output surface file **f**.

void [ami_]bcolorc(FILE* f, long r, long g, long b);

Set background color character in output surface file **f**. The background color is set to the red **r**, green **g** and blue **b** color. The colors are in 0..**LONG_MAX** ratios, with 0 = black, and **LONG_MAX** = saturated. The nearest color to the one given is found and set active as the foreground color. The exact color that results will depend on the total number of colors implemented in the system, and could well be black and white in a system so equipped.

This function gives you direct setting of character colors in RGB format if your system is so capable.

void [ami_]bcolorg(FILE* f, long r, long g, long b);

Set background color graphical in output surface file **f**. The background color is set to the red **r**, green **g** and blue **b** color. The colors are in 0..**LONG_MAX** ratios, with 0 = black, and **LONG_MAX** = saturated. The nearest color to the one given is found and set active as the foreground color. The exact color that results will depend on the total number of colors implemented in the system, and could well be black and white in a system so equipped.

void [ami_]binvis(FILE* f);

Set invisible mode background. Sets all new drawing to discard colors on the background in output surface file **f**.

void [ami_]blockcopyg(FILE* f, long s, long d, long sx1, long sy1, long sx2, long sy2, long dx1, long dy1, long dx2, long dy2);

Copy a pixel block between buffers in output surface file **f**. Copies a block of pixels from the source buffer **s** with the bounding box **sx1**, **sy1** to **sx2**, **sy2** to the destination buffer **d** in bounding box **dx1**, **dy1** to **dx2**, **dy2**. The current foreground write mode is used. The picture is stretched or compressed as required to fit into the destination bounding box. The method used to stretch or compress may depend on how much computing power is available on the system. There is no attention paid to aspect ratio. If the aspect ratio is to be preserved, it must be calculated. The current foreground mode is used.

void [ami_]bor(FILE* f);

Set or mode background. Sets all new drawing to or old colors with new colors on the background in output surface file **f**.

void [ami_]bover(FILE* f);

Set overwrite mode background. Sets all new drawing to overwrite old colors on the background in output surface file **f**.

void [ami_]bxor(FILE* f);

Set xor mode background. Sets all new drawing to xor old colors with new colors on the background in output surface file **f**.

long [ami_]chrpos(FILE* f, const char* s, long p);

Find pixel offset of character in output surface file **f**. Finds the pixel offset from the start of a string **s** in terms of x pixels, to the character by the index **p**.

long [ami_]chrsizx(FILE* f);

Find character size in x in output surface file **f**. Returns the x size, in pixels, of the characters in the current font. If the font is proportional, its x size will vary per character. The size will then be the space character, which is guaranteed to be the widest character in the font.

long [ami_]chrsizy(FILE* f);

Find character size in y in output surface file **f**. Returns the y size, in pixels, of the characters in the current font.

void [ami_]chrspcx(FILE* f, long s);

Set character x spacing in output surface file **f**. Sets the character to character extra spacing **s**, in pixels.

void [ami_]chrspcy(FILE* f, long s);

Set character y spacing in output surface file **f**. Sets the line to line extra spacing to **s**, in pixels.

void [ami_]condensed(FILE* f, long e);

Set condensed mode in output surface file **f**. Sets the current font to occupy a shorter baseline than normal. Note that this effect may not be implemented.

void [ami_]cursorg(FILE* f, long x, long y);

Position the cursor graphically in output surface file **f**. Moves the cursor to the pixel position **x** and **y**.

long [ami_]curxg(FILE* f);

Returns the current location of the cursor, in pixel units, for x in output surface file **f**.

long [ami_]curyg(FILE* f);

Returns the current location of the cursor, in pixel units, for y in output surface file **f**.

void [ami_]delpict(FILE* f, long p);

Remove logical picture from use in output surface file **f**. The logical picture **p** is removed from the picture queue, and will no longer take up memory space.

long [ami_]dpmx(FILE* f);

Find dots per meter x. Finds the dots per meter in x of the current display device in output surface file **f**.

long [ami_]dpmy(FILE* f);

Find dots per meter y. Finds the dots per meter in y of the current display device in output surface file **f**.

void [ami_]ellipse(FILE* f, long x1, long y1, long x2, long y2);

Draw an ellipse. Draws an ellipse bounded by a box whose opposite corners are **x1**,**y1** to **x2**,**y2**. Uses the current line width, color and mode in output surface file **f**.

void [ami_]extended(FILE* f, long e);

Set extended mode in output surface file **f**. Sets the current font to occupy a longer baseline than normal. Note that this effect may not be implemented.

void [ami_]fand(FILE* f);

Set and mode foreground. Sets all new drawing to and old colors with new colors on the foreground in output surface file **f**.

void [ami_]farc(FILE* f, long x1, long y1, long x2, long y2, long sa, long ea);

Draw a filled arc. Draws a filled arc on an ellipse, whose bounding box is **x1**,**y1** to **x2**,**y2**. The arc starts on the ellipse from the angle **sa**, to the angle **ea**. The angles are given in 360 degree to **LONG_MAX** ratio form. Uses the current color and mode in output surface file **f**.

void [ami_]fchord(FILE* f, long x1, long y1, long x2, long y2, long sa, long ea);

Draw a filled cord. Draws a filled cord on an ellipse, whose bounding box is **x1**,**y1** to **x2**,**y2**. The cord starts on the ellipse from the angle **sa**, to the angle **ea**. The angles are given in 360 degree to **LONG_MAX** ratio form. Uses the current color and mode in output surface file **f**.

void [ami_]fcolorc(FILE* f, long r, long g, long b);

Set foreground color character in output surface file **f**. The foreground color is set to the red **r**, green **g** and blue **b** color. The colors are in 0..**LONG_MAX** ratios, with 0 = black, and **LONG_MAX** = saturated. The nearest color to the one given is found and set active as the foreground color. The exact color that results will depend on the total number of colors implemented in the system, and could well be black and white in a system so equipped.

This function gives you direct setting of character colors in RGB format if your system is so capable.

void [ami_]fcolorg(FILE* f, long r, long g, long b);

Set foreground color graphical in output surface file **f**. The foreground color is set to the red **r**, green **g** and blue **b** color. The colors are in 0..**LONG_MAX** ratios, with 0 = black, and **LONG_MAX** = saturated. The nearest color to the one given is found and set active as the foreground color. The exact color that results will depend on the total number of colors implemented in the system, and could well be black and white in a system so equipped.

void [ami_]fellipse(FILE* f, long x1, long y1, long x2, long y2);

Draw a filled ellipse. Draws a solid ellipse bounded by a box whose opposite corners are **x1**,**y1** to **x2**,**y2**. Uses the current color and mode in output surface file **f**.

void [ami_]finvis(FILE* f);

Set invisible mode foreground. Sets all new drawing to discard colors on the foreground in output surface file **f**.

void [ami_]font(FILE* f, long fc);

Select logical font in output surface file **f**. Selects a font by logical number **fc**, where **fc** is 1..**fonts()**.

void [ami_]fontnam(FILE* f, long fc, char* fns, long fnsl);

Find name of logical font in output surface file **f**. Returns the name of the logical font **fc** in the string **fns**. The length of the buffer is passed in **fnsl**. The buffer must be long enough to include the whole string and a zero termination.

long [ami_]fonts(FILE* f);

Find number of fonts in output surface file **f**. Returns the number of fonts installed on the system.

void [ami_]fontsiz(FILE* f, long s);

Set font size in output surface file **f**. Sets the height of the character cell in pixels: every glyph fits the cell, the em square scaling down inside it, and chrsizy() returns exactly the height given. The point size that resulted is read with points().

void [ami_]for(FILE* f);

Set or mode foreground. Sets all new drawing to or old colors with new colors on the foreground in output surface file f. Note that the for_() name must be used in C++, as for is a reserved word there.

void [ami_]fover(FILE* f);

Set overwrite mode foreground. Sets all new drawing to overwrite old colors on the foreground in output surface file **f**.

void [ami_]frect(FILE* f, long x1, long y1, long x2, long y2);

Draw filled rectangle. Draws a solid rectangle, whose opposite corners are **x1**,**y1** and **x2**,**y2**. Uses the current color and mode in output surface file **f**.

void [ami_]frrect(FILE* f, long x1, long y1, long x2, long y2, long xs, long ys);

Draw filled rounded rectangle. Draws a rectangle with rounded corners. The opposite corners of the bounding box are specified as **x1**,**y1** to **x2**,**y2**. The ellipses that specify the rounded corners are **xs** and **ys**, which specify the width and height of the ellipse. Uses the current color and mode in output surface file **f**.

void [ami_]ftriangle(FILE* f, long x1, long y1, long x2, long y2, long x3, long y3);

Draw a filled triangle. Draws a filled triangle given the three points **x1**,**y1**,**x2**,**y2**, and **x3**,**y3**. Uses the current color and mode in output surface file **f**.

void [ami_]fxor(FILE* f);

Set xor mode foreground. Sets all new drawing to xor old colors with new colors on the foreground in output surface file **f**.

void [ami_]hollow(FILE* f, long e);

Set hollow mode in output surface file **f**. Sets the current font to hollow, or sunken look, printing. Note that this effect may not be implemented.

long [ami_]justpos(FILE* f, const char* s, long p, long n);

Find justified pixel off set of character in output surface file **f**. Finds the pixel offset from the start of a string **s**, in terms of x pixels, to the character by the index **p**, for a string justified to fit into **n** pixels. The rules of justification are the same as for **writejust()**.

void [ami_]light(FILE* f, long e);

Set light mode in output surface file **f**. Sets the current font to light printing. Note that this effect may not be implemented.

void [ami_]line(FILE* f, long x1, long y1, long x2, long y2);

Draw line. Draws a line between the point **x1**,**y1** to **x2**,**y2**, using the current line width, color and mode in output surface file **f**.

void [ami_]linestyle(FILE* f, [ami_]lstyle style);

Set line style. Chooses the style of the lines drawn after it, from lssolid, lsdash and lsdot, in output surface file f.

void [ami_]linewidth(FILE* f, long w);

Set line width. Sets the line drawing width at **w** pixels wide in output surface file **f**. Use of an odd number of pixels is recommended.

void [ami_]loadpict(FILE* f, long p, char* fn);

Load picture into output surface file **f**. Loads the picture from the filename **fn** to logical picture number **p**, which is 1..n. The file must be in a format that the system understands, and is converted to an in memory format that is optimal for the system, such as a direct match for the graphics device in use.

long [ami_]maxxg(FILE* f);

Returns the maximum pixel index x in output surface file **f**.

long [ami_]maxyg(FILE* f);

Returns the maximum pixel index y in output surface file **f.**

void [ami_]openprint(FILE** f, char* n);

Opens a print file to **f**. The name **n** is either a filename, which receives the finished document in .pdf form, or a printer, recognized by the ‘:’ in the name: **lp0:**, **lp1:**, and so on are the printers by number as the system lists them, and **lp:** is the system default printer. The print file is written with the normal terminal and graphics calls, one page at a time; outputting a form-feed (‘\f’) completes the page. There is no input side: the event and input device calls give errors. Closing the file completes the document, ejecting a partial page, and sends a printer document to the spooler as a single job.

void [ami_]path(FILE* f, long a);

Set character path in output surface file f. Sets the angle at which characters are drawn: 0 is up, running clockwise through the full circle as a ratio of LONG_MAX. The normal setting is LONG_MAX/4, 90 degrees, on the grid. The path cannot be changed while auto() is on.

long [ami_]pictsizx(FILE* f, long p);

Find picture size x in output surface file **f**. Returns the size, in pixels of x, of the logical picture **p**.

long [ami_]pictsizy(FILE* f, long p);

Find picture size y in output surface file **f**. Returns the size, in pixels of y, of the logical picture **p**.

void [ami_]picture(FILE* f, long p, long x1, long y1, long x2, long y2);

Draw picture into output surface file **f**. The logical picture **p** is drawn into the bounding box formed by **x1**,**y1** to **x2**,**y2**. The picture is stretched or compressed as required to fix into the destination bounding box. The method used to stretch or compress may depend on how much computing power is available on the system. There is no attention paid to aspect ratio. If the aspect ratio is to be preserved, it must be calculated. The current foreground mode is used.

float [ami_]points(FILE* f);

Find font size in points in output surface file **f**. Returns the current point size: the size **setpoints()** set, or the size a **fontsiz()** cell fit produced.

void [ami_]raised(FILE* f, long e);

Set raised mode in output surface file **f**. Sets the current font to raised, or relief look, printing. Note that this effect may not be implemented.

void [ami_]rect(FILE* f, long x1, long y1, long x2, long y2);

Draw rectangle. Draws a rectangle whose opposite corners are **x1**,**y1** and **x2**,**y2**. Uses the current line width, color and mode in output surface file **f**.

void [ami_]restabg(FILE* f, long t);

Reset graphical tab in output surface file **f**. The tab at pixel **t** is removed. If no tab is set at **t**, no error will result.

void [ami_]rrect(FILE* f, long x1, long y1, long x2, long y2, long xs, long ys);

Draw rounded rectangle. Draws a rectangle with rounded corners. The opposite corners of the bounding box are specified as **x1**,**y1** to **x2**,**y2**. The ellipses that specify the rounded corners are **xs** and **ys**, which specify the width and height of the ellipse. Uses the current line width, color and mode in output surface file **f**.

long [ami_]scalex(FILE* f, long x);

Maps the physical pixel coordinate x back to the logical drawing space, applying the inverse of the current view offset and scale, in output surface file f.

long [ami_]scaley(FILE* f, long y);

Maps the physical pixel coordinate y back to the logical drawing space, applying the inverse of the current view offset and scale, in output surface file f.

void [ami_]scrollg(FILE* f, long x, long y);

Scroll in arbitrary directions in output surface file **f**. The screen is scrolled according to the differences in **x** and **y**, which are in pixels. Uncovered areas on the screen appear in the current background color.

void [ami_]setpixel(FILE* f, long x, long y);

Set single pixel. Sets a single pixel at the point **x**,**y**, using the current color and mode in output surface file **f**.

void [ami_]setpoints(FILE* f, float ps);

Set font size in points in output surface file f. Sets the height of the current font to the point size ps, converting through the display metrics; 72 points are an inch.

void [ami_]settabg(FILE* f, long t);

Set graphical tab in output surface file **f**. Sets a tab to the pixel **t**.

void [ami_]sizbufg(FILE* f, long x, long y);

Sets the size of the buffer used to draw into, in pixels. x and y indicate the width and height of the buffer surface in window f.

long [ami_]strsiz(FILE* f, const char* s);

Find pixel size of string in output surface file **f**. Finds the total x size of the string **s**, in pixels. Accounts for all sizes and spacing.

void [ami_]viewoffg(FILE* f, long x, long y);

Sets the offset of the viewport into the logical drawing space, in pixels, from -LONG_MAX to LONG_MAX, in output surface file f.

void [ami_]viewscale(FILE* f, float x, float y);

Sets the scale of the view in x and y in output surface file f. A scale of 1.0 presents the logical space at actual size; smaller shrinks the presentation, larger expands it. Character sizes follow the scale.

void [ami_]writejust(FILE* f, const char* s, long n);

Write string justified in output surface file **f**. Writes the given string s into the number of x pixels n. If the space is more than is required, the extra space will be distributed between the characters. If the space given is insufficient for the characters to be drawn, it will be drawn in the minimum amount of space for the entire string.

void [ami_]xbold(FILE* f, long e);

Set extra bold mode in output surface file **f**. Sets the current font to extra bold printing. Note that this effect may not be implemented.

void [ami_]xlight(FILE* f, long e);

Set extra light mode in output surface file **f**. Sets the current font to extra light printing. Note that this effect may not be implemented.

6.25 Events In graphics

See the description of the event record (6.22 “Declarations”) for the format of the entire record. **graphics** events carry over the set of events from terminal, and add a small set of their own.

Event: [ami_]etmoumovg
Override: long evmoumovg(long m, long x, long y);

The mouse with handle **mmoung** has moved, to the graphical position indicated by **moupxg** and **moupyg**.

Event: [ami_]etusize
Override: long evusize(void);

The user asked for the displayed material to be enlarged, with control-+ on either the main board or the keypad. The window is not touched -- the user sets that themselves -- it is the material within that changes, which for most programs means taking the point size a step up and laying out again.

Event: [ami_]etdsize
Override: long evdsize(void);

The user asked for the displayed material to be reduced, with control-- on either the main board or the keypad. As with etusize the window is not touched; the program takes what it displays a step smaller and lays out again.

7 Graphical Windows Management Library



windows is upward compatible with **terminal** and **graphics**. Given a single fixed screen, **windows** subdivides the screen into virtual windows, which can be set to any size or position. A **terminal** compatible program sees its window as a "virtual screen", and does not know that some or all of the contents of that screen may be hidden. A **windows** aware program can participate in the benefits of a windowed environment.

7.1 Screen Appearance

The idea of windows management is to take a program that thinks it is talking to an ordinary terminal, and allow the presentation of multiple such windows within a single screen.

By default, **windows** emulates a **terminal** interface for each window that is identical to the behavior of a full screen window from the program's point of view. A standard size screen is implemented for the program of 80 characters by 25 lines (even if the real display has a different size). **windows** accepts program writes, and places that information in a buffer that looks like an ideal screen. Then, the contents of that buffer are placed on the screen in various arrangements to complete the desktop for the user.

7.2 Rooted vs. non-rooted windows surfaces

Graphical windows can appear in either a fixed display surface or as part of a managed window system. An example of the former would be an application that runs alone with a window manager on a stand-alone console display. An example of the latter is within a fully windows driven display with multiple programs running. A rooted display means the program window runs full screen without a window frame and cannot be resized or minimized. Any child windows or dialogs appear as windows within that rooted display surface.

On a non-rooted display, parentless windows appear as peer windows on the home desktop, as do dialogs.

7.3 Window Modes

A window can be any actual size on the display. A window can also be off the display entirely, so that it accepts changes to its buffer, but does not display those changes. In addition, a window can be active, inactive, or overlapped, or even completely covered by other windows on the display.

7.4 Buffered Mode

A window comes up by default in the "buffered" mode. In buffered mode, the program has a "virtual display surface" that it writes to. **windows** maintains this surface in memory, and maintains the actual onscreen window as a scrolled window on that. The program is unconcerned with what part of its screen is being displayed, or even if it is displayed at all. The user operates the window using the frame controls.

Buffered mode is designed to allow the program to be unconcerned with the management of the display. However, the program can set the size of the virtual display using **sizbuf[g](f, x, y)**. When a buffer is resized, the contents that are in the intersection of the new buffer and old buffer are kept, and all the parts of the new buffer that are outside the the intersection of the old and new buffer are cleared to the current background color.

The buffer is a display surface that the buffer handler keeps for the program. The buffered mode allows a program to "think" it has a window of size N, but the appearance of the window on the actual display is a window on that virtual display that is maintained by the buffer handler.

Because from the program's point of view the window size does not change, and is always a complete window, the resize events are performed transparently to the program. Similarly, the minimize and maximize events are handled for the program (see 7.5 “Unbuffered Mode”). Discovery is when a window or part of a window is uncovered on the display.

When in buffered mode, the window manager may use scroll bars to display within a screen view that is smaller than the buffer.

A window shows them when its buffer is larger than its client area: a bar on the right for a taller buffer, and one along the bottom for a wider one. The bars are the window system’s own, and the user scrolls the view over the buffer with them; the program receives no events from them. They can be turned off for a window with **autobuf(f, e)**, and off for every window by the option scrollbuffer under graphics in the configuration.

Although buffered mode automatically handles window status events (covered in 7.5 “Unbuffered Mode”), these events are still sent through. For maximum compatibility, the using program should ignore any events not relevant to the mode it is in.

7.5 Unbuffered Mode

The program can leave buffered mode by using **buffer(f, b)**. Unbuffered mode has no display buffer, and no default actions for events. Instead, the onscreen appearance of the window is set dynamically by the program.

When unbuffered mode is entered, the buffer for the window is discarded, and it becomes entirely up to the program to manage its own window by watching events. The size of the window no longer reflects the buffer, but instead is the size of the onscreen window. In contrast to buffered mode, this can change many times as the window is resized under user control.

Buffered mode can be reentered at any time. The contents of the buffer are cleared to spaces, or the background color.

When running unbuffered, it is assumed that the program will manage the update of the on-screen display surface itself. When running unbuffered, the program handles events that are normally handled automatically.

The **etredraw** event contains the limits of an update rectangle that shows what part of the client area needs to be redrawn on the screen. The program can ignore the update rectangle, and simply redraw the entire screen, or it may only redraw the figures that overlap the update rectangle. The latter is usually more time efficient. **etredraw** gives its update rectangle in character coordinates. There is also a pixel version of that, **etredrawg**, which uses pixel coordinates. On a graphical system, both messages will be sent, and they duplicate each other. The character coordinate **etredraw** will contain a rectangle of characters that at least covers the pixels that need to be redrawn in a graphical system. Since **etredraw** and **etredrawg** duplicate each other, only one of them should be obeyed, and the other ignored.

The **etresize** event indicates that the onscreen window has changed its size. Its purpose is to let the program update any calculations based on the size of the window. This event can be ignored, used to update stored **maxx[g]** and **maxy[g]** values, or it could even cause the entire arrangement of the screen to be reformed.

etresize is not used to redraw areas of the screen. If a resize event causes new screen area to be uncovered, then one or more **etredraw** events will also be sent. There is no way to determine exactly which redraw operations correspond to the resize event, so some redundancy is possible with applications that repack their window's contents on a resize.

The **etmin** event indicates that the window has been minimized. When a window is minimized, it will not be displayed, and won't receive **etredraw** events. On the desktop, the window is represented by a small icon. An alternative form of minimization is for the window to receive an **etmin** message, followed by an etresize message, and continued **etredraw** messages. This is a hint for the window to display vital information in a much smaller format that fits into the minimized icon.

The **etmax** event indicates that the window has been maximized, or covers the entire desktop. If the window needs to be updated, this event may be accompanied by **etresize** and **etredraw** events.

The **etnorm** event indicates a normal window mode has been resumed from **etmin** or **etmax**. And it will again get **etresize** and **etredraw** events.

In many modern computers, the screen buffers are in fast video memory and managed by graphics hardware. Thus it can be more efficient for the underlying system to process all writes to an offscreen buffer and then copy back to the user screen. Thus the unbuffered mode may have no real effect. The main difference for the client program is that **etredraw** events are rarely, if ever sent, since the system has a copy of all onscreen graphics.

7.6 Buffer Follow Mode

windows has an intermediate mode between buffered and unbuffered mode. With faster computers and more memory, programs often retain the screen buffers and simply resize them as the window is resized. To allow for this mode, the **etresize** message also contains rszx, rszy, rszxg and rszyg parameters that have the new size of the window. These parameters are normally ignored. For buffer follow mode, the rszx, rszy, rszxg and rszyg parameters are used to resize the buffer using **sizbuf[g](f,x,y)**. The new

buffer sizes are then reflected in **maxx[g](f)** and **maxy[g](f)** and the program can continue to use the screen buffers as before. Note that the **sizbuf[g](f,x,y)** call resizes all of the screen buffers and clears them to the background color.

Buffer mode gives client programs the ability to treat the screen as a virtual screen whose geometry does not change. The **etresize** message, which is normally ignored in buffered mode, gives the status of what the actual display window is doing, and the client program has the option of reconfiguring the buffer to "follow" what the display window is doing.

7.7 Defacto transparency

Using unbuffered mode, it is possible to achieve window transparency. Transparency means that some parts of the window will show through to the display surface underneath the window. This can be used to implement advanced effects like non-rectangular windows, and it is the basis for widgets (which appear in 8 "Widget Library"). For defacto transparency to work, the drawing order of windows must be tightly controlled.

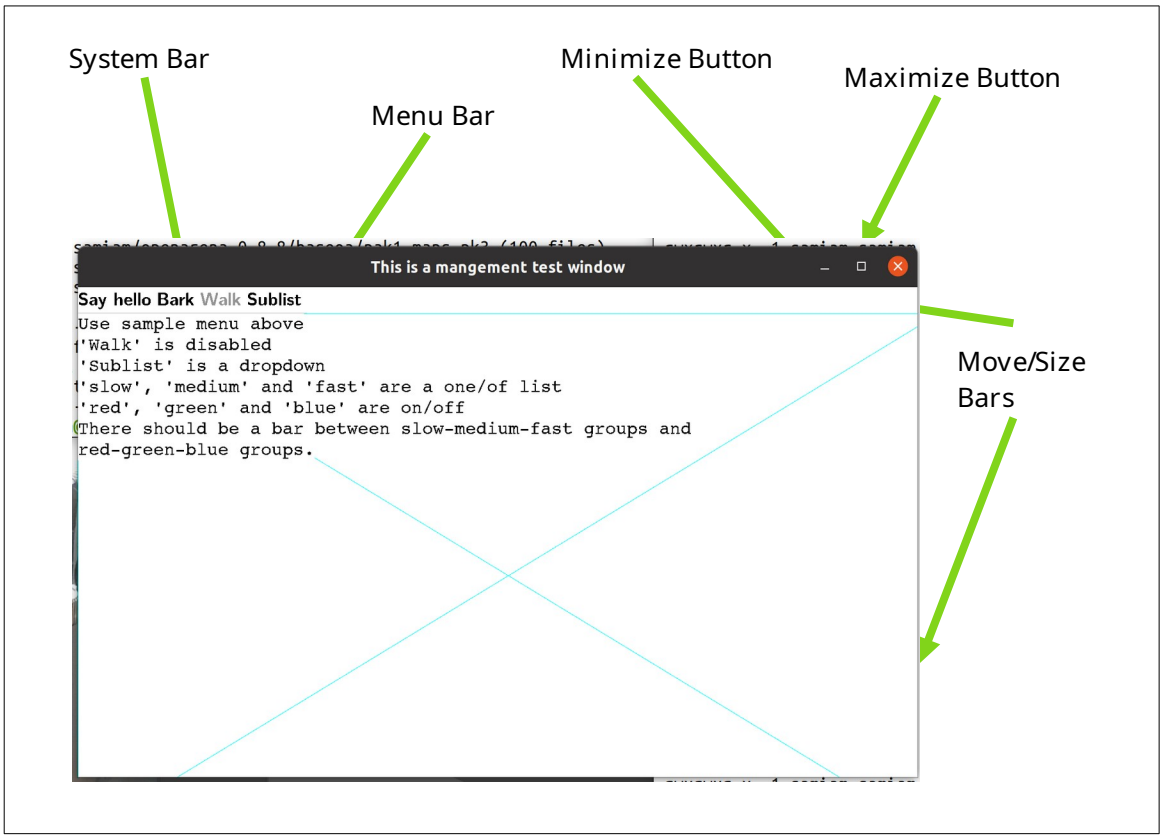
7.8 Delayed Window Display

When a window is created, it is not displayed until an actual write operation is made to it. This allows the window to be changed in configuration by the program before it is displayed, and prevents the window being rapidly changed on screen as that happens. For example, the program might want to take the window out of buffered mode, change its size, remove the frame, or add menus.

7.9 Window Frames

Under window systems, each window has a "window frame", which has various window management functions. These may include:

- Move bars to resize and move the window.
- A title bar that indicates what program is running in the window.
- A "minimize" button.
- A "maximize" button.
- A "menu" bar.



A window has several system components, not all of which are visible. Here the move bars are invisible unless you move the mouse cursor across them (Gnome desktop example).

When a window is in buffered mode, none of the frame features are under the control of the program, but instead are managed by the buffering software.

The appearance, or even the existence, of any of these items can be customized by the program. The frame can be enabled or disabled by **frame(f, e),** where **f** is the window file, and **e** is true to enable the frame. The title can be set by title(f, ts). The title, minimize and maximize buttons are considered part of the "system bar", which can be enabled or disabled by **sysbar(f, e)**. The resize bars are enabled or disabled by **sizable(f, e)**.

The frame control functions **frame()**, **title()**, **sysbar()** and **sizable()** are optional within an implementation. The system may not allow an application to, for example, disable its sizing bars, or it may not have a set of system controls on the window. The program must be ready for these commands not to have an effect. For example, after the **sizable()** function is set false, it should still be ready to receive a new size. If, for example, the size of the graphic to be presented is a fixed size, it would be filled out or clipped to fit the resulting window.

7.10 Multiple Windows

The **stdout** file is used as the output and the **stdin** file as input to the main window by default. A window can also be explicitly opened by **openwin(inw,outw,par,id)**. When a new window is opened,

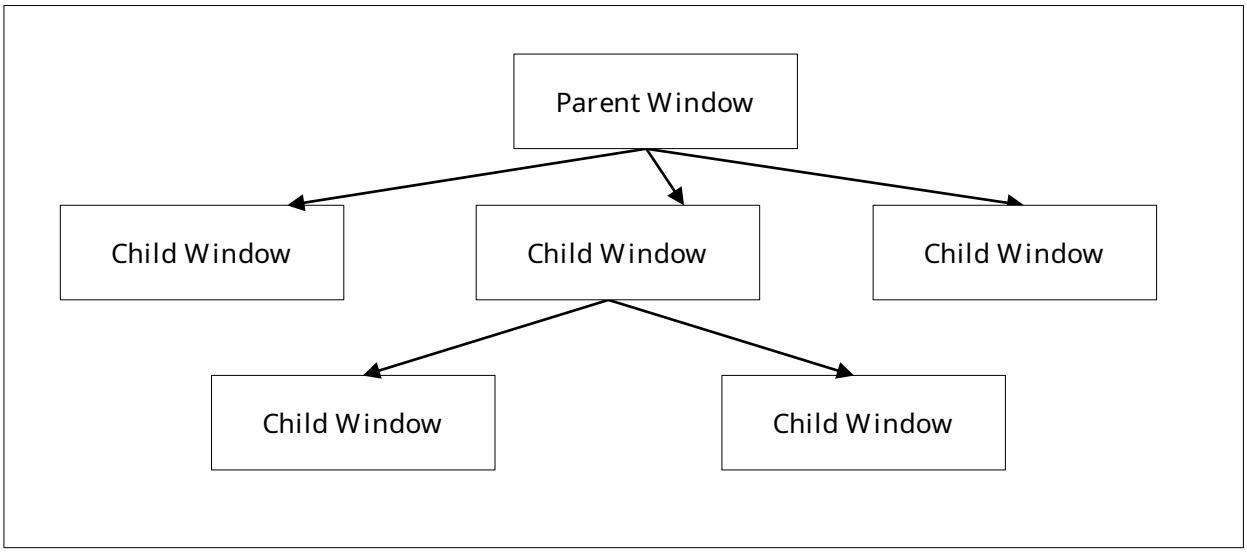
it is placed on the desktop with the other windows. The files **inw** and **outw** specify the input and output files attached to the window. The input file receives all user input and events from the window, and all of the write and other drawing operations are sent to the output file. The **id** is a number, from 1 to N, that specifies the logical window. The logical window number 1 is reserved for the **stdin** and **stdout** pair. The **par** parameter is NULL for a parentless window.

The output side of a window must always be unique, but the input side can be shared. Multiple windows can be attached to a single input file, and that input file will receive all of the input and events from all attached windows. The logical window number is returned with each event, so that the source of the input or event can be determined. This forms the basis of “class window handling” (7.14 “Class Window Handling”), or using one input handler to handle multiple windows.

Windows can be closed using the standard ANSI C **fclose()** call. Only the output side of a window is used to close that window. The input side is automatically closed when there is no longer a window that references it.

7.11 Parent/Child Windows

Windows systems are tree structured, and each window is a child of a parent window, with the exception of a "root" or master window, which is usually the window that contains the entire desktop. The methods introduced create windows that live side by side on the desktop, and have the desktop as their parents. However, it is possible to nest windows within each other.



A window is opened as a child by **openwin(inw, outw, par, id)**, which is the same **openwin()** call with a parent specified. The parent file **par** is the text file that is attached to the output window of the parent. To be a child window means to move as one within it. It maintains its position within the parent, even as the parent itself moves. It always is in front of the Z ordering for the parent. It is minimized, maximized and closed with the parent.

All windows have their position given in relative terms to the parent's client area origin. Any position operation is also relative to the parent.

The common use of the parent is to create "sibling" windows, or windows equivalent to the default program window in status, that are positioned independently within the parent. A program starts as the child of an external parent window. This could be an actual program, such as a program acting as a manager, or it could be the desktop root. There is no difference between these for the program.

Programs do not have direct access to the parent window in which the program was created. That parent window is usually the desktop.

7.12 Moving and Sizing Windows

Moving a window is done with the **setpos[g](f, x, y)** procedure[5]. The position is given in parent coordinates. When a window is on the desktop, it is necessary to find the size of the desktop, which is found with **scnsiz[g](f, x, y)**. When the desktop size is returned for character sizes, this is calculated using the current character sizes for the window. This is for comparative purposes only. The desktop may use a completely different set of characters and sizes. The purpose of giving the parent sizes is only to allow the program to determine the relative size and placement of the child window in the parent.

Sizing a window is done with **setsiz[g](f, x, y)** procedure. This sets the size of the entire window, and so does not indicate the resulting size of its client area. To find out what window size is needed for a given client area, before actually sizing the window, the function **winclient[g](f, cx, cy, wx, wy, ms)** is used. It returns the window size needed to contain that client. **winclient[g]** takes a set of the modes of the window to enable it to make the size determination.

The mode set declaration is:

```
/* windows mode sets */
typedef enum {

    ami_wmframe,   /* frame on/off */
    ami_wmsize,    /* size bars on/off */
    ami_wmsysbar,  /* system bar on/off */
    ami_wmmenu     /* menu on/off */

} ami_winmod;
typedef long ami_winmodset;
```

To find the current size of a window, in parent terms, the procedure **getsiz[g](f, x, y)** is used. This procedure is useful when you need to find the size of a window on the desktop.

7.13 Z Ordering

The Z ordering, or back to front ordering of windows, is determined by default to be newest created windows to the front, oldest to the back. This order can be modified by the users when they select windows towards the back to come to the front. The program can also reorder windows at will by sending them to the front or back. Note that the Z ordering is always relative within a parent.

To send a window to the front, the call **front(f)** is used. To send a window to the back, the call **back(f)** is used. To achieve a specific order within a set of windows, they should be sent to the front in the

[5]Many not work on top level window under desktop.

opposite order from which they are to appear, i.e., the backmost first, and the frontmost last. Alternately, the windows can be sent to the back in the order they appear.

There are other reasons besides reordering windows that these functions might be useful. When an error is encountered, it is common to send a window containing the error message to the front of the parent Z order, and to send the entire window to the front. Placing an active display or a background pattern in a frameless maximized window can create wallpaper. Placing the same window in the back can be a screen saver.

7.14 Class Window Handling

windows handles windows as a set of two files, the input and output. However, these don't need to always be a set. It is possible to reuse an already open input file as the input side of any new window. This is possible because the window id supplied when the window is opened is passed with all messages to the event procedure.

Allowing a single program thread to handle multiple open windows allows the creation of multiple window programs without the need to create a multitask program. For each message, the id is examined, and the action performed on the appropriate output file window.

Because the output file is unique, it is always the handle used to refer to the window in management calls.

7.15 Parallel Windows

Parallel tasking is a natural match to a multiwindowing system. With a task created to operate each window, the program is simplified, and user response is generally better.

To increase the responsiveness of user interfaces, the recommended program design for windowing programs is to create two tasks for each window, the "foreground" and the "background" tasks. The foreground task performs the event loop and the redraw, resize, move and other user interface tasks. The background task would handle computation tasks and other tasks related to performing actions or changing the drawing surface. The foreground task determines if an action that requires the background task is required, then sends a command to the background task via a queue or other IPC (InterProcess Communication).

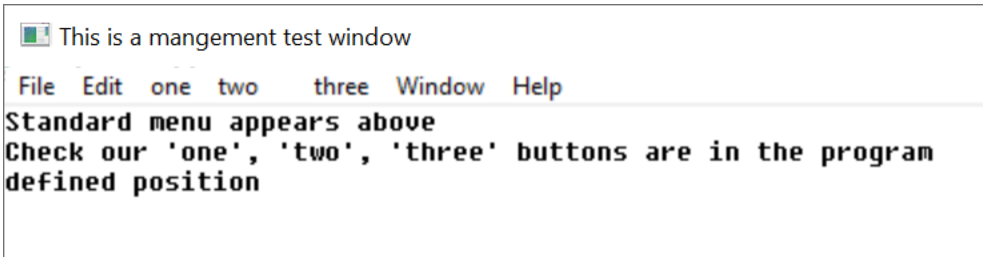
The reason for this construction is that the things the user sees, such as keeping the client area redrawn, or obeying a move, resize, minimize or similar command always have a task waiting to perform them. This means that elementary user desktop management appears responsive to the user, while data manipulation and computation tasks simply delay client area updates. Users will be much more willing to tolerate client area slowdowns than having a window lock stubbornly in position. Ideally, the client area should also have an indication that computation is taking place. The most modern method for this is a progress indicator that gives estimates as to when a task will complete, if the task will take longer than about 1 second.

If the work performed in a client area is trivial, only a foreground task need be provided. But be aware that it is very easy to slip into a mode where essential updates are held off. Calling a file procedure, waiting on a timer, or other simple task compromises the response time for window management. Better to leave the window in buffered mode, or structure with foreground/background from program creation than to have to add it later.

7.16 Menus

The menu is a series of buttons labeled with text for user action. The buttons can either have a direct effect on the program, or they can activate a series of submenus in a tree structure.

The exact location of a menu is system dependent. It may or may not affect the client area.



A menu on a window.

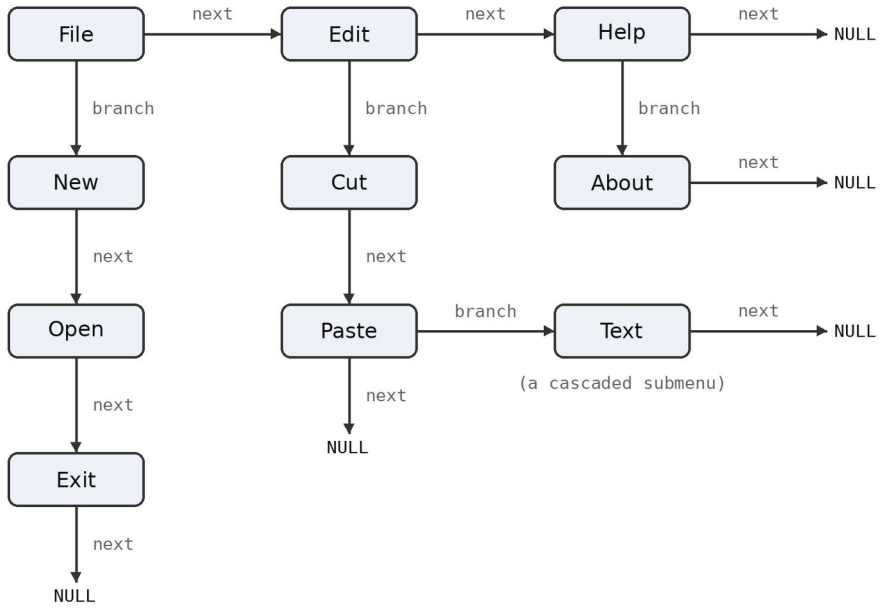
A menu is described to **windows** by constructing a data structure:

```
/* menu */
typedef struct ami_menurec* ami_menuptr;
typedef struct ami_menurec {

        ami_menuptr next;   /* next menu item in list */
        ami_menuptr branch; /* menu branch */
        long        onoff;  /* on/off highlight */
        long        oneof;  /* "one of" highlight */
        long        bar;    /* place bar under */
        long        id;     /* id of menu item */
        char*      face;   /* text to place on button */

} ami_menurec;
```

The elements tie together into a tree by their two pointers: **next** runs along the menu bar or down a pulldown, and **branch** hangs a pulldown below its bar item, or cascades a submenu from a pulldown item:



The menu is activated by **menu(f, m)** where **m** is the data structure for the menu.

Menu buttons are highlighted when pointed to. Additionally, a button can have "on/off" highlighting. In this mode, a state is kept for the button that is flipped on or off as the button is pressed. The highlighting for on and off states is different. The data structure that defines a menu is not used or updated after it is used to create a menu.

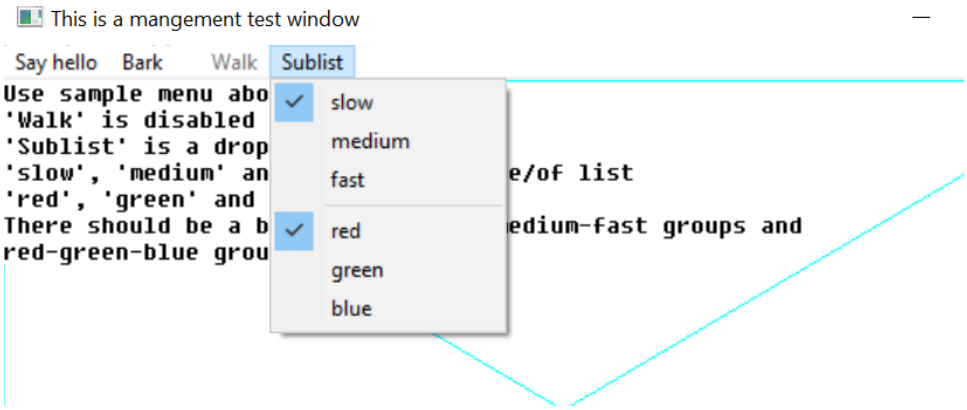
The **onoff** field true selects on/off highlighting. After the menu is activated, the state of the button can be changed with **menusel(f, id, e)**. The button state is not automatically changed by a user menu select. The program must specifically change the state of the button.

Another highlight mode is "one of" or "checklist" highlighting. In this case, a group of related buttons are joined by setting the **oneof** flag on each item in the list but the last. The highlighting for a button that is selected is different from one that is not, and only one item will be active in the list.

Like on/off buttons, user selection does not automatically change the state of the buttons in the list. It must be specifically changed by a **menusel()** call.

Typically, neither on/off nor one/of switching is available for top level (horizontal) menus, and the program should not count on them.

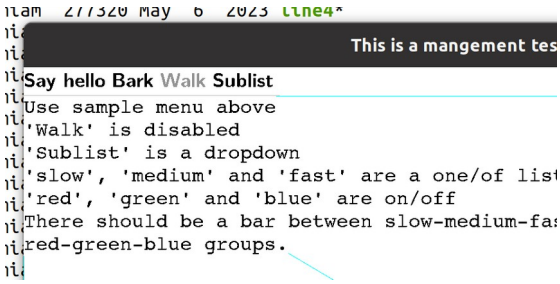
Menu items can be grouped in a vertical list by setting the **bar** field active. This causes a horizontal bar to be drawn under the menu item. Vertical lists are described next in "menu sublisting".



Selected menu items “slow” and “red”. Note bar between top and bottom groups.

A button can be enabled or disabled. A disabled button is highlighted specially, typically as greyed out. It indicates a button that is entirely inactive, because that function is not available. This is done to allow the same menu to be used regardless of the availability of the functions on the menu. Also, some functions come and go. For example, a "save" button (for "save file") might only be active if the file has changed, and needs to be saved.

The enable status can be changed after the menu is activated by **menuena()**.



The “walk” menu item is disabled.

Each button in a menu has a numeric **id**. This is used by several functions to change the state of buttons. It is an error to have two buttons with the same id.

The **face** text string indicates what text will appear on the face of the button. The system will automatically size the buttons to fit the largest text of a button, so care should be taken not to have a single button's text so long as to cause a lot of white space in the menu.

7.17 Setting Menu Active

A menu is constructed by the program as a list using the menu data structure, then set active by calling **menu()**, which can also turn the menu back off.

When a menu data structure is activated, all of the fields are copied and placed into an internal form. After the activation, the menu data has no function, and changing its fields will have no effect. The menu data structure can be recycled immediately after the **menu()** call.

7.18 Setting Menu States

The check state of a button is set with menusel(f, id, e) after the menu is active. For a button in a one of group, setting it also clears the other members of the group. Enabling and disabling a button of any kind, greyed out when disabled, is done with menuena(f, id, e).

7.19 Standard Menus

Besides presenting a menu in the method standard for the implementation, There exists a standard arrangement of commonly used buttons in the menu. **windows** defines a series of such standard buttons.

```
/* standardized menu entries */
#define AMI_SMNEW         1 /* new file */
#define AMI_SMOPEN        2 /* open file */
#define AMI_SMCLOSE       3 /* close file */
#define AMI_SMSAVE        4 /* save file */
#define AMI_SMSAVEAS      5 /* save file as name */
#define AMI_SMPAGESET     6 /* page setup */
#define AMI_SMPRINT       7 /* print */
#define AMI_SMEXIT        8 /* exit program */
#define AMI_SMUNDO        9 /* undo edit */
#define AMI_SMCUT        10 /* cut selection */
#define AMI_SMPASTE      11 /* paste selection */
#define AMI_SMDELETE     12 /* delete selection */
#define AMI_SMFIND       13 /* find text */
#define AMI_SMFINDNEXT   14 /* find next */
#define AMI_SMREPLACE    15 /* replace text */
#define AMI_SMGOTO       16 /* goto line */
#define AMI_SMSELECTALL  17 /* select all text */
#define AMI_SMNEWWINDOW  18 /* new window */
#define AMI_SMTILEHORIZ  19 /* tile child windows horizontally */
#define AMI_SMTILEVERT   20 /* tile child windows vertically */
#define AMI_SMCASCADE    21 /* cascade windows */
#define AMI_SMCLOSEALL   22 /* close all windows */
#define AMI_SMHELPTOPIC  23 /* help topics */
#define AMI_SMABOUT      24 /* about this program */
#define AMI_SMMAX        24 /* maximum defined standard menu entries
*/

/* standard menu selector */
typedef long ami_stdmenusel;
```

Standard menu button definitions should be used whenever possible. To create a menu using standard menu buttons, the procedure **stdmenu()** is used. The arrangement of the menu is chosen to match the implementation, and the non-standard buttons provided are integrated into the menu in the normal place for the implementation. When standard buttons are used, the button ids are set to the values shown above, so these values should not be used by the program.

Note that **stdmenu()** simply constructs a menu, it does not set it active.

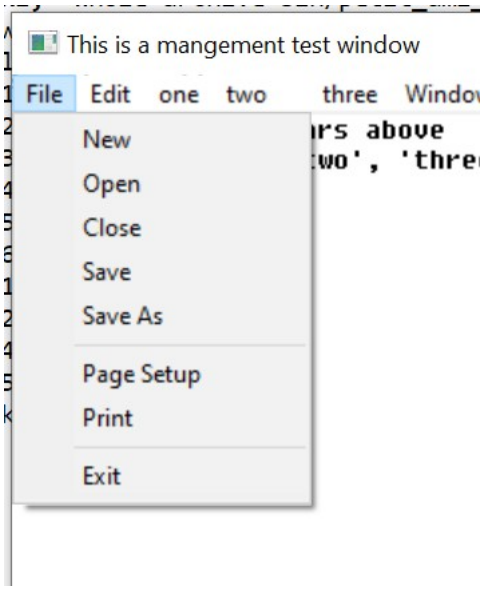
7.20 Menu Sublisting

When there is not enough room to represent the entire menu on a window, or anytime the size of a menu must be compressed, the tree structure of menus can be used with "pulldown" menus. A pulldown menu is a vertical menu list that appears under the selected button. This is done by placing a submenu in the **branch** pointer of the menu structure. The branch defines the pulldown menu items. The branch menu item is specially indicated to show that it is a branch, and not an immediate button, usually with an arrow pointing towards where the branch pulldown will appear. Any number of levels of pulldown

menus can be created. The levels under the topmost branch are generally placed to the right of the branch button.

If a branch pulldown cannot be placed where it should, due to the proximity of the button to the edge of the display, it instead goes back in the other direction, up or down, or both, until space is found for it. If the entire pulldown cannot be displayed, it is truncated. This would only happen if the pulldown was too large for the display.

When a branch button is pressed, no event is generated for it. The button is strictly used to activate the pulldown.



A pulldown menu.

7.21 Advanced Windowing

The features of a window besides the client area are designed to be the minimum features implemented across platforms. To implement more advanced windows appearances with custom menus, controls and status lines, the standard method is to turn off frames and menus for child windows, and use them as building blocks for more advanced client areas with multiple subwindows and features.

It is recommended that at least one menu appear for a program, even if the functions duplicate some of the advanced controls provided. The reason for this is that the menu has different presentation methods in different systems, so providing the standard "top" menu will help programs' portability.

7.22 Events

New event types are added for the management mode. As usual, events beyond the definition here should be ignored.

The event type and the event record are those of terminal (4.10 “Advanced Input”) with the graphical additions (6.21 “Declarations”); the codes and fields of the management level are already declared there. The events of the management level are:

```
/** window redraw character */ ami_etredraw,
/** window redraw graphical */ ami_etredrawg,
/** window minimized */ ami_etmin,
/** window maximized */ ami_etmax,
/** window normalized */ ami_etnorm,
/** menu item selected */ ami_etmenus,
```

The etresize record carries rszx and rszy, the new size in characters, and rszxg and rszyg, the new size in pixels. The redraw events carry the update rectangle in rsx, rsy, rex and rey, characters for etredraw and pixels for etredrawg, and etmenus carries the selected menu item in menuid. The widget codes that follow these in the declaration belong to the widget level, described in the widgets chapter.

The last two are the user asking for what is displayed to be made larger or smaller, with control-+ and control--, the way the browsers take those keys. The window is not touched -- the user sets that themselves -- it is the material in it that changes, which for most programs means taking the point size a step up or down and laying out again.

7.23 C++ windows interface

The window object is one window, and any number of window objects may exist. Constructing one attaches or opens it, and destroying it closes what it opened:

```
window w; /* attach to the main window */
window c(&w); /* open a child of w */
window t(NULL); /* open an independent top level */
```

Every drawing and configuration call is a method -- c.cursorg(x, y), c.fcolor(blue), c.setpoints(12.0) -- and the object converts to FILE*, so the stdio calls and the C interface speak to it directly: fprintf(c, "score: %ld", n) writes to the window c holds.

Every event names its window, and the wrapper routes by that name: each window object receives, through its virtual functions, the events of its own window and no other. A subclass overrides what it answers --

```
class reader: public window {

public:

reader(window* p): window(p) { }
    long evredraw(long x1, long y1, long x2, long y2)
    { redrawreader(); return (1); }
    long evmouba(long m, long b) { openhit(); return (1); }

};
```

-- and returns nonzero to consume the event, zero to pass it on. Events for windows that have no object fall through to the ordinary event loop untouched, so a program can keep some windows as objects and drive others procedurally, window by window. This is what the single graph object cannot do: its virtuals receive every window's events without the window id, so with more than one window it cannot tell whose event it holds. The window object is the answer: which window is answered by being asked.

The main window may be held by a plain window object or by a subclass; the id and the file are had from id() and the FILE* conversion. Make a window before what is made on it, and let what is on it die first: in a subclass holding widgets as members that order takes care of itself, members being destroyed before their base. A window destroyed first disarms the widgets it still carries.

7.24 Procedures and Functions in windows

void [ami_]autobuf(FILE* f, long e);

Enables or disables the automatic scroll bars for a buffered window **f**. If **e** is true, the default, the window shows scroll bars when its buffer is larger than its client area, a vertical bar for a taller buffer and a horizontal one for a wider buffer, and the user scrolls the view over the buffer with them. If **e** is false, the window never shows them. The bars belong to the window manager, and the program receives no events from them. A program that keeps its buffer the size of its client area has no use for the bars, and turns them off so that they do not appear for a moment while the window shrinks. The option scrollbuffer under graphics in the configuration turns the bars off for every window.

void [ami_]back(FILE* f);

Sends the window **f** to the back of the parent Z order.

void [ami_]buffer(FILE* f, long e);

Engages or removes window **f** from buffered mode, according to the boolean **e**. In buffered mode, all of the drawing for a window is performed on a memory buffer, then copied to the screen. The screen view of the buffer can be all or part of the buffer, and multiple buffers can be managed.

If the buffer mode is enabled, the size of the buffer is set to the previous size.

void [ami_]dragwin(FILE* f);

Drags window f under pointer control: the window follows the mouse until the button is released. This serves a window that draws its own drag handle, a frameless dialog with its own title bar being the common case. Graphical systems only.

void [ami_]focus(FILE* f);

Sends the input focus to window f: the keyboard speaks to that window until the focus moves again.

void [ami_]frame(FILE* f, long e);

Enables or disables the appearance of the frame in window **f**, according to Boolean **e**, which includes the minimize, maximize, title, size, move and close controls. If the frame is removed, the user will be unable to operate the frame controls.

void [ami_]front(FILE* f);

Sends the window **f** to the front of the parent Z order.

void [ami_]getsiz[g](FILE* f, long* x, long* y);

Finds the size of a window in parent coordinate terms for window **f**, to size **x** and **y**. **getsiz()** returns the character size, and **getsizg()** returns the pixel size.

If the parent has no character mode, then one is created that is compatible with other **windows** functions and procedures.

long [ami_]getwinid(void);

Returns an unused window id. The ids returned are negative, so they never collide with the ids a program chooses for openwin(), which are positive. The id is reserved until used.

void [ami_]menu(FILE* f, [ami_]menuptr m);

Sets up the menu bar for window **f** from the given list **m**, which contains a menu bar definition structure.

If the menu pointer is NULL, then the menu is removed.

The menu data structure is copied during the call, so the menu structure can be reused or freed.

void [ami_]menuena(FILE* f, long id, long onoff);

Enables or disables an on/off menu button for window **f**. **id** refers to a button **id** that was specified in the menu data structure. If onoff is true, the button is enabled, otherwise disabled. The highlighting of the button will change to match.

void [ami_]menusel(FILE* f, long id, long select);

Sets the check state of a menu button in window f. id refers to a button id that was specified in the menu data structure. If select is true, the button is checked, otherwise unchecked. For a button in a one of group, checking it clears the other members of the group. The highlighting of the buttons will change to match.

void [ami_]openwin(FILE** infile, FILE** outfile, FILE* parent, long wid);

Opens a new window. The input file **infile** will get messages pertaining to the window, and the output file **outfile** will be used to draw to it. The input file may already be open elsewhere, in which case it will get all messages for all open windows opened with it. If a previous **infile** is not used, it must be NULL. The window will be placed at 1,1 within the parent and held out of display. The output file is always used as the window handle, since it is unique to the window.

The window identifier **id** is a number from 1 to n that is returned in messages concerning the window.

Only the output side of a window pair is used in procedures or functions that operate on the window once opened with **openwin()**.

A window is closed with the standard ANSI C fclose() procedure, used on the output file for the window. The input side of the window is automatically closed when there are no longer any windows that reference it.

If the optional parent window is specified, the new window is created as a child of the given parent, otherwise it must be NULL. This means that it will always be displayed within the parent, and be clipped to it.

void [ami_]scncen[g](FILE* f, long* x, long* y);

Finds the center of the screen holding window f, to x and y. scncen() answers in characters, scnceng() in pixels. On a single display this is the display center; where displays join, it is the center of the display the window occupies, so a dialog placed by it lands whole on one screen.

void [ami_]scnsiz[g](FILE* f, long* x, long* y);

Finds the size of the user screen or desktop for window **f** to the size **x** and **y**. **scnsiz()** returns the size in character terms, and **scnsizg()** returns the size in pixel terms. If the desktop does not have a character mode, an arbitrary scale is created. What is important is the relative location within the desktop.

void [ami_]sendevent(FILE* f, [ami_]evtrec* er);

Sends the event er to window f: the event is placed in the queue of the window's input side, and arrives as any user event would. The window id in the record is replaced with the window's own.

void [ami_]setpos[g](FILE* f, long x, long y);

Sets the position, within the parent, of a child window **f**, using position **x** and **y**. If the window is on the desktop, then the window position is relative to the desktop. The **setpos()** procedure sets the position in terms of characters, and the **setposg()** procedure set the position in terms of pixels.

The position is set in terms of the parent's coordinates. The mode of the parent's coordinates may not be known. For example, the desktop may not even have a character mode, so the idea of a character size may be arbitrary. What matters in this case is the relative position and size in the desktop as determined by **scnsiz()**.

void [ami_]setsiz[g](FILE* f, long x, long y);

Sets the size of window **f** in parent coordinate terms, to size **x** and **y**. **setsiz()** sets the character size, and **setsizg()** sets the pixel size.

If the parent has no character mode, then one is created that is compatible with other **windows** functions and procedures.

void [ami_]sizable(FILE* f, long e);

Enables or disables the sizing bars for a window **f**. If **e** is true, the sizing bar is enabled, otherwise disabled. If the frame is not enabled, then the size bars will not appear regardless of the sizable status, but it will be recorded. If the size bars are removed, the user will be unable to resize the window.

void [ami_]sizbuf[g](FILE* f, long x, long y);

Sets the size of the buffer used to draw into. **x** and **y** indicate the width and height, respectively, of the buffer surface in window **f**. It is an error if buffering is not enabled. **sizbuf()** sets the buffer size in characters. **sizbufg()** sets the buffer size in pixels.

void [ami_]stdmenu([ami_]stdmenusel sms, [ami_]menuptr* sm, [ami_]menuptr pm);

Constructs a standard menu and returns that in **sm**. **sms** contains the set of desired standard buttons. **pm** contains a menu containing non-standard buttons to be added to the menu. The menu is constructed using the desired standard buttons, and the non-standard buttons placed into the menu at a standard location.

If **pm** contains no menu items, it should be NULL.

void [ami_]sysbar(FILE* f, long e);

Enables or disables the system control bar for a window **f**. If **e** is true, the system bar is enabled, otherwise disabled. The system bar normally includes the title, minimum, maximum, and other control buttons. If the frame is not enabled, then the system bar will not appear regardless of the **sysbar()** status, but the size of it will be recorded.

void [ami_]title(FILE* f, char* ts);

Sets the title of the window **f** to the string **ts**. If the title is too long for the current window size, an implementation defined method will be used to make it fit, for example, it is clipped.

void [ami_]winclient[g](FILE* f, long cx, long cy, long* wx, long* wy, [ami_]winmodset ms);

Determines the window size needed for a given client size within window **f**. Given a desired client size of **cx** and **cy**, in width and height, the necessary window size to achieve that will be returned in wx and wy. **winclient()** determines these measurements in character dimensions, and **winclientg()** determines them in pixel terms.

The set of modes **ms** is used to determine the needed window size.

If the parent of the window has no character mode, then one is created that will be acceptable to **setpos()**.

7.25 Events In windows

See the description of the event record (7.22”Events”) for the format of the event record.

Event: [ami_]etresize

The window was resized. The new size in parent terms can be found with **getsiz(f, x, y)**.

Event: [ami_]etredraw

Signifies that the rectangle represented by the starting point in the upper left hand corner (**rsx**, **rsy**) and the ending point in the lower right hand corner (**rex**, **rey**) should be redrawn. This redraw can be satisfied by either redrawing just the rectangle, or by redrawing the entire window client area.

It is possible that a complex area needing to be redrawn could be sent as a series of redraw commands.

Event: [ami_]etredrawg

As etredraw, with the update rectangle in pixels. On a graphical system both arrive; obey one and ignore the other.

Event: [ami_]etmin

Signifies that the window was minimized. This may not need any action, but is simply for information.

Event: [ami_]etmax

Signifies that the window was maximized. This may not need any action, but is simply for information. If the window needs to be redrawn as a result of the change, a separate redraw event will be sent

Event: [ami_]etnorm

The window was normalized back to its original size. This may not need any action, but is simply for information. If the window needs to be redrawn as a result of the change, a separate redraw event will be sent.

Event: [ami_]etmenus

A menu item was selected. The **id** parameter gives which menu item was selected, which is a logical identifier number selected when the menu structure was constructed.

8 Widgets Library



terminal and **graphics** define terminal and graphical operations on a fixed screen. **windows** defines its division into "virtual windows". **widgets** provides elements placed within those windows to allow user control. These include buttons, sliders, scroll bars, checkboxes, and similar user interface elements.

These are sometimes referred to as controls or widgets. Dialogs are predefined windows with widgets in them. What these user elements have in common is they all use **windows** elements to define the window they appear in, and **graphics** or **terminal** routines to draw their appearance.

widgets can be performed entirely in terms of **windows** with **graphics** or **terminal** calls. However, it is still dependent on a particular operating system because of its appearance. **widgets** maintains the "look and feel" of a particular operating system or desktop.

8.1 Tiles, Layers and Looks

Each widget is implemented in its own window. A layout of widgets occurs when several widgets are tiled side by side, or placed on top of each other (layered). For example, a window surface with text and scrollbars to control the positioning of that text is done by constructing a series of windows. The first window is the window that holds the presented text. The second and third are the windows that hold horizontal and vertical scrollbars.

Layering is done by defacto transparency. A series of widgets are placed one atop the other. For example, a text window can be laid on top of a background window.

The key to interface design is to think of a window as simply a building block for construction of the user interface.

Widgets are fundamental to the "look" of an interface. Because **widgets** uses or emulates the native widgets on the operating system it serves, the client program will pick up quite a bit of that look from just the use of the widgets.

There is more to an application than just the look of the widgets. There are layout conventions, actions, and other intangibles. The rule that applies to Ami portability is:

- Ami programs will be able to target a high percentage of simple applications just by use of its normal **widgets** components.
- Ami should be able to finish a high percentage of the work to create complex applications.

The typical cycle in designing an Ami application is to design an initial version that uses just **widgets** components, then finish the design with special layouts, actions and colors for a particular operating system that give it the "look and feel" of a native application.

8.2 Background Colors and Placement

Widgets are placed by specifying their "bounding box" or rectangle. This is the rectangle that contains the widget. A widget can occupy all of the specified bounding box, or just a part of it. Typically, the operating system will try its best to format the widget to fit within the space provided.

The color scheme for widgets can vary. However, widgets are designed to be placed against a background color that is system dependent. This is available as a new system defined color, **backcolor**. It can be selected by the standard color selection routines.

```
typedef enum { ami_black, ami_white, ami_red, ami_green,
               ami_blue, ami_cyan, ami_yellow, ami_magenta,
               ami_backcolor } ami_color;

ami_fcolor(ami_backcolor);
ami_bcolor(ami_backcolor);
```

There are many ways to group widgets so that they blend into your background. The surface they appear on can be specifically colored using the background color, or you can create a child window with that color. Also, there is a background widget that can color the background for widgets automatically.

8.3 Sizes

Using widgets can go a long way towards getting the look and feel of a system in an independent way. The other important factor for achieving system independence is to control sizing of widgets. For each widget there is a size routine. The size routine takes the particular features of that widget, such as face text or borders, and gives the best size for the widget, given the desired client area, contents, etc..

The size information must be considered against the particular widget to be created. Sizing is key to establishing the layout of widgets in a window, and key to producing a truly portable application.

Sizing typically considers the minimum size of widget needed to contain text, whether the widget minimum is in the horizontal or veritical direction, etc.

8.4 Logical Widget Identifiers

A widget is created with a logical number, from 1 to n, where n is a positive integer. You specify the number you wish to create the widget under. The id of a widget cannot be the same as any other active widget, but widget logical identifiers can be reused by killing the widget first.

8.5 Killing, Selecting, Enabling and Getting Text to and from Widgets

A widget is killed by killwidget(f, id).

Some widgets can be selected, which changes their appearance to the select state. A widget is selected by **selectwidget(f, id, e)**.

A selected widget can be a checkbox that is checked, a radio button that is pushed, or similar effect. The processing of selects, and keeping track of the state of widgets is up to you. However, this is a very flexible system. You can implement widgets that flip their state when clicked on, a series of widgets that are mutually exclusive, and many other combinations.

Similar to selection, widgets can be enabled or disabled by **enablewidget(f, id, e).** Widgets are enabled by default. When a widget is disabled, it has the disabled appearance, such as greyed out. The widget will not give click events when it is disabled. Disabling a widget is typically used when it is not needed or not available in the current context. For example, a "next" button could be disabled when there is no next item to process.

Not every widget takes these. A select sets a flag on the widget, and what is made of it belongs to the widget: a checkbox draws a check, a button draws as pressed, and a widget with no select appearance records the state and shows nothing. Enabling and disabling is particular to each widget, and a widget with no disabled state answers with an error rather than quietly doing nothing, since a program that disables a widget means it to be unusable and had better hear that it is not. Each widget below says which of the two it accepts, and asking a widget for what it does not have may bring an error.

Some widgets have text that can be set, read, or in some cases, both. The text in a widget is set by putwidgettext(f, id, s). The text in a widget can be read by getwidgettext(f, id, s, sl).

Setting and getting the widget text is used with widgets that allow the user to modify the text, such as an edit box.

8.6 Resizing and repositioning a widget

To prevent the need to remove and replace widgets each time a window is resized, use **sizwidget[g](f, id, x, y)**. To reposition the widget in the parent window, **poswidget[g](f, id, x, y)** exists.

8.7 Types of widgets

Widgets come in three different types, controls, components and dialogs. Controls are widgets that the user can manipulate, and these widgets issue events to the program that owns them.

Components are display widgets whose only job is to form part of a display to the user. A group box, and a background are examples of components. Some components have active displays, such as the progress bar. Components never issue events.

Dialogs are fully autonomous windows that exist apart from the applications windows. They can be very complex inside, having a whole system of layout, widgets and other features. They resemble entirely separate programs.

Dialogs take a series of parameters when they are called, and deliver those same parameters back to the caller, with any modifications the user performs on the data.

8.8 Z ordering

widgets uses “implicit Z ordering” for layered widgets. In order for widgets to properly layer, the drawing Z order must be controlled. For example, a background must be drawn first, followed by whatever is on top of it, or the drawing of the background will wipe out what is on the surface of the background.

To enable this, the surface of a window is divided into “Z order domains”:

1. The drawing on the surface of the window.
2. Components.
3. Widgets and child windows.

For each domain, the Z order can only be specified within that domain. Components cannot appear behind the parent window or its drawing. Widgets and child windows cannot appear behind components, etc.

frontwidget(f, id) and **backwidget(f, id)** order widgets within their own layer, and are not a way to put a control under a component.

Components are not meant to lie over one another either. Two components in the same space have no order between them, and what appears is not defined.

8.9 Controls

8.9.1 Buttons

A button can be created with **button[g](f, x1, y1, x2, y2, s, id)**. The button is drawn in window **f**, in the specified rectangle (**x1**, **y1**) to (**x2**, **y2**) with a label text string **s**, and logical widget number **id**. The text string will be a single line of text with no control characters, and will be presented on the face of the button. The font style and size will be the same as other buttons in the operating systems user interface.



A button has different appearances when pressed vs. not pressed

When a button is pressed, it will typically change its appearance to indicate that. The button will send an event **etbutton**. This event carries the id of the button that was asserted. Similarly, when a button is released, it changes appearance back from the pressed state.

Buttons can be any size, any height and width. The button face text does not get larger with the button, but remains centered. However, if the button too small, some of the face text will be clipped off. The size of the button can be determined before creating it by **buttonsiz[g](f, s, w, h)**. The size will be the minimum required to fit the face string in width and height, plus a reasonable border margin.

Buttons cannot be selected, but they can be disabled. The text in a button can be neither read nor written.



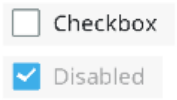
A disabled button.

8.9.2 Checkboxes

A checkbox is created with checkbox[g](f, x1, y1, x2, y2, s, id). When clicked, it gives a single event that indicates activation, **etchkbox**. The event contains the identifier of the widget.

Checkbox sizing is found with **checkboxsiz[g](f, s, w, h)**. Checkboxes are sized to minimum, but since they have no edges (like a button), there is typically no need to add space to them.

Checkboxes can be selected (checked). They can be enabled or disabled, and default to enabled. They cannot have their face text written or read.



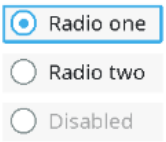
Checkboxes can be enabled, disabled or selected.

8.9.3 Radio Buttons

Radio buttons work identically to checkboxes, but have a different appearance. A radio button is created by **radiobutton[g](f, x1, y1, x2, y2, s, id).** Radio buttons, when hit, give a single event that indicates activation, **etradbut**. This event contains the id of the widget.

Radio button sizing is found with **radiobuttonsiz[g](f, s, w, h)**. Radio buttons are sized to minimum, but since they have no edges (like a button), there is typically no need to add space to them.

Radio buttons can be selected (checked). They can be enabled or disabled, and default to enabled. They cannot have their face text written or read.



Radio buttons can be enabled, disabled or selected.

8.9.4 Scrollbars

Scrollbar widgets are free floating, and can appear anywhere in the window, not just the sides. In addition, the height and width of them can be controlled, instead of being fixed to the window size.

Scrollbars are placed vertically by **scrollvert[g](f, x1, y1, x2, y2, id)**. Scrollbars are placed horizontally by **scrollhoriz[g](f, x1, y1, x2, y2, id)**.



Horizontal and vertical scrollbars.

Scroll bars can be placed using any dimensions, but the width of a vertical scroll bar, and the height of a horizontal scroll bar usually has a standard size. These can be determined by **scrollvertsiz[g](f, w, h)** and **scrollhorizsiz[g](f, w, h).**

A user movement of a scrollbar is given by the event **etsclpos**. This event does not move the scrollbar slider. This must be done by the program via **scrollpos(f, id, p)**.

When a user positions the scroll bar directly, it will follow the users mouse movements. However, it will return to the original position unless the **etsclpos** event is responded to and a **scrollpos(f, id, p)** call is made.

A slider moves itself and a scroll bar does not, and the difference is in what the two stand for. A slider carries a value, and where the user puts it is the answer. A scroll bar stands for a view onto something the program holds, and only the program knows what a movement comes to: how far the material scrolls, whether it can scroll that far, and how large the slider should be once it has. Where the user drops the slider is plain enough. What it means is not, so the scroll bar waits to be told.

Besides position events, scrollbars issue two types of button pushes, referred to as line and page button events. The line buttons are the arrow buttons at either side of the scroll bar. The page buttons are the space between the scrollbar slider and the line arrows. A press anywhere in this area generates a page button event.

The page button area may not appear at all if the slider is fully to one side of the scrollbar.

The page and line terminology for these buttons occurs because their most common use is to scroll documents. In this case, the line buttons would be used to move the document one line up or down. In the case of horizontal scrollbars, this would be one character left or right (despite the term "line" in the buttons name). The page refers to one screenful of text, in any direction. If the page up button is hit, for example, the document would move one screenful up.

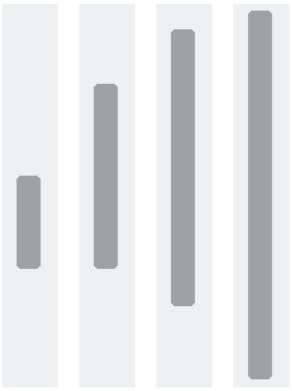
It's up to the program to implement the actions for line up/down and page up/down. In fact, these events can be used for any purpose in client programs.

Besides the position of the slider, its size can also be controlled by **scrollsiz(f, id, s)**. The value of s is ratioed from 0 to **LONG_MAX**.

The standard use of the scrollbar size is to indicate what proportion of the document or display is contained within the onscreen display, vs. the total size of the document. For example, if the document has 50% of its total displayed, then the size of the scrollbar is set to 50%, which is done by:

```
ami_scrollsiz(f, n, LONG_MAX/2);
```

Scroll bars cannot be selected, enabled or disabled, or have face text read or written.



Scrollbars with sliders of various sizes.

8.9.5 Number Select Box

A number can be selected in an edit box by **numselbox[g](f, x1, y1, x2, y2, l, u, id)**.

The first number that appears in the number select box is by default the lower bound.

Number select boxes are an easy way to enter numbers from the user, and automatically restrict the input to numbers only. The numbers are entered in decimal, and cannot be negative. The edit box allows the number to be directly edited. Also, there are up and down buttons that allow the user to count the number up or down by one.



A number select box.

The size of a number select box is found by **numselboxsiz[g](f, l, u, w, h)**.

Number select boxes cannot be selected, enabled or disabled. The number standing in the box is the face text, and is read with **getwidgettext(f, id, s, sl)** and written with **putwidgettext(f, id, s)**. A number select box opens showing its lower bound.

8.9.6 Edit Boxes

A general string can be edited with **editbox[g](f, x1, y1, x2, y2, id)**.

An empty edit box is placed, and the user has the ability to edit text into the box, with cursor movements, character delete, etc.



An edit box.

An edit box can be presented blank, or default text can be placed into the edit box. If the user presses enter to the box, it sends a **etedtbox** event. However, the program can use any method to signal done, such as a button next to the edit control. The resulting text can then be retrieved from the edit box.

The size of an edit box is found by **editboxsiz[g](f, s, w, h)**.

Edit boxes cannot be selected, enabled or disabled.

8.9.7 List Boxes

A list box is a series of items that can be selected. It is placed with **listbox[g](f, x1, y1, x2, y2, sp, id)**, where **sp** is a list of strings to display.

The string list definition appears as:

```
/* string set for list box */
typedef struct ami_strrec* ami_strptr;
typedef struct ami_strrec {

    ami_strptr next; /* next entry in list */
    char*     str;  /* string */

} ami_strrec;
```

The string pointer is a list of strings, each string of which describes an entry in the list box.

When the user selects an item from the list box, the **etlstbox** event is returned. This event gives the id of the widget, and the number of the select, from the top. The first item in the list will be 1, the second 2, etc.

The size of a list box is found by **listboxsiz[g](f, sp, w, h)**.



A list box.

List boxes cannot be selected, enabled or disabled, or have face text read or written.

8.9.8 Drop Down Box

The same multiple string selection can be done in a different way by **dropbox[g](f, x1, y1, x2, y2, sp, id)**.

A drop box only shows the whole list if the user selects it, otherwise only the currently selected entry is shown. The full list "drops down" from the selection box when the user selects it. A drop box takes less space than a list box when it is not selected. A drop box selection is signaled by the **etdrpbox** event, which gives the widget id, and the number of the selection, from 1 to n.



A drop box, in both closed and open states.

The size of a drop box is found by **dropboxsiz[g](f, sp, cw, ch, ow, oh)**. Because drop boxes have two appearances, one when open, and one when closed, both sizes are returned. The closed appearance gives the basic size of the widget, but the open size makes it possible to determine if the list will go beyond the edge of the window when open.

Drop boxes cannot be selected, enabled or disabled.

8.9.9 Drop/Edit Box

Very similar to a drop box, a drop/edit box allows selection from a list, but also allows the current selection string to be edited.

A drop/edit box is placed with **dropeditbox[g](f, x1, y1, x2, y2, sp, id)**. When a selection is made from the drop/edit box, the **etdrebox** event is sent, which includes the widget identifier. The selection data itself is a string, and is retrieved with **getwidgettext(f, id, s)**.





A drop/edit box in both closed and open states.

The size of a drop/edit box is found by **dropeditboxsiz[g](f, sp, cw, ch, ow, oh)**. Because drop/edit boxes have two appearances, one when open, and one when closed, both sizes are returned. The closed appearance gives the basic size of the widget, but the open size makes it possible to determine if the list will go beyond the edge of the window when open.

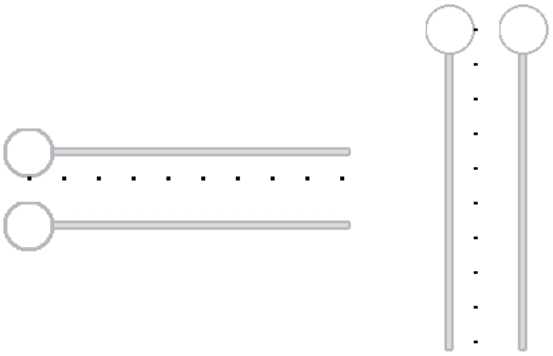
Drop/edit boxes cannot be selected, enabled or disabled.

8.9.10 Sliders

Sliders are linear controls that can be placed either horizontally or vertically.

A vertical slider is placed with **slidevert[g](f, x1, y1, x2, y2, m, id).** A horizontal slider can be placed by **slidehoriz[g](f, x1, y1, x2, y2, m, id).** The **m** parameter gives the number of tick marks to place on the slider, which could be zero.

Sliders indicate changes in their position with the event **etsldpos**. This gives the widget id, and a LONG_MAX ratioed position of the slider, from 0 to LONG_MAX. 0 is the top or leftmost position of the slider, and LONG_MAX is the bottom or rightmost position of the slider.



Horizonal and vertical sliders, both with and without tick marks.

The sizes of sliders are determined by slidevertsiz[g](f, w, h) and slidehorizsiz[g](f, w, h).

Sliders cannot be selected, enabled or disabled, or have face text read or written.

8.9.11 Tabbed Bars

Tab bars allow the user to select from a series of labeled tabs, usually to specify locations in a document. A **tabbar** is a group box with tabs on one edge. The tabs can be placed on the top, bottom, left or right side of the included client area.

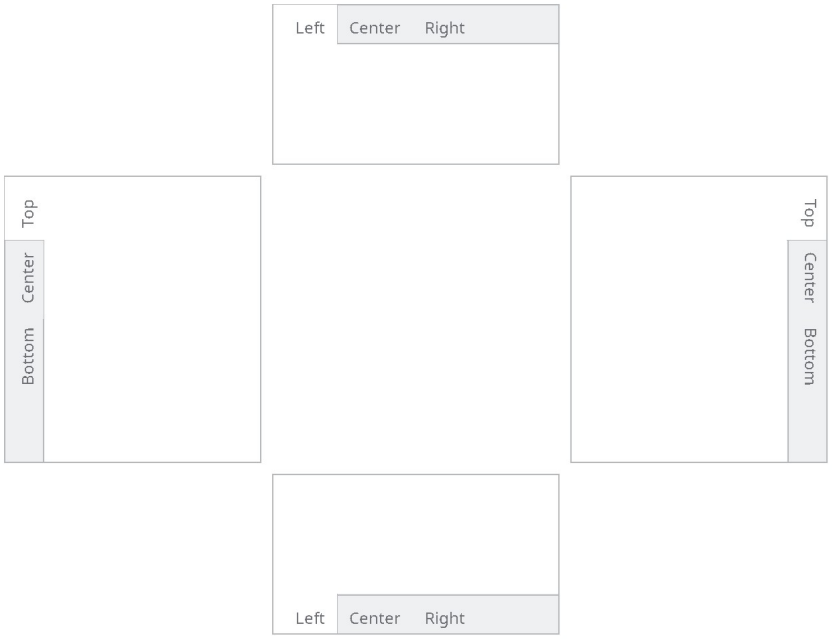
A tabbar is placed by **tabbar[g](f, x1, y1, x2, y2, sp, tor, id)**.

Tabbar selections are indicated by the event **ettabbar**, which gives the widget id, and the tab number selection, from 1 to n, counting from the first string entry in the list.

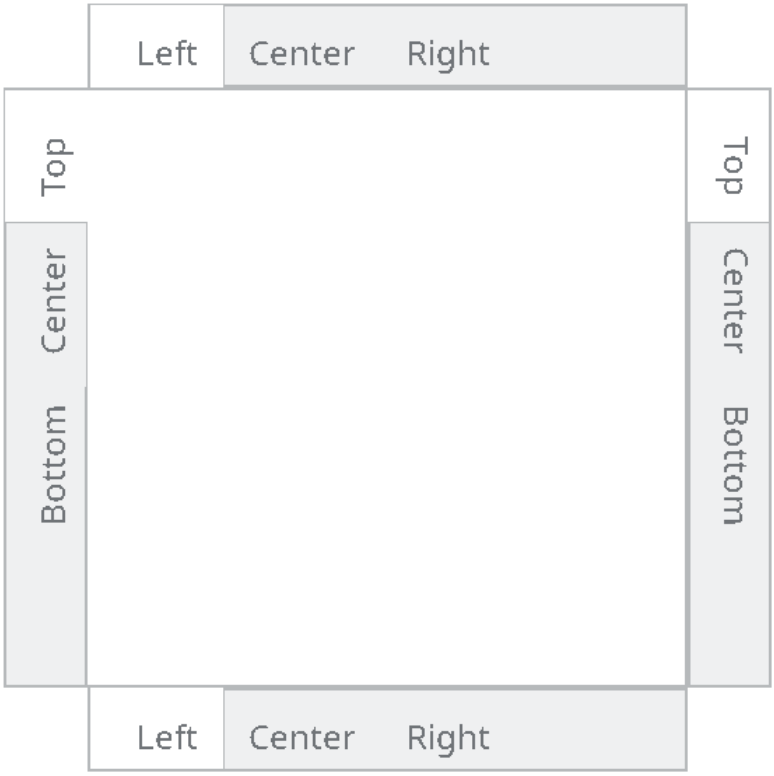
The size of a tabbar is found by **tabbarsiz[g](f, sp, tor, cw, ch, w, h, ox, oy)**, where **sp** is the same list of tab strings the tabbar itself is given. A **tabbar** acts like a group box, and has a client area to place child windows or widgets. The required client size can be specified, and the sizing call returns the offset required to find the client location within the **tabbar**.

If the **tabbar** must fit into a fixed window size, the size of the resulting client for a **tabbar** can be found with **tabbarclient[g](f, tor, w, h, cw, ch, ox, oy)**. This returns the client width and height, and its offset from the origin of the **tabbar**.

Tab bars cannot be selected, enabled or disabled, or have face text read or written.



Separate left, top, right and bottom tab bars.



Overlaid tab bars that share a client area.

When a tab bar with tabs on more than one side is needed, several tab bars with different orientations can be overlaid.

```
/* orientation for tab bars */
typedef enum { ami_totop, ami_toright, ami_tobottom, ami_toleft }
ami_tabori;
```

8.10 Components

8.10.1 Background Box

A background box is placed by **background[g](f, x1, y1, x2, y2, id)**. A background box is designed to serve as the background to a series of controls, and it has the standard color for such backgrounds.

A background component. It is more useful than a rectangle because it draws itself.

Background boxes have no sizing, because there are no borders or other content. They are just a colored rectangle. Background boxes cannot be selected, enabled or disabled, or have face text read or written.

8.10.2 Group Boxes

A group box is similar to a background box, but it has a label for the "group" of controls contained within it. It is placed by **group[g](f, x1, y1, x2, y2, s, id)**.



A group box, which is a labeled background.

The size of a group box is found by **groupsiz[g](f, s, cw, ch, w, h, ox, oy)**. A group box has a client area to place child windows or widgets. The required client size can be specified, and the sizing call returns the offset required to find the client location within the group box.

Group boxes cannot be selected, enabled or disabled, or have face text read or written.

8.10.3 Progress Bars

A progress bar is used to indicate the progress of a job completion, like installing software, saving a file, etc.

It is placed by **progbar[g](f, x1, y1, x2, y2, id)**. The initial progress indication is zero when placed.



A progress bar.

The size of a progress bars can be determined by **progbarsiz[g](f, w, h)**.

The position of the progress bar is set by **progbarpos(f, id, pos)**. The position is a ratio of LONG_MAX, from 0 for an empty bar to LONG_MAX for a full one, the same scale a scroll bar and a slider are placed on. A position below zero is an error.

Progress bars cannot be selected, enabled or disabled, or have face text read or written.

8.11 Dialogs

A dialog is a completely separate window which is preformatted with widgets. Dialogs introduce complex queries into a program, using the look of the native operating system.

Dialogs display a property known as modality. Since the dialog is a separate window, it can be independent of the other windows created by the calling task, or the dialog can be forced to appear at the top of the applications stacking order.

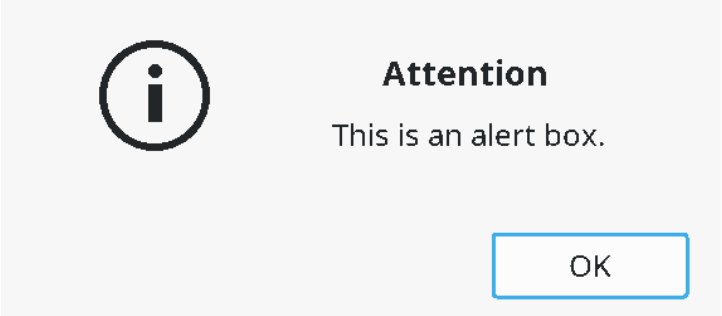
widgets uses two types of modality. If the task that created the dialog also created other windows, the dialog is forced to the front of the other windows, and selection of the other task windows is disabled. This indicates to the user that only the dialog is currently being managed by the task.

If windows are created by different threads, then the dialog will not be modal vs. the other thread's windows. This reflects the fact that the windows outside the dialog can run while the dialog does.

8.11.1 Alert Dialogs

An alert dialog is used to send errors or other important messages to the user. It has a window title, a message that constitutes the alert, and typically has an "ok" or "close" button for the user to indicate the user has seen it.

An alert is created by **alert(title, msg).** The alert call will not return until the user has clicked the OK button for the alert.



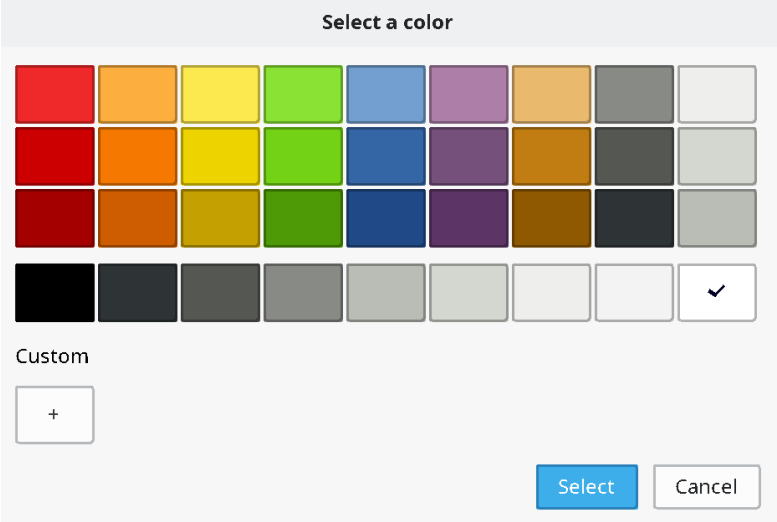
An alert box. One of the simplest and most useful widgets.

The query dialogs allow the user to select important information such as a file, a search string, or color or a font. They use a model called "flow through". The query may select several types of information, and will accept a default setting, allow the user to select a new setting, and return that. The flow through model means that several parameters are set up before the call, may or may not be modified by the query, then are returned to the caller. The calling thread stops until the dialog is completed by the user.

All of the parameters of a dialog may or may not be implemented in the actual system dialog. The flowthrough system allows for differences in systems. Since the variables are preinitialized with the defaults, if the dialog does not implement a particular parameter, the value will be left unchanged.

8.11.2 Color Query Dialogs

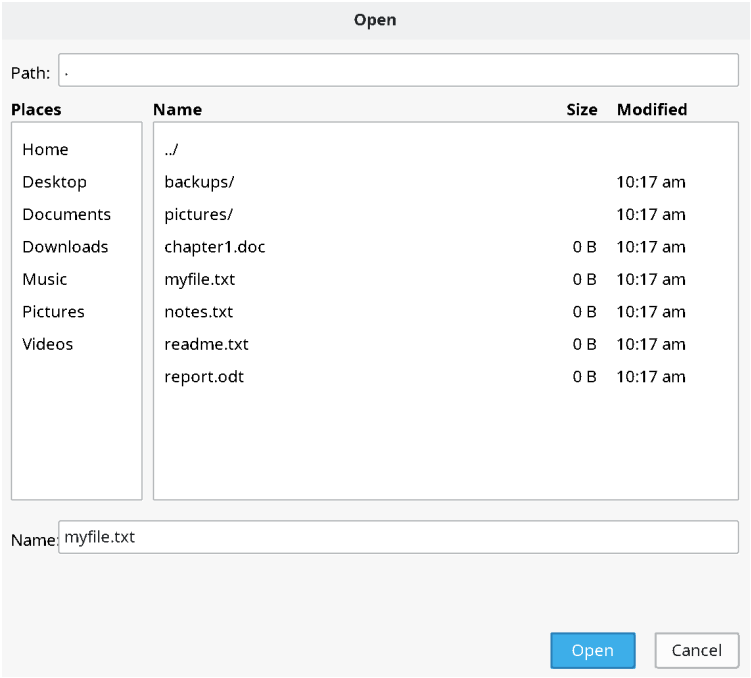
A color can be chosen by **querycolor(r, g, b).** The default color is set before the call, and the possibly changed color is returned by the call.



A color query dialog.

8.11.3 Open File Query Dialogs

A file to open name is selected by **queryopen(s, sl)**.

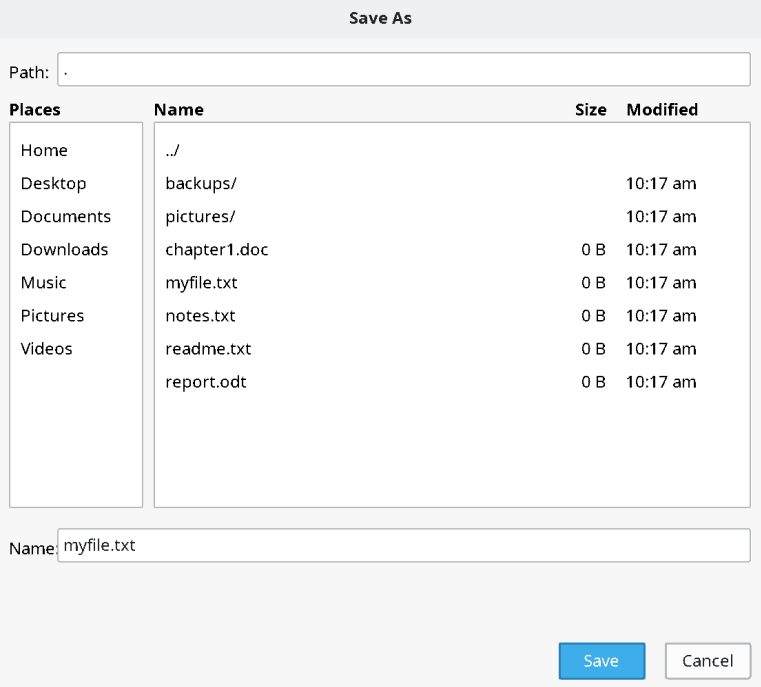


A file open dialog.

The default filename is passed in **s**, and the resulting filename string returned in **s** with maximum length **sl**. If the dialog is canceled instead of completed by the user, the string returned is empty. This means to not proceed with the open operation.

8.11.4 Save File Query Dialogs

A file to save name is selected by **querysave(s, sl)**.

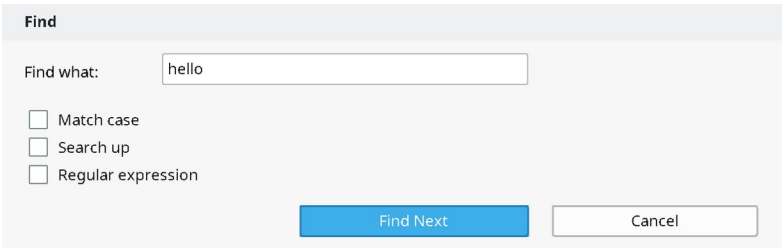


A file save dialog.

The default filename is passed in **s**, and the resulting filename string returned in **s** with maximum length **sl**. If the dialog is canceled instead of completed by the user, the string returned is empty. This means to not proceed with the save operation.

8.11.5 Find Query Dialogs

A string to search for is selected by **queryfind(s, sl, opt)**.



A find dialog.

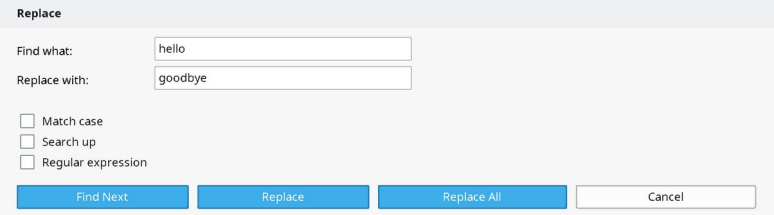
The option flags are given by a set of flags:

```
/* settable items in find query */
typedef enum { ami_qfncase, ami_qfnup, ami_qfnre } ami_qfnopt;
typedef long ami_qfnopts;
```

The default search string is passed in s, and the resulting search string returned in **s** with maximum length **sl**. If the dialog is canceled instead of completed by the user, the string returned is empty. This means to not proceed with the search operation.

8.11.6 Search and Replace Query Dialogs

A string to search for and replace is selected by **queryfindrep(s, sl, r, rl, opt)**.



A find/replace dialog.

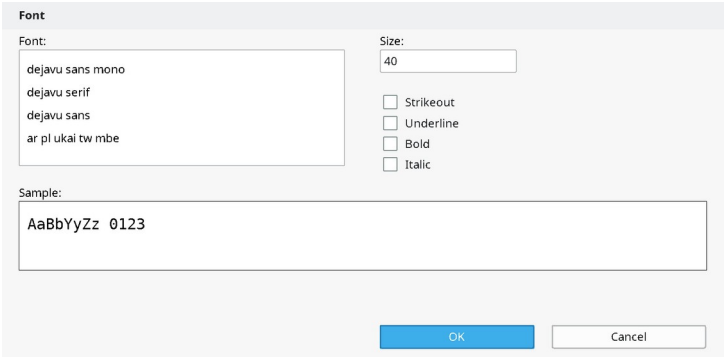
The option flags are given by a set of flags:

```
/* settable items in replace query */
typedef enum { ami_qfrcase, ami_qfrup, ami_qfrre, ami_qfrfind,
ami_qfrallfil, ami_qfralllin } ami_qfropt;
typedef long ami_qfropts;
```

The default search string and replacement strings are passed in **s** and **r**, and the resulting search string is returned in **s** with maximum length **sl**, and the resulting replacement string is returned in **r** with maximum length **rl**. If the dialog is canceled instead of completed by the user, both strings are returned empty. This means to not proceed with the search/replace operation.

8.11.7 Font Query Dialogs

Fonts are selected by **queryfont(f, fc, s, fr, fg, fb, br, bg, bb, effect)**.



A font query dialog.

The font effects are declared as:

```
/* effects in font query */
typedef enum { ami_qfteblink, ami_qftereverse, ami_qfteunderline,
               ami_qftesuperscript, ami_qftesubscript,
```

```
                ami_qfteitalic, ami_qftebold,ami_qftestrikeout,
                ami_qftestandout, ami_qftecondensed, ami_qfteextended,
                ami_qftexlight, ami_qftelight, ami_qftexbold,
                ami_qftehollow, ami_qfteraised
              } ami_qfteffect;
typedef long ami_qfteffects;
```

8.12 Events

The definition of an event record is upward compatible with the previous event record declarations from terminal, graphics and windows: the event type and the record are those declared before (4.10 “Advanced Input”, 6.21 “Declarations”, 7.21 “Events”). The widget level adds its own codes:

```
/** button assert */ ami_etbutton,
/** checkbox click */ ami_etchkbox,
/** radio button click */ ami_etradbut,
/** scroll up/left line */ ami_etsclull,
/** scroll down/right line */ ami_etscldrl,
/** scroll up/left page */ ami_etsclulp,
/** scroll down/right page */ ami_etscldrp,
/** scroll bar position */ ami_etsclpos,
/** edit box signals done */ ami_etedtbox,
/** number select box done */ ami_etnumbox,
/** list box selection */ ami_etlstbox,
/** drop box selection */ ami_etdrpbox,
/** drop edit box done */ ami_etdrebox,
/** slider position */ ami_etsldpos,
/** tab bar select */ ami_ettabbar,
```

and their fields in the event record:

```
/** etbutton */ long butid; /* button id */
/** etchkbox */ long ckbxid; /* checkbox id */
/** etradbut */ long radbid; /* radio button id */
/** etsclull */ long sclulid; /* scroll up/left line id */
/** etscldrl */ long scldrid; /* scroll down/right line id */
/** etsclulp */ long sclupid; /* scroll up/left page id */
/** etscldrp */ long scldpid; /* scroll down/right page id */
/** etsclpos */ long sclpid, sclpos; /* scroll bar id and position */
/** etedtbox */ long edtbid; /* edit box complete id */
/** etnumbox */ long numbid, numbsl; /* number box id and value */
/** etlstbox */ long lstbid, lstbsl; /* list box id and select */
/** etdrpbox */ long drpbid, drpbsl; /* drop box id and select */
/** etdrebox */ long drebid; /* drop edit box complete id */
/** etsldpos */ long sldpid, sldpos; /* slider id and position */
/** ettabbar */ long tabid, tabsel; /* tab bar id and select */
```

The scroll bar and slider positions are LONG_MAX ratios. The display size events etusize and etdsize belong to the graphical level, described with graphics (6.21 “Declarations”).

8.13 C++ widgets interface

A widget is made by constructing one of the typed widget classes on a window: button, checkbox, radiobutton, group, background, scrollvert, scrollhoriz, numselbox, editbox, progbar, listbox, dropbox, dropeditbox, slidehoriz, slidevert and tabbar. The logical id is allocated for you unless given, and the destructor kills the widget. The common operations are methods of every widget: select(), enable(), gettext(), puttext(), pos(), siz(), back(), front() and kill().

Widget events are routed to the widget object first. Each typed widget has the virtuals of its kind -- a button's pressed(), a checkbox's clicked(), an edit box's done(), a list box's selected(item), a

slider's moved(position), a scroll bar's upline(), downline(), uppage(), downpage() and moved() -- and an override that returns nonzero consumes the event. Unhandled, it goes to the window's virtual with the widget id -- evbutton(id) and its kin -- and then to the ordinary event loop, so a program may answer its widgets at the widget, at the window, or in the loop, as suits its size.

The pieces compose into the shape a custom dialog wants: a window subclass with its widgets as members, made in the constructor, answered in the virtuals, and taken down in the right order by C++ itself:

```
class serverdlg: public window {

public:

editbox host;
button ok;

serverdlg(window* p): window(p),
    host(*this, 20, 20, 300, 60),
    ok(*this, 20, 80, 120, 130, "ok")
{

    title((char*)"Server");

}

long evbutton(long id)
    { char s[100]; host.gettext(s, 100); use(s); return (1); }

};
```

One rule of order: make a widget's window before the widget, and let the widget die before its window. The member pattern above does both by itself: members are constructed after their base and destroyed before it. And one naming note: the widget creators are the classes, not free functions of the namespace -- C++ will not let a class and a function share a name -- while the C interface keeps its creator functions unchanged.

8.14 Procedures and functions in widgets

void [ami_]alert(char* title, char* message);

Creates an alert dialog, with window title **title**, and client message **msg**. The alert dialog is a freestanding window that is placed at the front of the desktop stacking order. It is generally used to display an error, warning or other attention condition. The window title should tell the user what application is generating the alert, and the client message should give the error or warning. The user dismisses the dialog, and the program holds until the dialog completes.

void [ami_]background[g](FILE* f, long x1, long y1, long x2, long y2, long id);

Creates a background box in the window **f**, with the bounding rectangle **x1**, **y1**, **x2**, **y2**, and the logical widget identifier **id**. A background is simply a rectangle with the standard background color. It is more convenient than simply painting a rectangle on the window with the background color because it handles its own redraws. Because a background box has no borders, widgets can be placed within it anywhere, and in any number. Any widgets to be placed within the group should be created after the group box is created, so that that they are on top of the group box in stacking order.

void [ami_]backwidget(FILE* f, long id);

Sends the widget with logical identifier id in window f to the back of the widget stacking order.

void [ami_]button[g](FILE* f, long x1, long y1, long x2, long y2, char* s, long id);

Places a button within window **f** in the bounding box formed by **x1**, **y1**, **x2**, **y2**, with the face text from the string **s**. The logical identifier is specified in **id**, which must be an integer from 1 to n, which is not currently in use in any other widget. The button drawn is not guaranteed to completely fill the bounding box. The button should be placed against a background that is colored with the standard background color. The button will be placed at the front of the window stacking order, and should be placed after any other widgets or child windows that the button is to appear in front of.

When the button is pressed, it will send an **etbutton** event, which contains the logical id of the button that was pressed. It is up to the system whether the event occurs when the button is depressed or released.

Buttons cannot be selected, or have their face text changed or read. Buttons can be enabled and disabled. If the button is disabled, it will not send **etbutton** events when pressed.

void [ami_]buttonsiz[g](FILE* f, char* s, long* w, long* h);

Finds the minimum size of a button within window **f**, with face text string **s**. The width to use is returned in **w**, and the height in **h**. Button sizing returns the minimum size in terms of the minimum amount of space needed to contain the face text, in the labeling font, and the border of the button itself. You will want to use the size as a guide to button sizing, and not for the actual size of the button. A good rule of thumb is to add 25% of the minimum height of the button to the actual height and width of the button to give the user sufficient area to click on the button. In addition, buttons should be "justified" when they appear in groups to appear as the same width. This is done by calculating a maximum size for a group of related buttons, then using the same width and height for all of them.

void [ami_]checkbox[g](FILE* f, long x1, long y1, long x2, long y2, char* s, long id);

Places a checkbox within window **f** in the bounding box formed by **x1**, **y1**, **x2**, **y2**, with the face text from the string **s**. The logical identifier is specified in **id**, which must be an integer from 1 to n, which is not currently in use in any other widget. The checkbox drawn is not guaranteed to completely fill the bounding box. The checkbox should be placed against a background that is colored with the standard background color. The checkbox will be placed at the front of the window stacking order, and should be placed after any other widgets or child windows that the checkbox is to appear in front of.

When the checkbox is clicked, it will send an **etchkbox** event, which contains the logical id of the checkbox that was pressed. It is up to the system whether the event occurs when the checkbox is depressed or released.

Checkboxes cannot have their face text changed or read. A checkbox can be selected or deselected, and can be enabled and disabled. If the checkbox is selected, it will appear with a selected face, which is typically a checkmark in a box. If the checkbox is disabled, it will not send **etchkbox** events when pressed.

A checkbox only changes its appearance in response to a select, and does not keep a state that can be read by the program. It's up to the program to keep track of the state of the checkbox, and how to handle it. In particular, if the **checkbox** is pressed, it is up to the program to change its select status, otherwise the press will have no effect. The program can implement many different effects for checkboxes. The **checkbox** can toggle, or it can be one of a series of mutually exclusive selections.

void [ami_]checkboxsiz[g](FILE* f, char* s, long* w, long* h);

Finds the minimum size of a checkbox within window **f**, with face text string **s**. The width to use is returned in **w**, and the height in **h**. Checkbox sizing returns the minimum size in terms of the minimum amount of space needed to contain the face text, in the labeling font, and the border of the checkbox itself, if it exists. You will want to use the size as a guide to checkbox sizing, and not for the actual size of the checkbox. A good rule of thumb is to add 25% of the minimum height of the checkbox to the actual height and width of the checkbox to give the user sufficient area to click on the checkbox.

void [ami_]dropbox[g](FILE* f, long x1, long y1, long x2, long y2, [ami_]strptr sp, long id);

Creates a dropbox in the window **f**, within rectangle **x1**, **y1**, **x2**, **y2**, for string list **sp**, with logical identifier **id**. The bounding rectangle for a drop box specifies its open mode, where the user has selected and dropped down the list of items. If the size specified is greater than or equal to the open size, as determined by **dropboxsiz()**, then the entire dropbox will be presented. If the size is between the closed size and the open size, the system will attempt to work around the fact that the entire list cannot be dropped down. This is typically done by allowing the user to scroll though the list. If the size is less than the closed size, the dropbox will be clipped.

When a string within the drop box is selected, it will send an **etdrpbox** event. It contains the sequential number of the string that was selected. For example, the first string sends 1, the second in the list sends 2, etc.

void [ami_]dropboxsiz[g](FILE* f, [ami_]strptr sp, long* cw, long* ch, long* ow, long* oh);

Finds the size of a drop box for window **f**, with string list sp. The closed width is returned in **cw**, and the closed height in **h**. The open width is returned in **ow**, and the height in **oh**. Drop boxes are used to display a list of selections as in a listbox, but they occupy less space than a listbox. Dropboxes have two bounding rectangles, one is its dimensions when closed, and another when the user drops it down, or opens it. Both sizes are returned. This allows layout planning for both modes of the widget. Generally, the closed size is used to plan placement, then the open size is used to check if the widget will extend past the edges of the window when open.void [ami_]dropeditbox[g](FILE* f, long x1, long y1, long x2, long y2, [ami_]strptr sp, long id);

Creates a drop edit box in the window **f**, within rectangle **x1**, **y1**, **x2**, **y2**, for string list **sp**, with logical identifier **id**. The bounding rectangle for a drop edit box specifies its open mode, where the user has selected and dropped down the list of items. If the size specified is greater than or equal to the open size, as determined by **dropboxsiz()**, then the entire dropbox will be presented. If the size is between the closed size and the open size, the system will attempt to work around the fact that the entire list cannot be dropped down. This is typically done by allowing the user to scroll though the list. If the size is less than the closed size, the dropbox will be clipped.

When a drop box string is selected, or enter is hit while editing, it sends the event **etdrebox**. There is no other information associated with this event. Since the text is editable, it could be anything, and may not match one of the list entries. Instead, the program should use **getwidgettext()** to retrieve the result of the edit.

Drop edit boxes default to a blank edit string. The idea of the drop edit box is that the user can simply use it as an edit box to enter the needed data, or drop down a list of preselected items. If you wish to make one of the string list items the default, or even a text that is not on the list, use the **putwidgettext()** to initialize the edit field.

void [ami_]dropeditboxsiz[g](FILE* f, [ami_]strptr sp, long* cw, long* ch, long* ow, long* oh);

Finds the size of a drop edit box for window **f**, with string list sp. The closed width is returned in **cw**, and the closed height in **ch**. The open width is returned in **ow**, and the height in **oh**. Drop edit boxes are used to display a list of selections, and acts as a combination of a list and edit box, but they occupy less space than a listbox. Drop edit boxes have two bounding rectangles, one is its dimensions when closed, and another when the user drops it down, or opens it. Both sizes are returned. This allows layout planning for both modes of the widget. Generally, the closed size is used to plan placement, then the open size is used to check if the widget will extend past the edges of the window when open.

void [ami_]editbox[g](FILE* f, long x1, long y1, long x2, long y2, long id);

Creates an edit box for window **f**, in rectangle **x1**, **y1**, **x2**, **y2**, with logical identifier **id**. Edit boxes can be used to allow the user to enter any text. The text within an edit box can be set by **putwidgettext()**, and retrieved by **getwidgettext()**. This can occur at any time. When the user presses enter in the edit box, it sends an **etedtbox** event. The program can then retrieve the text from the edit box.

void [ami_]editboxsiz[g](FILE* f, char* s, long* w, long* h);

Finds the size of an edit box for window **f**, with face text string **s**. The width is returned in **w**, and the height in **h**. The string passed is a dummy, and will not be used for any purpose other than as a reference to determine the width of the required edit box. The string should contain text that is representative of the string to be edited. This could be the string that you plan to place in the edit box as its default, or it could be the worst case contents of the edit box. For example, if 8 characters is the planned edit width, the string "WWWWWWWW" (8 "W" characters) would be the largest width of edit possible. The edit box is sized to be the minimum appropriate, and can be used without extra added space.

void [ami_]enablewidget(FILE* f, long id, long e);

The widget within window **f** by the logical identifier **id** will enter the enable state if **e** is true, otherwise the disable state is entered. The exact effect of the enable or disable state depends on the widget. See the individual widget to be selected for more information. It is an error to enable or disable a widget that has no such capability. The default state for all widgets is to be enabled. A widget that is disabled will stop sending events.

Disabling a widget is typically used to mark it as unusable or invalid in the current context. An example would be a "next" button where no next page or item exists. The standard method used to indicate disabled widgets is to give them a "greyed out" appearance, but the actual effect may depend on the operating system.

void [ami_]frontwidget(FILE* f, long id);

Sends the widget with logical identifier id in window f to the front of the widget stacking order.

void [ami_]getwidgettext(FILE* f, long id, char* s, long sl);

Retrieves the text contained by the widget with the logical identifier **id** within window **f**, and returns the text in string **s**, whose buffer length is **sl**. It depends on the widget as to if it has text that can be read. It is an error to get text from a widget that has no such capability.

Widgets can typically have their text read if the widget provides the user with the ability to modify or edit text. In this case, retrieving the text is required to obtain the new text.void [ami_]group[g](FILE* f, long x1, long y1, long x2, long y2, char* s, long id);

Creates a group box in window **f** within the bounding rectangle **x1**, **y1**, **x2**, **y2**, the face text given in string **s**, and with the logical identifier **id**. Group boxes are containers for other widgets, and consist of a border area, the face text, and an internal client area where other widgets are to be placed.

The entire client area of the group will have the standard background color, and widgets can be placed into the client in any arrangement or number. The program should be sure to create the client area widgets after the group is created, so that they will appear in front of the group in stacking order.

The location of the client area within a group box can be found with the group sizing call.

void [ami_]groupsiz[g](FILE* f, char* s, long cw, long ch, long* w, long* h, long* ox, long* oy);

Finds the required size of a group box in window **f**, with the face text given in string **s**, and the client area width **cw**, and client area height **ch**. The required width is returned in **w**, the height in **h**, and the offset to the client area in **ox** and **oy**. A group box consists of a border area, a label, and an internal client area. Group boxes are designed to be layered components. They contain other widgets, and provide a background for them. When a group size is found, the minimum size is found as what will contain all of the border, face text and the requested client area. The program will know where to place its widgets in the client area by the client offset, which is given as a difference between the bounding box origin, and the origin of the client rectangle.

void [ami_]killwidget(FILE* f, long id);

The widget within window **f** with the logical identifier **id** is removed from the system. It will be erased from the screen, and contents under it will be restored as required.void [ami_]listbox[g] (FILE* f, long x1, long y1, long x2, long y2, [ami_]strptr sp, long id);

Creates a listbox for window **f**, in rectangle **x1**, **y1**, **x2**, **y2**, with string list **sp**, and logical identifier **id**. A listbox contains a series of strings that can be selected by the user. When the user clicks a string, the event **etlstbox** will be sent, which contains the number of the selected string in list order. For example, the first string in the list would be 1, and second string in the list 2, etc. If there is not enough room in the height of the listbox for all strings in the list to be presented, then the widget will use a compression method to fit the available space. This is typically done by allowing the user to scroll through the selections. If there is not enough width for the strings in the list, they are typically clipped at the right.

void [ami_]listboxsiz[g](FILE* f, [ami_]strptr sp, long* w, long* h);

Finds the required size of a listbox for window **f**, with string list **sp**. The required width is returned in **w**, and the required height is returned in **h**. A listbox is sized such that all of the strings in the string list can be presented in it, with borders added. No extra space is required.

void [ami_]numselbox[g](FILE* f, long x1, long y1, long x2, long y2, long l, long u, long id);

Creates a number select box for window **f**, in the rectangle **x1**, **y1**, **x2**, **y2**, with lower number limit **l**, and upper number limit **u**. The default number appearing in the box is set to the lower limit. The number select box allows the user to edit the number, or use up/down arrow controls to select the number. Any digits typed into the edit section are limited to the digits 0-9, and negative numbers are not allowed. When the user presses enter to the number edit box, an event, **etnumbox** will be sent, which includes the number selected.

void [ami_]numselboxsiz[g](FILE* f, long l, long u, long* w, long* h);

Finds the width and height of a number select box for window **f**, with lower number limit **l** and upper number limit **u**. The width required is returned in **w**, and the height in **h**. The minimum width and height is determined by the maximum length of the number to be displayed, with borders and up/down arrows considered. This can be used without adding extra space.

void [ami_]poswidget[g](FILE* f, long id, long x, long y);

Reposition an existing widget. The widget with the logical identifier **id** is repositioned to be at the position **x** and **y**, in the parent window of the window f.

If the graphical version is used, the size is in pixels, otherwise, the size is in characters.

void [ami_]progbar[g](FILE* f, long x1, long y1, long x2, long y2, long id);

Creates a progress bar in window **f**, in rectangle **x1**, **y1**, **x2**, **y2**, with logical identifier **id**. The progress bar starts by default at 0, and is entirely operated by the program with **progbarpos()** calls.

void [ami_]progbarpos(FILE* f, long id, long pos);

Sets the progress bar in window **f**, with logical identifier **id**, to the position **pos**. The position is a ratioed **LONG_MAX** number, from 0 to **LONG_MAX**. 0 indicates "no progress", and **LONG_MAX** indicates "complete". Because of rounding, it is recommended that the program specifically set **LONG_MAX** at completion, instead of using a formula.

void [ami_]progbarsiz[g](FILE* f, long* w, long* h);

Finds the size of a progress bar for window **f**. The width is returned in **w**, and the height in **h**. For systems that can size progress bars arbitrarily, the height is returned as the size that matches others used in the system. The width is a suggestion, and can be ignored.

void [ami_]putwidgettext(FILE* f, long id, char* s);

Places text in the widget with the logical identifier **id** within window **f** from the string **s**. It depends on the widget as to if it can accept text placed in this manner. It is an error to place text in a widget that has no such capability.

A widget will have the ability to place text if it can edit text by the user. Placing text in the widget can be used to initialize the contents of such a widget, or as part of the overall interaction with the user.

void [ami_]querycolor(long* r, long* g, long* b);

Creates a color select dialog. The dialog "flows through" to set its parameters. When called, **r** contains the default red, **g** the default green, and **b** the default blue colors. These defaults are used to set the dialog default selection. When the user chooses a color, the same parameters return the new selection. If the user cancels, or leaves the default selection alone, the input colors will simply be left as the default output colors. The dialog is presented to the front of the desktop stacking order, and the program holds until the dialog is complete.

void [ami_]queryfind(char* s, long sl, [ami_]qfnopts* opt);

Creates a find string dialog. The dialog "flows through" to set its parameters. When called, **s** contains a default search string (which could be empty). This default is used to initialize the dialog default. When the user edits a search, that is then returned in **s** as well, with a maximum length **sl**. The dialog is presented to the front of the desktop stacking order, and the program holds until the dialog is complete.

The find dialog may set one or more of several options from the set provided in **opt**. These are set before the call, and they are used to initialize the defaults in the dialog. When the dialog terminates, the new state of the options are returned in **opt** as well. If the dialog does not implement a particular option, then the input value will simply be copied to the output value without change.

If the user cancels, an empty string is returned.

void [ami_]queryfindrep(char* s, long sl, char* r, long rl, [ami_]qfropts* opt);

Creates a find/replace string dialog. The dialog "flows through" to set its parameters. When called, **s** contains a default search string (which could be null), and **r** contains the default replacement string (which could be null). This default is used to initialize the dialog default. When the user edits a search, that is then returned in **s** and **r** as well, with maximum lengths **sl** and **rl**. The input and output strings are not the same even if the user chooses the default. The result strings must be disposed of by the caller, and the default strings supplied to the dialog are allocated and deallocated entirely by the caller as well. The dialog is presented to the front of the desktop stacking order, and the program holds until the dialog is complete.

The find dialog may set one or more of several options from the set provided in **opt**. These are set before the call, and they are used to initialize the defaults in the dialog. When the dialog terminates, the new state of the options are returned in **opt** as well. If the dialog does not implement a particular option, then the input value will simply be copied to the output value without change.

If the user cancels, an empty string is returned for both **s** and **r**.

void [ami_]queryfont(FILE* f, long* fc, long* s, long* fr, long* fg, long* fb, long* br, long* bg, long* bb, [ami_]qfteffects* effect);

Creates a query font dialog. The dialog "flows through" to set its parameters. When called, **fc** contains the font code, **s** contains the size, **fr**, **fg** and **fb** contain the foreground red, green and blue colors, **br**, **bg**, **bb** contains the background red, green and blue colors, and effect contains a set of font effects. These values are used to initialize the dialog defaults. The user then sets any or all of the parameters, and the results are copied back to the same output parameters. The dialog is presented to the front of the desktop stacking order, and the program holds until the dialog is complete.

If the dialog does not have a particular feature such as color set ability or one or more effects, the input for that parameter is simply copied to the output.

void [ami_]queryopen(char* s, long sl);

Creates an open file dialog. The dialog "flows through" to set its parameters. When called, **s** contains a default filename to open (which could be empty). This default is used to initialize the dialog default. When the user edits a filename, that is then returned in **s** as well, with maximum length **sl**. The input and output strings are not the same even if the user chooses the default. The result string must be disposed of by the caller, and the default string supplied to the dialog is allocated and deallocated entirely by the caller as well. The dialog is presented to the front of the desktop stacking order, and the program holds until the dialog is complete.

If the user cancels, an empty string is returned.

void [ami_]querysave(char* s, long sl);

Creates a save file dialog. The dialog "flows through" to set its parameters. When called, **s** contains a default filename to save (which could be empty). This default is used to initialize the dialog default. When the user edits a filename, that is then returned in **s** as well, with maximum length **sl**. The input and output strings are not the same even if the user chooses the default. The result string must be disposed of by the caller, and the default string supplied to the dialog is allocated and deallocated entirely by the caller as well. The dialog is presented to the front of the desktop stacking order, and the program holds until the dialog is complete.

If the user cancels, an empty string is returned.void [ami_]radiobutton[g](FILE* f, long x1, long y1, long x2, long y2, char* s, long id);

Places a radio button within window **f** in the bounding box formed by **x1**, **y1**, **x2**, **y2**, with the face text from the string **s**. The logical identifier is specified in **id**, which must be an integer from 1 to n, which is not currently in use in any other widget. The radio button drawn is not guaranteed to completely fill the bounding box. The radio button should be placed against a background that is colored with the standard background color. The radio button will be placed at the front of the window stacking order, and should be placed after any other widgets or child windows that the radio button is to appear in front of.

When the radio button is pressed, it will send an **etradbut** event, which contains the logical id of the radio button that was pressed. It is up to the system whether the event occurs when the radio button is depressed or released.

Radio buttons cannot have their face text changed or read. A radio button can be selected or deselected, and can be enabled and disabled. If the radio button is selected, it will appear with a selected face, which is typically a blacked out radio button. If the radio button is disabled, it will not sent **etradbut** events when pressed.

A radio button only changes its appearance in response to a select, and does not keep a state that can be read by the program. It’s up to the program to keep track of the state of the radio button, and how to handle it. In particular, if the checkbox is pressed, it is up to the program to change its select status, otherwise the press will have no effect. The program can implement many different effects for radio buttons. The radio button can toggle, or it can be one of a series of mutually exclusive selections.

void [ami_]radiobuttonsiz[g](FILE* f, char* s, long* w, long* h);

Finds the minimum size of a radio button within window **f**, with face text string **s**. The width to use is returned in **w**, and the height in **h**. Radio button sizing returns the minimum size in terms of the minimum amount of space needed to contain the face text, in the labeling font, and the border of the radio button, if it exists. You will want to use the size as a guide to radio button sizing, and not for the actual size of the checkbox. A good rule of thumb is to add 25% of the minimum height of the radio button to the actual height and width of the radio button to give the user sufficient area to click on the radio button.

void [ami_]scrollhoriz[g](FILE* f, long x1, long y1, long x2, long y2, long id);

Creates a horizontal scrollbar in window **f**, with bounding rectangle **x1**, **y1**, **x2**, **y2**, and logical widget identifier **id**. If possible, the scrollbar is made to fill the width and height requested. If this is not possible, then the largest scrollbar is created that fits within the bounding rectangle. There is no guarantee that the scrollbar will completely fill the rectangle. The area under the scrollbar should be drawn with the standard background color.

The scrollbar will generate several events when clicked. The **etsclull** event indicates the line left button of the scrollbar was pressed. The **etscldrl** event indicates the line right button of the scrollbar was pressed. The **etsclulp** event indicates the page left section of the scrollbar was pressed. The **etscldrp** event indicates the page right section of the scrollbar was pressed. It is system dependent as to whether the buttons generate their events on a button press or a button release.

The **etsclpos** event gives the position of the left of the slider after the user moves it. The position is returned as a ratioed **LONG_MAX** number, where 0 means the slider is at the left, and **LONG_MAX** means the slider is at the right. The number is affected by the size of the slider. If, for example, the slider occupies 50% of the scrollbar, then only the positions 0 to **LONG_MAX /** 2 will be generated. It is undefined as to exactly when **etsclpos** events occur. They may only be generated when the slider is moved and then released, or they may be generated continuously as the slider is moved. If the generation is continuous, then the movements are usually subject to "rate limiting" to keep them from generating events too fast to handle.

The scrollbar will not change position on its own. The scrollbar will generate events, and it is up to the program to use those to set the scrollbar slider position, and to take action on them, such as move the screen data left or right. The page left/right and line left/right terminology is suggestive of the use of these events, but it is up to the program exactly how to use or implement these functions. Typically, these are used to move the displayed area of a document one character left or right, and one page or screenful left or right.

The size of the scrollbar slider is set by default to small, but convenient for the user to press and manipulate. This can be left alone for programs that don't require sized scrollbar sliders. The size of the slider is set by **scrollsiz[g]()**. Typically, it is used to set the ratio of onscreen data shown to the entire document or other data available. For example, if 50% of the document is being displayed, then the slider should occupy 50% of the scrollbar.

void [ami_]scrollhorizsiz[g](FILE* f, long* w, long* h);

Finds the size for a horizontal scroll bar in window **f**. Returns the width in **w**, and the height in **h**. Scrollbars typically can be sized to any size, and the height of a horizontal scroll bar is a suggested width designed to match others used in the same system. The width of a horizontal scrollbar is simply a suggestion, and can be ignored.

If a scrollbar cannot be arbitrarily sized, then the width and height will reflect the dimensions of a fixed scrollbar.

void [ami_]scrollpos(FILE* f, long id, long r);

Sets the scrollbar slider position for window **f**, scrollbar identifier **id**, to the position r. The position is in ratioed **LONG_MAX** format. That is, 0 means to set the position to the top or left, and **LONG_MAX** means bottom or right. The position is affected by the size of the scrollbar slider. For example, if the slider occupies 50% of the scrollbar, then the range of positions would only be from 0 to **LONG_MAX** / 2. If the position given is beyond the maximum position possible, then the slider is set to the maximum travel position, and no error occurs. It is an error if the position is negative.

The program must specifically set the position of the scrollbar. The user moving the scrollbar slider may temporarily move the slider while it is being moved, but this will not remain in position after the user releases it. The program must set the scrollbar position in response to the event.

void [ami_]scrollsiz(FILE* f, long id, long r);

Sets the size of the scrollbar slider in window **f**, logical identifier id, to the size r. The size of the scrollbar slider is a **LONG_MAX** ratio, with 0 meaning infinitely small, and **LONG_MAX** meaning that it occupies the entire scrollbar. In practice, there is a practical limit to how small the slider can be. If the slider is set too small, it will be set to the minimum size, and no error will occur. If the size set is negative, then an error will result.

The size of the scrollbar is set to a reasonable default if it is never specifically set. This is typically a fairly small size that is still easy to press and manipulate by the user. This allows the scrollbar to be used when slider sizing is not supported by the program.

The meaning of the scrollbar slider size is up to the program. However, it is typically used to indicate how much of the data is onscreen. For example, if a document has 50% of its content currently displayed, then the slider would be set to 50% of the scrollbar.

void [ami_]scrollvert[g](FILE* f, long x1, long y1, long x2, long y2, long id);

Creates a vertical scrollbar in window **f**, with bounding rectangle **x1**, **y1**, **x2**, **y2**, and logical widget identifier **id**. If possible, the scrollbar is made to fill the width and height requested. If this is not possible, then the largest scrollbar is created that fits within the bounding rectangle. There is no guarantee that the scrollbar will completely fill the rectangle. The area under the scrollbar should be drawn with the standard background color.

The scrollbar will generate several events when clicked. The **etsclull** event indicates the line up button of the scrollbar was pressed. The **etscldrl** event indicates the line down button of the scrollbar was pressed. The **etsclulp** event indicates the page up section of the scrollbar was pressed. The **etscldrp** event indicates the page down section of the scrollbar was pressed. It is system dependent as to whether the buttons generate their events on a button press or a button release.

The **etsclpos** event gives the position of the top of the slider after the user moves it. The position is returned as a ratioed **LONG_MAX** number, where 0 means the slider is at the top, and **LONG_MAX** means the slider is at the bottom. The number is affected by the size of the slider. If, for example, the slider occupies 50% of the scrollbar, then only the positions 0 to **LONG_MAX /** 2 will be generated. It is undefined as to exactly when **etsclpos** events occur. They may only be generated when the slider is moved and then released, or they may be generated continuously as the slider is moved. If the generation is continuous, then the movements are usually subject to "rate limiting" to keep them from generating events too fast to handle.

The scrollbar will not change position on its own. The scrollbar will generate events, and it is up to the program to use those to set the scrollbar slider position, and to take action on them, such as move the screen data up or down. The page up/down and line up/down terminology is suggestive of the use of these events, but it is up to the program exactly how to use or implement these functions. Typically, these are used to move the displayed area of a document one line up or down, and one page or screenful up or down.

The size of the scrollbar slider is set by default to small, but convenient for the user to press and manipulate. This can be left alone for programs that don't require sized scrollbar sliders. The size of the slider is set by **scrollsiz[g]()**. Typically, it is used to set the ratio of onscreen data shown to the entire document or other data available. For example, if 50% of the document is being displayed, then the slider should occupy 50% of the scrollbar.

void [ami_]scrollvertsiz[g](FILE* f, long* w, long* h);

Finds the size for a vertical scroll bar in window **f**. Returns the width in **w**, and the height in **h**. Scrollbars typically can be sized to any size, and the width of a vertical scroll bar is a suggested width designed to match others used in the same system. The height of a vertical scrollbar is simply a suggestion, and can be ignored.

If a scrollbar cannot be arbitrarily sized, then the width and height will reflect the dimensions of a fixed scrollbar.

void [ami_]selectwidget(FILE* f, long id, long e);

The widget within window **f** by the logical identifier **id** will enter the select state if **e** is true, otherwise the select state is removed. The exact effect of the select state depends on the widget. See the individual widget to be selected for more information. It is an error to select a widget that has no selectability.

Selection is typically used to indicate that the widget is "on", by changing its face appearance. For example, it can be checked, pressed or a similar visual state change.

void [ami_]sizwidget[g](FILE* f, long id, long x, long y);

Resize an existing widget. The widget with the logical identifier **id** is resized to be the size in **x** and **y**, in the window **f**.

If the graphical version is used, the size is in pixels, otherwise, the size is in characters.

void [ami_]slidehoriz[g](FILE* f, long x1, long y1, long x2, long y2, long mark, long id);

Creates a horizontal slider in window **f**, in bounding rectangle **x1**, **y1**, **x2**, **y2**, with **mark** number of tick marks, and a logical identifier **id**. Sliders give a convenient way to select from a range of values. The system will either size the slider to fit the given rectangle, or choose the slider representation that is as large as possible, but still fits the given rectangle. It is not guaranteed that the slider will fill the entire rectangle. This means that it is important for the entire background under the rectangle to be set to the standard background color.

When the slider is moved by the user, it generates a **etsldpos** event. This event carries the new position of the slider, which is a ratioed **LONG_MAX** number, from 0 to **LONG_MAX**. 0 means the slider is at the extreme left, and **LONG_MAX** means the slider is at the extreme right.

There is no guarantee as to when a slider generates its events. It can generate them as the slider is moved, or it may wait until the user moves, and then releases the slider to generate events. Sliders automatically update the position of the slider, and do not need to be set by the program.

The number of tick marks given by **mark** are evenly distributed across the slider. If **mark** is zero, then no tick marks are placed at all. Tick marks have no other effect besides appearing on the slider.

void [ami_]slidehorizsiz[g](FILE* f, long* w, long* h);

Finds the size of a horizontal slider for window f. The required width is returned in **w**, and the required height in **h**. The height of a slider is chosen so that they match other slidebars used in the system. The width is a suggestion, and can be ignored.

void [ami_]slidevert[g](FILE* f, long x1, long y1, long x2, long y2, long mark, long id);

Creates a vertical slider in window **f**, in bounding rectangle **x1**, **y1**, **x2**, **y2**, with mark number of tick marks, and a logical identifier **id**. Sliders give a convenient way to select from a range of values. The system will either size the slider to fit the given rectangle, or choose the slider representation that is as large as possible, but still fits the given rectangle. It is not guaranteed that the slider will fill the entire rectangle. This means that it is important for the entire background under the rectangle to be set to the standard background color.

When the slider is moved by the user, it generates a **etsldpos** event. This event carries the new position of the slider, which is a ratioed **LONG_MAX** number, from 0 to **LONG_MAX**. 0 means the slider is at the extreme top, and **LONG_MAX** means the slider is at the extreme bottom.

There is no guarantee as to when a slider generates its events. It can generate them as the slider is moved, or it may wait until the user moves, and then releases the slider to generate events. Sliders automatically update the position of the slider, and do not need to be set by the program.

The number of tick marks given by **mark** are evenly distributed across the slider. If **mark** is zero, then no tick marks are placed at all. Tick marks have no other effect besides appearing on the slider.void [ami_]slidevertsiz[g](FILE* f, long* w, long* h);

Finds the size of a vertical slider for window **f**. The required width is returned in **w**, and the required height in **h**. The width of a slider is chosen so that they match other slidebars used in the system. The height is a suggestion, and can be ignored.

void [ami_]tabbar[g](FILE* f, long x1, long y1, long x2, long y2, [ami_]strptr sp, [ami_]tabori tor, long id);

Creates a tab bar, in the window **f**, in the rectangle **x1**,**y1** to **x2**,**y2**, with string list **sp**, tab orientation tor, and logical identifier **id**. A tabbar gives the user a paradigm of a book with tabs on the side. The list of tabs, which are specified by the program, each generate events. The program establishes the view to be selected in the client area at the center of the tabbar, then it uses the tab events to switch the views in the client. If there is not enough area to display the full list of tabs, the system will allow the user to scroll through them with arrows.

To locate the tabbar and client, the **tabbarsiz()** call is used to establish the size and client offset. Then, the client is offset into the tabbar. The client widgets or child windows must be created after the tabbar in order to be placed to the front of the stacking order.

When a tab is selected, it generates an **ettabbar** event, which contains both the logical tabbar id and the number of the string list that was selected. This number will be the number in the string list. For example, the first string in the list will be 1, the second 2, etc.

A tab bar can have tabs on any of its sides. If a tab bar needs tabs on more than one side, the standard method is to overlay multiple tabs.

void [ami_]tabbarclient[g](FILE* f, [ami_]tabori tor, long w, long h, long* cw, long* ch,long* ox, long* oy);

Finds the size of a tabbar client area, in window **f**, with tab orientation **tor**, tabbar width **w**, and tab bar height **h**. The client width is returned in **cw**, the height in **ch**, and the client offset in **ox** and **oy**. This procedure is used to find the client area for a specific size of tabbar. It takes no string list: what the client loses to the tabbed edge is the thickness of that edge, which is the labeling font's business rather than the tabs'.

void [ami_]tabbarsiz[g](FILE* f, [ami_]strptr sp, [ami_]tabori tor, long cw, long ch, long* w, long* h, long* ox, long* oy);

Finds the size of a tabbar, in window **f**, with tab string list **sp**, tab orientation **tor**, client width **cw**, and client height **ch**. The required width is returned in **w**, the height in **h**, and the client offset in **ox** and **oy**. The size of a tabbar is enough to hold the height of the labeling font (in whatever orientation), plus border areas, any selection scrolling arrows, and the client area. It is enough to hold the tabs as well: the strings are given so the call can add up what they take along the tabbed edge, and what comes back is the larger of that and the client asked for. A tab bar sized without its strings cannot know whether its own tabs fit, which is the use a sizing call is put to.

void [ami_]tabsel(FILE* f, long id, long tn);

Select tab in tab bar. Causes the tab **tn** in the tab bar **id** in the window **f**, to enter the selected state. **tn** is the number of the string list item to select. For example, the first string in the list will be 1, the second 2, etc.

8.15 Events In widgets

See the description of the event record (8.12 “Events”) for the format of the entire record.

Event: [ami_]etbutton

The button with identifier **butid** was pressed.

Event: [ami_]etchkbox

The checkbox with identifier ckbxid was selected.

Event: [ami_]etradbut

The radio button with identifier **radbid** was selected.

Event: [ami_]etsclull

The scrollbar with identifier **sclulid** had its up or left line button pressed.

Event: [ami_]etscldrl

The scrollbar with identifier **scldrid** had its down or right line button pressed

Event: [ami_]etsclulp

The scrollbar with identifier **sclupid** had its up or left page button pressed.

Event: [ami_]etscldrp

The scrollbar with identifier **scldpid** had its down or right page button pressed.

Event: [ami_]etsclpos

The scrollbar with identifier sclpid was repositioned to sclpos. The value sclpos is a LONG_MAX ratio, with 0 indicating top or left, and LONG_MAX indicating bottom or right.

Event: [ami_]etedtbox

The editbox with identifier **edtbid** was given an enter key. This means that the text in the editbox is complete, and can be retrieved by getwidgettext().

Event: [ami_]etnumbox

The numselbox with the identifier **numbid** was entered with the number **numbsl**. **numbsl** directly corresponds to the number selected.

Event: [ami_]etlstbox

The listbox with the identifier **lstbid** was entered with the logical select **lstbsl**. The logical select **lstbsl** gives the number of the string selected in the order used to create the listbox, with 1 indicating the first string, 2 the second, etc.

Event: [ami_]etdrpbox

The dropbox with the identifier drpbid was entered with the logical select drpbsl. The logical select drpbsl gives the number of the string selected in the order used to create the dropbox, with 1 indicating the first string, 2 the second, etc.

Event: [ami_]etdrebox

The dropeditbox with identifier **drebid** was given an enter key. This means that the text in the editbox is complete, and can be retrieved by getwidgettext().

Event: [ami_]etsldpos

The slider with identifier sldpid was repositioned to sldpos. The value sldpos is a LONG_MAX ratio, with 0 indicating top or left, and LONG_MAX indicating bottom or right.

Event: [ami_]ettabbar

The tabbar with the identifier **tabid** had a tab selected with the string number tabsel: 1 is the first string of the bar's list, 2 the second, and so on.

9 Sound Library



sound adds both a synthesizer interface via the MIDI standard, and the ability to play or input wave files. It implements a sequencer that programs the exact time at which each of the events occurs to make a complex combination of sounds.

Generation of sound for games and other uses can use the MIDI interface or the waveform interface. MIDI gives much more compact descriptions of complex sounds than waveforms, but in today's computers, that is not an issue. A game or other program may use waveforms for all of its sound outputs, even if MIDI was originally used to generate the waveform.

The MIDI interface is defined as a serial interface, but the target of a MIDI port can be a standard serial MIDI daisy chain, another type of interface that carries MIDI commands (such as USB), or simply terminate in an internal sound card or a software synthesizer.

The sound module presents a logical abstraction of the MIDI interface.



Older keyboards with MIDI acted as both controllers and synthesizers.



Now with software based synthesizers, hardware that is a MIDI controller only is gaining popularity, including compact controllers with drumpads and rotary controllers.

9.1 Ports

sound has both MIDI synthesizer ports and waveform device ports, and input and output ports of each. The number of synthesizer and wave ports are found by

synthout() The number of synthesizer output ports.
synthin() The number of synthesizer input ports.
waveout() The number of waveform output ports.
wavein() The number of waveform input ports.

The ports are arranged in order so that port 1 is the default input or output port. These are also found as:

```
#define AMI_SYNTH_OUT 1 /* the default output synth for host */
#define AMI_SYNTH_IN  1 /* The default input from external synth */
#define AMI_WAVE_IN   1 /* the default wave input for host */
#define AMI_WAVE_OUT  1 /* the default output wave for host */
```

Each port has a name, descriptive of its function or place in the computer. This is used to present a list of ports to the user, who selects which one is wanted. The name of each port is found as:

synthoutname(p,n,l) Return the name of a synthesizer output port.
synthinname(p,n,l) Return the name of a synthesizer input port.
waveoutname(p,n,l) Return the name of a waveform output port.
waveinname(p,n,l) Return the name of a waveform input port.

Where **p** is the port number, **n** is a string buffer, and **l** is the length of the buffer.

9.2 MIDI Output: Composition Interface

sound has a composition interface for MIDI output devices. A computer may have several output ports, for example the sound card internal to the computer with an onboard synthesizer, and the external MIDI jack and others. The sound card synthesizer may be replaced by a software synthesizer. A synthesizer output is opened with **opensynthout(p)**, where **p** is the synthesizer port. It can be closed with

closesynthout(p), where **p** is the synthesizer port. All synthesizer ports are automatically closed when the program closes.

9.3 Notes

```
typedef long ami_note; /* 1..128 note number for midi */
```

The basic work of making music is playing notes. MIDI can play 128 notes, numbered from 1 to 128. This ranges in frequency from 8 Hertz, or cycles per second, to 12 Kilohertz. This covers the range of human hearing associated with music. MIDI can also change each note in frequency enough to move it to the note next to it (and then some), so MIDI is able to reach any frequency desired.

The ear hears pitch by ratio and not by difference. The step from 100 to 200 Hertz is the same interval to the ear as the step from 1,000 to 2,000, though one gap is ten times the other in Hertz. A doubling of frequency is that interval, the octave, and a note at twice the frequency is heard as the same note an octave higher. An octave divides into twelve notes, each one the twelfth root of two, about 1.0595, times the note below it. That is why the numbers below run twelve to an octave, and why the octave bases that follow them stand twelve apart. In the lowest octave, the notes are:

Symbol	Number
note_c	1
note_c_sharp	2
note_d_flat	2
note_d	3
note_d_sharp	4
note_e_flat	4
note_e	5
note_f	6
note_f_sharp	7
note_g_flat	7
note_g	8
note_g_sharp	9
note_a_flat	9
note_a	10
note_a_sharp	11
note_b_flat	11
note_b	12

The bases of the octaves are:

Symbol	Number
octave_1	0
octave_2	12
octave_3	24

octave_4	36
octave_5	48
octave_6	60
octave_7	72
octave_8	84
octave_9	96
octave_10	108
octave_11	120

So any note in any octave can be found by:

note+octave

For example, C in the 6th Octave:

AMI_NOTE_C+AMI_OCTAVE_6

Notes are activated in MIDI by the **noteon(p,t,c,n,v)** and notes are deactivated by **noteoff(p,t,c,n,v)**. Each of these calls takes:

- A Port **p.**
- A time **t.**
- A channel **c.**
- A note **n.**
- A volume **v.**

Each of these parameters will be presented separately. The time to play will be discussed below. For now, it can be zero, which means "play it now". The channel is the instrument type to play it to, for instance, a piano, or an organ, or a tuba. The note is the logical note number we saw above, one of the 128 MIDI notes. The volume gives the volume the particular note is to be played at. A piano note can be louder if hit harder.

A note can either last forever, until turned off with **noteoff()**, or it can stop on its own. For example, an organ plays as long as you hold the key down, but a string instrument plays a note when the string is plucked, then dies away. **noteoff()** need not be used for these instruments, but can still be used to cause the note to be "clipped" off early, much as if the player put a hand on the string to stop it. Similarly, a **noteon()** can be used to restart the note, even while it is playing.

9.4 Channels and Instruments

```
typedef long ami_channel;    /* 1..16   channel number */
typedef long ami_instrument; /* 1..128  instrument number */
```

MIDI has from 1 to 16 logical channels, indexed by a logical channel number. Although there are 128 instruments, only one can be played at any one time. To play an instrument, it must be assigned to a channel. This is done with **instchange(p,t,c,i),** where i is the instrument.

The instruments that can be assigned to a given channel range from1 to 128, and **sound** provides symbols for each of them. The instruments are grouped, with 8 instruments per group, for a total of 16 groups.

Timing

S = Self timed
C = Configurable timing via noteon/noteoff

9.4.1 Piano Group

Instrument	Symbol	Number	Timing
Acoustic Grand	inst_acoustic_grand	1	S
Bright Acoustic	inst_bright_acoustic	2	S
Electric Grand	inst_electric_grand	3	S
Honky Tonk	inst_honky_tonk	4	S
Electric Piano 1	inst_electric_piano_1	5	S
Electric Piano 2	inst_electric_piano_2	6	S
Harpsichord	inst_harpsichord	7	S
Clavinet	inst_clavinet	8	S

9.4.2 Chromatic percussion Group

Instrument	Symbol	Number	Timing
celesta	inst_celesta	9	S
glockenspiel	inst_glockenspiel	10	S
music box	inst_music_box	11	S
vibraphone	inst_vibraphone	12	S
marimba	inst_marimba	13	S
xylophone	inst_xylophone	14	S
tubular bells	inst_tubular_bells	15	S
dulcimer	inst_dulcimer	16	S

9.4.3 Organ Group

Instrument	Symbol	Number	Timing
drawbar organ	inst_drawbar_organ	17	C
percussive organ	inst_percussive_organ	18	C
rock organ	inst_rock_organ	19	C
church organ	inst_church_organ	20	C
reed organ	inst_reed_organ	21	C
accoridan	inst_accoridan	22	C
harmonica	inst_harmonica	23	C
tango accordion	inst_tango_accordian	24	C

9.4.4 Guitar Group

Instrument	Symbol	Number	Timing
nylon string guitar	inst_nylon_string_guitar	25	S
steel string guitar	inst_steel_string_guitar	26	S
electric jazz guitar	inst_electric_jazz_guitar	27	S
electric clean guitar	inst_electric_clean_guitar	28	S
electric muted guitar	inst_electric_muted_guitar	29	S
overdriven guitar	inst_overdriven_guitar	30	S

distortion guitar	inst_distortion_guitar	31	S
guitar harmonics	inst_guitar_harmonics	32	S

9.4.5 Bass Group

Instrument	Symbol	Number	Timing
acoustic bass	inst_acoustic_bass	33	S
electric bass finger	inst_electric_bass_finger	34	S
electric bass pick	inst_electric_bass_pick	35	S
fretless bass	inst_fretless_bass	36	S
slap bass 1	inst_slap_bass_1	37	S
slap bass 2	inst_slap_bass_2	38	S
synth bass 1	inst_synth_bass_1	39	S
synth bass 2	inst_synth_bass_2	40	S

9.4.6 Solo strings Group

Instrument	Symbol	Number	Timing
violin	inst_violin	41	S
viola	inst_viola	42	S
cello	inst_cello	43	S
contrabass	inst_contrabass	44	S
tremolo strings	inst_tremolo_strings	45	S
pizzicato strings	inst_pizzicato_strings	46	S
orchestral strings	inst_orchestral_strings	47	S
timpani	inst_timpani	48	S

9.4.7 Ensemble Group

Instrument	Symbol	Number	Timing
string ensemble 1	inst_string_ensemble_1	49	S
string ensemble 2	inst_string_ensemble_2	50	S
synthstrings 1	inst_synthstrings_1	51	S
synthstrings 2	inst_synthstrings_2	52	S
choir aahs	inst_choir_aahs	53	S
voice oohs	inst_voice_oohs	54	S
synth voice	inst_synth_voice	55	S
orchestra hit	inst_orchestra_hit	56	S

9.4.8 Brass Group

Instrument	Symbol	Number	Timing
trumpet	inst_trumpet	57	C
trombone	inst_trombone	58	C
tuba	inst_tuba	59	C
muted trumpet	inst_muted_trumpet	60	C

french horn	inst_french_horn	61	C
brass section	inst_brass_section	62	C
synthbrass 1	inst_synthbrass_1	63	C
synthbrass 2	inst_synthbrass_2	64	C

9.4.9 Reed Group

Instrument	Symbol	Number	Timing
soprano sax	inst_soprano_sax	65	C
alto sax	inst_alto_sax	66	C
tenor sax	inst_tenor_sax	67	C
baritone sax	inst_baritone_sax	68	C
oboe	inst_oboe	69	C
english horn	inst_english_horn	70	C
bassoon	inst_bassoon	71	C
clarinet	inst_clarinet	72	C

9.4.10 Pipe Group

Instrument	Symbol	Number	Timing
piccolo	inst_piccolo	73	C
flute	inst_flute	74	C
recorder	inst_recorder	75	C
pan flute	inst_pan_flute	76	C
blown bottle	inst_blown_bottle	77	C
skakuhachi	inst_skakuhachi	78	C
whistle	inst_whistle	79	C
ocarina	inst_ocarina	80	C

9.4.11 Synth lead Group

Instrument	Symbol	Number	Timing
lead 1 square	inst_lead_1_square	81	C
lead 2 sawtooth	inst_lead_2_sawtooth	82	C
lead 3 calliope	inst_lead_3_calliope	83	C
lead 4 chiff	inst_lead_4_chiff	84	C
lead 5 charang	inst_lead_5_charang	85	C
lead 6 voice	inst_lead_6_voice	86	C
lead 7 fifths	inst_lead_7_fifths	87	C
lead 8 bass lead	inst_lead_8_bass_lead	88	C

9.4.12 Synth pad Group

Instrument	Symbol	Number	Timing
pad 1 new age	inst_pad_1_new_age	89	C
pad 2 warm	inst_pad_2_warm	90	C

pad 3 polysynth	inst_pad_3_polysynth	91	C
pad 4 choir	inst_pad_4_choir	92	C
pad 5 bowed	inst_pad_5_bowed	93	C
pad 6 metallic	inst_pad_6_metallic	94	C
pad 7 halo	inst_pad_7_halo	95	C
pad 8 sweep	inst_pad_8_sweep	96	C

9.4.13 Synth effects Group

Instrument	Symbol	Number	Timing
fx 1 rain	inst_fx_1_rain	97	C
fx 2 soundtrack	inst_fx_2_soundtrack	98	C
fx 3 crystal	inst_fx_3_crystal	99	C
fx 4 atmosphere	inst_fx_4_atmosphere	100	C
fx 5 brightness	inst_fx_5_brightness	101	C
fx 6 goblins	inst_fx_6_goblins	102	C
fx 7 echoes	inst_fx_7_echoes	103	C
fx 8 sci fi	inst_fx_8_sci_fi	104	C

9.4.14 Ethnic Group

Instrument	Symbol	Number	Timing
sitar	inst_sitar	105	S
banjo	inst_banjo	106	S
shamisen	inst_shamisen	107	S
koto	inst_koto	108	S
kalimba	inst_kalimba	109	S
bagpipe	inst_bagpipe	110	S
fiddle	inst_fiddle	111	S
shanai	inst_shanai	112	S

9.4.15 Percussive Group

Instrument	Symbol	Number	Timing
tinkle bell	inst_tinkle_bell	113	S
agogo	inst_agogo	114	S
steel drums	inst_steel_drums	115	S
woodblock	inst_woodblock	116	S
taiko drum	inst_taiko_drum	117	S
melodic tom	inst_melodic_tom	118	S
synth drum	inst_synth_drum	119	S
reverse cymbal	inst_reverse_cymbal	120	S

9.4.16 Sound effects Group

Instrument	Symbol	Number	Timing
guitar fret noise	inst_guitar_fret_noise	121	C
breath noise	inst_breath_noise	122	C
seashore	inst_seashore	123	C
bird tweet	inst_bird_tweet	124	C
telephone ring	inst_telephone_ring	125	C
helicopter	inst_helicopter	126	C
applause	inst_applause	127	C
gunshot	inst_gunshot	128	C

9.4.17 Drum Channel

When MIDI starts up, all channels are assigned logical instrument number 1, an acoustical grand piano, with the exception of channel 10.

The MIDI channel system allows an “arrangement” to be created from different instruments. Each channel is configured with an instrument, then used to play a sequence. The computer can quickly change instruments during the music, and start an entirely different kind of music without skipping a beat. It is good practice not to count on an instrument being able to complete a note that is playing if it is swapped out of its channel for another instrument.

Channel 10 is an exception. This channel is always reserved for percussion (or drum) sounds. In this channel, the notes sent have a special meaning. Each note selects a different instrument. The instruments in the drum channel are always self timed.

Instrument	Symbol	Note number
Acoustic bass drum	note_acoustic_bass_drum	35
Bass drum 1	note_bass_drum_1	36
Side stick	note_side_stick	37
Acoustic snare	note_acoustic_snare	38
Hand clap	note_hand_clap	39
Electric snare	note_electric_snare	40
Low floor tom	note_low_floor_tom	41
Closed hi hat	note_closed_hi_hat	42
High floor tom	note_high_floor_tom	43
Pedal hi hat	note_pedal_hi_hat	44
Low tom	note_low_tom	45
Open hi hat	note_open_hi_hat	46
Low mid tom	note_low_mid_tom	47
Hi mid tom	note_hi_mid_tom	48
Crash cymbal 1	note_crash_cymbal_1	49
High tom	note_high_tom	50

Ride cymbal 1	note_ride_cymbal_1	51
Chinese cymbal	note_chinese_cymbal	52
Ride bell	note_ride_bell	53
Tambourine	note_tambourine	54
Splash cymbal	note_splash_cymbal	55
Cowbell	note_cowbell	56
Crash cymbal 2	note_crash_cymbal_2	57
Vibraslap	note_vibraslap	58
Ride cymbal 2	note_ride_cymbal_2	59
Hi bongo	note_hi_bongo	60
Low bongo	note_low_bongo	61
Mute hi conga	note_mute_hi_conga	62
Open hi conga	note_open_hi_conga	63
Low conga	note_low_conga	64
High timbale	note_high_timbale	65
Low timbale	note_low_timbale	66
High agogo	note_high_agogo	67
Low agogo	note_low_agogo	68
Cabasa	note_cabasa	69
Maracas	note_maracas	70
Short whistle	note_short_whistle	71
Long whistle	note_long_whistle	72
Short guiro	note_short_guiro	73
Long guiro	note_long_guiro	74
Claves	note_claves	75
Hi wood block	note_hi_wood_block	76
Low wood block	note_low_wood_block	77
Mute cuica	note_mute_cuica	78
Open cuica	note_open_cuica	79
Mute triangle	note_mute_triangle	80
Open_triangle	note_open_triangle	81

The same instrument can be assigned to multiple channels. This allows an instrument harmonize with itself, playing overlapping notes.

9.5 Volume

The volume can be set for each individual note. Volume can be set for each channel by **volsynthchan(p,t,c,v)**.

The volume is "**LONG_MAX** ratioed". It exists as a value from 0 to **LONG_MAX**, where 0 off (no volume) and **LONG_MAX** is full on. It is not decibel compensated, meaning that **LONG_MAX** div 2 is not half volume.

Balance between left and right can be set for each channel with **balance(p,t,c,b)**. It's still **LONG_MAX** ratioed, except that 0 means middle, **LONG_MAX** means full right, and -**LONG_MAX** means full left.

9.6 Time and the Sequencer

sound MIDI does not have a concept of time built into the protocol. All notes or events sent to the MIDI port are assumed to happen "Now", unless an external sequencer is installed.

sound has sequencer support that is used by setting a time on each event call. If the time is 0, it means to send the note or event to the MIDI port immediately, otherwise the sequencer schedules the event to occur at the indicated time.

To start **sound**'s sequencer, the **starttimeout()** is used, which starts a 100us counter running (it ticks every 10,000th of a second). Then, each time is specified relative to that running timer. The current time on the sequencer can be found with **curtimeout()**, so the required time can be specified as an offset from that:

```
ami_curtimeout()+10000
```

means a time that is one second in the future.

This example dumps a "fanfare" into the MIDI port using the sequencer. It will be played using the time specified in the **noteon()** and **noteoff()** calls.

```
#include <sound.h>

#define OSEC 10000 /* one second */

int main(void)

{

   ami_opensynthout(AMI_SYNTH_OUT); /* open the synthesizer */
   ami_starttimeout(); /* start the sequencer */

   ami_noteon( AMI_SYNTH_OUT,  0,                          1,
               AMI_NOTE_C+AMI_OCTAVE_6, LONG_MAX);
   ami_noteoff(AMI_SYNTH_OUT, ami_curtimeout()+OSEC*2,  1,
               AMI_NOTE_C+AMI_OCTAVE_6, LONG_MAX);
   ami_noteon( AMI_SYNTH_OUT,  ami_curtimeout()+OSEC*3,  1,
               AMI_NOTE_D+AMI_OCTAVE_6, LONG_MAX);
   ami_noteoff(AMI_SYNTH_OUT, ami_curtimeout()+OSEC*4,  1,
               AMI_NOTE_D+AMI_OCTAVE_6, LONG_MAX);
   ami_noteon( AMI_SYNTH_OUT,  ami_curtimeout()+OSEC*5,  1,
               AMI_NOTE_E+AMI_OCTAVE_6, LONG_MAX);
   ami_noteoff(AMI_SYNTH_OUT, ami_curtimeout()+OSEC*6,  1,
               AMI_NOTE_E+AMI_OCTAVE_6, LONG_MAX);
   ami_noteon( AMI_SYNTH_OUT,  ami_curtimeout()+OSEC*7,  1,
               AMI_NOTE_F+AMI_OCTAVE_6, LONG_MAX);
   ami_noteoff(AMI_SYNTH_OUT, ami_curtimeout()+OSEC*8,  1,
               AMI_NOTE_F+AMI_OCTAVE_6, LONG_MAX);
   ami_noteon( AMI_SYNTH_OUT,  ami_curtimeout()+OSEC*9,  1,
               AMI_NOTE_E+AMI_OCTAVE_6, LONG_MAX);
   ami_noteoff(AMI_SYNTH_OUT, ami_curtimeout()+OSEC*10, 1,
               AMI_NOTE_E+AMI_OCTAVE_6, LONG_MAX);
   ami_noteon( AMI_SYNTH_OUT,  ami_curtimeout()+OSEC*11, 1,
               AMI_NOTE_D+AMI_OCTAVE_6, LONG_MAX);
   ami_noteoff(AMI_SYNTH_OUT, ami_curtimeout()+OSEC*13, 1,
               AMI_NOTE_D+AMI_OCTAVE_6, LONG_MAX);

}
```

The fanfare plays, and the program goes on to other work, or waits for the sequenced time to pass by setting a timer to the time required to finish it, or uses **waitsynth(p)**, which returns when the synthesizer has no messages left to play.

If a note is output (or other action) with a 0 time while the sequencer is running, it will still occur immediately. Time 0 always means "now". The sequencer is a "flow through" model. Actions and notes can be timed with the sequencer, or by the program, or any combination thereof.

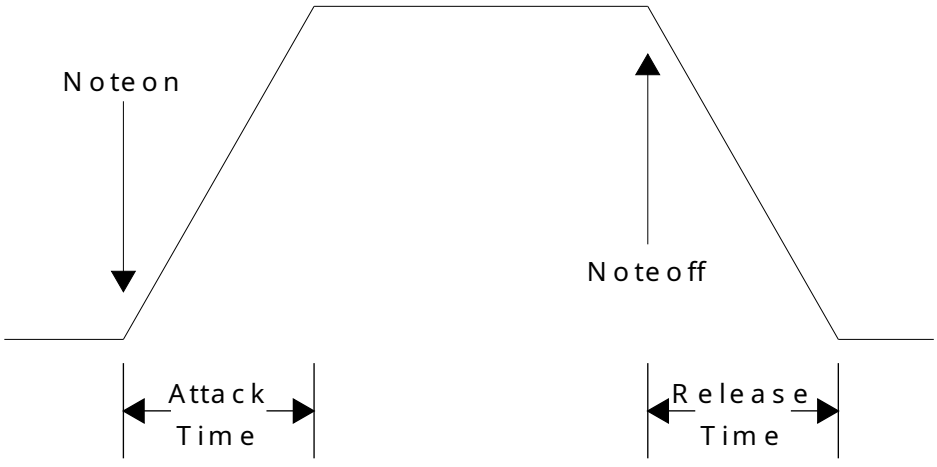
When the sequencer is no longer required, **stoptimeout()** stops it. Doing that can save processor time, and free up system timers.

9.7 Effects

There are many effects in MIDI that can be applied to output notes. However, there is no requirement for the system to implement them. Few of the effects are implemented on most computer sound cards or software synthesizers.

Function	Description
attack(p,t,c,at)	Adjusts the "attack time" **at** of each note.
release(p,t,c,rt)	Adjusts the release or "decay" time **rt** of the note.
reverb(p,t,c,r)	Sets the amount of reverberation **r**, or a series of repetitions of the note with delay.
vibrato(p,t,c,v)	Sets the vibrato **v**, which is a pulsating pitch change.
chorus(p,t,c,r)	Sets the chorus effect **r,** which is an echo of the same note with a delay and possible pitch change..
phaser(p,t,c,ph)	Sets the phaser effect **ph**, which is a series of peaks and valleys in the frequency spectrum of the note.
brightness(p,t,c,b)	Sets the brightness **b**, or VCF cutoff frequency.
timbre(p,t,c,tb)	Sets the “tone color” **tb**.
aftertouch(p,t,c,n,at)	Sets the amount of time or pressure used to sustain a key pressed. **at** is the time or pressure value.
pressure(p,t,c,pr)	Sets the amount of pressure **p** applied to a key.
legato(p,t,c,b)	Sets the note to be played shorter than normal. **b** is the value.
portamento(p,t,c,b)	Sets the note to “slide” or smoothly change into the next note. **b** is the value.

Where **p** is the port, **t** is the time, and **c** is the channel. Note that all linear values are LONG_MAX ratioed.



Attack and release times control the basic envelope of notes.

Some missing effects can be simulated by other means. As an example, **release** control can be emulated by putting the instrument to control in its own channel, sounding the note, then lowering the volume in steps until 0, then turning the note off.

9.8 Pitch Changes

If a frequency is needed that is not exactly on a note, it can be “bent” with **pitch(p,t,c,pt),** where **p** is the port, **t** is the time, **c** is the channel, and **pt** is the pitch offset. The pitch change is none for 0, and by default, one note up or down. In other words, the default pitch change range is one note up or down. A D note can be bent downwards to C, or upwards to E. The term "bend" comes from bending a string to change the note.

The default range of pitch changes can also be changed, by **pitchrange(p,t,c,v),** where p is the port, t is time, c is the channel, and v is the pitch range value. The range is a ratioed 0..**LONG_MAX**. 0 means no pitch range at all (disabled), and **LONG_MAX** means the full 128 notes worth of pitch range. What you pick up with total range, you lose in fine control. If the pitch range is **LONG_MAX**, each step of pitch change is going to be very coarse.

9.9 MIDI Input/Output: Structure Interface

The other way to format MIDI I/O is by keeping the MIDI messages in structures or records. This is the method you use when you are just dealing with MIDI messages as data to be transferred from place to place. When reading from a MIDI interface, you have to use this format, since you don’t know what the incoming message is. With the composition interface, you know beforehand what message you are going to construct. The MIDI structure is:

```
typedef long ami_note;       /* 1..128  note number for midi */
typedef long ami_channel;    /* 1..16   channel number */
typedef long ami_instrument; /* 1..128  instrument number */

/* sequencer message types. each routine with a sequenced option has
   a sequencer message assocated with it */
typedef enum {
    st_noteon, st_noteoff, st_instchange, st_attack, st_release,
    st_legato, st_portamento, st_vibrato, st_volsynthchan,
    st_porttime, st_balance, st_pan, st_timbre, st_brightness,
    st_reverb, st_tremulo, st_chorus, st_celeste, st_phaser,
    st_aftertouch, st_pressure, st_pitch, st_pitchrange, st_mono,
    st_poly, st_playsynth, st_playwave, st_volwave
} ami_seqtyp;

/* sequencer message */
typedef struct ami_seqmsg {

    struct ami_seqmsg* next; /* next message in list */
    long               port; /* port to which message applies */
    long               time; /* time to execute message */
    ami_seqtyp         st;   /* type of message */
    union {

        /* st_noteon st_noteoff st_aftertouch st_pressure */
        struct { ami_channel ntc; ami_note ntn; long ntv; };
        /* st_instchange */
        struct { ami_channel icc; ami_instrument ici; };
        /* st_attack, st_release, st_vibrato, st_volsynthchan,
           st_porttime, st_balance, st_pan, st_timbre, st_brightness,
           st_reverb, st_tremulo, st_chorus, st_celeste, st_phaser,
           st_pitch, st_pitchrange, st_mono */
        struct { ami_channel vsc; long vsv; };
        /* st_poly */ ami_channel pc;
        /* st_legato, st_portamento */
        struct { ami_channel bsc; long bsb; };
        /* st_playsynth */ long sid;
        /* st_playwave */ long wt;
        /* st_volwave */ long wv;

    };

} ami_seqmsg;

/* pointer to message */
typedef ami_seqmsg* ami_seqptr;
```

To write a MIDI message structure to an output port, use **wrsynth(p, sp)**, where **p** is the port, and **sp** is a pointer to a MIDI message structure. To read from a MIDI input port to a MIDI message structure, use **rdsynth(p, sp)**, where **p** is the port, and **sp** is a pointer to a MIDI message structure.

Although MIDI outputs will tolerate any speed of messages on output, input MIDI devices will generate messages at whatever the rate the external device requires. Its possible that the input device will wait for your program to read messages, say if the input is supplied from the file, otherwise your code must be prepared to deal with real time devices that need to be read at any possible rate.

MIDI devices are considered to have a maximum message rate of about 1000 messages per second. Thus your program should ideally be able to process messages in no less than a millisecond. This is certainly within the capability of modern computers, which can execute from 1,000 to as much as 1,000,000 assembly instructions per millisecond. When reading from MIDI inputs, simply be aware that failure to read in a timely manner may cause data loss.

9.10 Prerecorded MIDI Files

We don't have to make all our MIDI commands on the fly. In fact, we can forget doing any MIDI, and just play back prerecorded MIDI files. To reduce latency with playing MIDI files, the file must be preloaded with **loadsynth(s, sf)**, where **s** is the logical synthesizer cache number, and **sf** is a string with the name of the MIDI file. To load a MIDI file can mean anything from just storing the name of the file, to reading and converting the MIDI messages the file contains into memory. There are at least 10 positions in the MIDI synthesizer cache, numbered 1 to n. The cached file is played with **playsynth(p,t,s).** where **p** is the output port, **t** is the time to play it, and **s** is the logical cache number. The format is system defined. Note that even though the prerecorded MIDI file has its own timing, it is played relative to the clock start position that is indicated for it. To remove a MIDI file from the cache, use **delsynth(s)**, where **s** is the logical cache number of the MIDI file.

Because it is difficult or impossible to determine when a played MIDI file will end, the function **waitsynth(p)** can be used, where **p** is the synthesizer output port. It blocks until the given synthesizer completes, that is, runs out of messages to process. Thus after scheduling a MIDI file to play, waiting for the synthesizer playing it to finish will play the entire file.

9.11 Waveform Input/Output

Waveform files are how we get arbitrary sounds into and out of the computer. Anything, for any length, can be played via the waveform files. Waveform outputs go out via their own ports separate from MIDI ports. Like MIDI, however, they are selected via logical numbers from 1 to n, where n is the maximum number of waveform output devices on the computer.

Waveforms can be both input and output with sound. To directly input or output waveforms, the I/O must be performed in real time, that is, the program must keep up with the speed of external devices.

The basic calls for waveform devices are:

Function	Description
wavein()	Find number of waveform input devices.
waveout()	Find number of waveform output devices.
openwavein(p)	Open Waveform input device.
closewavein(p)	Close waveform input device.
openwaveout(p)	Open waveform output device.
closewaveout(p)	Close waveform output device.

Where **p** is the port number for the waveform device.

Waveform devices have several important characteristics:

- Number of channels (mono, stereo, quad, etc).
- Sample rate.
- Number of bits used for each sample.
- Signed or unsigned format of each sample.
- Endian (byte order) of each sample.
- Floating point vs. integer format of samples.

You can count on a single sound card or device having a set format or a number of set formats, and having the input formats match the output formats. However, different hardware inputs and outputs on different devices or cards may have different formats. Its also possible for the computer to do “on the fly” conversion of sample formats for you. But if both the inputs and the outputs are capable of such conversions, how to you pick the appropriate sample format? You may specify conversion to/from 8 bits on both input and output, when both input and output are capable of 16 bits, thus not only cutting down the sound quality, but creating more processing work for no reason.

Because of this, sound uses the rules that:

- You examine the format of samples the input device can provide.
- You set the format of samples going to the output device.

Thus **sound** usually picks the best format for input, usually the one that involves no conversion, and has the highest quality sound samples, then your program will set what it wants for the output. If your program is copying from input to output, it should read the input sample format values, then set those same values to the output device.

Function to read input values	Function to set output values	Value
chanwavein(p)	**chanwaveout(p, c)**	Number of channels.
ratewavein(p)	**ratewaveout(p, r)**	Sample rate per second.
lenwavein(p)	**lenwaveout(p, l)**	Length of sample in bits.
sgnwavein(p)	**sgnwaveout(p, s)**	Signed or unsigned (**s**=1, **s**=0).
endwavein(p)	**endwaveout(p, e)**	Endian (**e**=1=big endian, **e**=0=little endian).
fltwavein(p)	**fltwaveout(p, f)**	Floating point (**f**=1=floating point, **f**=0=integer).

The output port must be open to set parameters. For input ports, the parameters can be read at any time.

Some of the parameters have no effect on sound quality. Endian mode and signed mode have no effect on quality. Again the idea is to match the formats to evade the need for conversion.

If you are only outputting samples, you can choose your format. The best format matches the characteristics of the machine you are working on. A little endian format for a little endian machine, etc. The best sound cards today will convert samples in hardware, on the fly.

For most uses, the gold standard is CD disc audio, which is 44,100 samples per second, 16 bits, signed integer samples, and stereo.

Waveform data is output by **wrwave(p, b, l)**, where **p** is the port, **b** is the sample buffer, and **l** is the length. The data in the buffer must be formatted according to the parameters set by the previous parameter calls. The basic calculation for this is:

```
length=bit_length/8
if (bit_length%8) length++;
sample_size = length*channels;
```

The buffer size should be divisible by the sample size.

To keep up with the sample rate defined by the output channel, you have to supply data at least at the rate specified. For example, for the standard CD disc rate of 44.1 khz, with stereo 16 bit samples, you need:

16/8*2*44,100

or 176400 bytes per second. If you supply 512 sample buffers of 2,048 bytes, thats 87 buffers per second, which is a rate today's computers can easily handle. To not experience interruptions or silent periods in the output, there must be no delay in outputting these buffers.

When **wrwave()** is called, the data is automatically buffered for you. This must be, since if the sound is being played from the sample buffer, and **wrwave()** does not return until all of the samples are played, you would have only a single sample time to prepare another full sample buffer.

If you are outputting prerecorded material, the buffer could be any length. For applications such as computer telephony, the latency, or amount of time that passes between the time a user says a word at

the source and the time the receiving user hears it, must be tightly controlled. In telephony, the latency must be kept down below about 50ms. For the 2,048 byte buffer above, the latency is:

2048/(16/8*2)=512 samples
1/44100=0.000022676 second
0.000022676*512=0.011610 second latency.

So well below the 0.050 second requirement. To reduce the overhead in making repetitive calls, but still achieve the target latency, we can do:

0.050/0.000022676=2205 samples per buffer.

or:

2205*4=8820 bytes.

And still meet the latency requirements.

To read waveform data use **rdwave(p,b,l)** where **p** is the input wave port, **b** is the sample buffer, and **l** is the length. It returns the length of data read, but this is only different from the buffer length if the channel is closed during the read, which does not apply for most devices.

The buffer sample format for input wave channels is the same as for output wave channels. Putting this all together, we can show a program that copies the standard input wave port to the standard output wave port:

```
#define BUFLEN 2048

unsigned char buff[BUFLEN]; /* sample buffer */

/* open target ports */
ami_openwavein(AMI_WAVE_IN);
ami_openwaveout(AMI_WAVE_OUT);

/* transfer input parameters to output port */
ami_chanwaveout(AMI_WAVE_OUT, ami_chanwavein(AMI_WAVE_IN));
ami_ratewaveout(AMI_WAVE_OUT, ami_ratewavein(AMI_WAVE_IN));
ami_lenwaveout(AMI_WAVE_OUT, ami_lenwavein(AMI_WAVE_IN));
ami_sgnwaveout(AMI_WAVE_OUT, ami_sgnwavein(AMI_WAVE_IN));
ami_endwaveout(AMI_WAVE_OUT, ami_endwavein(AMI_WAVE_IN));
ami_fltwaveout(AMI_WAVE_OUT, ami_fltwavein(AMI_WAVE_IN));

/* transfer data continuously */
while (1) {

    ami_rdwave(sport, buff, BUFLEN);
    ami_wrwave(dport, buff, BUFLEN);

}
```

This is from the sample program connectwave.c.

You are not limited to just moving sound data to and from output or input sound ports. Computers have sufficient bandwidth to create, filter or perform complex operations on audio in real time. For example, this code generates a sine wave to the standard output wave device:

```
#define SIZEBUF 2048
#define PI 3.14159

double angle; /* start sine angle */

rate = 44100; /* set sample rate */

ami_openwaveout(AMI_SYNTH_OUT);          /* open output wave port */
ami_chanwaveout(AMI_SYNTH_OUT, 1);       /* one channel */
ami_ratewaveout(AMI_SYNTH_OUT, 44100);   /* CD sample rate */
ami_lenwaveout(AMI_SYNTH_OUT,  16);      /* 16 bits */
ami_sgnwaveout(AMI_SYNTH_OUT,  TRUE);    /* signed */
ami_endwaveout(AMI_SYNTH_OUT,  FALSE);   /* little endian */
ami_fltwaveout(AMI_SYNTH_OUT,  FALSE);   /* integer */

while (1) {

    for (i = 0 ; i < SIZEBUF ; i++) {

        buf[i] = (short)(SHRT_MAX*sin(angle));
        angle += 2*PI*freq/rate;
        if (angle > 2 * PI) angle -= 2*PI;

    }
    ami_wrwave(AMI_SYNTH_OUT, (byte*)buf, SIZEBUF);

}
```

This is from the sample program genwave.c.

9.12 Prerecorded Waveform Files

An entire waveform file can be played into an output waveform device. Waveform files are usually very system dependent, so the exact format of the file will be different for different systems.

Waveform files are cached to reduce latency in start time. Wave cache numbers are from 1 to n, where n is the maximum number of cached waveform files. At least 10 such files can be loaded into cache at once.

A waveform file is loaded with **loadwave(w,s)** where **w** is the logical cache number, and **s** is the string containing the wave file name. The waveform file is played with **playwave(p,t,w),** where **p** is the waveform output port, **t** is the time to play it, and **w** is the cache number. A waveform file can be deleted from cache with **delwave(w)**, where **w** is the logical cache number.

A wave id can also be taken with **getwaveid()**, which returns an id the program did not name. These ids are negative, counting down from -1, and an id is held from the moment it is taken until **delwave(w)** releases it, whether or not a file was loaded to it. A library that plays a sound of its own takes its id this way, so its wave and any the program loads never collide.

As with MIDI files, waveforms have their own timing, and are simply played relative to the indicated start time.

The playback volume for waveform files is adjusted with **volwave(p,t,v),** where **p** is the output wave port, **t** is the time to change the volume, and **v** is the **LONG_MAX** ratioed volume value.

It is possible that the implementation will only be able to play one waveform file at a time. In this case, the behavior will be to stop any currently playing waveform file and start the new one, if a new waveform play is ordered before the previous one has finished. High quality implementation will be capable of playing multiple waveform files at once, typically via mixing of the output.

When a waveform file is played, it is difficult or impossible to determine when the waveform playback will be done. For this the **waitwave(p)** function exists. It will wait until all wave output activity on the given wave output port **p** is complete, then return.

If multiple waveforms are being played on a single output port, **waitwave()** will wait until all waveforms are complete.

9.13 C++ sound interface

Sound has a C++ wrapper, in cpp/sound.cpp with its declarations in hpp/sound.hpp, following the pattern of the terminal and graphics wrappers. A C++ program includes <sound.hpp> and works inside the sound namespace, and the wrapper is carried in the standard libraries, so nothing extra is linked.

The namespace carries the whole of the sound interface with the prefixes stripped: every function without its ami_, and the two hundred and eight constants of the header in lower case as C++ constants rather than macros, note_c to inst_gunshot, octave_1 to octave_11, chan_drum, and the default ports synth_out, synth_in, wave_out and wave_in. The sequencer types seqtyp, seqmsg and seqptr appear the same way, laid out identically to their C forms.

Every call that takes a port also has an overload without one, meaning the default port, the way the terminal wrapper defaults stdout: noteon(0, 1, note_c+octave_6, v) plays on the default synthesizer.

Beyond the procedural calls, two objects hold ports. A synth holds a synthesizer output port and a synthesizer input port; a wave holds the same pair of waveform ports. Opening sets the port the object speaks for, and every call after that leaves the port off:

```
synth s;

s.opensynthout(); /* the default port, or opensynthout(2) */
s.instchange(0, 1, inst_acoustic_grand);
s.noteon(0, 1, note_c+octave_6, v);
s.noteoff(0, 1, note_c+octave_6, 0);
```

An object holds one port of each direction, so a single synth can serve a keyboard in and a synthesizer out, and a single wave can stand for a recording and its playback, the format of one copied to the other from the same object:

```
wave w;
```

```
w.openwavein();
w.openwaveout();
w.chanwaveout(w.chanwavein()); /* the input format, copied over
*/
w.ratewaveout(w.ratewavein());
```

The destructor closes whatever the object still holds, so a port cannot leak past an early return, and opening over a held port lets go of the old one first. A call on a direction that was never opened is an error, caught the way the C interface catches any unopened port. Unlike the term and graph objects of the other wrappers, these hook nothing: any number of them may exist at once, one per port in use, so two synthesizers can be played at the same time from two objects.

The calls that take no port -- the sequencer clocks, the device counts, and the wave and MIDI file caches loadsynth, delsynth, loadwave and delwave -- are functions of the namespace alone.

Functions and Procedures in sound

void [ami_]aftertouch(long p, long t, [ami_]channel c, [ami_]note n, long at);

Set aftertouch. Sets aftertouch amount, for synthesizer output port p, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **at**.

void [ami_]attack(long p, long t, [ami_]channel c, long at);

Set attack time. Sets the attack time for synthesizer output port p, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed time **at**.

void [ami_]balance(long p, long t, [ami_]channel c, long b);

Set channel balance. Sets the right left balance for synthesizer output port **p**, at time **t**, for channel **c**, to -**LONG_MAX**..**LONG_MAX** ratioed value **b**. -**LONG_MAX** is full left, **LONG_MAX** is full right, and 0 is centered.

void [ami_]brightness(long p, long t, [ami_]channel c, long b);

Set brightness. Sets brightness amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **b**.

void [ami_]celeste(long p, long t, [ami_]channel c, long ce);

Set celeste. Sets celeste amount, for synthesizer output port p, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **ce**.

long [ami_]chanwavein(long p);

Returns the number of channels in samples from waveform input port **p**.

void [ami_]chanwaveout(long p, long c);

Sets the number of channels **c** on an output waveform device **p**. The device must be open.

void [ami_]chorus(long p, long t, [ami_]channel c, long cr);

Set chorus. Sets chorus amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **cr**.

void [ami_]closesynthin(long p);

Close input synthesizer. Closes the input synthesizer by the logical number **p**.

void [ami_]closesynthout(long p);

Close output synthesizer. Closes the output synthesizer by the logical number **p**.

void [ami_]closewavein(long p);

Closes the wave input device **p**.

void [ami_]closewaveout(long p);

Close waveform device. Closes the logical waveform device **p**.

long [ami_]curtimein(void);

Get current input sequencer time. Returns the current sequencer time, in 100 Microsecond counts. The count is a long, and does not wrap in any span of time worth planning for.

long [ami_]curtimeout(void);

Get current output sequencer time. Returns the current sequencer time, in 100 Microsecond counts. The count is a long, and does not wrap in any span of time worth planning for.

void [ami_]delsynth(long s);

Delete synthesizer file from cache. **s** is the logical synthesizer id. The indicated cache entry is cleared, and the logical id can be reused.

void [ami_]delwave(long w);

Delete waveform file from cache. w is the logical wave cache id. The indicated cache entry is cleared, and the logical id can be reused.

long [ami_]endwavein(long p);

Returns the big or little endian format of samples from waveform input port **p**. Returns 1 is the samples are big endian, and 0 if little endian.

void [ami_]endwaveout(long p, long e);

Sets the endian format **e** for the waveform port **p**. **e** is 1 for big endian, and 0 for little endian. Note that it makes no difference to the bit length what the endian format is. The device must be open.

The endian format should be set to match either the incoming stream or the natural endian mode of the current machine.

long [ami_]fltwavein(long p);

Returns the floating point format of samples from waveform input port **p**. Returns 1 if the samples are floating point, and 0 if integer.

void [ami_]fltwaveout(long p, long f);

Sets the floating point or integer format **f** of the waveform port **p**. **f** is 1 if the samples are in floating point format, and 0 if in integer format. Note that it makes no difference to the bit length what the floating point or integer format is. The device must be open.

Commonly, only binary power multiples have floating point formats, IE, 2, 4, 8 bytes, etc. The most common formats in use are the IEEE 754 formats for 2 and 4 bytes (16 bits and 32 bits).

void [ami_]getparamsynthin(long p, string name, string value, long len);

As getparamsynthout(), for a synthesizer input port.

void [ami_]getparamsynthout(long p, string name, string value, long len);

Get a device parameter from synthesizer output port p: the value of the parameter named name is returned in the buffer value, with length len. The result follows the critical buffer convention: a value that exactly fills the buffer is not zero terminated, a shorter one is, and a value that cannot fit is an error. A device that does not know the name returns an empty string, which is all the ALSA devices ever return.

void [ami_]getparamwavein(long p, string name, string value, long len);

As getparamsynthout(), for a waveform input port.

The string returned will include the name of the device connected to the port, and may include descriptive text as well. It will fit on a single line.

void [ami_]getparamwaveout(long p, string name, string value, long len);

As getparamsynthout(), for a waveform output port.

long [ami_]getwaveid(void);

Take a wave id the program did not name. The ids run -1, -2 and so on down to -n, the first free one each time, and the id is held until delwave() releases it. A library that plays a sound of its own takes its id this way, so its wave and any the program loads never collide.

void [ami_]instchange(long p, long t, [ami_]channel c, [ami_]instrument i);

Change instrument. Changes the instrument assigned to a channel, for output port **p**, at time **t**, for channel **c**, to instrument **i**.

void [ami_]legato(long p, long t, [ami_]channel c, long b);

Set legato. Sets legato mode on or off, for synthesizer output port **p**, at time **t**, for channel **c**, to on/off value **b**.

long [ami_]lenwavein(long p);

Returns the bit length in samples from waveform input port **p**.

void [ami_]lenwaveout(long p, long l);

Sets the bit length of samples to **l** for the output waveform device **p**. The bit length of samples is rounded up to the next 8 bits for the sample size in bytes. The total sample size is found by round(l, 8)*channels, where round is a function to round up to the nearest 8 bits. The device must be open.

void [ami_]loadsynth(long s, string sf);

Load synthesizer file into cache. **s** is the logical number of the cache position, from 1 to n where n is the maximum number of cache positions, and **sf** is a string giving the full filename and path of the synthesizer file. The synthesizer file remains in the cache until it is specifically removed by **[ami_]delsynth()**.

Caching of synthesizer files allows the system to remove any possible latency from the time the playing of a synthesizer file is ordered to the time it actually starts playing. The function may completely load the file into memory, or it may just save the filename.

From 1 to 10 synthesizer cache positions are guaranteed, but the implementation may provide more.

The extension, type and format of the synthesizer file is system dependent. More that one format may be allowed.

The filename may be fully pathed.

void [ami_]loadwave(long w, string fn);

Loads a waveform file with the name **fn** to the logical waveform cache **w**. The waveform cache logical numbers are from 1 to n, where n is the maximum number of waveform files that can be cached. This is guaranteed to be at least 10 files.

The purpose of caching waveform files is to allow the system to reduce or eliminate latency of the start time of the waveform output. **loadwave()** may do anything from completely load the waveform file into the memory, to simply saving the name of the file for later load.

The filename for the waveform file may be fully pathed.

void [ami_]mono(long p, long t, [ami_]channel c, long ch);

Set mono mode. Sets mono mode for synthesizer output port **p**, at time **t**, for channel **c**, for the number of channels **ch**. See MIDI specification for details.

void [ami_]noteoff(long p, long t, [ami_]channel c, [ami_]note n, long v);

Stop note. Stops a note for synthesizer **p**, in channel **c**, with note, and 0..**LONG_MAX** ratioed volume **v**. **v** is usually ignored on a **noteoff**.

void [ami_]noteon(long p, long t, [ami_]channel c, [ami_]note n, long v);

Start note. Starts a note for synthesizer **p**, in channel **c**, with note n, and 0..**LONG_MAX** ratioed volume **v**.

void [ami_]opensynthin(long p);

Open input synthesizer. Opens the input synthesizer by the logical number **p**, where p is 1..**synthin()**.

void [ami_]opensynthout(long p);

Open output synthesizer. Opens the output synthesizer by the logical number **p**, where p is 1..**synthout()**.

void [ami_]openwavein(long p);

Opens the wave input device **p**.

void [ami_]openwaveout(long p);

Open waveform device. Opens the logical waveform device **p**, where **p** is 1..**waveout()**.

void [ami_]pan(long p, long t, [ami_]channel c, long b);

Set channel pan. Sets the right left pan for synthesizer output port **p**, at time **t**, for channel **c**, to -**LONG_MAX**..**LONG_MAX** ratioed value. -**LONG_MAX** is full left, **LONG_MAX** is full right, and 0 is centered.

void [ami_]phaser(long p, long t, [ami_]channel c, long ph);

Set phaser. Sets phaser amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **ph**.

void [ami_]pitch(long p, long t, [ami_]channel c, long pt);

Set pitch bend. Sets the pitch "bend", or change amount, for synthesizer output port **p**, at time **t**, for channel **c**, to -**LONG_MAX**..**LONG_MAX** ratioed value **pt**. **pt** value is -**LONG_MAX** for full down range, **LONG_MAX** for full up range, and 0 for neutral (on note) pitch. The amount of pitch range is set by the pitchrange procedure, and defaults to one note down and one note up.

void [ami_]pitchrange(long p, long t, [ami_]channel c, long v);

Set pitch bend range. Sets the total amount of pitch change that can be reached by the pitch command, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **v**. 0 disables the pitch command, and **LONG_MAX** allows it to reach all 128 notes of MIDI. Note that increasing the range of the pitch command decreases its resolution.

void [ami_]playsynth(long p, long t, long s);

Play MIDI synthesizer file. Plays the MIDI instructions from the file by the cache position by logical number **s**, for output synthesizer **p**, at time **t**.

void [ami_]playwave(long p, long t, long w);

Play waveform file. Plays the waveform file by the logical cache number **w**, for output waveform device **p**, at time **t**. If the time **t** is left off, it defaults to 0.

void [ami_]poly(long p, long t, [ami_]channel c);

Set polyphonic mode. Sets polyphonic mode for synthesizer output port **p**, at time **t**, for channel **c**. Reverses the effect of a mono operation.

void [ami_]portamento(long p, long t, [ami_]channel c, long b);

Set portamento. Sets portamento mode on or off, for synthesizer output port **p**, at time **t**, for channel c, to on/off value **b**.

void [ami_]porttime(long p, long t, [ami_]channel c, long v);

Set portamento time. Sets portamento time, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **v**.

void [ami_]pressure(long p, long t, [ami_]channel c, long pr);

Set pressure. Sets pressure amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **pr**.

long [ami_]ratewavein(long p);

Returns the sample rate, in samples per second, from waveform input port **p**.

void [ami_]ratewaveout(long p, long r);

Sets the rate **r**, in samples per second, for the output waveform device **p**.

void [ami_]rdsynth(long p, [ami_]seqptr sp);

Read a synthesizer structure at pointer **sp** from synthesizer port **p**. The function does not allocate the structure, it only fills it out. The time provided in the structure is that of the input synthesizer clock, which may be zero if it has not been started. The timing provided on input is up to the system. It can be provided by the synthesizer input clock, if that has been started. Alternately it could be set at the MIDI input device (so called “device timing”), or it could read it in with the MIDI stream. Pitch bend read from a device is expanded to the full API range, the inverse of the reduction made on output: what pitch() sent reads back as the same value to within the fourteen bit step of the wire.

If the synthesizer input clock is not started, the input time is set to zero on all samples.

long [ami_]rdwave(long p, byte* buff, long len);

Read a group of waveform samples into buffer b, with length l, from the input waveform port p. The length is in samples, one sample being one frame of every channel, and the number of samples read is returned: the buffer holds l*round(b, 8)/8*c bytes, where b is the bit length of samples, c is the number of channels, and round rounds up to the nearest 8 bits. The device must be open.

If the input wave device is real time, the caller is responsible for reading buffers from the device at a rate sufficient to satisfy the input without overruns in the input stream (and resulting loss of data). At the same time, the caller is responsible for insuring that the latency of the stream is sufficiently low, typically less than 50 milliseconds for telephony applications.

The caller allocates the buffer. If the function itself buffers data, it will copy the data to the caller provided buffer.

rdwave() will block or not block the caller to satisfy its own input stream requirements. Typically this will be to copy the incoming data from a buffer that is sized for latency concerns, and the caller will only be blocked if there is no data ready from the input device. Thus the size of the buffer used to call **rdwave()** may or may not have any relationship to the internal buffer size.

void [ami_]release(long p, long t, [ami_]channel c, long rt);

Set release time. Sets the release time for synthesizer output port **p**, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed time **at**.

void [ami_]reverb(long p, long t, [ami_]channel c, long r);

Set reverb. Sets reverb amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **r**.

long [ami_]setparamsynthin(long p, string name, string value);

As setparamsynthout(), for a synthesizer input port.

long [ami_]setparamsynthout(long p, string name, string value);

Set a device parameter on synthesizer output port p: the parameter is named by the string name and given the string value. The parameters a device carries are its own: a plug-in defines what may be set (see “Sound module plugins”). Returns nonzero if the parameter was not accepted. The ALSA devices carry no parameters, and decline every set.

long [ami_]setparamwavein(long p, string name, string value);

As setparamsynthout(), for a waveform input port.

long [ami_]setparamwaveout(long p, string name, string value);

As setparamsynthout(), for a waveform output port.

long [ami_]sgnwavein(long p);

Returns the signed or unsigned format of samples from waveform input port **p**. Returns 1 is the samples are signed, and 0 if unsigned.

void [ami_]sgnwaveout(long p, long s);

Sets the signed format **s** for output waveform device **p**. **s** is 1 is the sample format is signed, and 0 if it is not. Note that it makes no difference to the bit length what the signed or unsigned format is. The device must be open.

void [ami_]starttimein(void);

Start time for the input MIDI sequencer. Starts the sequencer running/ If the sequencer is already running, it will be restarted at 0.

void [ami_]starttimeout(void);

Start time for the output MIDI sequencer. Starts the sequencer running. If the sequencer is already running, it will be restarted at 0.

void [ami_]stoptimein(void);

Stop input MIDI sequencer. Halts the sequencer timer, and releases it.

void [ami_]stoptimeout(void);

Stop output MIDI sequencer. Halts the sequencer timer, and releases it.

long [ami_]synthin(void);

Find number of input synthesizers. Returns the total input synthesizers in the system.

void [ami_]synthinname(long p, string name, long l);

Returns the name of the synthesizer input port **p** in the string **n**, with buffer length **l**. The name is returned by the critical buffer convention: a name that exactly fills the buffer is not zero terminated, a shorter one is, and a name that cannot fit at all is an error.

The string returned will include the name of the device connected to the port, and may include descriptive text as well. It will fit on a single line.

long [ami_]synthout(void);

Find number of output synthesizers. Returns the total output synthesizers in the system.

void [ami_]synthoutname(long p, string n, long l);

Returns the name of the synthesizer output port **p** in the string **n**, with buffer length **l**. The name is returned by the critical buffer convention: a name that exactly fills the buffer is not zero terminated, a shorter one is, and a name that cannot fit at all is an error.

The string returned will include the name of the device connected to the port, and may include descriptive text as well. It will fit on a single line.

void [ami_]timbre(long p, long t, [ami_]channel c, long tb);

Set timbre. Sets timbre amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **tb**.

void [ami_]tremulo(long p, long t, [ami_]channel c, long tr);

Set tremulo. Sets tremulo amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **tr**.

void [ami_]vibrato(long p, long t, [ami_]channel c, long v);

Set vibrato. Sets vibrato amount, for synthesizer output port p, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **v**.

void [ami_]volsynthchan(long p, long t, [ami_]channel c, long v);

Set volume for channel. Sets volume for channel, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..**LONG_MAX** ratioed value **v**.

void [ami_]volwave(long p, long t, long v);

Set waveform volume. Sets the output waveform device volume for logical device **p**, at time **t**, to 0..**LONG_MAX** ratioed value **v**.

void [ami_]waitsynth(long p);

Waits until all pending activity on synthesizer port **p** has finished, then returns.

void [ami_]waitwave(long p);

Waits until all waveform activity on waveform output port **p** is complete. This function is typically used to wait until the end of a **playwave()**, which is difficult or impossible to know when the playback is complete.

long [ami_]wavein(void);

Find number of waveform input devices. Returns the total number of waveform input devices in the system.

void [ami_]waveinname(long p, string name, long l);

Returns the name of the waveform input port **p** in the string **n**, with buffer length **l**. The name is returned by the critical buffer convention: a name that exactly fills the buffer is not zero terminated, a shorter one is, and a name that cannot fit at all is an error.

long [ami_]waveout(void);

Find number of waveform output devices. Returns the total number of waveform output devices in the system.

void [ami_]waveoutname(long p, string name, long l);

Returns the name of the waveform output port **p** in the string **n**, with buffer length **l**. The name is returned by the critical buffer convention: a name that exactly fills the buffer is not zero terminated, a shorter one is, and a name that cannot fit at all is an error.

The string returned will include the name of the device connected to the port, and may include descriptive text as well. It will fit on a single line.void [ami_]wrsynth(long p, [ami_]seqptr sp);

Write a synthesizer structure at pointer **sp** to synthesizer port **p**. The time for the structure will be examined, and if zero, it will be output immediately. Otherwise it is inserted into the synthesizer pending list.

If the function will store the structure for later output, a copy of it will be created. The structure can be reused immediately by the caller.

void [ami_]wrwave(long p, byte* b, long l);

Write a group of waveform samples from buffer b, with length l, to the output waveform port p. The length is in samples, one sample being one frame of every channel: the buffer holds l*round(b, 8)/8*c bytes, where b is the bit length of samples, c is the number of channels, and round rounds up to the nearest 8 bits. The device must be open.

If the output wave device is real time, the caller is responsible for writing buffers to the device at a rate sufficient to satisfy the output without gaps or silent periods in the output stream. At the same time, the caller is responsible for insuring that the latency of the stream is sufficiently low, typically less than 50 milliseconds for telephony applications.

After the function is called, the buffer is free to be changed by the caller.

wrwave() will block or not block the caller to satisfy its own output stream requirements. Typically this will be to copy the incoming data to a buffer that is sized for latency concerns, and the caller will only be blocked if the internal buffer overflows. Thus the size of the buffer used to call **wrwave()** may or may not have any relationship to the internal buffer size.

10 Networking Library



network gives ANSI C the ability to transfer data over a network such as the internet. It does this by connecting ANSI C files to network resources. Because of the use of standard file mechanisms, few added calls are needed.

network supports both continuous streams of data as well as messaging. It supports both plain text and encrypted communications via SSL/TLS. It supports both configuration as a client and a server.

10.1 IPv4 and IPv6 addresses

IPv4 and IPv6 addresses have formats unique from each other. IPv4 addresses are represented in 32 bit unsigned longs, and IPv6 addresses are represented in 128 bits as two unsigned long long (64 bits each). This format was chosen because 128 bit variables are not universally implemented in ANSI C. 64 bit is implemented widely even on 32 bit machines. It is the most universal format.

10.2 Standard stream channels

Streaming data is a series of bytes sent to and from a network connection. The data is secured in that any errors in the data are either fixed automatically or the program is stopped. The data sent or received is guaranteed both to be correct and in the same order as sent. When data is received, the call will wait until all data requested is received. Any amount of time may pass before that occurs, which is why network connections typically use with multiple threads, with one for each input and output.

To open a new network connection, **opennet(a,p,s)** is used, where **a** is the IP address, **p** is the port, and **s** indicates the connection is to be secured. The file pointer used to send and receive data from the connection is returned. To close network connections, the standard ANSI C **fclose()** is used. **opennet()** uses an address/port pair to indicate the network address of the server, and the port within the server. The address of a server, as determined from its name in characters, is found with **addrnet(n,a)** where **n** is the name of the server or its IP address in dotted form, and **a** is the resulting IPv4 address.

When a remote network port is opened, **network** treats the connection as a single synchronous channel going to and from the remote resource. All of the standard ANSI C I/O functions can be applied to such network connections, including **printf()**, **fprintf()**, **fscanf()**, **fgets()**, etc.

When working with IPv6 addresses, the calls **opennetv6(ah,al,p,s)** and **addrnetv6(n,ah,al)** are used. This is necessary because IPv6 addresses are not a superset of IPv4 addresses, but have a new and different format.

10.3 Secure channels

A secure channel with streaming data encodes all of the data passing over the interface so that, even though the client and the server can read it, third parties on the network cannot. To set a network connection as secure, the “secure” flag is simply set in **opennet(a,p,s)** or **opennetv6(ah,al,p,s)**, that’s all that is required. There is also a set of certificate functions if you need to verify or examine the contents of certificates.

10.4 Message based communications

An alternative to stream based network communication is message based. In this model, a fixed length of data is sent and received. It may or may not be guaranteed to be delivered. The order in which it is received is also not guaranteed. All that is guaranteed is that if a message contains data errors, it will not be delivered. The sender is responsible for handling interruptions in the data flow.

Message based communication can be a better match for certain communications types. For example, an audio call or broadcast would not be a good match for a stream, because an error in the data might stall the channel attempting to fix data corruption, and that would corrupt the timing of the data, causing it to fall behind the sender. Using message based instead allows the application to implement its own error handling procedure[6].

A message channel is opened with **openmsg(a,p,s)**, where **a** is the address, **p** is the port, and **s** indicates the channel is to be secured. The function returns the logical number of the message file. If IPv6 is to be used, the call is **openmsgv6(ah,al,p,s)**.

Message channels do not use ANSI C file functions. Reading a message is done with **rdmsg(fn,m,l)** where **fn** is the logical message channel id, **m** is a pointer to the message buffer, and **l** is the maximum byte length of the buffer. It returns the actual number of bytes read. Writing a message is done with **wrmsg(fn,m,l)** where **fn** is the logical message channel id, **m** is the message buffer, and l is the size of the data in the buffer.

Message channels are closed with **clsmsg(fn)**, where **fn** is the logical message channel id.

The maximum possible size of messages for a given address is found by **maxmsg(a,s)**, where **a** is the IP address and **s** indicates a secured channel. The two differ: a secured message carries the framing of its record inside the datagram, and what is left over is what the message may fill. For IPv6, the maximum is found with **maxmsgv6(ah,al,s)**. For standard IP, 1500 bytes is a common maximum, but so called “jumbo packets” can reach 64kb.

10.5 Reliable messaging

Messaging is widely used for interprocessor communication in computer clusters. The network carrying the messages may only exist in the same machine, or same internal network. The hardware may implement guaranteed messaging, so that the application does not have to deal with handling errors in the data, or reordering of messages. The program can determine if this is true with the call **relymsg(a)**, where **a** is the IP address. It returns 1 if the message channel is reliable, and 0 if not. For IPv6 connections, **relymsgv6(ah,al)** is used.

Another way of looking at reliable messaging is that if an error were to occur in data transfer, it would be fatal, and not just discarding a bad message, and cause the program to terminate.

[6]Which can include simply dropping the message. This may be appropriate for audio or video communications.

network assumes that if a message will be sent and received on the current machine without being transmitted over a wire, that it will be reliable. Outside of that, it needs an indication that the hardware supports reliable messaging.

10.6 Serving Connections

So far we have talked about establishing connections to outside servers. We can also act as a server, waiting for inbound connections on a port. After such a connection is established, the server can send and receive data just as for outbound connections.

To wait on a stream connection, the function **waitnet(p,s)** is used, where **p** is the port number, and **s** indicates the connection is to be secured. The file pointer used to talk to the connection is returned. The IP address of the connection is the IP address of the machine the program is being run on, and it can be either IPv4, IPv6, or both. Any number of threads in the same application can open server connections on the same port, but other programs in the same system will be blocked from opening connections with that port.

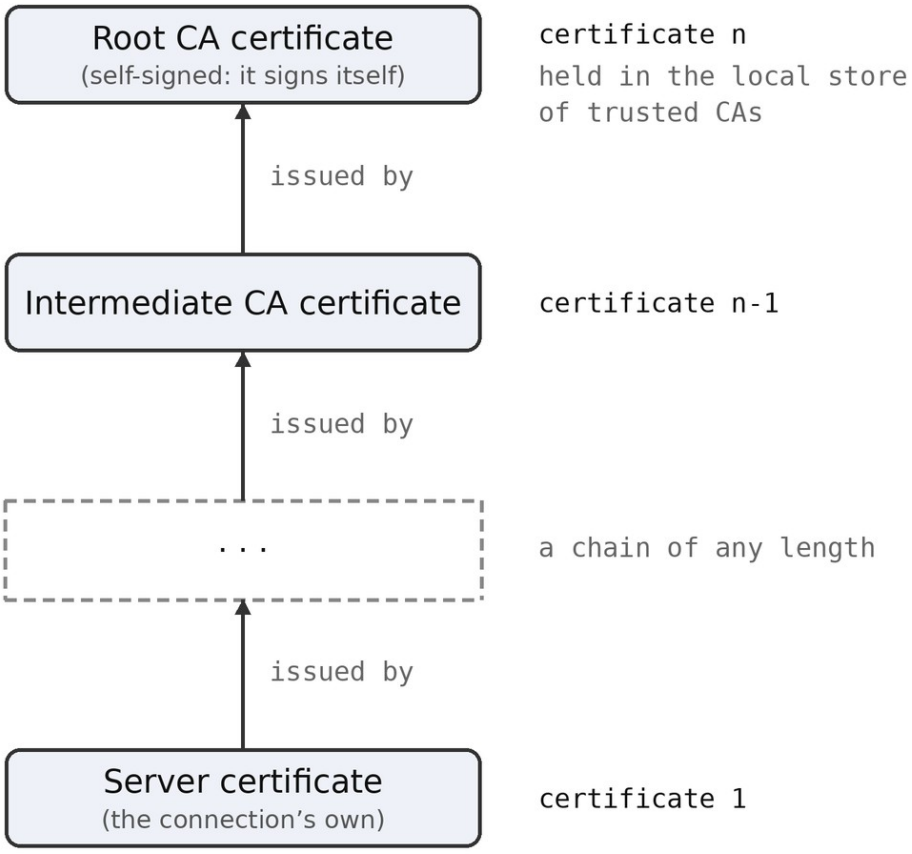
For serving message based connections, the function **waitmsg(p,s)** is used. It returns the logical channel number. The connection is then read and written with **rdmsg()** and **wrmsg()**.

10.7 Managing certificates

When a secure connection is created, the connection is secured by public key encryption. To make a channel, either stream or message, both the public keys and a “certificate” are exchanged. The key makes sure that only data from the given server or client is being received, and the certificate allows verification of who is on the other side of the communication. It is essentially a series of fields of data from the creator of the certificate, such as its IP address, the name of the organization, etc., that has been signed using the communications keys.

To check that the certificate is valid, it can be checked against a known database of so called “trusted” certificates. This would be the case for say, a company you know and trust with a privately created certificate. For others, say an arbitrary web site, the method used is a “chain of trust”. A chain of trust is a series of certificates traceable back to a central “certificate authority”. Each company that participates in a chain of trust purchases a certificate from a provider who can vouch for the fact that the company purchasing the certificate is who they say they are. Thus you would get both the certificate for the company that purchased it, and the certificate for the issuing authority. Issuing authorities can issue certificates to companies that are themselves authorized to issue certificates, and so on. These certificate “chains” can be any length. The benefit of the system is that only a few certificates ultimately need be held by the program accessing the servers, and a connection can be verified by following the chain of certificates.

The chain runs from the connection’s own certificate back to a root the program already trusts:



How you manage certificates depends on your type of program. It is possible to create your own certificates, and you would keep a cache or local store of certificates for such "private" machines. Programs that use many machines on the public internet would store a list of so-called "Certificate Authorities" or CAs certificates, then you would traverse the chain of certificates and check those certificates exist in the local cache. **network** does not manage a certificate cache. This is typically done with a hashing scheme.

A certificate for streams can be obtained by **certnet(f, w, s, l)**, where **f** is the connection file, **w** is which certificate in the chain to get, from 1 to n, where n is the last certificate in the chain. **s** is the buffer for the certificate, and **l** is the length of the buffer. The function returns the number of bytes read for the certificate. If there is no certificate by that number, 0 is returned. For message based channels, **certmsg(fn,w,s,l)** is used.

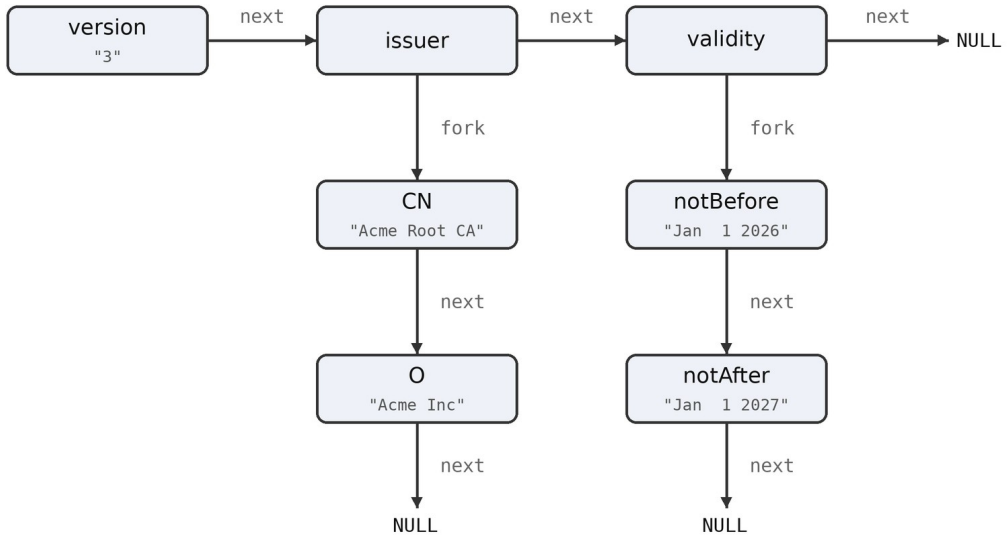
Certificates for the previous calls are returned as a block of encoded data. To actually examine the certificate data, it is broken down into a tree structured series of data fields, of the format:

```
/* name - value pair list */
typedef struct ami_certfield {

    string                 name;     /* name of field */
    string                 data;     /* content of field */
    long                   critical; /* is a critical X509 field */
    struct ami_certfield* fork;      /* sublist */
    struct ami_certfield* next;      /* next entry in list */

} ami_certfield, *ami_certptr;
```

The tree such a list forms: **next** chains the fields of one level, and **fork** opens a field's sublist:



Thus each data item in the certificate contains a name and data for that field. It may also be a branch (a subcategory) which contains any number of sublists. It may also be marked as an essential X509 data field.

A certificate tree for a given certificate in a stream connection can be read by **certlistnet(f,w,l)** where **f** is the connection file, **w** is the number of the certificate in the chain, 1 to n, and **l** is a pointer to the resulting certificate data tree. For message connections, use the function **certlistmsg(fn,w,l)**, where **fn** is the file id, **w** indicates which certificate, and **l** is the data tree.

After a certificate data tree is used, it can be recycled by **certlistfree(l)**, where **l** is a pointer to the certificate tree. This will recycle all of the data entries in the tree.

10.8 C++ network interface

Network has a C++ wrapper, in cpp/network.cpp with its declarations in hpp/network.hpp, following the pattern of the other wrappers. A C++ program includes <network.hpp> and works inside the network namespace, and the wrapper is carried in the standard libraries, so nothing extra is linked.

The namespace carries the whole of the network interface with the prefixes stripped, and the certificate field record laid out identically to its C form. The secure flag is defaulted off everywhere, so a plain connection asks only for an address and a port: opennet(addr, 80).

Two objects hold connections. A net holds one stream connection, opened as a client by address or by name -- an overload makes the addrnet lookup on the way -- or as a server with waitnet. It converts to FILE*, so the stdio calls read and write it directly, which is the file paradigm of the C interface kept whole:

```
net c;

c.opennet("www.example.com", 443, 1); /* by name, secured */
fputs("GET / HTTP/1.0\r\n\r\n", c);
fflush(c);
while (fgets(buff, sizeof(buff), c)) { ... }
```

The certificate queries are methods of the connection: c.certnet(1, cert, len) fetches the raw certificate, c.certlistnet(1, &list) the decoded one. A msg object holds one message channel the same way, with openmsg or waitmsg to hold it, wrmsg and rdmsg to speak, clsmsg to let it go, and certmsg and certlistmsg for its certificates.

The destructor closes whatever the object still holds, so a connection cannot leak past an early return, and opening over a held connection lets the old one go. Close a net with close() or the destructor, never with fclose through the converted pointer, or it would be closed a second time. The objects hold handles, not hooks, so any number may exist: a mail program holds one net for its IMAP connection and another for its SMTP.

Functions and Procedures in network

void [ami_]addrnet(const string name, unsigned long* addr);

addrnet takes the logical name of a server in string **name** and finds the address number **addr** for it. Such names are formatted according to the needs of the network.

Network connections are ended by the end of the program, or by using the standard file **close()** procedure on the handle of the connection.

void [ami_]addrnetv6(const string name, unsigned long long* addrh, unsigned long long* addrl);

addrnet takes the logical name of a server in string **name** and finds the IPv6 address number **addrh** and **addrl** for it. Such names are formatted according to the needs of the network.

Network connections are ended by the end of the program, or by using the standard file **close()** procedure on the handle of the connection.

void [ami_]certlistfree([ami_]certptr* list);

Recycles the given certificate name/value format tree **list**.

void [ami_]certlistmsg(long fn, long which, [ami_]certptr* list);

Reads a certificate from the given message file **fn**. The certificate is parsed into a data tree in name/value format, and returned as a pointer to **list**.

The tree is allocated from heap store, and should be recycled when it is no longer needed.

void [ami_]certlistnet(FILE *f, long which, [ami_]certptr* list);

Reads a certificate from the given stream file **f**. The certificate is parsed into a data tree in name/value format, and returned as a pointer to **list**.

The tree is allocated from heap store, and should be recycled when it is no longer needed.

long [ami_]certmsg(long fn, long which, string cert, long len);

Reads a certificate from the given message file **fn**. The certificate is placed in the buffer **cert**, with length **len**. The actual length of the certificate is returned. Certificates are strings, and may include line feeds to break it up into lines.

long [ami_]certnet(FILE* f, long which, string cert, long len);

Reads a certificate from the given stream file **f.** The certificate is placed in the buffer **cert**, with length **len**. The actual length of the certificate is returned. Certificates are strings, and may include line feeds to break it up into lines.

void [ami_]clsmsg(long fn);

Closes the message channel by logical id **fn**.

long [ami_]maxmsg(unsigned long addr, long secure);

Returns the maximum size of a message for the IP address **addr**.

long [ami_]maxmsgv6(unsigned long long addrh, unsigned long long addrl, long secure);

Returns the maximum size of a message for the IPv6 address **addrh** and **addrl**.

long [ami_]openmsg(unsigned long addr, long port, long secure);

Opens a message channel with the address **addr**. If **secure** is true, then the messages will be encrypted.

long [ami_]openmsgv6(unsigned long long addrh, unsigned long long addrl, long port, long secure);

Opens a message channel with the IPv6 address **addrh and addrl**. If **secure** is true, then the messages will be encrypted.

FILE* [ami_]opennet(unsigned long addr, long port, long secure);

The server is indicated by a logical address number **addr**, whose exact meaning and format is dictated by the network itself. A logical port number **port** selects which resource within the server is being accessed. For the internet, this is a fixed constant that gives the exact service being asked of the far server.

The file used to read and write the connection is returned.

FILE* [ami_]opennetv6(unsigned long long addrh, unsigned long long addrl, long port, long secure);

The server is indicated by a logical IPv6 address number **addrh** and **addrl**, whose exact meaning and format is dictated by the network itself. A logical port number **port** selects which resource within the server is being accessed. For the internet, this is a fixed constant that gives the exact service being asked of the far server.

The file used to read and write the connection is returned.

opennet() uses IPv4 addressing. **opennetv6()** uses IPv6 addressing.

long [ami_]rdmsg(long fn, void* msg, unsigned long len);

Read a message from the message channel by logical id **fn**. The message will be placed in the buffer **msg**, whose maximum length is **len**. The actual length of the message is returned.

If the incoming message exceeds the buffer length, an error will result.

long [ami_]relymsg(unsigned long addr);

Returns true if a connection to the given address **addr** is “reliable” or without data loss or reordering of messages. If a data loss or reorder error does happen, however unlikely, the program will halt with error.

long [ami_]relymsgv6(unsigned long long addrh, unsigned long long addrl);

Returns true if a connection to the given IPv6 address **addrh** and **addrl** is “reliable” or without data loss or reordering of messages. If a data loss or reorder error does happen, however unlikely, the program will halt with error.

long [ami_]waitmsg(long port, long secure);

Waits for a message connection on the given port **p**. If the **s** flag is enabled, the connection will be encrypted. The file pointer for the connection is returned.

Any number of connections can occur on a single port within the program, but connections to the same port by other programs result in an error.

FILE* [ami_]waitnet(long port, long secure);

Waits for a stream connection on the given port **p**. If the **s** flag is enabled, the connection will be encrypted. The file pointer for the connection is returned.

Any number of connections can occur on a single port within the program, but connections to the same port by other programs result in an error.

void [ami_]wrmsg(long fn, void* msg, unsigned long len);

Write a message to the message channel by logical id **fn**, The message is in the buffer **msg**, and the length of the message is in **len**. The message must be less than or equal to **maxmsg()** or **maxmsgv6()** in size.

11 Option: Command Line Option Processing



Parsing options can be done in a non-OS specific way by the **option** package. It does both the work of accommodating different OS option conventions, as well as providing easy table lookup of options, and processes their parameters.

Presently, option accommodates two different convention for options, that of Unix/Linux/Mac OS X, or Windows:

Format	Operating system
/option	Windows
-o	Unix/Linux/Mac OS X
--option	Unix/Linux/Mac OS X

The option character is set from **optchr()** in services. In all conventions, multiple character options are the normal case, but option will also support single character options if:

- The operating system is Unix/Linux/Mac OS X.
- The single parameter is set on the option call.

The call to parse a series of options is **options(argi, argc, argv, opts, single)**. **argi** is the index of command line words being processed, and **argc** and **argv** are pointer references to the standard C command line arguments. **opts** is a table to look up options, and single indicates that single character options are to be allowed.

Options uses a “transparent” model to parse options. When called, it will check command line words for options, and if found, parse them and skip over any options found. Thus the **options()** call can be made any number of times during processing of command line words, so that options can appear anywhere on the line. Other models are to allow options only at the start of the command line or only at the end, as directed by the program using **options**.

The option table is of the format:

```
/* option record */
```

```
typedef struct {

    string   name; /* name of option */
    long*    flag; /* flag encounter */
    long*    ival; /* integer value */
    float*   fval; /* floating point value */
    string   str;  /* string value */

} ami_optrec, *ami_optptr;
```

The fields are:

Name	The character name of the option, a string.
flag	A pointer to Boolean, if not NULL, the flag will be set if the option is encountered.
ival	A pointer to integer, if not NULL, an integer value is parsed.
fval	A pointer to float, if not NULL, a floating point value is parsed.
str	A pointer to a string buffer, if not NULL, a string value is placed after removing any quotes.

If an option has a parameter associated with it, it can be specified as follows:

--option=42

or

/option:42

(with the leading character matching the operating system in use).

If single character options are in effect, the parameters are gathered from succeeding words:

-ox 42 “hi there”

Option will fill any or all of the option types asked for. For example, if the option was:

--option=42

And all of integer, floating point, and string were specified, then the results would be:

42	Integer.
42.0	Floating point.
“42”	String.

The table ends when the name is specified as NULL.

Putting all this together, an example option use would be (from the program “genwave.c”):

```
long dport = AMI_SYNTH_OUT; /* set default synth out */
long freq = 440; /* set default frequency */
long square = FALSE; /* set not square wave */

ami_optrec opttbl[] = {

    { "port",   NULL,    &dport, NULL, NULL },
    { "p",      NULL,    &dport, NULL, NULL },
    { "freq",   NULL,    &freq,  NULL, NULL },
    { "f",      NULL,    &freq,  NULL, NULL },
    { "square", &square, NULL,   NULL, NULL },
    { "s",      &square, NULL,   NULL, NULL },
    { NULL,     NULL,    NULL,   NULL, NULL }

};

int main(int argc, char **argv)
{

    long argi = 1;
    long argcl = argc;

    /* parse user options */
    ami_options(&argi, &argcl, argv, opttbl, TRUE);

    if (argcl != 1) {

        fprintf(stderr,
            "Usage: genwave [--port=<port>|--p=<port>|"
            "--freq=<freq>|--f=<freq>|\n");
        fprintf(stderr,
            "                --square|--sq|--sine|-s]\n");

        exit(1);

    }
    ...
}
```

In this program the options are **port**, **freq** and **square**. **port** and **freq** are integers, but square is a simple Boolean, we either encountered it or not. Each option has a single character equivalent:

Multicharacter Option	Single Character Option
port	p
freq	f
square	s

and the call to **options()** specifies that Unix/Linux/MAC OS X style single character options are allowed . In Unix/Linux/MAC OS X programs can use the next word in the command line to satisfy single character options with parameters.

There are two ways that parsing through the **argc/argv** command words in a C program are done. The first is by incrementing the argv array pointer and decrementing the **argc** value. The second is to keep an index number for the **argv** array, and increment that and decrement the **argc** value. The first method is simpler, but keeping an **argi** or **argv** index makes it clear exactly what word we are parsing in the command line. **option** supports this second method.

For string arguments, option will accept quoted strings and automatically remove the quotes when the string (or integer or float) is stored. Thus:

```
--mystring=”This is a long string with spaces”
```

Is possible. **option** will accept either single (‘) or double (“) quotes, as long as they are matched.

The return value for **options()** is 0 if no error was met, which includes the case of no options being present at all, and 1 if an option was found that could not be parsed: it reports errors, not discoveries. If no option was found, the **argi** and **argc** values are not changed. If a list of correct options is followed by one with an error, then the **argi** and **argc** indexes stop at the incorrect option.

Option has another function that does not use **argi** and **argc**, but just takes an option string, **option(s,opts,single)**. Since function is not tied to a **argc**/**argv** word array, it can be used to form your own option parsing system.

Finally, the function **dequote(s)** can be used to remove the quotes from a string. It accepts either single or double quotes, and removes them and puts the string back into the buffer provided on input.

11.1 Functions and Procedures in Option

void [ami_]dequote(string s);

Remove quotes from string. Leading and trailing quotes are removed from the provided string. **s** contains the string. Either single (‘) or double (“) quotes can be removed, but leading and trailing quotes must match.

long [ami_]option(string s, [ami_]optrec opts[], long single);

Parse a single option from a string. **s** contains the string to parse. **opts** contains a table of options. **single** is a flag indicating if single character options are to be allowed.

If the option is successfully parsed, a 0 is returned, otherwise 1.

long [ami_]options(long* argi, long* argc, char** argv, [ami_]optrec opts[], long single);

Parse a series of options from an **argc**/**argv** array. **argi** contains the index of the current **argv** array word position. **argc** contains the number of arguments left. **argv** contains the word array. **opts** contains a table of options. **single** is a flag indicating if single character options are to be allowed.

Any of 0 to n options can be parsed. If one or more options is successfully parsed, a 0 is returned, otherwise 1. The integer pointed to by **argi** is incremented for each option successfully parsed, and the integer pointed to by **argc** is decremented. **argv** is not changed.

12 Config: The configuration database



There are two ways to pass configuration into a running program:

- Specify options on the command line.
- Use environmental strings inside the program.

Ami has a third, and more general method, a configuration database or page. It supports, and is supported by, the program/user/current path method introduced by **services**. It is also the method used by Ami modules itself. Ami supports all three methods. The advantages and disadvantages of each are:

Mode	Advantage	Disadvantage
Command line option	Can configure per program run.	Have to repeatedly specify configuration.
Environmental strings	Configuration applies to every run.	Configuration exposed to all programs. Loads up the environment with various program configurations.
Configuration Pages	Configuration applies to every run. Supports per-feature configurations. Supports different configurations for all users, a single user, or a single run instance or directory.	The effort to construct configuration pages.

A configuration page is a file containing a series of “value sets” of the form:

name value

The name can be any name starting with the characters A..Z, a..z and _, and continuing with the characters A..Z, a..z, _ and 0..9. This is the “lookup key” for the value. The value itself can be any character sequence, including spaces, but excluding ‘\n’ (end of line). Thus, the entire series of characters after the name and trailing space are kept as the value. The value can also be blank, containing no characters or only spaces.

Configuration files can also contain blank lines, and a “comment line” of the form:

hi there

Comments can be placed anywhere except in the value field, because they would be added into the value itself (this may or may not be a problem depending on the program that uses the value).

Thus a typical configuration file could be:

```
# my configuration file

myname herman
myipaddr 1.2.3.4
...
```

Etc.

All of your configuration files can be “flat” as shown, but config also supports tree structure in configuration files. For each “branch” of the tree, a **begin** statement appears:

```
begin mybranch

    leaf
    ...

end
```

The meaning of this is that a new sub-catagory “mybranch” is created, and all of the definitions between the **begin** and **end** statements are placed under that branch. Note that neither **begin** nor **end** can be the name of a configuration branch.

Here is an example for a cut-down version of Ami’s own configuration file, **ami.cfg**:

```
#
# Configuration file for Ami and submodules
#

#
# Definitions for services module
#
begin services
end

#
# Definitions for terminal mode
#
# Terminal mode applies to several modules, including console and
graph.
#
begin terminal

    #
    # Set default dimensions of main window.
    #
    maxxd 80
    maxyd 25

    #
    # Enable/disable mouse
    #
    mouse 1

    #
    # Enable/disable joystick
    #
    joystick 1

end

#
# Definitions for graphical mode
#
# Graphical mode can apply to multiple modules, but usually is
reserved for just
# one main module.
#
begin graphics

    #
    # Send runtime errors to dialog/parent console window
    #
    dialogerr 0
```

```
    #
    # definitions for windows version of graph
    #
    begin windows

        #
        # Definitions for windows diagnostic settings
        #
        begin diagnostics

            #
            # Dump messages (windows messages posted to us)
            #
            dump_messages 0

        end

    end

end
```

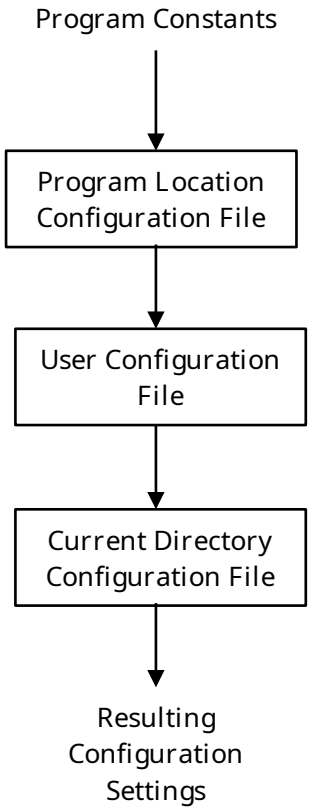
When you parse configuration files, this is done in the series of locations:

- The program location or path.
- The user location or path.
- The current location or path (current directory).

The result of each configuration file parse is a tree of configuration options. Each new tree is merged with the last, with any new definitions replacing the previous definitions, but otherwise each definition is left alone. Thus the resulting tree is the merge of each of the program path, user path and current path trees merged, starting with a merge to a empty list.

The result of this is that the starting basis for the option tree is the option file provided with the program, and available to all users. These definitions are overridden, or replaced, by definitions unique to each user, then possibly overridden by definitions unique to the current directory. Any one (or all) of these configuration files may or may not exist.

Typically the program using the configuration will transfer the definitions it uses to internal configuration parameters. The effect of this is that the default settings for the program exist before being overridden (or not) by each of the program, user and current configuration trees.



Many programs will also provide one or more local options via the command line that can be configured. These will either be configured before or after the configuration tree parse, meaning they may override or be overridden by the configuration pages. Ami cannot do this because it does not have access to the command line.

For any configuration file(s) a utility exists to parse the file(s) and print the resulting tree:

```
$ prtconfig
Petit-Ami configuration tree:

services              (null)
terminal
    mouse                 1
    joystick              1
    dump_event            0
console               (null)
graphics
    dialogerr             1
    windows
        diagnostics
            dump_messages         0
    xwindow
```

```
        diagnostics
            dump_messages        0
            print_font_metrics   0
    macosx
        diagnostics
            dump_messages        0
            print_font_metrics   0
widgets
    theme                (null)
sound               (null)
network             (null)
```

This is the configuration tree for Ami itself.

To read a config tree, use **config(r)** where **r** is a pointer to the root of the tree. It needs to be set null first, or contain a tree to be merged. **config()** looks for the file(s) by he name of **ami.cfg** or **.ami.cfg** (either the visible or invisible file). **config()** looks through the program, user and current path for these files.

To use an alternative filename or search procedure, the function **configfile(fn,r)** is used, where **fn** is the filename (including extension) and **r** is the root, which should be null or contain a starting tree as with **config()**. **configfile()** uses the name as is, complete with path, so it is a building block for your own way of using configuration trees.

To perform your own configuration tree merges, the function **merge(or, nr)** is used, where **or** is a pointer to the old root, as with **config()**, and **nr** is the new root itself. As before, the new tree definitions replace the old tree definitions of the same name and branch(s).

To find definitions in the resulting configuration trees, the function **schlst(id,r)** is used, where **id** contains the key name to be found, and **r** contains the tree. It returns a pointer to the value structure if found, or NULL if none is found.

In practice, this simple search function works well even if multiple levels of nesting are present. An example of this is the code from the Windows graphics module:

```
ami_valptr config_root; /* root for config block */
ami_valptr term_root;   /* root for terminal block */
ami_valptr graph_root;  /* root for graphics block */
ami_valptr diag_root;   /* root for diagnostics block */
ami_valptr win_root;    /* root for windows */
ami_valptr vp;

/* get setup configuration */
config_root = NULL;
ami_config(&config_root);

/* find "terminal" block */
term_root = ami_schlst("terminal", config_root);
if (term_root && term_root->sublist) term_root = term_root->sublist;

/* find x and y max if they exist */
```

```
vp = ami_schlst("maxxd", term_root);
if (vp) { maxxd = strtol(vp->value, &errstr, 10);
          if (*errstr) error(ecfgval); }
vp = ami_schlst("maxyd", term_root);
if (vp) { maxyd = strtol(vp->value, &errstr, 10);
          if (*errstr) error(ecfgval); }

/* find graphics block */
graph_root = ami_schlst("graphics", config_root);
if (graph_root) {

    vp = ami_schlst("dialogerr", graph_root->sublist);
    if (vp) { dialogerr = strtol(vp->value, &errstr, 10);
              if (*errstr) error(ecfgval); }

    /* find windows subsection */
    win_root = ami_schlst("windows", graph_root->sublist);
    if (win_root) {

        /* find diagnostic subsection */
        diag_root = ami_schlst("diagnostics", win_root->sublist);
        if (diag_root) {

            vp = ami_schlst("dump_messages", diag_root->sublist);
            if (vp) { dmpmsg = strtol(vp->value, &errstr, 10);
                      if (*errstr) error(ecfgval); }

        }

    }

}
```

The code works on the idea that the **schlst()** routine can find either value entries or branch entries, and once a branch is found, searching within that sublist is easy.

To print out configuration trees of any depth, use **prttre(l)**, where **l** is the configuration tree root. This prints a tree structured list of entries as shown before with **prtconfig()**.

12.1 Functions and Procedures in config

A configuration tree is made of value records, one to a name:

```
/* Tree structured value record */

typedef struct ami_value {

    struct ami_value* next;    /* next value in list */
    struct ami_value* sublist; /* new begin/end block */
    string name;                 /* name of node */
    string value;                /* value of this node */

} ami_value, *ami_valptr;
```

Every record carries its name and its value as strings. A record that opens a begin block has its contents on its sublist, and the value of such a record has no meaning. The next field carries the rest of the list at that level.

void [ami_]config([ami_]valptr* root);

Parse the configuration files of the program, user and current paths, in that order, and merge them into the tree at root. root must be NULL for a fresh tree, or hold a tree the files are merged into. The files are looked for under the names ami.cfg and .ami.cfg.

void [ami_]configfile(string fn, [ami_]valptr* root);

Parse one configuration file by name and merge it into the tree at root. The name is used as given, path and all, so this is the building block for a search of your own. root is treated as it is by config().

void [ami_]merge([ami_]valptr* root, [ami_]valptr newroot);

Merge the tree newroot into the tree at root. Definitions in the new tree replace those of the same name and branch in the old, and the rest are left alone. Note that root is a pointer to the tree and newroot is the tree.

void [ami_]prttre([ami_]valptr list);

Print the tree at list, indented by depth, in the form the prtconfig utility prints.

[ami_]valptr [ami_]schlst(string id, [ami_]valptr root);

Find the record named id in the tree root and return it, or NULL if there is none. It finds value records and branch records alike, so a branch is found and then searched by passing its sublist as the root of the next search.

13 Print modules



The ability to print from applications is covered by two plug in modules:

1. pdfgraph – Produces .pdf files in full graphics.
2. txtterminal – Produces ASCII text files.

These modules are portable and work on any operating system. Printing to a file is portable outright; reaching a printer device goes through the system print spooler. Print modules only print from the set of graphics functions in graphics, or text functions in terminal, the base APIs. No windows or widgets. Also, the print modules do not work by copying whatever is on the display surface to a printer. You must replay each of the graphics or text functions into the printer after opening it.

The printer subsystem is opened with **openprint(f, n)**, where **f** is a pointer to the FILE that is opened, and **n** names what is opened. The name is either a file in the filesystem, fully pathed or not, or a printer device, which is recognized by the ‘:’ at the end of the name.

The printer devices the print modules take are:

lp: the system default printer

lp0:, **lp1:** the printers by number, in the order the system lists them

If a filename is specified, the entire print output is written to the indicated file. If a print device is indicated, then the entire print output is sent to the printer.

Printers are page oriented. Each page is built up until a \f or form feed is output, which completes it and begins the next. A page that is still open when the print file closes is completed as it stands.

13.1 Including a print subsystem

In order to produce printout, a print module must be included in the link for your progam.

The print systems are mutually exclusive, and whichever one is last in the link order will intercept all print traffic.

Both print modules are operating system independent, and will work on any system.

12.5 pdfgraph

Pdfgraph can work on either terminal level output or fully graphic level output. It produces .pdf standard files, or can go direct to a .pdf capable printer. Most systems are able to view .pdf files, and if not there are many programs available for that purpose.

Pdfgraph is capable of printing text, text with colors and attributes, and full graphics. Because of this, this is usually the default print system.

Pdfgraph holds the pages until the print file is closed, and assembles the document then: a .pdf carries its cross reference table at the end, so it cannot be written page by page. A printer device receives that document from the spooler as a single job.

12.6 txtterminal

txtterminal prints only plain text from a terminal module. If any colors or attributes of text are output, they are ignored. All of the written text to print is stored in a buffer before printing. Specifically this means that text in the buffer can be overwritten.

txtterminal is useful if you want to write plain text files but still need the terminal command set. This would be true if, for example, you are using special layouts and wanted an image of that in a file. txtterminal also presents the printer as a simple output device (fprintf) which often does not exist on modern windowed operating systems.

For general purpose text output, we recommend using pdfgraph if possible, even if you are only using plain text. The reason is that pdfgraph is capable of printing text colors and attributes.

14 Remote mode



Ami normally runs with the display, sound and the devices such as the mouse and joystick all on the same machine. However it is also capable of running the display, sound and devices on a different machine that the program is running on. The protocol itself is described in the next chapter, and in doc/remote.pdf, which is the same text as a paper.

The remote mode is accomplished by two modules that are portable to all operating systems:

graph_server Waits for network connections and displays graphics, produces sound and handles devices.

graph_client Attaches to the remote program and converts all of its graphics calls to remote requests.

Note the server/client relationship descriptions here are opposite of X11. The server is what executes requests, the client is what makes them.

The graph_server is a complete stand-alone program. It runs on the display machine, and waits on the command network port for a new client. The ports used are:

Port	Channel
4901	commands: every drawing, sound and device call the program makes
4902	events: keys, mouse, resize and terminate, back to the program
4903	file transfer: fonts and pictures the program sends over

The command and event channels are message channels, carried on UDP, and each message crosses as one datagram. They are opened when the client arrives and stay open for the length of the session. The file transfer channel is a stream, carried on TCP, and it is opened only when a file has to cross and closed when it has. A firewall between the two machines needs the two message ports and the stream port passed.

The client finds the server by the environment. GRAPH_SERVER holds the address of the display machine, either a name or a dotted address, and is 127.0.0.1 if it is not set, which puts both sides on the same machine. GRAPH_PORT holds the command port, and the event and file transfer ports follow it at plus one and plus two. The server reads GRAPH_PORT as well, and the two ends must agree.

The channels are secured by default. The message channels use DTLS and the file transfer stream uses TLS, so what crosses between the machines can be neither read nor altered on the way. Either end can be told to run in the clear instead, the server by its options below and the client by GRAPH_PLAIN in its environment, but both ends must agree: a client that wants a secure channel where the server has none is refused.

Security costs little enough not to be worth avoiding. Measured against a server over the loopback, with 20,000 lines drawn and 2,000 round trips made, encryption adds about 3.5 microseconds to a round trip, which is under a thousandth of what a network hop costs, and nothing measurable to drawing, where the time is in the rendering.

14.1 graph_server

The server is a program in its own right, and takes no arguments but its options. It is started as:

graph_server [options]

The options are:

-p, --plain — Run the channels in the clear, without encryption. The client must be told the same, by GRAPH_PLAIN in its environment, or the two ends will not meet.

-s, --secure — Secure the channels, the message channels with DTLS and the file transfer stream with TLS. This is what happens anyway, the option saying what is already so.

--trace — Print every message that crosses, in either direction, to the error channel. It is slow, and it is how a session that goes wrong is read. **GRAPH_TRACE** in the environment does the same.

The port the server waits on comes from GRAPH_PORT, and is 4901 if that is not set.

The graph_client is a module, and it is linked with the program that runs remotely. The program runs unmodified as a client. The calls it uses are all in graphics.h. It thinks it is talking to the local operating system. But graph_client packages all of its calls and sends them to graph_server to be executed.

The result of this is that graph_server is a complete program and can be run stand-alone on the user’s machine, while the client side program is linked with graph_client and run on the server.

Sound crosses with the rest. The synthesizer, MIDI and wave calls travel on the command channel and are played on the display machine, so a program that makes sound is heard on the machine the user is sitting at. This is worth saying plainly, because the other ways of moving a display to another machine, X11, Wayland and waypipe among them, carry the picture only.

The server serves one program at a time. When the program ends, or its client goes away, the server winds the session down: the session's windows are closed, its widgets and its sound are released, and the main window returns to the state it started in. Then it waits for the next client. The server is meant to be left running, as a service on the display machine, rather than started for each program.

The server's own window is the way to stop it. A terminate reaches it from that window's close button, from control-c in it, or from control-c in the shell that started it, all three arriving as the same event. Standing idle, the server goes down at once. With a program up, the terminate goes to the program over the event channel, and the program winds down in its normal way, exactly as it would had the user

closed its window locally; the bye it sends on its way out ends the server with it. Asked again before that finishes, the server goes down where it stands.

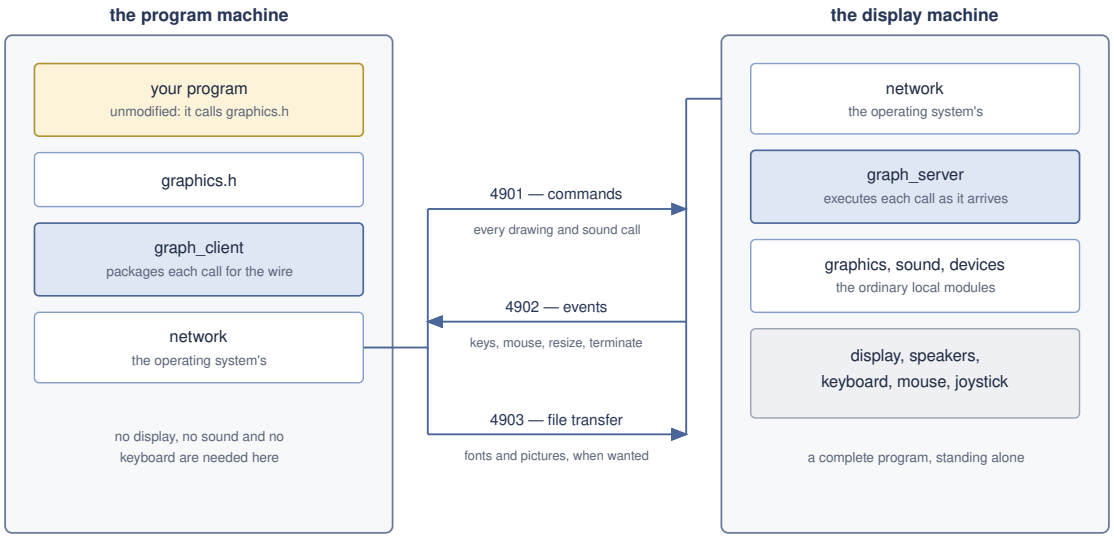
A program is made remote by linking it with graph_client in place of the local display module, and is not otherwise changed. The demonstration programs breakoutgr and breakoutwgr are built this way, as are the remote forms of the tests, graphics_testr, window_testr, widget_testr and sound_testr. That the local and the remote program are built from one source, and differ only in what they are linked with, is the whole of the idea.

For a program of your own, from the root of the distribution:

```
gcc -Iinclude myprog.c portable/graph_client.o linux/stdio.o \
   linux/services.o linux/network.o -lssl -lcrypto -lm -lpthread \
   -o myprog
```

The source is not changed, and nothing in it says whether it is local or remote. Built against the local display module instead, the same source is the local program.

Printing does not cross. The protocol reserves the print calls for a later version, and graph_client does not provide openprint(). A remote program that prints links a print module, pdfgraph or txtterminal, and the printing then happens on the machine the program runs on rather than on the display machine. A program that calls openprint() with no print module linked will not link at all.



4901 and 4902 are message channels, secured with DTLS unless plain is asked for. 4903 is a stream, opened only when a file has to cross.

The same program, linked with graph_client in place of the local display module: nothing above graphics.h knows the difference.

Remote mode: the same program, linked with graph_client in place of the local display module.

15 The remote display protocol



Remote display mode separates a Petit-Ami program from its display: the program runs on one machine, the client side, and its windows, drawing, widgets and input devices live on another, the server side. This chapter defines the wire protocol between the two, carried on the message channels of the network library. Every call of the graphical API has a message, and the sound API travels the same channel in its own partition of the message space; a continuous event stream flows back. The protocol is defined by the header **include/graph_remote.h**, of which this chapter is the narrative companion; where the two disagree, the header governs.

12.7 Introduction

The terminal and graphics libraries have always reserved a place for “remote display”: the output of a program encoded and carried over a communications channel, with input events carried back, so that the full functionality of the API survives the trip. The manual states the two conditions that make this possible: all output must be applied in the order issued, and all input events must be received in the order generated, with the rule that outstanding output completes before input is taken.

Two modules implement the mode:

graph_client
links with the program. It exposes the complete graphical API; each call composes a message and sends it, and **event()** returns the next inbound event from the client's queue, fed by a receiver thread.

graph_server
owns the display. It plugs into the standard graphics package as an overrider, executes inbound call messages against it, and sends the resulting events back out.

The display, the mouse, the keyboard and the joysticks face the user on the server side, while the program logic runs on the client side. Relative to the usual naming, where the machine in front of the user is the client, this arrangement is upside down: the “server” is the screen on the desk, and the “client” may be a machine in another room entirely. The naming here follows the connection roles: the server waits for a connection, the client makes one.

12.8 Architecture

The link is two message channels on adjacent ports, plus a third port reserved for file transfer:

port p	command channel	client → server calls, and server → client replies
port p+1	event channel	server → client events, continuously
port p+2	file port	stream connection, opened per transfer

The default command port is 4901.

The server divides its work between two threads. The main thread runs the command loop: wait for a message, execute it against the display library, reply if the call returns values. A second thread runs the event pump: call **event()** forever and send each event out on the event channel. The two directions are asynchronous, which is exactly why they are separate threads; neither ever waits on the other.

The client runs one thread beside the program: a receiver, which takes event messages from the event channel as they arrive and appends them to the client's event queue. Calls stay on the program's thread: they go out on the command channel as they are made, in order, and a query sends its request and waits for the reply before anything else is issued. **event()** takes from the queue, waiting if it is empty.

12.9 Transport

The protocol rides the message channels of the network library: **openmsg()** on the client side, **waitmsg()** on the server side, **rdmsg()** and **wrmsg()** to move discrete messages. A channel message carries one or more protocol messages, back to back, their header lengths delimiting them; the receiver executes them in order, and no protocol message is split across channel messages. Batching is the sender's option, and it saves the per-datagram cost on the hot paths; but nothing ever waits to fill a batch. Every boundary, a query, an event wait, the sync, the bye, sends what has gathered at once, and a lone trailing command is bounded by a flusher of a couple of milliseconds.

Message channels bound the size of a message, found from **maxmsg()** for the address. The write stream chunks itself to fit; every other message must fit the bound with its payload, which in practice bounds string lengths. Senders are responsible for staying under the bound.

The protocol requires a reliable channel in the sense of **relymsg()**: no loss, no duplication, no reordering. The local machine connection of the standard test arrangement is reliable by definition. Operation over an unreliable network needs a retransmission layer beneath this protocol; that layer is future work, and nothing in the message design precludes it.

The secure flag of the message channel calls passes through: a secured deployment secures both channels and the file connections alike.

12.10 Wire format

All multibyte values are little endian. The scalar and composite notations used throughout the catalog:

i64	signed 64 bit integer, the API **long**
f64	IEEE 754 double, carrying the API float values
str	i32 byte length, then the bytes, no terminator
blk	i32 byte length, then the bytes
slst	string list: i32 count, then count **str**
menu	menu tree, defined below
evt	event record, defined below

Every message begins with a fixed sixteen byte header:

i32	**len**	total message length in bytes, header included
i16	**mid**	message id, from the catalog
i16	**seq**	request serial, echoed by the reply; 0 where not meaningful
i64	**wid**	window handle the call acts on; 0 where not meaningful

The payload follows, holding the call's parameters in the order the API declares them, with the window file parameter excluded: it is the header **wid**.

12.11 Menu trees

A menu serializes depth first: an **i32** count, then that many entries, where each entry is an **i64** flag word (bit 0 the on/off capability, bit 1 the one-of membership, bit 2 the bar), the **i64** id, the **str** face text, and then a nested menu for the branch, empty if the entry has none. An empty tree is a single zero count, and removes the menu where that has meaning.

stdmenu() is the one call that returns a menu: the client sends the standard selections and the program's additions, and the server returns the combined tree as it built it. The client materializes that tree as real menu records for the program, so the program can hand it back to **menu()** later, as the API contract requires.

12.12 Event records

An event is a fixed record of eleven **i64**: the window handle the event belongs to, the event code, the handled flag, and eight parameter slots holding the fields of the event union in declaration order. A character rides in the first slot. The slots not used by an event code are zero. The fixed layout spends a few bytes to buy a single marshaling routine for every event.

12.13 Session

The first message on the command channel is the hello: the client sends its protocol version and the server replies with its own. A server that does not speak the client's version replies with its version and closes; the client raises the failure to the program. The first message on the event channel is the event-channel hello from the client, which is how the server learns where events go; it carries the version again and has no reply.

The client announces the end of the session with a bye on the command channel; the server drops the connection. A server error that cannot be returned as a reply travels to the client as an error message on the event channel, carrying the error text; the client raises it to the program.

12.14 Calls, queries and ordering

Calls divide in two. A command carries no result: it is sent and the client proceeds. A query returns values: the client sends the request and waits on the command channel for the reply before issuing anything else.

There is therefore at most one request outstanding at any time. Replies cannot interleave with each other or with commands, and the server may process every message in arrival order with no reassembly. The **seq** serial in the header is echoed by the reply purely as a consistency check; a mismatch is a protocol failure.

The ordinary output of a program, the characters written to a window file, travels as the write stream message: a block of bytes exactly as they would have gone to the window, chunked by the sender to the channel bound. Order is preserved end to end: the write stream and the drawing calls arrive in the order the program issued them, which satisfies the manual's first condition for remote display. The event stream preserves generation order, which satisfies the second.

12.15 Windows

Window handles are small integers assigned by the client. Handle 1 is the main window, the standard input and output pair, and exists from the hello. **openwin()** carries the parent handle, the new handle, and the logical window id; the server maps each handle to a window file of its own. Every subsequent call names its window by handle in the message header. Closing a window file on the client side sends the close message, and the server closes its side.

12.16 Events and client-local operations

Events are pushed, never polled. A pull model, where the program asks for the next event and waits for it, would degenerate to an ask and answer protocol: a full round trip across the wire per event, which for a stream of mouse movements is a performance disaster. Instead the source runs ahead of the consumer at both ends. The server's event pump thread calls **event()** on the display library without pause, translating each event to the wire record and sending it the moment it is ready. The client's receiver thread takes each event message as it arrives and appends it to the client's event queue, so the transport is always drained; **event()** takes from the queue, translates back, and returns the event to the program, replacing the wire handle with the window identification the API promises. The program regulates its own consumption from the queue.

Redundant events are thinned at both ends, on the principle that of a movement stream only the latest matters. The server sends at most one movement per short interval per device, mouse, graphical mouse, joystick, and likewise one resize per window; between sends the latest holds as pending, and a pending movement flushes before any other event goes out, so a click always follows the position it happened at. On the client, a movement or resize arriving at the queue replaces an unconsumed one it supersedes at the tail instead of stacking behind it. Neither side changes the protocol on the wire.

Three operations act on the client's own event stream and put nothing on the wire: **eventover()** and **eventsover()**, which hook event delivery, hook the client's delivery; and **sendevent()**, which injects an event, injects into the client's queue. The input device queries, the number of mice, buttons, joysticks and function keys, are synchronous queries like any others; the devices themselves are the server's, and their activity arrives as events.

12.17 File transfer

Files named in calls live where the program lives, on the client; the picture files of **loadpict()** are the case in point. The server holds a cache, its current directory. When a call names a file, the server looks it up: first the name as given, then the base name in the cache. If it holds the file, the call proceeds.

If not, the server requests the file, on the model of ftp's control and data split, in ftp's passive form. The request goes by message on the command channel, inside the reply window of the pending call, so the client is guaranteed to be listening; it names the file and the server's file port, the command port plus two. The client opens that port as a stream connection with **opennet()** and streams the file in as raw bytes, close delimited, exactly as an ftp data connection carries one file; an empty stream is a file the

client does not hold either. The server stores the file in its cache under its base name, never under a path the client supplied, and the pending call then proceeds against the cache and sends its reply.

The passive form, the client connecting rather than the server, is forced by an asymmetry: the message channels never disclose the client's address to the server, while the client always knows the server's. It is also the direction that crosses network address translation toward the server. The active form, where the server would connect to a port the client names, keeps its reserved message for a later version. One transfer at a time falls out of the one outstanding request rule.

The exchange is deliberately not tied to pictures: any call that names a file uses it, and it stands as the protocol's general bulk transfer mechanism.

12.18 The module partition

The sixteen bit message id space divides into four modules of 8192 codes each: graphics holds 0..8191, sound holds 8192..16383, and two partitions remain for future modules; the negative half of the space is reserved. A module's calls may grow within its partition without disturbing its neighbors, which is the reason for partitioning rather than packing: the graphics catalog and the sound catalog will both grow, and neither should renumber the other.

12.19 Sound

The sound API of **sound.h** carries in the sound partition, complete: the synthesizer channel voice messages, the stored sequences, the wave channels, the device names and parameters. There is no window; every sound message goes with a zero window handle in the header.

The devices live at the server, which is where the speakers and the microphones are, matching the placement of the display and the input devices. Sequencer times are interpreted against the server's time bases, and the time queries return them, so a schedule computed on the client lands where it should. The input calls, sequencer reads and wave reads, block the caller until the far device delivers, exactly as the native calls do; a program that did not want to wait natively will not want to wait remotely either.

The files of **loadsynth()** and **loadwave()** live where the program lives, and travel by the file transfer mechanism, which is why those calls are queries: the transfer request must arrive inside the reply window, with the client listening. The name in a transfer request now travels already under the extension rule of the call that asked, **.bmp** for pictures, **.wav** for waves, **.mid** for stored sequences, and the client serves exactly the name asked. Wave sample streams, in either direction, chunk to the channel bound like the write stream.

12.20 Future work

The protocol as specified assumes a reliable channel and same width, little endian hosts at both ends, which covers the intended deployments and the standard test arrangement. Three extensions are anticipated and none disturb the message catalog: a retransmission layer beneath the protocol for unreliable networks; print files (**openprint()**), reserved in the catalog, whose composition would run on either side; and big endian hosts, which would swap on the wire boundary. Multiple clients to one server, and compression of the write stream, remain open questions.

12.21 Message catalog

The catalog below is generated from **include/graph_remote.h**, which is the normative definition. Payload notation is as defined in the wire format section; "→" introduces the reply payload where the call is a query.

Message	Id	Payload → reply
GR_MHELLO	1	first on the command channel: i64 version → i64 version. The server refuses a version it does not speak by replying its own and closing.
GR_MEVOPEN	2	first on the event channel, from the client, so the server learns where events go: i64 version. No reply.
GR_MBYE	3	client is done; the server drops the connection. No payload.
GR_MERROR	4	server → client on the event channel: str message. The client raises the error to the program.
GR_MFILEREQ	5	server → client on the command channel, inside the reply window of a pending call that names a file the server does not hold: str name, i64 port, the server's file port. The client opens the port as a stream connection and streams the file in, raw bytes, close delimited; the pending call then completes and replies.
GR_MFILEPORT	6	reserved: the active form of the transfer, where the server would connect to a port the client names. The passive form above is the form of this protocol version.
GR_MSYNC	7	client → server: → i64 0. The flow fence: the client bounds its outstanding bytes by syncing once per window, so a command burst cannot outrun the server's socket into datagram loss. Any query is also a fence, being a round trip.
GR_MREPLY	9	the reply to any query: the request's seq, and the payload the request's catalog entry gives.
GR_MWRITE	10	the write stream of the window: blk of characters, as they would go to the window file. Chunked by the sender to the channel maximum. No reply.
GR_MCLOSEWIN	11	the window file closes: the server closes its side. No payload.
GR_MCURSOR	20	i64 x, i64 y
GR_MMAXX	21	→ i64
GR_MMAXY	22	→ i64
GR_MHOME	23	(no payload)
GR_MDEL	24	(no payload)
GR_MUP	25	(no payload)
GR_MDOWN	26	(no payload)
GR_MLEFT	27	(no payload)
GR_MRIGHT	28	(no payload)
GR_MBLINK	29	i64 e
GR_MREVERSE	30	i64 e
GR_MUNDERLINE	31	i64 e

GR_MSUPERSCRIPT	32	i64 e
GR_MSUBSCRIPT	33	i64 e
GR_MITALIC	34	i64 e
GR_MBOLD	35	i64 e
GR_MSTRIKEOUT	36	i64 e
GR_MSTANDOUT	37	i64 e
GR_MFCOLOR	38	i64 color
GR_MBCOLOR	39	i64 color
GR_MAUTO	40	i64 e
GR_MCURVIS	41	i64 e
GR_MSCROLL	42	i64 x, i64 y
GR_MCURX	43	→ i64
GR_MCURY	44	→ i64
GR_MCURBND	45	→ i64
GR_MSELECT	46	i64 u, i64 d
GR_MTIMER	47	i64 i, i64 t, i64 r
GR_MKILLTIMER	48	i64 i
GR_MMOUSE	49	→ i64
GR_MMOUSEBUTTON	50	i64 m → i64
GR_MJOYSTICK	51	→ i64
GR_MJOYBUTTON	52	i64 j → i64
GR_MJOYAXIS	53	i64 j → i64
GR_MSETTAB	54	i64 t
GR_MRESTAB	55	i64 t
GR_MCLRTAB	56	(no payload)
GR_MFUNKEY	57	→ i64
GR_MFRAMETIMER	58	i64 e
GR_MAUTOHOLD	59	i64 e; wid 0, the hold is global
GR_MWRTSTR	60	str
GR_MWRTSTRN	61	str
GR_MSIZBUF	62	i64 x, i64 y
GR_MTITLE	63	str
GR_MFCOLORC	64	i64 r, i64 g, i64 b
GR_MBCOLORC	65	i64 r, i64 g, i64 b
GR_MMAXXG	100	→ i64
GR_MMAXYG	101	→ i64
GR_MCURXG	102	→ i64
GR_MCURYG	103	→ i64
GR_MLINE	104	i64 x1, y1, x2, y2
GR_MLINEWIDTH	105	i64 w
GR_MLINESTYLE	106	i64 style
GR_MRECT	107	i64 x1, y1, x2, y2
GR_MFRECT	108	i64 x1, y1, x2, y2
GR_MRRECT	109	i64 x1, y1, x2, y2, xs, ys
GR_MFRRECT	110	i64 x1, y1, x2, y2, xs, ys
GR_MELLIPSE	111	i64 x1, y1, x2, y2

GR_MFELLIPSE	112	i64 x1, y1, x2, y2
GR_MARC	113	i64 x1, y1, x2, y2, sa, ea
GR_MFARC	114	i64 x1, y1, x2, y2, sa, ea
GR_MFCHORD	115	i64 x1, y1, x2, y2, sa, ea
GR_MFTRIANGLE	116	i64 x1, y1, x2, y2, x3, y3
GR_MCURSORG	117	i64 x, i64 y
GR_MBASELINE	118	→ i64
GR_MSETPIXEL	119	i64 x, i64 y
GR_MFOVER	120	(no payload)
GR_MBOVER	121	(no payload)
GR_MFINVIS	122	(no payload)
GR_MBINVIS	123	(no payload)
GR_MFXOR	124	(no payload)
GR_MBXOR	125	(no payload)
GR_MFAND	126	(no payload)
GR_MBAND	127	(no payload)
GR_MFOR	128	(no payload)
GR_MBOR	129	(no payload)
GR_MCHRSIZX	130	→ i64
GR_MCHRSIZY	131	→ i64
GR_MFONTS	132	→ i64
GR_MFONT	133	i64 fc
GR_MFONTNAM	134	i64 fc → str
GR_MFONTSIZ	135	i64 s
GR_MSETPOINTS	136	f64 ps
GR_MPOINTS	137	→ f64
GR_MCHRSPCY	138	i64 s
GR_MCHRSPCX	139	i64 s
GR_MDPMX	140	→ i64
GR_MDPMY	141	→ i64
GR_MSTRSIZ	142	str → i64
GR_MCHRPOS	143	str, i64 p → i64
GR_MWRITEJUST	144	str, i64 n
GR_MJUSTPOS	145	str, i64 p, i64 n → i64
GR_MCONDENSED	146	i64 e
GR_MEXTENDED	147	i64 e
GR_MXLIGHT	148	i64 e
GR_MLIGHT	149	i64 e
GR_MXBOLD	150	i64 e
GR_MHOLLOW	151	i64 e
GR_MRAISED	152	i64 e
GR_MSETTABG	153	i64 t
GR_MRESTABG	154	i64 t
GR_MFCOLORG	155	i64 r, i64 g, i64 b
GR_MBCOLORG	156	i64 r, i64 g, i64 b
GR_MLOADPICT	157	i64 p, str name → i64 0. The server fills its cache by the

		file transfer exchange if it does not hold the file; the reply comes after the picture is loaded.
GR_MPICTSIZX	158	i64 p → i64
GR_MPICTSIZY	159	i64 p → i64
GR_MPICTURE	160	i64 p, x1, y1, x2, y2
GR_MDELPICT	161	i64 p
GR_MSCROLLG	162	i64 x, i64 y
GR_MPATH	163	i64 a
GR_MVIEWOFFG	164	i64 x, i64 y
GR_MVIEWSCALE	165	f64 x, f64 y
GR_MSCALEX	166	i64 x → i64
GR_MSCALEY	167	i64 y → i64
GR_MBLOCKCOPYG	168	i64 s, d, sx1, sy1, sx2, sy2, dx1, dy1, dx2, dy2
GR_MOPENWIN	200	open a window: i64 parent handle (0 for none), i64 new handle, i64 logical window id. The header wid is 0.
GR_MBUFFER	201	i64 e
GR_MSIZBUFG	202	i64 x, i64 y
GR_MGETSIZ	203	→ i64 x, i64 y
GR_MGETSIZG	204	→ i64 x, i64 y
GR_MSETSIZ	205	i64 x, i64 y
GR_MSETSIZG	206	i64 x, i64 y
GR_MSETPOS	207	i64 x, i64 y
GR_MSETPOSG	208	i64 x, i64 y
GR_MDRAGWIN	209	(no payload)
GR_MSCNSIZ	210	→ i64 x, i64 y
GR_MSCNSIZG	211	→ i64 x, i64 y
GR_MSCNCEN	212	→ i64 x, i64 y
GR_MSCNCENG	213	→ i64 x, i64 y
GR_MWINCLIENT	214	i64 cx, cy, ms → i64 wx, wy
GR_MWINCLIENTG	215	i64 cx, cy, ms → i64 wx, wy
GR_MFRONT	216	(no payload)
GR_MBACK	217	(no payload)
GR_MFRAME	218	i64 e
GR_MSIZABLE	219	i64 e
GR_MSYSBAR	220	i64 e
GR_MMENU	221	menu; an empty tree removes the menu
GR_MMENUENA	222	i64 id, i64 e
GR_MMENUSEL	223	i64 id, i64 e
GR_MSTDMENU	224	i64 standard menu set, menu the user additions → menu, the combined menu as the server built it, which the client materializes for the program
GR_MGETWINID	225	→ i64; wid 0, ids are global
GR_MFOCUS	226	(no payload)
GR_MGETWIGID	300	→ i64
GR_MKILLWIDGET	301	i64 id
GR_MSELECTWIDGET	302	i64 id, i64 e
GR_MENABLEWIDGET	303	i64 id, i64 e

GR_MGETWIDGETTEXT	304	i64 id → str
GR_MPUTWIDGETTEXT	305	i64 id, str
GR_MSIZWIDGET	306	i64 id, i64 x, i64 y
GR_MSIZWIDGETG	307	i64 id, i64 x, i64 y
GR_MPOSWIDGET	308	i64 id, i64 x, i64 y
GR_MPOSWIDGETG	309	i64 id, i64 x, i64 y
GR_MBACKWIDGET	310	i64 id
GR_MFRONTWIDGET	311	i64 id
GR_MFOCUSWIDGET	312	i64 id
GR_MBUTTONSIZ	313	str → i64 w, i64 h
GR_MBUTTONSIZG	314	str → i64 w, i64 h
GR_MBUTTON	315	i64 x1, y1, x2, y2, str, i64 id
GR_MBUTTONG	316	i64 x1, y1, x2, y2, str, i64 id
GR_MCHECKBOXSIZ	317	str → i64 w, i64 h
GR_MCHECKBOXSIZG	318	str → i64 w, i64 h
GR_MCHECKBOX	319	i64 x1, y1, x2, y2, str, i64 id
GR_MCHECKBOXG	320	i64 x1, y1, x2, y2, str, i64 id
GR_MRADIOBUTTONSIZ	321	str → i64 w, i64 h
GR_MRADIOBUTTONSIZG	322	str → i64 w, i64 h
GR_MRADIOBUTTON	323	i64 x1, y1, x2, y2, str, i64 id
GR_MRADIOBUTTONG	324	i64 x1, y1, x2, y2, str, i64 id
GR_MGROUPSIZG	325	str, i64 cw, i64 ch → i64 w, h, ox, oy
GR_MGROUPSIZ	326	str, i64 cw, i64 ch → i64 w, h, ox, oy
GR_MGROUP	327	i64 x1, y1, x2, y2, str, i64 id
GR_MGROUPG	328	i64 x1, y1, x2, y2, str, i64 id
GR_MBACKGROUND	329	i64 x1, y1, x2, y2, i64 id
GR_MBACKGROUNDG	330	i64 x1, y1, x2, y2, i64 id
GR_MSCROLLVERTSIZG	331	→ i64 w, i64 h
GR_MSCROLLVERTSIZ	332	→ i64 w, i64 h
GR_MSCROLLVERT	333	i64 x1, y1, x2, y2, i64 id
GR_MSCROLLVERTG	334	i64 x1, y1, x2, y2, i64 id
GR_MSCROLLHORIZSIZG	335	→ i64 w, i64 h
GR_MSCROLLHORIZSIZ	336	→ i64 w, i64 h
GR_MSCROLLHORIZ	337	i64 x1, y1, x2, y2, i64 id
GR_MSCROLLHORIZG	338	i64 x1, y1, x2, y2, i64 id
GR_MSCROLLPOS	339	i64 id, i64 r
GR_MSCROLLSIZ	340	i64 id, i64 r
GR_MNUMSELBOXSIZG	341	i64 l, i64 u → i64 w, i64 h
GR_MNUMSELBOXSIZ	342	i64 l, i64 u → i64 w, i64 h
GR_MNUMSELBOX	343	i64 x1, y1, x2, y2, l, u, id
GR_MNUMSELBOXG	344	i64 x1, y1, x2, y2, l, u, id
GR_MEDITBOXSIZG	345	str → i64 w, i64 h
GR_MEDITBOXSIZ	346	str → i64 w, i64 h
GR_MEDITBOX	347	i64 x1, y1, x2, y2, i64 id
GR_MEDITBOXG	348	i64 x1, y1, x2, y2, i64 id
GR_MPROGBARSIZG	349	→ i64 w, i64 h

GR_MPROGBARSIZ	350	→ i64 w, i64 h
GR_MPROGBAR	351	i64 x1, y1, x2, y2, i64 id
GR_MPROGBARG	352	i64 x1, y1, x2, y2, i64 id
GR_MPROGBARPOS	353	i64 id, i64 pos
GR_MLISTBOXSIZG	354	slst → i64 w, i64 h
GR_MLISTBOXSIZ	355	slst → i64 w, i64 h
GR_MLISTBOX	356	i64 x1, y1, x2, y2, slst, i64 id
GR_MLISTBOXG	357	i64 x1, y1, x2, y2, slst, i64 id
GR_MDROPBOXSIZG	358	slst → i64 cw, ch, ow, oh
GR_MDROPBOXSIZ	359	slst → i64 cw, ch, ow, oh
GR_MDROPBOX	360	i64 x1, y1, x2, y2, slst, i64 id
GR_MDROPBOXG	361	i64 x1, y1, x2, y2, slst, i64 id
GR_MDROPEDITBOXSIZG	362	slst → i64 cw, ch, ow, oh
GR_MDROPEDITBOXSIZ	363	slst → i64 cw, ch, ow, oh
GR_MDROPEDITBOX	364	i64 x1, y1, x2, y2, slst, i64 id
GR_MDROPEDITBOXG	365	i64 x1, y1, x2, y2, slst, i64 id
GR_MSLIDEHORIZSIZG	366	→ i64 w, i64 h
GR_MSLIDEHORIZSIZ	367	→ i64 w, i64 h
GR_MSLIDEHORIZ	368	i64 x1, y1, x2, y2, i64 mark, i64 id
GR_MSLIDEHORIZG	369	i64 x1, y1, x2, y2, i64 mark, i64 id
GR_MSLIDEVERTSIZG	370	→ i64 w, i64 h
GR_MSLIDEVERTSIZ	371	→ i64 w, i64 h
GR_MSLIDEVERT	372	i64 x1, y1, x2, y2, i64 mark, i64 id
GR_MSLIDEVERTG	373	i64 x1, y1, x2, y2, i64 mark, i64 id
GR_MTABBARSIZG	374	slst, i64 tor, i64 cw, i64 ch → i64 w, h, ox, oy
GR_MTABBARSIZ	375	slst, i64 tor, i64 cw, i64 ch → i64 w, h, ox, oy
GR_MTABBARCLIENTG	376	i64 tor, i64 w, i64 h → i64 cw, ch, ox, oy
GR_MTABBARCLIENT	377	i64 tor, i64 w, i64 h → i64 cw, ch, ox, oy
GR_MTABBAR	378	i64 x1, y1, x2, y2, slst, i64 tor, i64 id
GR_MTABBARG	379	i64 x1, y1, x2, y2, slst, i64 tor, i64 id
GR_MTABSEL	380	i64 id, i64 tn
GR_MALERT	420	str title, str message → nothing; the reply comes when the dialog is dismissed, which is what makes the call modal at the far end
GR_MQUERYCOLOR	421	i64 r, g, b → i64 r, g, b; wid 0
GR_MQUERYOPEN	422	str → str; wid 0
GR_MQUERYSAVE	423	str → str; wid 0
GR_MQUERYFIND	424	str, i64 opt → str, i64 opt; wid 0
GR_MQUERYFINDREP	425	str, str, i64 opt → str, str, i64 opt; wid 0
GR_MQUERYFONT	426	i64 fc, s, fr, fg, fb, br, bg, bb, effect → the same nine, as chosen
GR_MCAPTURENAME	480	str: the file the pictures go to
GR_MCAPTURELABEL	481	str: the next picture's title
GR_MCAPTURE	482	→ i64 0, when the picture is written
GR_MSEAT	483	str: a rig line; its pointer coordinates are in the session window's client area, and the server places them on its display

GR_MAUTOBUF	484	i64 e
GR_MEVENT	500	an event, server → client on the event channel: evt
	The sound partition (base 8192)	
GR_MSTARTTIMEOUT	8192	–
GR_MSTOPTIMEOUT	8193	–
GR_MCURTIMEOUT	8194	→ i64 time
GR_MSTARTTIMEIN	8195	–
GR_MSTOPTIMEIN	8196	–
GR_MCURTIMEIN	8197	→ i64 time
GR_MSYNTHOUT	8198	→ i64 count
GR_MSYNTHIN	8199	→ i64 count
GR_MOPENSYNTHOUT	8200	i64 p
GR_MCLOSESYNTHOUT	8201	i64 p
GR_MOPENSYNTHIN	8202	i64 p
GR_MCLOSESYNTHIN	8203	i64 p
GR_MNOTEON	8204	i64 p, t, c, n, v
GR_MNOTEOFF	8205	i64 p, t, c, n, v
GR_MINSTCHANGE	8206	i64 p, t, c, i
GR_MATTACK	8207	i64 p, t, c, at
GR_MRELEASE	8208	i64 p, t, c, rt
GR_MLEGATO	8209	i64 p, t, c, b
GR_MPORTAMENTO	8210	i64 p, t, c, b
GR_MVIBRATO	8211	i64 p, t, c, v
GR_MVOLSYNTHCHAN	8212	i64 p, t, c, v
GR_MPORTTIME	8213	i64 p, t, c, v
GR_MBALANCE	8214	i64 p, t, c, b
GR_MPAN	8215	i64 p, t, c, b
GR_MTIMBRE	8216	i64 p, t, c, tb
GR_MBRIGHTNESS	8217	i64 p, t, c, b
GR_MREVERB	8218	i64 p, t, c, r
GR_MTREMULO	8219	i64 p, t, c, tr
GR_MCHORUS	8220	i64 p, t, c, cr
GR_MCELESTE	8221	i64 p, t, c, ce
GR_MPHASER	8222	i64 p, t, c, ph
GR_MAFTERTOUCH	8223	i64 p, t, c, n, at
GR_MPRESSURE	8224	i64 p, t, c, pr
GR_MPITCH	8225	i64 p, t, c, pt
GR_MPITCHRANGE	8226	i64 p, t, c, v
GR_MMONO	8227	i64 p, t, c, ch
GR_MPOLY	8228	i64 p, t, c
GR_MLOADSYNTH	8229	i64 s, str fn → i64 0
GR_MPLAYSYNTH	8230	i64 p, t, s
GR_MDELSYNTH	8231	i64 s
GR_MWAITSYNTH	8232	i64 p → i64 0, on completion
GR_MWRSYNTH	8233	i64 p + seq: i64 port, time, st, p1, p2, p3
GR_MRDSYNTH	8234	i64 p → seq; blocks the caller until input arrives, as the

		native call does
GR_MWAVEOUT	8235	→ i64 count
GR_MWAVEIN	8236	→ i64 count
GR_MOPENWAVEOUT	8237	i64 p
GR_MCLOSEWAVEOUT	8238	i64 p
GR_MLOADWAVE	8239	i64 w, str fn → i64 0
GR_MPLAYWAVE	8240	i64 p, t, w
GR_MDELWAVE	8241	i64 w
GR_MVOLWAVE	8242	i64 p, t, v
GR_MWAITWAVE	8243	i64 p → i64 0, on completion
GR_MCHANWAVEOUT	8244	i64 p, c
GR_MRATEWAVEOUT	8245	i64 p, r
GR_MLENWAVEOUT	8246	i64 p, l
GR_MSGNWAVEOUT	8247	i64 p, s
GR_MFLTWAVEOUT	8248	i64 p, f
GR_MENDWAVEOUT	8249	i64 p, e
GR_MWRWAVE	8250	i64 p, i64 frames, blk samples → i64 0. The sender chunks to the channel bound on frame boundaries; each chunk is acknowledged, which paces the stream to the consumer and keeps it whole.
GR_MOPENWAVEIN	8251	i64 p
GR_MCLOSEWAVEIN	8252	i64 p
GR_MCHANWAVEIN	8253	i64 p → i64
GR_MRATEWAVEIN	8254	i64 p → i64
GR_MLENWAVEIN	8255	i64 p → i64
GR_MSGNWAVEIN	8256	i64 p → i64
GR_MENDWAVEIN	8257	i64 p → i64
GR_MFLTWAVEIN	8258	i64 p → i64
GR_MRDWAVE	8259	i64 p, frames → i64 got, blk samples; the server clamps the answer to the channel bound and the caller loops. Lengths are in samples, one frame of every channel, as the API counts.
GR_MSYNTHOUTNAME	8260	i64 p → str
GR_MSYNTHINNAME	8261	i64 p → str
GR_MWAVEOUTNAME	8262	i64 p → str
GR_MWAVEINNAME	8263	i64 p → str
GR_MSETPARAMSYNTHIN	8264	i64 p, str, str → i64
GR_MSETPARAMSYNTHOUT	8265	i64 p, str, str → i64
GR_MSETPARAMWAVEIN	8266	i64 p, str, str → i64
GR_MSETPARAMWAVEOUT	8267	i64 p, str, str → i64
GR_MGETPARAMSYNTHIN	8268	i64 p, str → str
GR_MGETPARAMSYNTHOUT	8269	i64 p, str → str

GR_MGETPARAMWAVEIN	8270	i64 p, str → str
GR_MGETPARAMWAVEOUT	8271	i64 p, str → str
GR_MGETWAVEID	8272	→ i64 w, negative

16 Widget Creation



A widget of your own gives a program a control the stock set does not have, or a look that is the program's own.

On Windows and macOS the graphics module gives the standard widgets with the operating system's own. Linux has no standard set to give, so Ami's stock sets for it, **gnome_widgets** and **plasma_widgets**, are written in Ami itself, on the widget base this chapter describes. On every system a program can add widgets of its own the same way.

The widget base is portable, and a widget written on it runs on every system. There are two reasons to write one on it even on Windows or macOS, where the stock set is native:

1. A widget written on the base is the same on every operating system.
2. It is simpler to write than a native one. A widget is a window, drawn with the drawing calls and driven by the events, and nothing else.

A package reaches into Ami through its overrides, the vectors behind every call that a module can take for itself. The first section is about those; the second is about the base; the last is a whole package, the smallest that can be written.

16.1 How the overrides work on stock widgets

Every call in **graphics.h** is reached through a vector, and a module can take one for itself. The override calls are named for the call they take, **_pa_button_ovr()**, **_pa_killwidget_ovr()** and so on, and each installs a new function and hands back the one that was there. For example:

```
_pa_button_ovr(mybutton, &button_vect);
```

This installs **mybutton** as the function **button()** now calls, and leaves the one it replaced at **button_vect**. The module answers the calls that are its own and passes the rest to the function it replaced, so nothing above it knows it happened.

The stock widget sets work this way, because they implement the standard widget set. **gnome_widgets.c** and **plasma_widgets.c** each override button, checkbox, **killwidget()**, enablewidget() and the rest of the standard calls, and which of them is in the link decides what a button looks like. The program calls **button()** either way.

16.2 New widgets

A widget of your own has nothing to override. There is no standard call for a custom widget, and there does not need to be. A widget routine is a call in your own header and your program calls it by name.

What every package does need is the events of its widget windows, and the base takes them for all packages at once. The first package to register puts the base's handler ahead of the program's with **ami_eventsover()**, and takes the **close()** vector with it, so that a window closed with widgets on it takes them down too. Each package is handed the events for its own windows, everything else passes on down the chain, and the last package to leave puts the vectors back.

16.3 The widget base

A widget is a frameless subwindow of the window that owns it. This means that all drawing in the widget is relative to that window, and it gets only the events for that window. All the other characteristics of the window are handled for it, buffering, clipping, etc. These are handled by **widget_base.c**.

A package declares its widget record with the base head first, the record **wbwig** as a member named **head**, and its own state after it:

```
typedef struct wigrec* wigptr;
typedef struct wigrec {

     wbwig head; /* the base head, first */
     long playing; /* your own state follows */
     long frame;

} wigrec;
```

The head carries what the base needs of every widget: the widget's own window and its parent, the id the client gave it and the id of its window, whether it is enabled, focused or hovered, where it sits in the parent, and its place in the tab order. The package reads the head through that member, as **wg->head.wf** for the widget's window. The package's fields follow, and the base never looks at them.

The package registers itself once, giving the size of its record and its callbacks:

wb_init(&pkg, sizeof(wigrec), dispatch, wiginit, wigkill, wigfree, errrep);

The callbacks are:

dispatch(ev, wg)	Handle an event addressed to one of the package's widgets. This is the only one that must be given.
wiginit(wg)	Fill in the package's own fields of a fresh record. The base has already set the head.
wigkill(wg)	The widget is about to be taken down: stop its timers, take down anything it built.

wigfree(wg)	Release whatever the widget holds.
errrep(s)	Report an error, and do not return. Errors go to stderr if this is not given.

The calls a package makes are:

wb_getwig	Take a fresh widget record: the base fills the head and calls wiginit for the package's own fields. This is how a package sets its state before the widget is made, and so before it first paints.
wb_widget	Make a widget: a frameless subwindow of the parent over the given rectangle, in graphical coordinates, registered under the client's id. The last argument carries the record: pass a pointer to a null one and the base takes one for you, or pass a record from wb_getwig that you have already filled in. The record comes back through the same argument.
wb_killwidget	Take the widget down.
wb_fndwig	Find a widget by its parent window and its id.
wb_getwigid	Allocate an anonymous id within a window, for a widget the client did not name.

The two of those together are how a widget is given state before it is seen. For example a stock slider does it for it’s tick count and for a dropbox, the list a drop box drops: take the record, set the field, make the widget, and the first paint has it. The same pair builds a compound widget, one whose parts are widgets of its own window, which is what a drop box and a number select box are.

Two fields of the head are the package’s to set. A fresh widget is not **live**: the base dispatches no events to it until the package sets the field at the end of its creation, so a widget is never asked to paint before it is whole. Every widget is a tab stop, meaning a tab character stops on that widget, unless the package clears **tabstop**. The base walks the tab key among the live, enabled stops of a window in the order they were made, giving each the keyboard focus in turn, and the package draws the focus as it sees fit.

Widget ids divide in two. The ids a program names are positive, 1 to n, and are the ones the widgets chapter describes. The ids a package takes for itself with **wb_getwigid()** are negative, counting down from -1, so a widget the client never named can never collide with one it did. An anonymous id is held from the moment it is allocated until the widget carrying it is killed. The drop box is the plain example: the list it drops is a widget of the parent window under an anonymous id, and no program need know it is there.

A widget speaks to the program by putting an event on the parent window's queue with **sendevent()**. A widget standing in for a standard kind reports the standard code, and the program cannot tell it from the stock one. A widget with no standard analogue defines a code of its own, from **etuser** on up, which are reserved for programs and packages to define, and the union fields of the event record are its own to assign.

16.4 An example: the kick button

portable/widget_demo.c is a small widget package: one widget, on the same base the stock set uses. The widget is a kick button, a button whose face is a photograph of a soccer player. Press it and he kicks the ball, twenty four frames at seventy milliseconds each, with the sound of the kick at the frame where his

foot meets it. It is deliberately not useful, and deliberately a widget no stock set would ever give you, which is the point of writing your own.

Its record adds two fields after the head, which is all the state a kick button has: whether the animation is running, and the frame it has reached.

It draws with the ordinary Ami drawing calls. The face is a picture, loaded with **loadpict()** and drawn with **picture()**, and around it is a rectangle whose colour says whether the mouse is over the widget or the widget holds the focus. Anything the drawing calls can do can be the face of a widget; the stock widget set draws its own with lines, rectangles and text.

It processes events in its dispatch callback. A press starts the animation, and a timer on the widget's own window steps it a frame at a time. At frame twelve of twenty four, where the foot meets the ball, it plays the kick sound and reports ETKICKED to the parent window. ETKICKED is defined in widget_demo.h as **etuser**+0, and carries the widget's id in the event record. The event is not at the time of the press: it is the time of the kick.

Its entry calls are its own, and there are three of them:

```
void kickbutton(FILE* f, long x1, long y1, long x2, long y2, long id);
void kickbuttonkill(FILE* f, long id);
void kickbuttonenable(FILE* f, long id, long e);
```

A program makes one and waits for its event, as it would for any other widget. test_demo.c is that program, and is built with the package beside it:

```
gcc test_demo.c portable/widget_demo.c <the graphics libraries> -o test_demo
```

The animation frames live in tests/widget_demo as kick01.bmp through kick24.bmp, and the sound beside them as kick.wav. Pictures load as twenty four bit uncompressed .bmp, The frames are taken from a video by Nesnad on Wikimedia Commons, under the Creative Commons CC BY 4.0 license.

The kick sound is loaded under a wave id from **getwaveid()**, an id the program did not name, so the package’s wave and any the program loads never collide.

```
Press the button: the player kicks the ball.
He kicked the ball!
```



The kick button at the moment of contact, with the program's own answer to the widget's event printed above it.

17 Sound module plugins



The sound module has the ability to accept plug-ins. This was necessitated by the fact that Linux does not have a standard midi synthesizer built into the operating system. See “Linux details” for more on this.

The sound plug-in facility gives external modules the ability to declare pseudo-devices, and intercept I/O calls, for any of:

- Synthesizer/MIDI input.
- Synthesizer/MIDI output.
- PCM wave input.
- PCM wave output.

A different call is provided for each:

_pa_synthoutplug()	For a synthesizer output pseudo-device.
_pa_synthinplug()	For a synthesizer input pseudo-device.
_pa_waveoutplug()	For a PCM wave output pseudo-device.
_pa_waveinplug()	For a PCM wave input pseudo-device.

A plug-in wishing to expose a pseudo-device gives the name of the device, and a series of vectors to the routines that will handle calls for that device. The call registers the name of the device and assigns it a device number. Any calls using that device will then be re-routed to the plug-in code.

In addition to the I/O functions for the plug-in device, a plug-in has the ability to both read and write string data parameters. These are values specific to the pseudo-device being implemented by the plug-in. A plug-in has two ways to have parameters within it set. The first way is to parse configuration strings (see 12 ” Config: The configuration database”). The second is to implement read and write data parameters via the plug-in interface. The first method gives the user the ability to configure the plug-in. The second gives a program using sound the ability to configure the plug-in.
One example of a pseudo-device given above is a software synthesizer. Another that is provided with Ami is a MIDI dumper that can decode and print all midi messages that pass through the device[7]. Other

[7] The dump plug-in is operating system independent, and is described in the Linux section because that is where it is built. Linux and Mac OS X both implement the plug-in calls; Windows does not.

possibilities include MIDI recorders, PCM wave echo or other effects generators.

Operating systems often provide their own sound plug-in facilities. Further, some programs such as DAWs (Digital Audio Workstations) may have their own plug-in format. The main advantage of using Ami's plug-in format is that it is portable to anywhere Ami goes.

17.1 Functions in sound plug-ins

void _pa_excseq(long p, [ami_]seqptr sp);

Hand a sequencer message back for the main code to carry out on device **p**. A plug-in is given every message sent to the device it stands for, and some of them are not its business: the sequencer messages that play a stored MIDI file or a stored wave are not MIDI instructions at all, and a synthesizer plug-in has nothing to do with them. It passes those to **_pa_excseq()**, which does with them what the module would have done had no plug-in been there. The fluidsynth plug-in shipped with Ami does exactly that for **st_playsynth** and **st_playwave**.

void _pa_synthinplug(long addend, string name, void (*opnseq)(long p), void (*clsseq)(long p),
void (*rdseq)(long p, [ami_]seqptr sp), long (*setparam)(long p, string name, string value),
void (*getparam)(long p, string name, string value, long len));

Declares a synthesizer/MIDI input pseudo-device. The **addend** Boolean indicates if the device is to be inserted at the front or the end of the device list. If true, it is inserted at the end, otherwise at the front.

The **opnseq(p)** vector will be called with the device number as parameter **p** when the device is opened, and the **clsseq(p)** vector will be called with the device number as parameter **p** when the device is closed. The device number may be ignored, or it may be used to differentiate between multiple devices if the same routine is used for the different devices.

The **rdseq(p, sp)** vector is called with a MIDI message structure **seqmsg** pointer **sp**. The next MIDI message is read into the the structure by the pseudo-device **p**.

The vector **setparam(p, n, v)** sends a string value to device **p** with name **n**, and value **v** to the pseudo device. The set of values the pseudo-device implements is specific to the pseudo-device.

The vector **getparam(p, n, v, l)** gets a string from device **p** with name **n**, and value **v** with length **l**. The buffer **v** is filled with the resulting string. The set of values the pseudo-device implements is specific to the pseudo-device.

void _pa_synthoutplug(long addend, string name, void (*opnseq)(long p),
void (*clsseq)(long p), void (*wrseq)(long p, [ami_]seqptr sp),
long (*setparam)(long p, string name, string value),
void (*getparam)(long p, string name, string value, long len));

Declares a synthesizer/MIDI output pseudo-device. The **addend** Boolean indicates if the device is to be inserted at the front or the end of the device list. If true, it is inserted at the end, otherwise at the front.

The **opnseq(p)** vector will be called with the device number as parameter **p** when the device is opened, and the **clsseq(p)** vector will be called with the device number as parameter **p** when the device is closed. The device number may be ignored, or it may be used to differentiate between multiple devices if the same routine is used for the different devices.

The **wrseq(p, sp)** vector is called with a MIDI message structure **seqmsg** pointer **sp**. The next MIDI message is written from the the structure to the pseudo-device **p**.

The vector **setparam(p, n, v)** sends a string value to device **p** with name **n**, and value **v** to the pseudo device. The set of values the pseudo-device implements is specific to the pseudo-device.

The vector **getparam(p, n, v, l)** gets a string from device **p** with name **n**, and value **v** with length **l**. The buffer **v** is filled with the resulting string. The set of values the pseudo-device implements is specific to the pseudo-device.

void _pa_waveinplug(long addend, string name, void (*open)(long p), void (*close)(long p), long (*chanwavin)(long p), long (*ratewavin)(long p), long (*lenwavin)(long p), long (*sgnwavin)(long p), long (*fltwavin)(long p), long (*endwavein)(long p), long (*rdwav)(long p, byte* buff, long len), long (*setparam)(long p, string name, string value), void (*getparam)(long p, string name, string value, long len));

Declares a PCM wave input pseudo-device. The **addend** Boolean indicates if the device is to be inserted at the front or the end of the device list. If true, it is inserted at the end, otherwise at the front.

The **open(p)** vector will be called with the device number as parameter **p** when the device is opened, and the **close(p)** vector when it is closed. The device number may be ignored, or it may be used to differentiate between multiple devices if the same routine is used for the different devices.

Six vectors carry the sample format, and a wave input device exists largely to implement them: **chanwavin(p)** returns the number of channels, **ratewavin(p)** the sample rate, **lenwavin(p)** the bits in a sample, **sgnwavin(p)** whether samples are signed, **fltwavin(p)** whether they are floating point, and **endwavein(p)** the byte order. They are the input side of the format, so each is asked and answers: the device says what it has, where an output device is told what to take.

The **rdwav(p, b, l)** vector is called with a buffer for wave samples **b** with length **l**. The wave samples are read from the pseudo-device.

The vector **setparam(p, n, v)** sends a string value to device **p** with name **n**, and value **v** to the pseudo device. The set of values the pseudo-device implements is specific to the pseudo-device.

The vector **getparam(p, n, v, l)** gets a string from device **p** with name **n**, and value **v** with length **l**. The buffer **v** is filled with the resulting string. The set of values the pseudo-device implements is specific to the pseudo-device.

void _pa_waveoutplug(long addend, string name, void (*open)(long p), void (*close)(long p), void (*chanwavout)(long p, long c), void (*ratewavout)(long p, long r), void (*lenwavout)(long p, long l), void (*sgnwavout)(long p, long s), void (*fltwavout)(long p, long f), void (*endwaveout)(long p, long e), void (*wrwav)(long p, byte* buff, long len), long (*setparam)(long p, string name, string value), void (*getparam)(long p, string name, string value, long len));

Declares a PCM wave output pseudo-device. The **addend** Boolean indicates if the device is to be inserted at the front or the end of the device list. If true, it is inserted at the end, otherwise at the front.

The **open(p)** vector will be called with the device number as parameter **p** when the device is opened, and the **close(p)** vector when it is closed. The device number may be ignored, or it may be used to differentiate between multiple devices if the same routine is used for the different devices.

Six vectors carry the sample format, and a wave output device exists largely to implement them: **chanwavout(p, c)** sets the number of channels, **ratewavout(p, r)** the sample rate, **lenwavout(p, l)** the bits in a sample, **sgnwavout(p, s)** whether samples are signed, **fltwavout(p, f)** whether they are floating point, and **endwaveout(p, e)** the byte order. They are the output side of the format, so each is told a value and keeps it; the buffers that follow are in the format they last agreed.

The **wrwav(p, b, l)** vector is called with a buffer containing wave samples **b** with length **l**. The wave samples are written to the pseudo-device.

The vector **setparam(p, n, v)** sends a string value to device **p** with name **n**, and value **v** to the pseudo device. The set of values the pseudo-device implements is specific to the pseudo-device.

The vector **getparam(p, n, v, l)** gets a string from device **p** with name **n**, and value **v** with length **l**. The buffer **v** is filled with the resulting string. The set of values the pseudo-device implements is specific to the pseudo-device.

18 Example Applications



graph_games	Example graphical games.
breakoutg.c	Break the brick wall game.
breakoutwg.c	Breakout with a window, menus and help.
backgammon.c	Backgammon, with a resizable window and menus.
checkers.c	Checkers, with a resizable window and menus.
chess.c	Chess, with a resizable window and menus.
defenders.c	Space invaders, with original artwork.
pong.c	Pong handball game.
graph_programs	Example graphical programs.
ball1.c	“dazzlers” using balls.
ball2.c	“”
ball3.c	“”
ball4.c	“”
ball5.c	“”
ball6.c	“”
clock.c	A resizable clock program.
line1.c	“dazzlers” using lines.
line2.c	“”
line4.c	“”
line5.c	“”

pixel.c	Pixel set demo.
calc.c	A desk calculator with a resizable window.
disp.c	Connects standard input to a window and displays it.
mail.c	Reads mail from an IMAP server and shows it.
spreadsheet.c	A scrolling grid of cells on Ami graphics.
network_programs	Network example programs.
getmail.c	Get mail from server.
getpage.c	Get page from http[s] server.
gettys.c	Example server.
msgclient.c	Example message client.
msgserver.c	Example message server.
prtcertmsg.c	Print ssl certificate for messages.
prtcertnet.c	Print ssl certificate for TCP/IP.
connectnet.c	Telnet, with TLS to secure the connection.
fakemail.c	Test server for getmail: one fixed message.
listcertnet.c	Fetch a certificate from the chain by number.
sound_programs	Example sound programs.
connectmidi.c	Connect MIDI input to output.
connectwave.c	Connect PCM wave input to output.
genwave.c	Generate sine/square wave.
keyboard.c	Use keyboard as MIDI controller.
play.c	Play songs in MS Basic format.
playmidi.c	Play MIDI file.
playwave.c	Play PCM wave file.
printdev.c	Print sound devices.
random.c	Play random notes.
playtextmidi.c	Play a MIDI file written as text.

terminal_games	Example terminal games.
breakout.c	Break the brick wall game.
mine.c	Mine sweeper game.
pong.c	Pong game.
snake.c	Snake game.
breakoutw.c	Breakout with a window, in characters.
terminal_programs	Example terminal programs.
editor.c	Text editor.
editor.not	Text editor notes.
wator.c	Wator visual demo.
mailc.c	Reads mail from an IMAP server, in characters.
tests	Test programs.
event.c	Event display.

19 Libc and alternatives



19.1 stdio

stdio.c is an implementation of the standard I/O functions:

```
FILE *fopen(const char* filename, const char* mode);
FILE *freopen(const char* filename, const char* mode, FILE* stream);
FILE *fdopen(int fd, const char *mode);
int fflush(FILE* stream);
int fclose(FILE* stream);
int remove(const char *filename);
int rename(const char *oldname, const char *newname);
FILE *tmpfile(void);
char *tmpnam(char s[]);
int setvbuf(FILE* stream, char *buf, int mode, size_t size);
void setbuf(FILE* stream, char *buf);
int fprintf(FILE* stream, const char *format, ...);
int printf(const char* format, ...);
int sprintf(char* s, const char *format, ...);
int vprintf(const char* format, va_list arg);
int vfprintf(FILE* stream, const char *format, va_list arg);
int vsprintf(char* s, const char *format, va_list arg);
int fscanf(FILE* stream, const char *format, ...);
int scanf(const char* format, ...);
int sscanf(const char* s, const char *format, ...);
int fgetc(FILE *stream);
int getc(FILE *stream);
char *fgets(char *s, int n, FILE *stream);
int fputc(int c, FILE *stream);
int fputs(const char *s, FILE *stream);
int putc(int c, FILE *stream);
int getchar(void);
char *gets(char *s);
int putchar(int c);
int puts(const char *s);
int ungetc(int c, FILE *stream);
```

```
size_t fread(void *ptr, size_t size, size_t nobj, FILE *stream);
size_t fwrite(const void *ptr, size_t size, size_t nobj, FILE
*stream);
int fseek(FILE* stream, long offset, int origin);
long ftell(FILE* stream);
void rewind(FILE* stream);
int fgetpos(FILE* stream, fpos_t *ptr);
int fsetpos(FILE* stream, const fpos_t *ptr);
void clearerr(FILE* stream);
int feof(FILE* stream);
int ferror(FILE* stream);
void perror(const char *s);
int fileno(FILE* stream);
int snprintf(char* s, size_t n, const char *format, ...);
int vsnprintf(char* s, size_t n, const char *format, va_list arg);
int fseeko(FILE* stream, off_t offset, int whence);
off_t ftello(FILE* stream);
ssize_t getline(char** lineptr, size_t* n, FILE* stream);
ssize_t getdelim(char** lineptr, size_t* n, int delim, FILE* stream);
FILE *fmemopen(void* buf, size_t size, const char* mode);
FILE *open_memstream(char** bufp, size_t* sizep);
```

These are implemented local to Ami. The purpose of this is that those calls can be redirected to be presented as a stream within the Peitit-Ami package. This is done for several reasons:

- The text is presented as a screen surface in Terminal.
- The text is presented in a graphical surface in Graphics.
- The I/O text is sent via a TCP/IP connection.

The stdio calls are implemented compatibly with standard ANSI C stdio calls. The difference is that the lowest level I/O that goes through the calls:

```
static ssize_t read(int fd, void* buff, size_t count);
static ssize_t write(int fd, const void* buff, size_t count);
static int open(const char* pathname, int flags, int perm);
static int close(int fd);
static int unlink(const char* pathname);
static off_t lseek(int fd, off_t offset, int whence);
```

can be overridden by the calls:

```
void ovr_read(vt_read_t nfp, vt_read_t* ofp) { *ofp = vt_read; vt_read = nfp; }
void ovr_write(vt_write_t nfp, vt_write_t* ofp) { *ofp = vt_write; vt_write = nfp; }
void ovr_open(vt_open_t nfp, vt_open_t* ofp) { *ofp = vt_open; vt_open = nfp; }
void ovr_close(vt_close_t nfp, vt_close_t* ofp) { *ofp = vt_close; vt_close = nfp; }
void ovr_unlink(vt_unlink_t nfp, vt_unlink_t* ofp)
      { *ofp = vt_unlink; vt_unlink = nfp; }
void ovr_lseek(vt_lseek_t nfp, vt_lseek_t* ofp) { *ofp = vt_lseek; vt_lseek = nfp; }
```

Which allows other modules (or even your modules) to redirect I/O, perhaps selectively, to other purposes. Each of the overriders places their own calls into vectors in the Ami stdio package, and gives

the caller back a vector to the old function. This means the overrider can perform the action itself, send it back to the original call, or modify the data and then send it on. This is a classic override in the sense of object oriented languages.

19.2 Linker overriding

Using either gcc or llvmgcc, when stdio is properly linked to the target program, the Ami stdio module replaces the functions in stdio from the ANSI C layer in the target operating system. For Linux this is glibc. For windows it is msvcrt. For Mac OS X it is libSystem, the system library that folds in the BSD C library. Thus Ami does not have to replace the entire libc collection of functions, but only the code within stdio.

19.3 The STDIO_BYPASS flag

In previous versions it was found that Linux had issues with programs that did backdoor accesses to the FILE structure, assuming that it had fields that only glibc had internally. This is no longer an issue due to:

1. stdio.c emulates many or most of these fields.
2. The link order in Linux arranges for glibc and associated programs to only call glibc routines.
3. The reliance on backdoor accesses is reduced in later versions of linux.

If linker overriding still causes issues, there is a backup method to use, which is the STDIO_BYPASS compile flag. This flag has two effects:

1. It changes the stdio.c library to use ami_ as the preamble for all stdio calls and data.
2. It changes user programs to use ami_ as the preamble for all stdio calls and data.

Usually these two are used together. The net effect of that is that the stdio calls are all redirected toward the Ami stdio layer. Since this can be applied on a file by file basis, this allows you to “sculpt” exactly which calls go to the Ami layer and which go to the existing libc layer.

Optionally, you can apply this flag to stdio.c only. The effect of this is that you would need to place the ami_ preamble on stdio calls you want to specifically go to the Ami stdio package. This mode would be used if you wan to mix existing libc calls with Ami stdio calls.

20 Directory layout



The directory format for the Ami project generally follows the standard for GNU project spaces. The directories are:

Directory	Contents
api_html	The API reference, as generated from the headers.
apis	Interfaces of other systems built on Ami, curses among them.
bin	Binaries and scripts.
cpp	The C++ wrappers.
curses	BSD curses games and nano, built on the curses in apis.
doc	Documentation: this manual, the protocol paper and their sources.
graph_games	Example graphical games.
graph_programs	Example graphical programs.
hello	A hello world program and the makefile that builds it.
hpp	The C++ wrapper headers.
html	The manual as generated HTML.
include	The API headers.
lib	Built libraries and objects.
libc	Ami's own stdio.
linux	Linux specific source files.
macosx	Mac OS X specific source files.
midi	MIDI example files.
network_programs	Example network programs.
portable	The modules that are the same on every system.
sound_programs	Example sound programs.
soundfont	Sound fonts for the software synthesizer.
stub	No-operation API code for Ami.
terminal_games	Example terminal games.
terminal_programs	Example terminal programs.
tests	Test programs.
tools	Tools used on the source itself.
utils	Utility modules and the programs that use them.
videos	Recordings of the programs running.
wave	Wave example files.

windows	Windows specific source files.

21 Installing and building Ami



The project lives on GitHub, and the tree carries submodules, the curses ports of nano, htop and the BSD games[8]. Clone the repo as follows:

```
git clone –recurse-submodules https://github.com/samiam95124/amitk.git
cd amitk
```

A tree cloned without them is filled in with **git submodule update --init**.

Ami follows the GNU guidelines for configure and build steps. Steps to install:

Linux:

```
. setpath
./configure
make
```

Windows:

```
setpath
configure
make
```

setpath puts the tree's bin directory on the path, so what is built there runs by name. It is the first of the steps for that reason.

On BSD and systems like it, gmake stands in for make.

configure installs what Ami builds against, by **sudo apt-get install** on Linux: alsa and its development files, openssl, fluidsynth, gawk and the rest. It does not ask first, so run it on a machine where that is welcome.

[8]The curses ports can be skipped if you don’t plan to use them.

The make step makes all Ami libraries and target programs. If you want to only build a particular program or component, execute:

```
make <program/module>
```

21.1 Standard make targets

all Makes all libraries and target programs.

clean Removes all binaries and intermediate files.

22 Building user applications



An Ami program is an ordinary C or C++ program linked against one of the Ami libraries. The tree carries two examples of that build, the directories **hello** and **hellocpp**, each holding the program, a makefile and a readme. The makefile uses the **include**, **hpp** and **lib** directories of a built tree and nothing else, so it is the pattern for a program of your own, and it builds wherever the directory is placed.

The program in hello is the standard hello, world:

```
#include <stdio.h>

int main(void)

{

    printf("hello, world\n");

    return (0);

}
```

hellocpp is the same program on the C++ standard streams:

```
#include <iostream>

int main()

{

    std::cout << "hello, world" << std::endl;

    return (0);

}
```

22.1 Models and forms

The same program builds on any of the three Ami models, chosen by **MODEL** on the make command line:

plain — No terminal handler. The system, sound and network interfaces are available, and output goes to standard output as it does for any program. Several parts of Ami rely on neither terminal nor graphical output, so there is good reason to build plain programs.

term — The terminal model takes over the console as the program starts, and hello, world appears on a cleared screen.

graph — The graphical model opens a window as the program starts, and hello, world appears in it. The window stays until it is closed.

and against either form of the library, chosen by **LINK_TYPE**:

static — The archive **lib/libami_<model>.a**. This is the default.

dynamic — The shared library **lib/petit_ami_<model>.so**.

So the build lines are:

```
make                                  plain, static archive
make MODEL=plain LINK_TYPE=static     plain, static archive
make MODEL=plain LINK_TYPE=dynamic    plain, shared library
make MODEL=term  LINK_TYPE=static     terminal, static archive
make MODEL=term  LINK_TYPE=dynamic    terminal, shared library
make MODEL=graph LINK_TYPE=static     graphical, static archive
make MODEL=graph LINK_TYPE=dynamic    graphical, shared library
```

Either option left out takes its default, so a bare make is the first line. The program carries no record of the options it was built with, so after changing either, make clean before making again.

22.2 Building outside the tree

The makefile finds the Ami tree through the environment variable **AMIDIR**. Sourcing **setpath** at the root of the tree sets it, and puts the tree's bin directory on the path:

```
cd amitk
. setpath
```

Then copy the example directory wherever the program is to live, and make there:

```
cp -r $AMIDIR/hello ~/myprog
cd ~/myprog
make MODEL=graph
```

The libraries the makefile links are made by the build at the root of the tree, in the same **LINK_TYPE**. The root compiles its objects for one link type or the other, so a tree built in one form is cleaned at the root before it is built in the other:

```
cd $AMIDIR
make clean
make LINK_TYPE=dynamic
```

The program runs from any directory. In the dynamic form it carries the path of the tree's lib directory, as an rpath, so the shared library is found wherever the program runs from.

The libraries Ami depends on must be present on the machine: OpenSSL, ALSA and fluidsynth, and for the graphical model the libraries of the display the graphical library was built for. **configure** at the root of the tree installs them.

A program of your own replaces hello.c in the directory and in the makefile, and takes the same line. A program of several files lists them where hello.c stands.

22.3 How the makefile links the program

The link line of the C example is:

```
gcc -g3 -I$(AMIDIR)/include $(AMICFLAGS) hello.c \
    $(KEEP) $(AMILINK) $(LIBS) -o hello
```

and its parts are these.

The system libraries every model wants come first, in **LIBS**: sound, SSL, math and threads, **-lasound -lfluidsynth -lssl -lcrypto -lm -lpthread**. The sound and network parts of the library are drawn in only by a program that calls them.

The graphical model adds the libraries of the display backend the graphical library was built with. The makefile chooses as the root build does: Wayland when a Wayland session is up, which is when **WAYLAND_DISPLAY** is set, **-lwayland-client -lwayland-cursor -lxkbcommon -lfreetype -lfontconfig**, and X11 otherwise, **-lX11 -lXtst -lfreetype -lfontconfig**. A library built for the frame buffer wants **-lfreetype -lfontconfig -lpng -lz**.

The terminal and graphical models take over standard output as the program starts, so a program that calls nothing in them still wants the model linked. A library is drawn in only by the calls made to it, so **KEEP** asks the linker for one of the model's functions by name, **-Wl,-u,ami_maxx**. The plain model is drawn in by the program's own calls.

AMILINK is the library itself. In the static form it is the archive. In the dynamic form it is the sound object **lib/sound.o**, then the shared library, then the rpath of the lib directory. Sound links as an object of its own in that form, because of a bug in ALSA, and it stands before the shared library on the line: the linker keeps a shared library for the references it has already seen, and the sound object's are the first.

The two forms meet the program's printf differently, which is what **AMICFLAGS** carries. The archive holds Ami's stdio under the standard names, so the program compiles against the system's stdio.h and its printf lands in the model when it is linked. The shared library holds Ami's stdio under its own names beside the system's, so the dynamic form compiles the program against Ami's stdio header with **-I$**

(AMIDIR)/libc -DSTDIO_BYPASS, which gives printf that name and puts the line in the terminal or the window.

22.4 C++ programs

The hellocpp makefile is the C example's with g++ for gcc, the **hpp** directory on the include path, and one more linker request:

```
g++ -g3 -I$(AMIDIR)/include -I$(AMIDIR)/hpp hello.cpp \
    $(KEEP) $(AMILINK) $(LIBS) -o hello
```

The request is **-Wl,-u,ami_iostream**, which names Ami's stream binding, **cpp/stdio.cpp** in the tree. The binding is a member of each model's library, and it puts the four standard streams, std::cin, std::cout, std::cerr and std::clog, on Ami's stdin, stdout and stderr as the program starts. A program that uses one of Ami's C++ classes, terminal::term or graphics::graph, gets the binding with the class. A program like hello.cpp calls nothing in it, so the makefile asks for it by name, and its std::cout lands where the model puts standard output, in step with any printf the program also makes.

A C++ program compiles against the system's headers in both forms of the library, since <iostream> brings the system's stdio.h with it, and the streams reach the model through the binding in the library. In the static form the archive supplies the stdio names to the whole program, so the program's printf and its std::cout land together. In the dynamic form the streams land in the model and the program's C stdio calls, printf and the rest, land on the system's stdout.

The binding covers the narrow streams, and it stands as long as the streams keep their default synchronization: std::ios_base::sync_with_stdio(false) replaces the bound streams with the C++ library's own.

The binding also gives file streams on Ami's stdio, stdio::ifstream, stdio::ofstream and stdio::fstream, declared in **hpp/stdio.hpp**, for files opened through Ami and for the FILE* Ami hands out, such as a window or a network connection:

```
#include <graphics.hpp>
#include <stdio.hpp>

graphics::window w;      /* a window; it converts to FILE* */
stdio::ofstream win(w);
win << "text in the window" << std::endl;
```

22.5 Platforms

Linux builds either form, as described above.

Mac OS X builds the static form. The example makefiles carry a branch for it: the OpenSSL libraries come from Homebrew and the rest from the system frameworks; the library carries its own stdio, so the program compiles against the tree's header with **-I$(AMIDIR)/libc**; and the graphical model loads the whole archive and enters at the library's entry point, **-Wl,-force_load,<archive> -Wl,-e,_pa_entry**. Mach-O symbols carry a leading underscore, so the model request there reads **-Wl,-u,_ami_maxx**.

On Windows every program links the static archives, and the examples build from the root of the tree as the tree's own programs do: setpath, then **make hello**, **make hellog**, **make hellocpp** or **make hellocppg**, which leave the programs in bin.

23 Testing Ami



The tests for Ami are kept in the **tests** directory, and are built along with everything else by **make**. Most of them judge themselves: they run without anyone watching, compare what they produced against a standard kept in the tree, and say whether it matched. The rest want eyes or ears, and are run by hand.

23.1 The regression

bin/regress runs the tests that judge themselves. With the tree's **bin** on the path, from anywhere in it:

```
regress
```

It finds the root of the tree from its own location and works there, so where it is started from does not matter. Each test is run in turn, and the result is one line each:

```
stdio_test: pass
services_test: pass
network_test: pass
txttest: pass
pdftest: pass
terminal_test: pass
terminal_testc: pass
window_testc: pass
graphics_test: pass
window_test: pass
widget_test: pass
graphics_testr: pass
window_testr: pass
widget_testr: pass
```

Named on the command line, only those tests run:

```
regress stdio_test network_test
```

The exit status is the number of tests that failed, so zero is a pass and a build script can act on it without reading the text. A test that cannot run where it is, a screen test with no display or a terminal test with no xterm, says so and is skipped, which is not a failure.

23.2 The tests

Fourteen tests make up the regression:

Test	What it covers	Produces
stdio_test	the standard I/O library: the printf and scanf conversions, open, close, read and write, positioning, character and line I/O with pushback, buffering, temporary files, and the direct FILE accessors	text
services_test	the services module: directory lists with their attributes and permissions, times and dates in every form, the environment, path and file name handling, and program execution	text
network_test	the network module, against servers run on threads of the test itself: name lookup, connections and messages, in the clear and secured, IPv4 and IPv6, certificates, and the corners of the contract	text
txttest	the text print module: a page printed into a file	a text page
pdftest	the PDF print module: a document printed into a file, its text set in the standard PDF fonts	a document
terminal_test	the terminal level: rows and columns numbered in turn, the printable set, fills, a bounced ball, scrolling in each direction, attributes, colors, the mouse and the joystick	terminal listings
terminal_testc	the same test through the character window manager, which presents the terminal transparently when the program opens no windows	terminal listings
window_testc	window management in character mode, rooted on an xterm of its own, every test applying to a child window on it	terminal listings
graphics_test	the drawing calls of a single unmanaged window: lines, rectangles, ellipses, arcs, chords, triangles, text in every font and attribute, colors, pictures, tabs, scrolling, the viewport and the mouse	screens
window_test	window management: opening windows, size and position, buffers, frames, titles, the system bar, menus, front and back, and the alert dialog	screens
widget_test	every widget and every dialog: buttons, checkboxes, radio buttons, groups, backgrounds, edit boxes, number selectors, lists, drop boxes, sliders, scroll bars, tab bars, progress bars, and the color, file, font and find dialogs	screens
graphics_testr, window_testr, widget_testr	the three screen tests again, linked with the remote client and drawn through graph_server on this machine	a clean run

They are judged four ways: text by a masked compare, terminal listings the same way, screens byte for byte, and the remote forms by walking out clean.

23.3 Judging text

A text test prints what it did and what it should have been, side by side:

```
test 100: "12345" s/b "12345"
```

Some judge themselves as well. **stdio_test** ends with “All self checking tests passed” and returns a status, and **network_test** says pass or fail beside every case it runs. The regression does not take their word for it: it compares the whole listing against the standard, so a change in what a test prints is caught along with a change in what it does.

The compare is **dif -q**. **dif** is the difference utility in **utils**, and **-q** is the option the regression wants: a **?** in either file matches any single character, and a ***** matches any run of them. The fields that cannot be the same twice are masked with those in the standard: dates, times, the elapsed tick count of the timing test, the process id in a temporary file name, the daylight savings flag, and the address a live name lookup returns. Everything else has to match to the character. The print tests are judged the same way: **txttest** prints a page into **tests/txttest.lst** and it is compared by **dif**; **pdftest** prints a document into **tests/pdftest.pdf**, named so that a reader opens it, and the file is compared whole, since nothing in it varies from run to run.

What the compare found is left in **tests/<test>.dif**. A test passes when that file is empty.

23.4 Judging terminal listings

terminal_test is judged on characters, not pixels. The regression runs it in an xterm of its own on a private X server, Xvfb, 80 by 24 whatever the desktop, and at each frame the test asks the terminal to print its screen. The xterm’s printer command appends the page to a listing, **tests/terminal_test.lst**, a page per frame with a form feed after each, and the test waits for each page to be in the file before it goes on, so the pages are in order and all there at the end. The terminal’s font, size and scale never reach the listing, so one standard serves every machine; it is compared by **dif** like the text tests, with the timing figures masked. The test runs from the sandbox **tests/terminal**, whose configuration turns the joysticks off, so the standard is of no machine in particular.

terminal_testc is the same test through the character window manager, which presents the terminal transparently when the program opens no windows of its own, and is judged against **terminal_test**’s standard; **--update** on either regenerates the one standard. **window_testc**, the window management test in character mode, runs the same way in a larger xterm, 132 by 50: it is rooted, the xterm being the desktop and every test applying to an 80 by 25 child window on it, and needs the room. Its standard is its own, **tests/window_testc.cmp**. These three are skipped where there is no xterm or Xvfb.

23.5 Judging screens

graphics_test, **window_test** and **widget_test** draw. What they produce is not text but the screens themselves, and every screen is kept.

Each of the three takes **auto** on the command line. It then walks its screens with no hand on the desk: the waits for the user are answered by the event file described below or, where it has nothing for a frame, at once; the benchmarks, whose output is timings and never the same twice, are left out; and the program ends when the screens end. A second argument names the file the screens go to, which is how the regression gives each test one of its own:

```
graphics_test auto tests/graphics_test.img
```

The screens come from the screen capture module, which the tests are linked with. Where the interactive form waits for a keypress, and after each line of the event file has done its work, the test calls **screen_capture()**, and the module appends the window’s picture to the file as a PNG. The PNGs sit back to back and each is self delimiting, so the file is a slide show of the run, in the order it happened: 456 screens for **graphics_test**, 89 for **window_test** and 656 for **widget_test** on Plasma over

Wayland as they stand. Each carries its label as the PNG title, “frame N” for a screen taken at the end of a pattern and “frame N.S” for a step within it. What is captured is the client area and not the frame around it. The frame is drawn focused or unfocused at the desktop’s discretion, and a standard carrying it would fail whenever the user happened to be looking somewhere else.

The compare is byte for byte, over the whole file. A picture has no field that varies from run to run to be masked: either the run drew what the standard drew, or it did not.

Either file is walked with the viewer:

```
testviewer tests/linux/plasma/wayland/widget_test.cmp
```

Right arrow for the next screen, left arrow for the one before, the shifted arrows for the next or previous whole frame, **q** to leave. That is how a new standard is reviewed before it is committed, and how a failure is read afterwards. The **.dif** of a screen test gives the size of each file and the number of screens in it, and where the first difference falls, and then leaves you to look. The file of the failed run is kept as well, as **tests/<test>.fail-<stamp>.img** with the date and time of the run in the name, so the screens that differed survive the re-run that overwrites the **.img**. What the test printed while it drew, the event file’s report among it, is in **tests/<test>.out**.

The remote forms, **graphics_testr**, **window_testr** and **widget_testr**, are the same three linked with **graph_client** in place of the local display module. The regression starts **graph_server** on this machine and runs each against it, and there are no pictures to compare: the display is the server’s. A remote test passes when it walks out clean and says nothing on the way. See the chapter on remote mode.

23.6 Driving the tests: the event file

A pattern that a user drives, the mouse over a widget, keys into an edit box, a menu chosen, cannot be walked by answering its waits. So each screen test, and the terminal test, has an event file: a text file of the events the run is to be given, in the order it is to be given them, and at which frame. The module that reads it is **tests/auto_event.c**, linked into the tests, and a test asks it for events with **auto_event()** in place of **ami_event()**: the next event from the file when one is ready for the frame the test is on, and the next event from Ami otherwise. The files are **<test>.evt**, kept in the tree of standards described in the next section and committed as inputs.

A line holds one statement, and **#** starts a comment:

Statement	What it does
<event> [parameters]	an event, named as in **include/graphics.h** without the **ami_et** prefix, its parameters in the order of the fields of the event record: **char 'h'**, **moumovg 1 400 300**, **mouba 1 1**, **enter**, **button 5**. A character is written **'x'** or as a number. A number is written plain, or as **max**, **-max**, **max/n** or **-max/n**, the largest value of the type and fractions of it, which is the range of a joystick’s axes
sync <frame>[.<step>]	hold the events after it until the test is on that frame, and that step of it if given
window <id>	the window the events that follow carry, 1 to start
wait <ms>	let the program run for that long, its events its own, then take a step: what time did, a progress bar or a clock
keyboardoff, mouseoff,	drop the events of the real keyboard, mouse or joystick, so a device

joystickoff	on the desk cannot join a test of its events from the file; **keyboardon**, **mouseon** and **joystickon** let them through again
mousenum <n>, joysticknum <n>	the count of mice or joysticks the test is told of, through **auto_mouse()** and **auto_joystick()**, which the test uses in place of **ami_mouse()** and **ami_joystick()**: the file's count when set, the desk's otherwise
mousebutton <n>, joybutton <n>, joyaxis <n>	the buttons of every mouse, and the buttons and axes of every joystick, through **auto_mousebutton()**, **auto_joybutton()** and **auto_joyaxis()** the same way

The device lines go at the start of the file, before any sync, so they hold for the run. A sync names a frame, and frames are numbered as the tests number them: the number advances at the start of each pattern, so the title while a pattern runs, the steps taken within it and the screen at its end all carry the same number, and that is the number a sync names. The events after a sync are held until the test reaches that frame, then given one by one as the test asks; each pattern's events end with the **enter** that ends the pattern for a user. A frame the test passes without running its pattern leaves its events in the file; they are skipped to the next sync and counted at the end, and a sync the run never reaches is reported on standard error, with the line it stopped at, when the run ends. A run that skipped or was left waiting is a cut run, and the regression refuses to take a standard from one.

This is the start of **widget_test**'s file:

```
# The real joystick is set aside; the keyboard and mouse are the
file's
joystickoff

# Terminal buttons test: each button pressed, then again with the
# middle one disabled, which does nothing
sync 2
moumovg 1 264 251
mouba 1 1
moubd 1 1
moumovg 1 221 359
mouba 1 1
moubd 1 1
moumovg 1 700 600
mouba 1 1
moubd 1 1
enter
```

Where the display carries a seat rig, the mouse and key events of the file are not handed to the program but put in at the seat, as if a person made them. The Wayland layer has one, a fifo named to it in **PD_INPUT**, made for headless compositors; the X module has one through the XTest extension. A move goes to the point named, in the client area of the main window; a button press or release goes to the pointer's place; a key goes to whatever holds the keyboard focus. The display routes them the way it routes a real seat's input: the hit test finds the window under the point, a widget's face or the client area, with hover, focus and crossings as for a person, and the program gets what the widget sends. So the three lines at 264,251 above press whatever is there, and the file works the buttons, scroll bars and number boxes that way, the coordinates read off the pictures of the standard. Keys go in as keycodes, so a character is a key of the US layout, letters, digits and the unshifted punctuation, with a held shift for the rest. Focus follows the seat's clicks as it does a person's: a press leaves the keyboard with the widget

pressed, so a file clicks the main window before the **enter** that is to end a chapter, and clicks a number box's field before the **enter** that is to send its value. Where there is no rig, the remote client, the terminal, the framebuffer, the events go to the program directly, as the widget and menu events always do.

After each seat line the module waits a fixed moment, 200 milliseconds, handing the program every event the line provokes, and then says a step: **auto_event_step()** is true once the line has run its course, and that is when a test captures a screen labelled with the frame and the step. The moment is fixed rather than a quiet interval because a program with the frame timer on is never quiet. While the Wayland rig's fifo is open the rig holds the seat: the display ignores the desk's own pointer and keyboard, crossings and focus included, so a run on a live desktop is not joined by the hand on it. XTest cannot hold the seat, and under Xwayland its pointer motion moves the desktop's own pointer, so an X run is best left alone while it goes.

Making a file means running the test once to find its frame numbers, and reading the coordinates of what is to be clicked off the standard's pictures. With **AUTO_EVENT_TRACE** set in the environment each event given is reported on standard error with the frame and step it went to, which is how the numbers are found and how a file that goes wrong is read. **graphics_test auto** reads the file; **graphics_test events** reads it in an interactive run as well, so a file can be watched. Every event the file can name, the test can be given from the keyboard and mouse in an interactive run; the file only replaces the hand.

23.7 The standards

The standard for a test is the file with the **.cmp** extension, and where it lives says what it is a standard of. A test whose output is the same everywhere keeps one at the top: **tests/stdio_test.cmp**, **tests/network_test.cmp**, **tests/terminal_test.cmp**, **tests/txttest.cmp**, **tests/window_testc.cmp**. Some text standards are of a platform's paths, environment and directory listing, and the document standard is of the platform's arithmetic, so those tests keep one per platform, in the platform's directory: **tests/linux**, **tests/windows**, **tests/macosx**. The screen standards are of a desktop's look, the frame the tests size themselves against, the menus, the theme, and of a backend's, since the Wayland module draws its own frames at the buffer's scale and the X module has the window manager's. So under **tests/linux** the tree forks again, by desktop and then backend:

```
tests/linux/gnome/wayland/widget_test.cmp
tests/linux/gnome/x11/widget_test.cmp
tests/linux/plasma/wayland/widget_test.cmp
tests/linux/plasma/x11/widget_test.cmp
```

A standard, or an event file, is looked for from the most particular directory that applies up to **tests** itself, and the first found serves. So a file that holds for every backend of a desktop sits at the desktop, one that holds everywhere sits at the top, and one that is a backend's own sits at the backend; the event file whose points were read off the Wayland pictures sits at the top today, and serves X11 only until one read off the X11 pictures is put beside the X11 standards. The regression picks the desktop the way the decorations choose themselves, from **XDG_CURRENT_DESKTOP**, and the backend from the symbols the test's binary links, which is what a build with **GRAPHICS_BACKEND=x11** changes. A desktop and backend that have no standard yet fail their screen tests with a line saying so, and **--update** makes one, in the backend's directory.

The standards are committed with the source, and they are the only files of a test run that are. The capture of the run (**.img** or **.lst**), what the compare found (**.dif**) and what a test printed while it ran (**.out**) are made fresh every time.

When a change to Ami is meant to change what a test produces, the standard is remade from a run of the new code:

```
regress --update
regress --update widget_test
```

Each standard is written from a fresh run, with the masks applied to the text tests as it goes, and a picture standard is refused from a run the event file did not carry through, since a standard taken from a cut run passes a cut run. What comes out is then reviewed, the text read, the screens walked with **testviewer**, before it is committed. An **--update** that is not reviewed commits the fault along with the fix.

A standard is a record of one machine. The text ones carry the home directory, the country and the timezone of the machine that made them. The screen ones are pictures of that screen, at that size, in those fonts, drawn in that desktop's look through that backend, which is why each keeps a set of its own. Moving the tree to another machine means making the standards again there and reviewing them, which is expected and not a fault.

23.8 The tests that want a person

terminal_testg is the terminal test built on the graphics library rather than the terminal module: one source, and the two should look the same. It is driven by hand and watched, since its screens are pictures of a graphical window drawing characters and there is no listing to judge.

sound_test goes through the sound library, and much of it is listening: the MIDI tests exercise the synthesizer as well as the library, and only an ear can say whether the note was the right one. The sections after the listening tests cover the rest of the interface, the devices and their names, the parameters, the sequencer, wave in and out and MIDI files. The virtual MIDI loop in it runs by itself: the library's encoder goes on one end of an ALSA wire and its decoder on the other, and each kind of message is sent and read back.

widget_testc is the widget test in character mode, over the character window manager, and **sound_testr** the sound test over the remote display protocol.

random_test is a torture test: it runs procedures of Ami chosen at random, figures at random points and colors, child windows at random places with random tests run inside them, network messages against an echo server of its own, and with **-s** sounds, on several threads at once, and it keeps a count in its title. It has no standard, since no two runs are alike; what it finds is a hang, which it reports as a stall and leaves running for a debugger, or a fault. See **doc/random_test.md**.

Besides the tests, it is customary to run through the example programs during a series: the games and programs in **terminal_games**, **terminal_programs**, **graph_games**, **graph_programs**, **sound_programs** and **network_programs**. They use Ami the way a program uses it, and they turn up what a test written against the interface does not.

23.9 When a test dies

A program built on Linux carries a crash dump. On a segmentation fault, a bus error, an illegal instruction, an arithmetic fault or an abort, the thread that took it writes to standard error what the signal was, the address it faulted on, which thread and process it was, and the stack of calls that led there, innermost first, then lets the signal take the process down as it would have, so the core file is still made. The stack is given twice: as the program exports its names, and, for the program's own frames, by function and source line, static functions included. A stack overflow is reported like any other fault,

from a stack the handler keeps for itself. The module is **linux/crashdump.c**; it is compiled in when the Makefile has **CRASHDUMP=1**, which is the default, and **CRASHDUMP=0** leaves it out. **AMI_CRASHDUMP=0** in the environment turns it off in a program built with it, for a debugger that wants the signal raw.

The library's own errors, "File is not attached to a window" and the rest, are not signals: each module's error routine says its piece and exits, so there is no dump and no core. With **AMI_ERRABORT** set in the environment every module aborts there instead, and the abort comes through the dump like any other fault, with the stack of the thread that raised the error. That is the switch for an error that only turns up in a long run:

```
AMI_ERRABORT=1 random_test -s
```

crash_test dies on purpose to show the dump: by itself it divides by zero three calls deep; **crash_test null** reads through a null pointer, **crash_test abort** aborts, **crash_test stack** recurses without end, and **crash_test error** asks the services module to run an empty command, which is a library error and shows what **AMI_ERRABORT** does.

23.10 The test report

The record of a full series is kept by hand in the file **TESTREPORT** at the root of the tree: which tests were run, on which systems, whether each passed or failed, when the series finished and on what version. The regression says pass or fail for the fourteen it runs; **TESTREPORT** is where the rest of a series, the tests that want a person and the walk through the examples, is written down.

24 Windows Specific Details



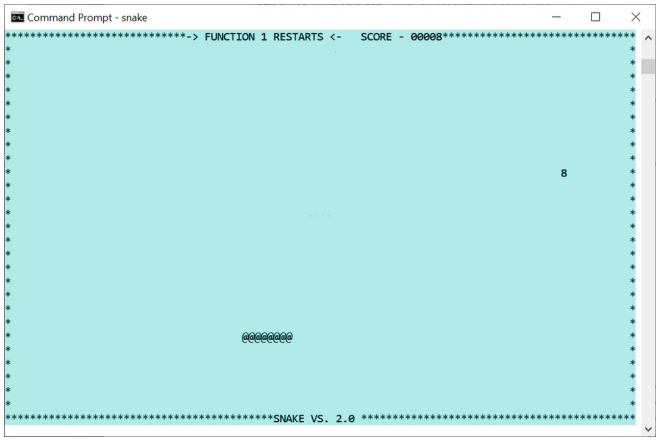
24.1 Terminal

The **terminal** module in Windows is implemented with the Windows Console API. By default, **terminal** takes the display and buffer sizes that exist when **terminal** starts.

24.1.1 Transparent mode

Windows console has two surfaces of importance, the current buffer size and the current display window. The display window is a subset of the buffer, and it moves down as lines are printed. The text scrolled off the top of the display region stay in the buffer, acting as a “history” of printed lines.

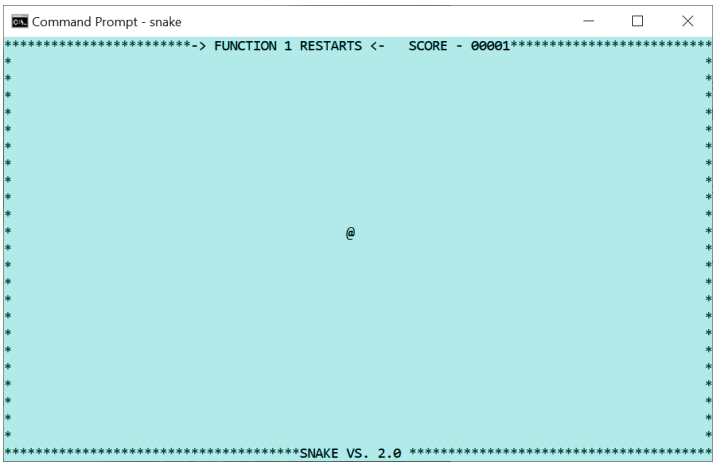
Windows **terminal** has an ability called “transparent mode”. If the selected screen is left at the default 1, and no control features executed (reposition cursor, etc.), then the console window will behave just as if the program did not use the **terminal** module. Scroll down will behave as normal. A program that takes advantage of cursor positioning will operate in a fixed window defined by the current display region:



A console program presenting in the middle of the buffer.

A program like “play” (in the sound_programs directory) does not do any display manipulation, but only uses the **terminal** module for its input event capability and timers. It does not flip the display screen. Thus it appears to function as a console line oriented program, even while it processes events.

A **terminal** program that does move to a different screen:



A console program using a new buffer.

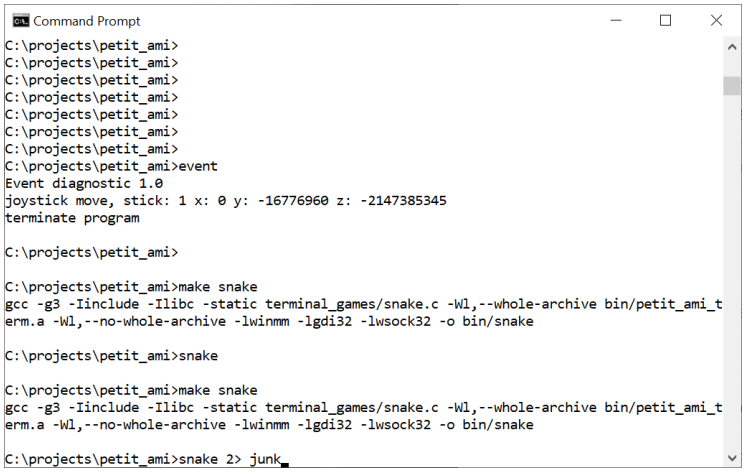
Gets a buffer that is the same as the display area. If it restores the screen to 1,1, as is the custom with Ami programs, the old buffer, display area, and screen contents will be restored.

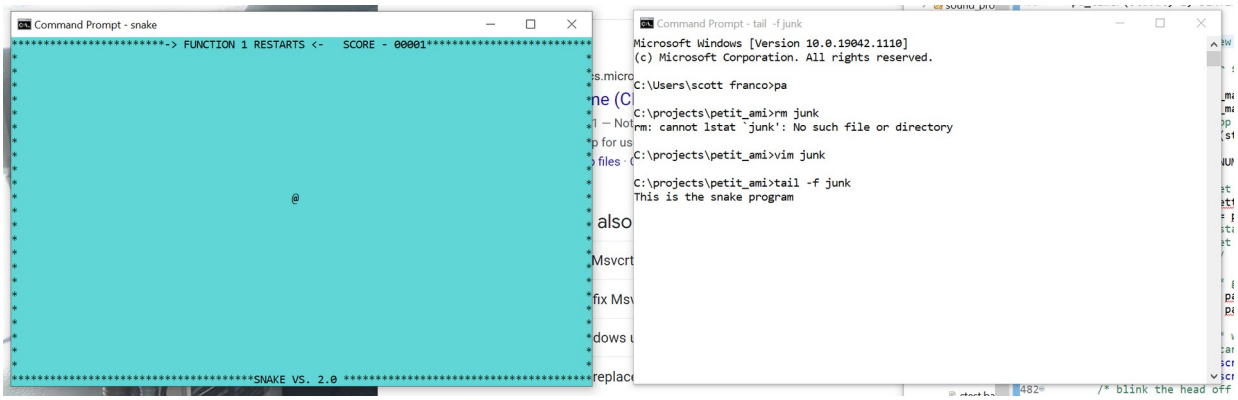
24.1.2 Configuration settings

At this writing, Windows terminal does not implement any configuration settings.

24.1.3 Redirected files

The files **stdin** and **stdout** are redirected to the terminal surface. Any other files, including the **stderr** file, bypass the terminal module and print directly. This can cause the terminal display module to appear to malfunction. I recommend instead that you reroute the messages from **stderr** to another window:





Using a program such as “tail” from the Mingw toolset. This gives you a good debug method without interfering with the main display.

24.1.4 Bypass input

If the input from **stdin** is directly read, as opposed to using the **event()** call, terminal will automatically perform echoing of all input characters, and will buffer the characters up into lines. The line is edited while it is typed: characters go in at the cursor or over what is already there, according to the insert toggle; the cursor moves by character, by word, and to either end of the line; a character is deleted forward or back, and the whole line at a stroke; and a click of the left button inside the line places the cursor. Which key does which is the event mapping of the terminal chapter, so the editing follows the keyboard the user has set rather than a list fixed here. Enter ends the line.

24.2 Graphics

The graphics module is implemented with the Win32 API. It comes up by default in 80x25 character mode. The main (default) window will be opened as a separate window, even if it were started from a console window.

AMI_FONT_TERM is the stock system fixed font. That is a raster font, which GDI can neither scale nor rotate, so it stands at its natural size on a normal text path. At any other size, or on a rotated path, a scalable fixed pitch font takes its place at the size asked for, so that sizing and text angle work with the terminal font as they do elsewhere. On a display scaled past 96 dots per inch it is sized that way from the start, at **console_points** points of the display's own resolution, which is what the other platforms do: the stock font's pixel size does not scale with the display, and left alone it gives an undersized window.

24.2.1 Configuration settings

graphics implements several configuration settings:

Section	Name	Function
terminal	maxxd	Sets the default character width of the main window.
terminal	maxyd	Sets the default line height of the main window.
terminal	joystick	Enables (1) or disables (0) any joystick attached.
terminal	mouse	Enables (1) or disables (0) any mouse attached.
terminal	dump_event	Dumps petit_ami event codes to stderr. 1 enables, 0 disables.
graphics	dialogerr	Sends all errors to a dialog. Default is stderr. 1 enables, 0 disables.

graphics	console_points	Sets the point size of the terminal font on a display scaled past 96 dots per inch. The default is 11.
graphics/ windows/ diagnostics	dump_messages	Dumps windows messages to stderr. 1 enables, 0 disables.

24.2.2 Bypass input

As with **terminal**, any input from **stdin** that is read directly gets echoed and line buffered.

24.2.3 Delayed output

New windows that are created will not display until output is written to them. The reason for this is to hold all geometry changes to the window (size, position, etc.) until actual presentation of the window is required.

This can have the side effect that if the program does not output anything, it can appear to malfunction by not presenting a window. For example, a program that only waits for characters to be entered into a window without first outputting any text or graphics might appear to hang.

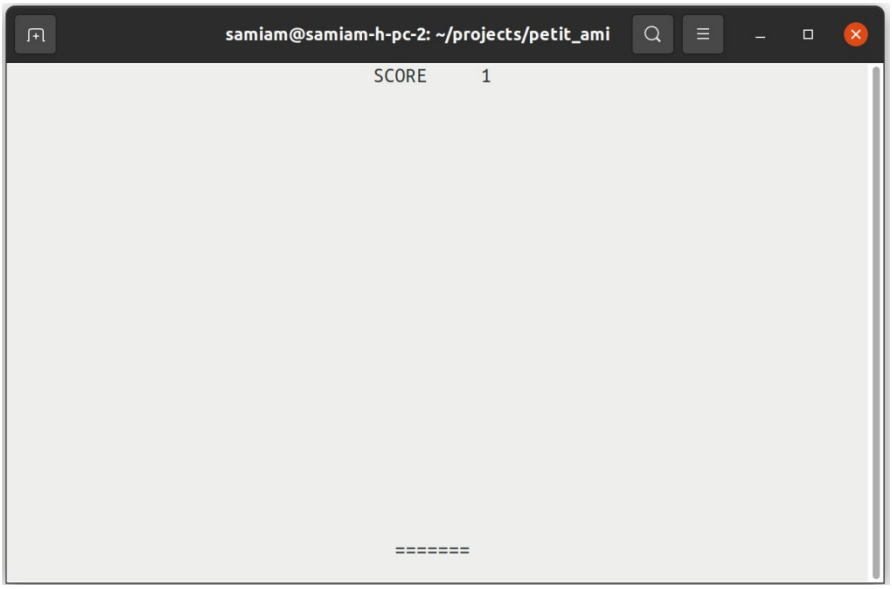
24.3 Printing

The print modules, pdfgraph and txtterminal, print to a file on Windows as on any system. A printer device is not reached yet: the modules hand a printer document to the system print spooler, and that is the Unix and macOS one. The presumed form of a Windows printer name is **lpt1:** and so on, where Unix has **lp:** and the printers by number.

25 Linux Specific details



25.1 Terminal



A Linux terminal program (xterm).

Linux uses XTERM serial control codes to function. XTERM is based on the VT terminal series from DEC (Digital Equipment Corporation), which were based on ANSI X3.64 (ISO 6429) standardized terminal control codes. XTERM supports codes from a series of VT terminals, from the VT102 onward. It also supports additional control codes to allow things such as remote operation of a mouse.

Linux terminal uses the display size it is given on startup.

At this writing, XTERM is not capable of supporting transparent mode. Instead, terminal always flips the screen buffer to a new, blank screen and the original XTERM screen is preserved. When terminal exits, the original screen is restored.

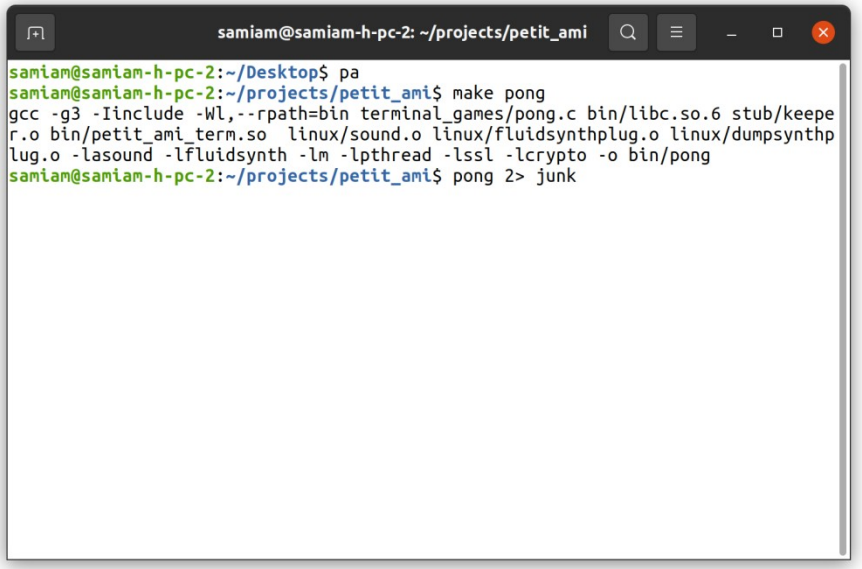
25.1.1 Configuration settings

terminal reads these settings, all of them in the **terminal** section:

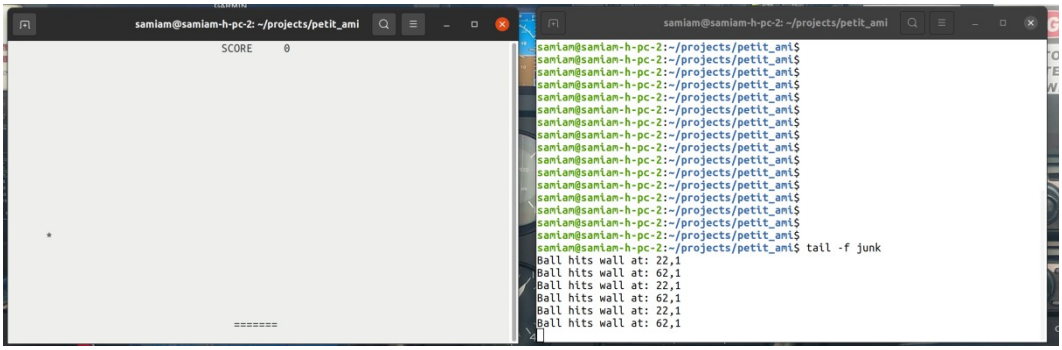
Section	Name	Function
terminal	**maxxd**	Sets the character width of the terminal buffer. The default is the width of the display at startup.
terminal	**maxyd**	Sets the line height of the terminal buffer. The default is the height of the display at startup.
terminal	**joystick**	Enables (1) or disables (0) any joystick attached.
terminal	**mouse**	Enables (1) or disables (0) any mouse attached.
terminal	**dump_event**	Dumps petit_ami event codes to stderr. 1 enables, 0 disables.
terminal	**unresponsive**	Flags a program that has not taken an event within unresponsive_time, by the terminal's title. 1 enables, 0 disables. The default is 1.
terminal	**unresponsive_kill**	Lets ctrl-c end a program flagged unresponsive. 1 enables, 0 disables. The default is 1.
terminal	**unresponsive_time**	The time without an event taken before a program is flagged unresponsive, in milliseconds. The default is 15000.
terminal	**title**	Uses the terminal's title bar for the unresponsive flag and the finished message. 1 enables, 0 disables. Off, the finished message is shown on the top line instead. The default is 1.
terminal	**hover_time**	The time the mouse rests before a hover event, in milliseconds. The default is 1000.
terminal	**escape_time**	The time a lone escape waits before it is taken as the escape key rather than the head of a sequence, in milliseconds. The default is 50.

25.1.2 Redirected files

The files **stdin** and **stdout** are redirected to the terminal surface. Any other files, including the **stderr** file, bypass the terminal module and print directly. This can cause the terminal display module to appear to malfunction. I recommend instead that you reroute the messages from **stderr** to another window:



Redirecting stderr to file “junk”.



Now prints to stderr go to a separate window.

Using a program such as “tail”. This gives you a good debug method without interfering with the main display.

25.1.3 Bypass input

If the input from **stdin** is directly read, as opposed to using the **event()** call, terminal will automatically perform echoing of all input characters, and will buffer the characters up into lines. The line is edited while it is typed: characters go in at the cursor or over what is already there, according to the insert toggle; the cursor moves by character, by word, and to either end of the line; a character is deleted forward or back, and the whole line at a stroke; and a click of the left button inside the line places the cursor. Which key does which is the event mapping of the terminal chapter, so the editing follows the keyboard the user has set rather than a list fixed here. Enter ends the line.

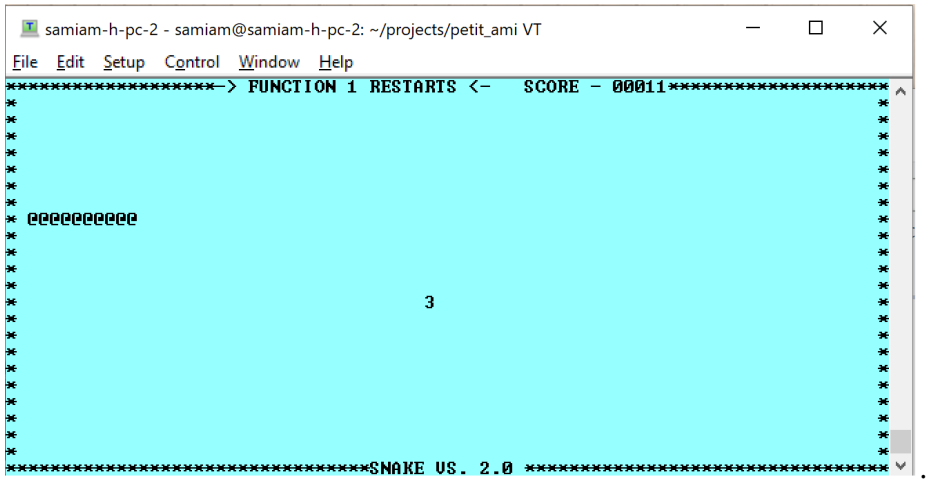
25.1.4 Remote operation

xterm control codes are widely emulated. Even the remote operation of a mouse can function on a remote computer. A ssh/ssl link that does not interfere with the control characters it is streaming can carry a remote connection. No protocol exists at this time to remote operate the joystick.

The following systems have been found to work remotely:

- Linux ssh with xterm, mouse supported.
- Windows with correct ssh in command shell, mouse not supported.
- Windows Teraterm, mouse supported.

Of course, the XWindow protocol can always be exported remotely to run any graphics window remotely, including xterm



An xterm session running on Windows TeraTerm, but hosted on a remote Linux system.

25.2 Graphics

Linux graphics has two backends. Under an X session it draws with Xlib. Under a Wayland session it speaks the Wayland protocol itself, and draws its own window frames, a compositor not usually drawing them for the client. Which one a program gets is settled when Ami is built: the Makefile takes the backend from the session that builds it, a Wayland display present choosing Wayland and otherwise Xlib, and **GRAPHICS_BACKEND=x11** or **GRAPHICS_BACKEND=wayland** on the make command line settles it by hand. The dynamically linked libraries follow the same choice. The two present the same interface, and no program is written for one or the other.

Either of them sizes the main window to 80x25 characters by default, and the terminal font to **console_points** points, which is 11. The terminal font is “courier 10 pitch”, and “courier new” where that is not installed. Fonts come from fontconfig and are drawn with FreeType, and only scalable fonts are taken: Ami sizes text freely and turns it to any angle, which a bitmap font cannot follow.

25.2.1 The X backend

Under X, Ami is an Xlib client and the window manager owns the top level window. The frame, the title bar and the buttons on it are the desktop’s: Ami neither draws them nor has any say in what they look like. Frames on child windows are Ami’s own, and are drawn the same way under either backend.

The resolution of the display comes from the **Xft.dpi** resource where the desktop declares one, and from the screen’s declared size where it does not. The resource is read because the toolkits read it: under XWayland the core geometry is synthesized for 96 dots per inch whatever the real display is, and the

effective resolution reaches X clients through the root resource database. Without it, text and dialogs come out undersized on a scaled display.

Being an X client, a program can run on one machine and display on another by exporting **DISPLAY**, as any X program can. Ami's own remote mode, in the chapter on it, is a different thing: there the program carries no display library at all.

The dynamically linked Ami libraries are built on this backend.

25.2.2 The Wayland backend

Under Wayland, Ami speaks the protocol itself. Windows are xdg-shell surfaces, and the drawing, the input and the presentation sit behind a platform display layer of Ami's own, **linux/wayland/pdisplay.c**, which the graphics module talks to in place of a library like Xlib.

A Wayland compositor does not usually decorate its clients, so Ami draws the whole frame itself, on the main window as much as on the children: the border, the title bar, the title, the buttons, and the edges the mouse takes hold of to resize. What that frame looks like is the desktop's business, and is what the two sections below are about.

Two things a program asks for do not always arrive as asked. The compositor places top level windows: a position set on the main window is a wish, and where the window lands is the compositor's decision. Child windows move within their parent exactly as they are placed. And a compositor may hand a window back a pixel or two off the size that was asked for, to suit its own geometry, so a size set and then read is not always the same number.

25.2.2.1 Gnome

On GNOME the frames are Adwaita: the title centered on the header, and round minimize, maximize and close buttons at the right, in the light or the dark header color the desktop is set to. The menu bar and the pulldowns follow, flat, with an accent underbar beneath the pressed entry.

The color scheme is the desktop's own setting, **org.gnome.desktop.interface color-scheme**, and it is watched while the program runs with **gsettings monitor**, so turning the desktop dark takes the frames of a running program with it. **frame_theme** in the configuration pins it instead.

25.2.2.2 Plasma

On KDE Plasma the frames are Breeze: a flat header with the title centered, and the minimize, maximize and close buttons drawn as plain glyphs, set adjacent, at the right. Breeze wears a slimmer title bar than Adwaita. In the menus the pressed entry is filled with the desktop's accent color.

Plasma keeps its colors in **kdeglobals**. That file is read at startup and watched with inotify, so a change of color scheme reaches the frames of a running program. Where it cannot be read, Breeze's own colors stand in.

25.2.3 Configuration settings

graphics implements several configuration settings:

Section	Name	Function
terminal	maxxd	Sets the default character width of the main window.
terminal	maxyd	Sets the default line height of the main window.
terminal	joystick	Enables (1) or disables (0) any joystick attached.
terminal	mouse	Enables (1) or disables (0) any mouse attached.
terminal	dump_event	Dumps petit_ami event codes to stderr. 1 enables, 0 disables.
graphics	dialogerr	Sends all errors to a dialog. Default is stderr. 1 enables, 0 disables.
graphics	console_points	Sets the point size of the terminal font. The default is 11.
graphics	frame_theme	Forces the window frame Ami draws under Wayland light or dark. The values are “light” and “dark”. Unset, the frame follows the desktop.
graphics/ xwindow/ diagnostics	dump_messages	Dumps display layer events to stderr, X events under Xlib and Wayland events under Wayland. 1 enables, 0 disables.
graphics/ xwindow/ diagnostics	print_font_metrics	Prints the font metrics of the terminal type font.

25.2.4 The desktop packages

The look of the widgets, and of the window frames under Wayland, follows the desktop the program finds itself on. Both packages of each pair are linked into every program, and each decides at startup whether the session is its own: **XDG_CURRENT_DESKTOP** naming KDE gives the Plasma package, and anything else gives the Gnome one. Nothing is settled when the program is built and nothing is asked of the program, so one binary carried to another desktop dresses itself that way when it starts.

The widget packages are **gnome_widgets** and **plasma_widgets**, both in **portable**, and they serve either backend. The frames Ami draws under Wayland are chosen the same way, by a decoration package for each of the two desktops.

25.2.5 The widget theme

The colors the widget packages draw with are a table, and the table can be overridden. The **widgets** section of the configuration carries a **theme** block, and each entry in it names one point of the table and gives it a value: a color as three decimal components, red green blue from 0 to 255, or as a hex triplet with an optional leading **#**, and the points that are not colors as a plain number. Points not named keep their built in values, and a name that is not a theme point is passed over, so a configuration written for a later version still loads here.

```
begin widgets
   begin theme
       back      252 252 252
       backhover 230 230 230
       focus     53 132 228
   end
end
```

The names are the **th_** list in **portable/gnome_widgets.c** with the **th_** dropped, and the **ami.cfg** of the distribution carries the common ones as comments.

A theme is not usually written by hand. **css2theme** reads the palette out of a GTK theme and writes the matching Ami theme into the configuration file:

`css2theme`
With no argument it takes the theme the desktop is using now and updates **ami.cfg** in the user's home directory, which reaches all of that user's programs. Given a name it converts an installed theme, **Yaru-dark** for one, and given a file ending in **.css** it reads that instead. **-o** names another configuration file to update, **-p** prints the theme block rather than writing anything, and **-n** prints the names it found and what each one mapped to.

GTK themes declare their palette with **@define-color** lines, and that is what is read; the thousands of compiled rules that follow carry literal colors and are not a palette. A theme whose stylesheet lives inside a compiled gresource is followed there. Adwaita, the stock GNOME theme, keeps its stylesheet inside the GTK library itself, where there is nothing to read, and the built in theme stays in use.

25.2.6 Redirected files

The **stdin** and **stdout** are redirected to the main window. All other files, including **stderr**, are left to their default stream locations. This can be useful. For example all stderr prints from a program started in a xterm window print to the xterm window:



25.2.7 Bypass input

As with terminal, any input from **stdin** that is read directly gets echoed and line buffered, and edited the same way.

25.2.8 Delayed output

New windows that are created will not display until output is written to them. The reason for this is to hold all geometry changes to the window (size, position, etc.) until actual presentation of the window is required.

This can have the side effect that if the program does not output anything, it can appear to malfunction by not presenting a window. For example, a program that only waits for characters to be entered into a window without first outputting any text or graphics might appear to hang.

25.3 Sound

The sound module implements the plug-in standard (17 ”Sound module plugins”). The reason for this is that under Linux, even though the ALSA interface supports a plug in synthesizer, that is not set up by default in most or all distributions, and having it defined as a plug-in means your program can ship with a synthesizer installed with it. At this writing, I included a plug-in for Fluidsynth, a wave table based synthesizer that uses the Sound Fonts format. The default sound font is:

/usr/share/sounds/sf2/FluidR3_GM.sf2

The Fluidsynth plug-in creates 4 instances of the synthesizer by default, and these will appear in the front of the MIDI synthesizer output device table.

The Fluidsynth plug-in does not take any parameter sets at this writing.

Sound also has a “dump” plug-in. This plug-in will print a decoded dump of any MIDI messages that pass through it, and then send the MIDI messages on to another input port. This is done by a parameter set:

connect <input port>

Where <input port> is another MIDI input port.

25.3.1 ALSA device names

For the most part, sound takes the device names given by ALSA. The exception would be for **sound** module plug-ins. The name of the device also includes details about the device itself. The important thing when selecting a device is the device number and kind (MIDI in/out, wave in/out). This unambiguously selects the device.

Care must be taken when selecting an input wave device. ALSA provides a lot of automatic software format conversions, but at this writing, there is no automatic method in use to determine the native parameters of an input device. This can lead to connecting an input device with conversions to an output device with conversions, which both wastes CPU time, creates unnecessary duplicate data, and possibly degrades the sound itself.

sound chooses the best format possible for any input. The total number of channels is limited to 8, and the sample rate is limited to 44100 (CD quality sound). ALSA specifies inputs with conversions as having more than 100 channels, which it does by duplicating samples. This is obviously a waste of time and space.

Thus when choosing inputs, look for the devices that say "without any conversions". These devices will have the actual capabilities of the input device instead of "pseudo-parameters", which give the maximum possible parameters that ALSA can fit the input to, with conversions.

One other quirk of ALSA is that it will classify even single channel inputs like a single microphone as

two channel devices by sample duplication. ALSA will throw an error for attempts to specify these as single channel devices.

25.4 Network

Network can be used to both connect to remote computers as well as local computers. The local address will be 127.0.0.1, and this will be the address of any server connection. To use network for program to program communication, you can use either TCP/IP or message based communication, and you can use plain text or TLS encrypted communications. A communication to the same machine is considered a reliable message link.

26 Mac OS X specific details



26.1 Contents of the Ami Mac OS X port

Half of Ami on the Mac is the code the other Unix builds use. What is particular to the Mac is the display, the sound and the system event layer:

Module	Where it comes from
stdio	libc/stdio.c, the same source every platform uses.
services	linux/services.c, compiled for Darwin. The calls are POSIX.
network	linux/network.c, compiled for Darwin, over the same sockets and OpenSSL.
terminal	linux/terminal.c, compiled for Darwin.
graphics	macosx/graphics.c with the Objective-C shim macosx/graphics_cocoa.m. The Mac's own.
sound	macosx/sound.c. The Mac's own.
system event	macosx/system_event.c. The Mac's own.
screen capture	macosx/screen_capture.c, which writes the same concatenated PNGs the other platforms do.

26.2 Terminal

terminal on the Mac is the Linux module built for Darwin. It drives the terminal with the same xterm control codes, and reads the same five configuration settings. Use the comments in the chapter 25 Linux Specific details.

26.3 Graphics

graphics draws with CoreGraphics and sets text with CoreText. The windows and the events are Cocoa, reached through an Objective-C shim, **macosx/graphics_cocoa.m**, which is the only Objective-C in Ami.

The default for windows comes up 80x25 characters. The four standard fonts are taken from the Mac's own: the terminal font is the system monospaced face, SF Mono, falling back through Menlo, Monaco, Courier New and Courier; the book font through Georgia, Palatino, Times New Roman and Times; the sign and technical fonts through Helvetica Neue, Helvetica, Arial and Verdana. The default height is 16 points, and **console_points** sets it otherwise. The graphical pixel size in x and y of the window is

determined by the font size in 80x25. This of course can be changed by the program before the window is displayed.

26.3.1 Widgets

The widgets on the Mac are Cocoa's own controls: a button is an **NSButton**, a list is an **NSTableView**, and so on down the set. There is no widget package to choose and no theme table to override, as there is on Linux; the widgets look like the desktop's because they are the desktop's.

26.3.2 Configuration settings

Graphics reads the same settings the other platforms do, with its diagnostics in a section of its own:

Section	Name	Function
terminal	**maxxd**	Sets the default character width of the main window.
terminal	**maxyd**	Sets the default line height of the main window.
terminal	**joystick**	Enables (1) or disables (0) any joystick attached.
terminal	**mouse**	Enables (1) or disables (0) any mouse attached.
terminal	**dump_event**	Dumps petit_ami event codes to stderr. 1 enables, 0 disables.
graphics	**dialogerr**	Sends all errors to a dialog. Default is stderr. 1 enables, 0 disables.
graphics	**console_points**	Sets the point size of the terminal font. The default is 16.
graphics/macosx/ diagnostics	**dump_messages**	Dumps display layer events to stderr. 1 enables, 0 disables.
graphics/macosx/ diagnostics	**print_font_metrics**	Prints the font metrics of the terminal type font.

26.4 Sound

sound is built on CoreMIDI for the MIDI ports, AudioToolbox for the rest, and MusicPlayer for MIDI file playback. Synthesizer output on port 1 is the built in Apple DLS software synthesizer, so a program has a synthesizer without anything being installed for it, which is what the Fluidsynth plug-in provides on Linux. The synthesizer ports past the first are the CoreMIDI destinations the machine has. Wave output is streamed through AudioQueue.

The Mac does not implement the sound plug in standard.

27 License



The code of Ami is under the BSD license, which is also the file **LICENSE** at the root of the tree and the notice at the head of every source file:

BSD LICENSE INFORMATION

Copyright (C) 2021 - Scott A. Franco

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

3. Neither the name of the project nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE PROJECT AND CONTRIBUTORS ``AS IS'' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE PROJECT OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

27.1 Code in Ami that is not mine

Two files carry someone else's work. The DTLS part of network, in **linux/network.c** and **windows/network.c**, is built on example code by Robin Seggelmann, Michael Tuexen, Felix

Weinrank and the OpenSSL project authors, under the same BSD terms. The notice naming them stands at the head of both files and stays there.

27.2 What the tree carries besides Ami

The distribution pulls three programs as submodules, each from its own upstream and each under its own license:

- nano, the text editor, under the GNU General Public License, version 2 or later.
- htop, the process viewer, under the GNU General Public License, version 2.
- The BSD games, whose terms are set out game by game in the **COPYING** file of that tree.

They are ports, and they are not part of the library. Each is taken unmodified at a fixed version and built against Ami’s curses layer, **apis/curses.c**, which is Ami’s own and BSD like the rest of it. Nothing of theirs is in Ami and nothing of Ami’s is in them. A tree cloned without its submodules carries none of it.

27.3 What a program links

A program built with Ami links libraries that are not Ami’s and are not covered by anything here. On Linux those are ALSA and Fluidsynth for sound, OpenSSL for the secure side of network, FreeType and fontconfig for text, and then either Xlib or the Wayland client libraries with xkbcommon; libpng and zlib come in with the screen capture. Windows and the Mac link their own systems’ libraries instead. Each of them carries its own terms, and anyone shipping a program built with Ami wants to know what those are for what they ship.